# Università degli Studi di Napoli Federico II

Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione


---

*Corso di Laurea Magistrale in Ingegneria Informatica*

---



# Elaborato di Architettura dei Sistemi Digitali

a.a. 2024-25


Studenti:

Anna Flavia De Rosa M63001797

Elisa Cervone M63001799

Francesco Avallone M63001682

# Sommario

**Capitolo 1: reti combinatorie elementari..... 6**
Esercizio 1: Multiplexer 16:1..... 6
Esercizio 1.1..... 6
Progetto e architettura..... 6
Implementazione..... 8
Simulazione..... 11
Esercizio 1.2..... 14
Progetto e architettura..... 14
Implementazione..... 15
Simulazione..... 18
Esercizio 1.3..... 22
Progetto e architettura..... 22
Implementazione..... 23
Sintesi su board di sviluppo..... 26
Esercizio 2: Sistema ROM+M..... 28
Esercizio 2.1..... 28
Progetto e architettura..... 28
Implementazione..... 28
Simulazione..... 31
Esercizio 2.2..... 32
Progetto e architettura..... 32
Implementazione..... 32
Sintesi su board di sviluppo..... 32
**Capitolo 2: reti sequenziali elementari..... 34**
Esercizio 3: Riconoscitore di sequenze..... 34
Esercizio 3.1..... 34
Progetto e architettura..... 34
Implementazione..... 35
Simulazione..... 38
Esercizio 3.2..... 41
Progetto e architettura..... 41
Implementazione..... 41
Sintesi su board di sviluppo..... 47
Esercizio 4: Shift Register..... 49
Esercizio 4.1..... 49
Approccio comportamentale..... 49
Progetto e architettura..... 49
Implementazione..... 50
Simulazione..... 52
Approccio strutturale..... 54
Progetto e architettura..... 54

Implementazione..... 55
Simulazione..... 58
Esercizio 5: Cronometro..... 59
Esercizio 5.1..... 59
Progetto e architettura..... 59
Implementazione..... 60
Simulazione..... 66
Esercizio 5.2..... 69
Progetto e architettura..... 69
Implementazione..... 69
Sintesi su board di sviluppo..... 78
Esercizio 5.3..... 81
Progetto e architettura..... 81
Implementazione..... 82
Simulazione..... 89
Progetto e architettura - Parte Opzionale..... 91
Implementazione..... 93
Sintesi su board di sviluppo..... 111
Esercizio 6: Sistema di lettura-elaborazione-scrittura PO_PC..... 113
Esercizio 6.1..... 113
Progetto e architettura..... 113
Implementazione..... 114
Simulazione..... 120
Esercizio 6.2..... 122
Progetto e architettura..... 122
Implementazione..... 123
Sintesi su board di sviluppo..... 127
**Capitolo 3: Macchine Aritmetiche..... 129**
Esercizio 7: Moltiplicatore di Booth..... 129
Esercizio 7.1..... 129
Progetto e architettura..... 129
Implementazione..... 131
Simulazione..... 140
Esercizio 7.2..... 145
Progetto e architettura..... 145
Implementazione..... 145
Sintesi su board di sviluppo..... 147
Timing analysis..... 149
**Capitolo 4: Comunicazione con handshaking..... 151**
Esercizio 8..... 151
Esercizio 8.1..... 151
Progetto e architettura..... 151
Implementazione..... 153

Simulazione..... 164
**Capitolo 5: Processore..... 169**
Esercizio 9..... 169
Analisi dell'architettura..... 169
Istruzioni ISUB e ISTORE..... 172
Modifica della IAND..... 180
**Capitolo 6: Interfaccia seriale..... 185**
Esercizio 10..... 185
Progetto e architettura..... 185
Implementazione..... 187
Simulazione..... 197
**Capitolo 7: Switch Multistadio..... 202**
Esercizio 11.1..... 202
Progetto e architettura..... 202
Implementazione..... 206
Simulazione..... 214
**Capitolo 8: Prova di esame 19 dicembre 2024..... 218**
Esercizio 12..... 218
Progetto e architettura..... 218
Implementazione..... 220
Simulazione..... 231
**Appendice..... 236**
Multiplexer indirizzabile 2:1..... 236
Progetto e architettura..... 236
Implementazione..... 236
Multiplexer indirizzabile 4:1..... 237
Progetto e architettura..... 237
Implementazione..... 238
Demultiplexer indirizzabile 2:1..... 239
Progetto e architettura..... 239
Implementazione..... 239
Button Debouncer..... 241
Progetto e architettura..... 241
Implementazione..... 241
Ripple Carry Adder..... 244
Progetto e architettura..... 244
Implementazione..... 245
Carry Look Ahead..... 247
Progetto e architettura..... 247
Implementazione..... 248
Display a sette segmenti..... 252
Progetto e architettura..... 252
Implementazione..... 253

Contatore modulo N.....261
Progetto e architettura.....261
Implementazione.....261

# Capitolo 1: reti combinatorie elementari

## Esercizio 1: Multiplexer 16:1

### Esercizio 1.1

Progettare, implementare in VHDL e testare mediante simulazione un **multiplexer indirizzabile 16:1**, utilizzando un approccio di progettazione per composizione a partire da **multiplexer 4:1.**

#### *Progetto e architettura*

La rete che vogliamo implementare è la seguente: realizziamo un multiplexer 16:1 (con quindi 16 fili in ingresso e 1 in uscita) seguendo un **approccio strutturale**. A partire da componenti più semplici, realizziamo il multiplexer 16:1 per composizione; in particolare, i componenti che utilizziamo sono i seguenti:

- 5 multiplexer **4:1**
    - a loro volta costituiti ciascuno da 3 multiplexer **2:1**

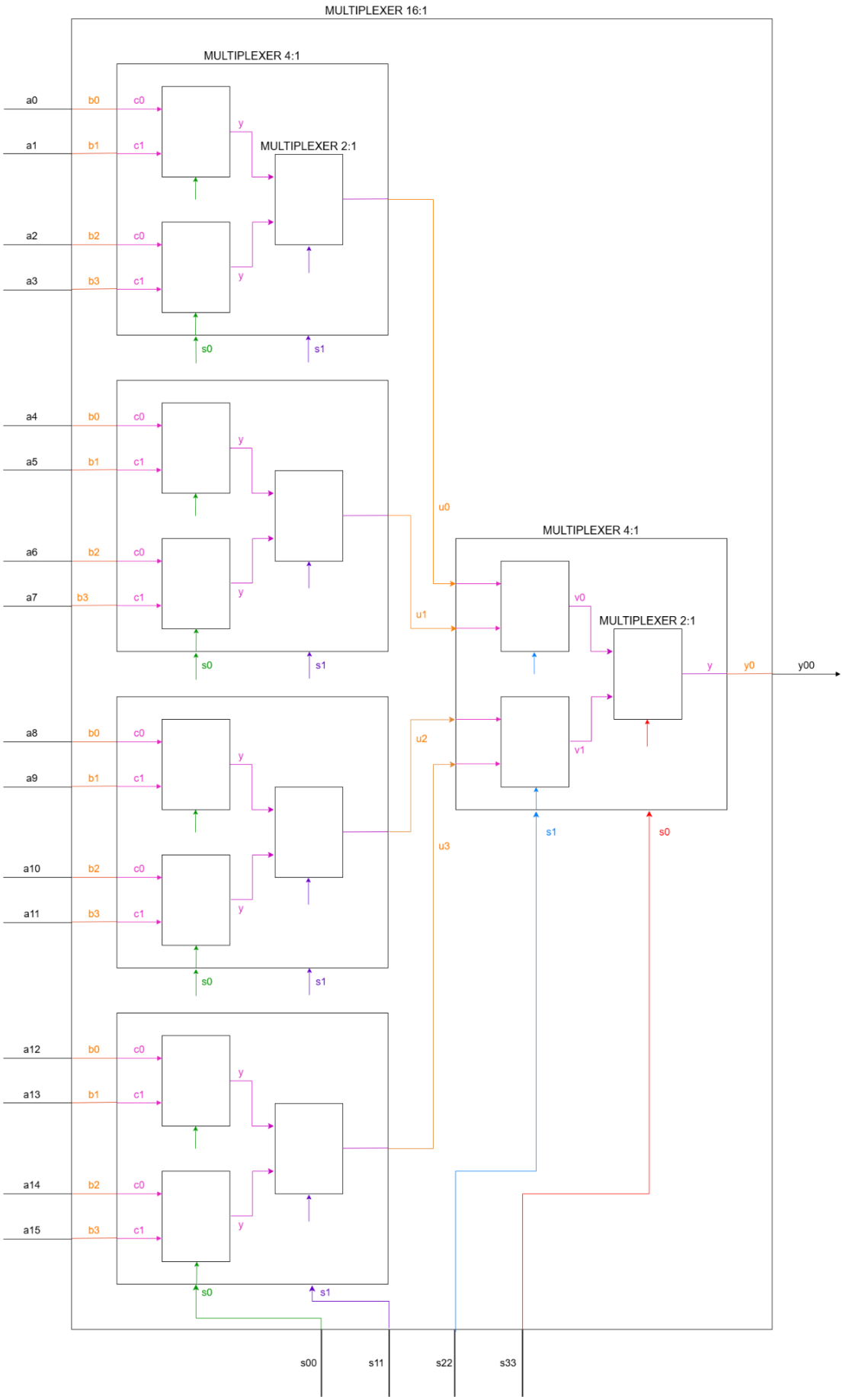



*Figura 1.1. Multiplexer 16:1 realizzato per composizione*

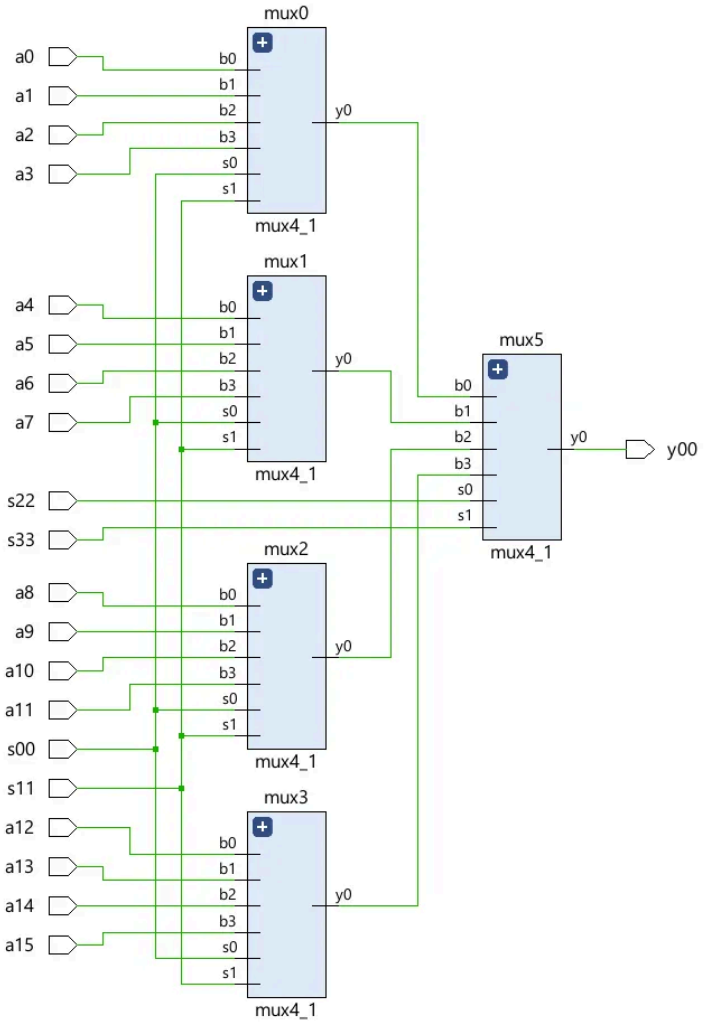



*Figura 1.2 Schematic del sistema implementato in Vivado*

### *Implementazione*

Il **top module** è il seguente:

```
-- Definizione dell'interfaccia del modulo mux16_1
entity mux16_1 is
    Port ( a0 : in STD_LOGIC;
           a1 : in STD_LOGIC;
           a2 : in STD_LOGIC;
           a3 : in STD_LOGIC;
           a4 : in STD_LOGIC;
           a5 : in STD_LOGIC;
           a6 : in STD_LOGIC;
           a7 : in STD_LOGIC;
           a8 : in STD_LOGIC;
           a9 : in STD_LOGIC;
           a10 : in STD_LOGIC;
           a11 : in STD_LOGIC;
           a12 : in STD_LOGIC;
           a13 : in STD_LOGIC;
```

```
           a14 : in STD_LOGIC;
           a15 : in STD_LOGIC;
           s00 : in STD_LOGIC;
           s11 : in STD_LOGIC;
           s22 : in STD_LOGIC;
           s33 : in STD_LOGIC;
           y00 : out STD_LOGIC);
end mux16_1;

-- Definizione architettura del modulo mux16_1 con una descrizione strutturale
architecture structural of mux16_1 is

    signal u0 : STD_LOGIC := '0';
    signal u1 : STD_LOGIC := '0';
    signal u2 : STD_LOGIC := '0';
    signal u3 : STD_LOGIC := '0';

    component mux4_1
        port(  b0 : in STD_LOGIC;
               b1 : in STD_LOGIC;
               b2 : in STD_LOGIC;
               b3 : in STD_LOGIC;
               s0 : in STD_LOGIC;
               s1 : in STD_LOGIC;
               y0 : out STD_LOGIC
           );
    end component;

    begin

        mux0: mux4_1
            Port map(   b0 => a0,
                        b1 => a1,
                        b2 => a2,
                        b3 => a3,
                        s0 => s00,
                        s1 => s11,
                        y0 => u0
                    );

        mux1: mux4_1
            Port map(   b0 => a4,
                        b1 => a5,
                        b2 => a6,
                        b3 => a7,
                        s0 => s00,
                        s1 => s11,
                        y0 => u1
                    );

        mux2: mux4_1
            Port map(   b0 => a8,
                        b1 => a9,
                        b2 => a10,
                        b3 => a11,
                        s0 => s00,
```

```
                        s1 => s11,
                        y0 => u2
                    );

        mux3: mux4_1
            Port map(   b0 => a12,
                        b1 => a13,
                        b2 => a14,
                        b3 => a15,
                        s0 => s00,
                        s1 => s11,
                        y0 => u3
                    );

        mux5: mux4_1
            Port map(   b0 => u0,
                        b1 => u1,
                        b2 => u2,
                        b3 => u3,
                        s0 => s22,
                        s1 => s33,
                        y0 => y00
                    );

end structural;
```

Il **multiplexer 4:1** è implementato anch'esso per composizione nel modo seguente:

```
entity mux4_1 is
    Port ( b0 : in STD_LOGIC;
           b1 : in STD_LOGIC;
           b2 : in STD_LOGIC;
           b3 : in STD_LOGIC;
           s0 : in STD_LOGIC;
           s1 : in STD_LOGIC;
           y0 : out STD_LOGIC);
end mux4_1;

architecture structural of mux4_1 is

    signal v0 : STD_LOGIC := '0';
    signal v1 : STD_LOGIC := '0';

    component mux2_1
        port(  c0 : in STD_LOGIC;
               c1 : in STD_LOGIC;
               s : in STD_LOGIC;
               y : out STD_LOGIC
            );
    end component;

begin

    mux0: mux2_1
        Port map(   c0 => b0,
                    c1 => b1,
```

```
                    s => s0,
                    y => v0
                );

    mux1: mux2_1
        Port map(   c0 => b2,
                    c1 => b3,
                    s => s0,
                    y => v1
                );

    mux2: mux2_1
        Port map(   c0 => v0,
                    c1 => v1,
                    s => s1,
                    y => y0
                );

end structural;
```

L'implementazione del componente **multiplexer 2:1** è in Appendice.

*Simulazione*

Il **testbench** utilizzato per simulare il componente mux16_1 è il seguente:

```
entity tb_mux16_1 is
end tb_mux16_1;

architecture tb of tb_mux16_1 is

    component mux16_1
        port (a0  : in std_logic;
              a1  : in std_logic;
              a2  : in std_logic;
              a3  : in std_logic;
              a4  : in std_logic;
              a5  : in std_logic;
              a6  : in std_logic;
              a7  : in std_logic;
              a8  : in std_logic;
              a9  : in std_logic;
              a10 : in std_logic;
              a11 : in std_logic;
              a12 : in std_logic;
              a13 : in std_logic;
              a14 : in std_logic;
              a15 : in std_logic;
              s00 : in std_logic;
              s11 : in std_logic;
              s22 : in std_logic;
              s33 : in std_logic;
              y00 : out std_logic);
    end component;

    signal a0  : std_logic;
    signal a1  : std_logic;
```

```
    signal a2  : std_logic;
    signal a3  : std_logic;
    signal a4  : std_logic;
    signal a5  : std_logic;
    signal a6  : std_logic;
    signal a7  : std_logic;
    signal a8  : std_logic;
    signal a9  : std_logic;
    signal a10 : std_logic;
    signal a11 : std_logic;
    signal a12 : std_logic;
    signal a13 : std_logic;
    signal a14 : std_logic;
    signal a15 : std_logic;
    signal s00 : std_logic;
    signal s11 : std_logic;
    signal s22 : std_logic;
    signal s33 : std_logic;
    signal y00 : std_logic;

begin

    dut : mux16_1
    port map (a0  => a0,
              a1  => a1,
              a2  => a2,
              a3  => a3,
              a4  => a4,
              a5  => a5,
              a6  => a6,
              a7  => a7,
              a8  => a8,
              a9  => a9,
              a10 => a10,
              a11 => a11,
              a12 => a12,
              a13 => a13,
              a14 => a14,
              a15 => a15,
              s00 => s00,
              s11 => s11,
              s22 => s22,
              s33 => s33,
              y00 => y00);

    stimuli : process
    begin
        -- EDIT Adapt initialization as needed
        a0 <= '0';
        a1 <= '0';
        a2 <= '0';
        a3 <= '0';
        a4 <= '0';
        a5 <= '0';
        a6 <= '0';
        a7 <= '0';
```

```
        a8 <= '0';
        a9 <= '0';
        a10 <= '0';
        a11 <= '0';
        a12 <= '0';
        a13 <= '0';
        a14 <= '0';
        a15 <= '0';
        s00 <= '0';
        s11 <= '0';
        s22 <= '0';
        s33 <= '0';

        -- EDIT Add stimuli here
        wait for 100 ns;

        a1 <= '1';
        s33 <= '1';

        wait for 10 ns;

        s00 <= '1';
        s22 <= '1';
        s33 <= '0';
        a1 <= '0';
        a5 <= '1';

        wait;
    end process;

end tb;

-- Configuration block below is required by some simulators. Usually no need to edit.

configuration cfg_tb_mux16_1 of tb_mux16_1 is
    for tb
    end for;
end cfg_tb_mux16_1;
```

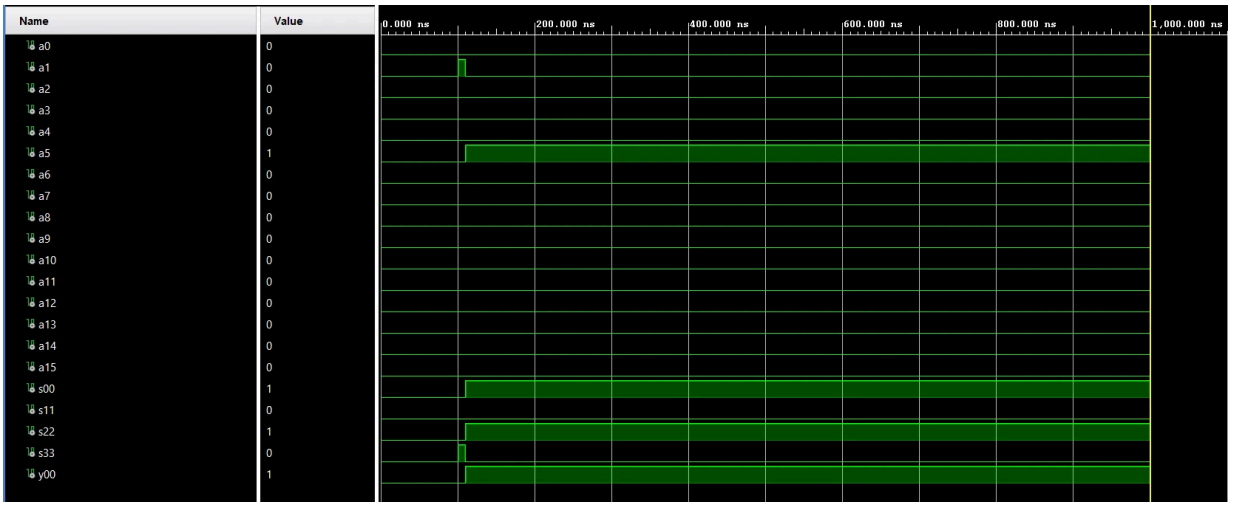



*Figura 1.3 Simulazione*

## Esercizio 1.2

Utilizzando il componente sviluppato al punto precedente, progettare, implementare in VHDL e testare mediante simulazione una rete di interconnessione a 16 sorgenti e 4 destinazioni.

### *Progetto e architettura*

Realizziamo la rete di interconnessione 16:4 **per composizione** nel modo seguente.

La rete prende in ingresso 16 sorgenti; queste vanno in ingresso ad un **multiplexer 16:1** (lo stesso sviluppato al punto precedente) che restituisce quindi un unico segnale in uscita.

L'uscita del multiplexer 16:1 la poniamo in ingresso ad un **demultiplexer 1:4**, così da connettere le 16 sorgenti con 4 destinazioni. Il demultiplexer 1:4 è implementato secondo un approccio strutturale per composizione, utilizzando **demultiplexer 1:2**.

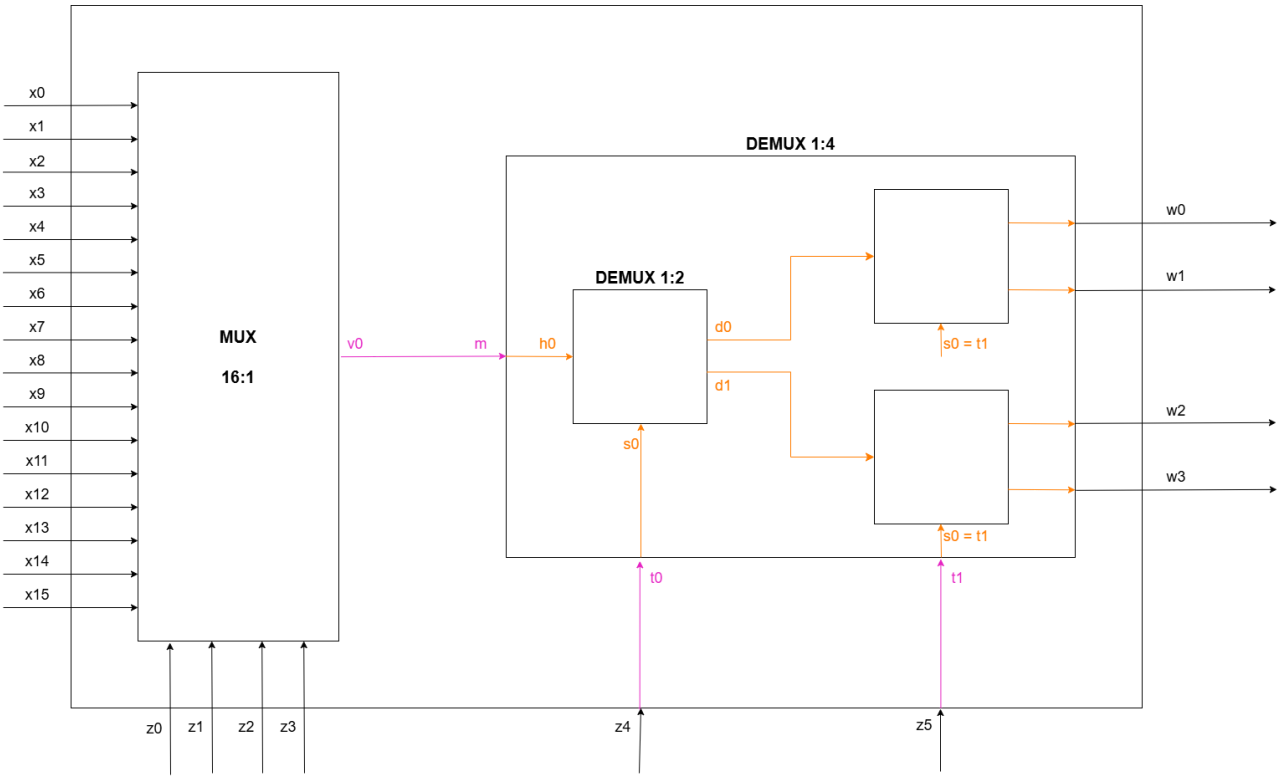



*Figura 1.4 Architettura della rete di interconnessione 16:4*

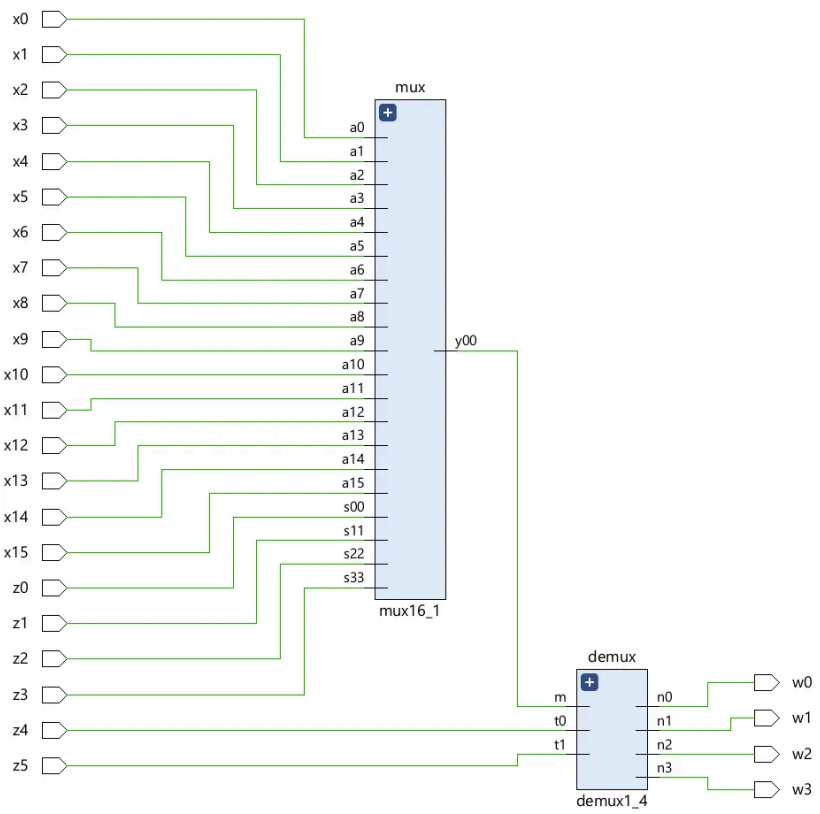



*Figura 1.5 Schematic del sistema complessivo implementato in Vivado*

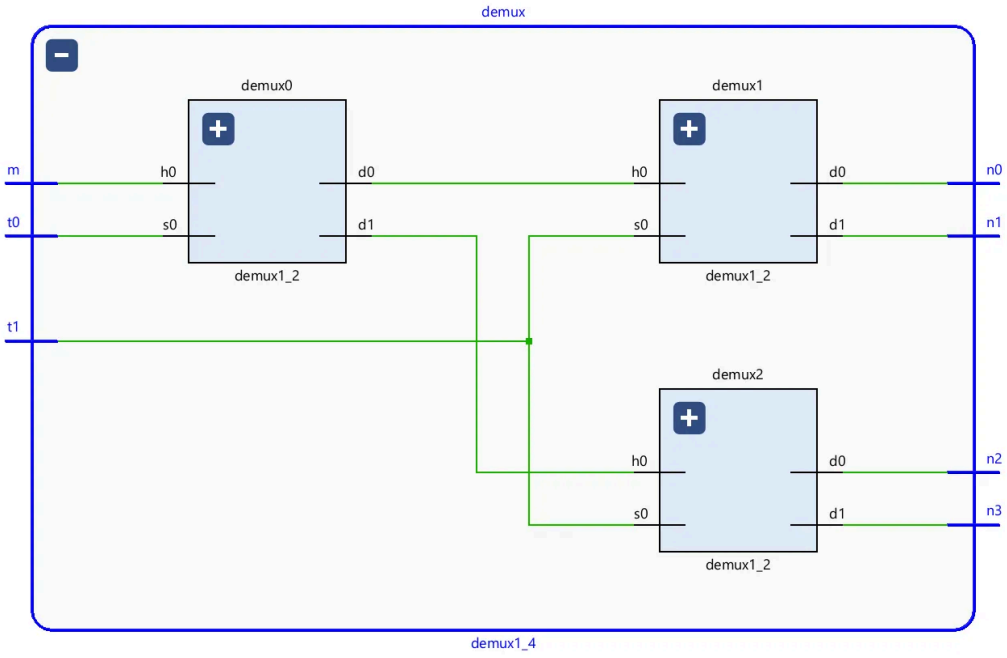



*Figura 1.6 Schematic del demultiplexer 1:4*

## *Implementazione*

Il top module, **rete16_4** è implementato secondo un approccio strutturale nel modo seguente:

```
entity rete16_4 is
    Port ( x0 : in STD_LOGIC;
           x1 : in STD_LOGIC;
```

```
           x2 : in STD_LOGIC;
           x3 : in STD_LOGIC;
           x4 : in STD_LOGIC;
           x5 : in STD_LOGIC;
           x6 : in STD_LOGIC;
           x7 : in STD_LOGIC;
           x8 : in STD_LOGIC;
           x9 : in STD_LOGIC;
           x10 : in STD_LOGIC;
           x11 : in STD_LOGIC;
           x12 : in STD_LOGIC;
           x13 : in STD_LOGIC;
           x14 : in STD_LOGIC;
           x15 : in STD_LOGIC;
           z0 : in STD_LOGIC;
           z1 : in STD_LOGIC;
           z2 : in STD_LOGIC;
           z3 : in STD_LOGIC;
           z4 : in STD_LOGIC;
           z5 : in STD_LOGIC;
           w0 : out STD_LOGIC;
           w1 : out STD_LOGIC;
           w2 : out STD_LOGIC;
           w3 : out STD_LOGIC);
end rete16_4;

architecture Structural of rete16_4 is
    signal v0 : STD_LOGIC := '0';

    component demux1_4
        port(  m : in STD_LOGIC;
               n0 : out STD_LOGIC;
               n1 : out STD_LOGIC;
               n2 : out STD_LOGIC;
               n3 : out STD_LOGIC;
               t0 : in STD_LOGIC;
               t1 : in STD_LOGIC
                );
    end component;

     component mux16_1
        port(  a0 : in STD_LOGIC;
               a1 : in STD_LOGIC;
               a2 : in STD_LOGIC;
               a3 : in STD_LOGIC;
               a4 : in STD_LOGIC;
               a5 : in STD_LOGIC;
               a6 : in STD_LOGIC;
               a7 : in STD_LOGIC;
               a8 : in STD_LOGIC;
               a9 : in STD_LOGIC;
               a10 : in STD_LOGIC;
               a11 : in STD_LOGIC;
               a12 : in STD_LOGIC;
               a13 : in STD_LOGIC;
               a14 : in STD_LOGIC;
```

```
                a15 : in STD_LOGIC;
                s00 : in STD_LOGIC;
                s11 : in STD_LOGIC;
                s22 : in STD_LOGIC;
                s33 : in STD_LOGIC;
                y00 : out STD_LOGIC
                );
    end component;

begin
    mux: mux16_1
        Port map(   a0=>x0,
                    a1=>x1,
                    a2=>x2,
                    a3=>x3,
                    a4=>x4,
                    a5=>x5,
                    a6=>x6,
                    a7=>x7,
                    a8=>x8,
                    a9=>x9,
                    a10=>x10,
                    a11=>x11,
                    a12=>x12,
                    a13=>x13,
                    a14=>x14,
                    a15=>x15,
                    s00=>z0,
                    s11=>z1,
                    s22=>z2,
                    s33=>z3,
                    y00=>v0
                    );

    demux: demux1_4
        Port map(   m=>v0,
                    n0=>w0,
                    n1=>w1,
                    n2=>w2,
                    n3=>w3,
                    t0=>z4,
                    t1=>z5
                    );


end Structural;
```

Il componente **mux16_1** è lo stesso implementato al punto precedente.

Il componente **demux1_4** è implementato per composizione nel modo seguente:

```
entity demux1_4 is
    Port ( m : in STD_LOGIC;
           n0 : out STD_LOGIC;
           n1 : out STD_LOGIC;
           n2 : out STD_LOGIC;
```

```
           n3 : out STD_LOGIC;
           t0 : in STD_LOGIC;
           t1 : in STD_LOGIC);
end demux1_4;

architecture Structural of demux1_4 is

    signal v0 : STD_LOGIC := '0';
    signal v1 : STD_LOGIC := '0';

    component demux1_2
        port(  h0 : in STD_LOGIC;
                        d0 : out STD_LOGIC;
                        d1 : out STD_LOGIC;
                        s0 : in STD_LOGIC
                        );
    end component;

begin

    demux0: demux1_2
        Port map(   h0 => m,
                    d0 => v0,
                    d1 => v1,
                    s0 => t0
                             );

    demux1: demux1_2
        Port map(   h0 => v0,
                    d0 => n0,
                    d1 => n1,
                    s0 => t1
                             );
    demux2: demux1_2
        Port map(   h0 => v1,
                    d0 => n2,
                    d1 => n3,
                    s0 => t1
                             );


end Structural;
```

L'implementazione del componente **demultiplexer 2:1** è in Appendice.

*Simulazione*

Il **testbench** utilizzato per simulare la rete 16:4 è il seguente:

```
entity tb_rete16_4 is
end tb_rete16_4;

architecture tb of tb_rete16_4 is

    component rete16_4
        port (x0  : in std_logic;
              x1  : in std_logic;
```

```
            x2  : in std_logic;
            x3  : in std_logic;
            x4  : in std_logic;
            x5  : in std_logic;
            x6  : in std_logic;
            x7  : in std_logic;
            x8  : in std_logic;
            x9  : in std_logic;
            x10 : in std_logic;
            x11 : in std_logic;
            x12 : in std_logic;
            x13 : in std_logic;
            x14 : in std_logic;
            x15 : in std_logic;
            z0  : in std_logic;
            z1  : in std_logic;
            z2  : in std_logic;
            z3  : in std_logic;
            z4  : in std_logic;
            z5  : in std_logic;
            w0  : out std_logic;
            w1  : out std_logic;
            w2  : out std_logic;
            w3  : out std_logic);
    end component;

    signal x0  : std_logic;
    signal x1  : std_logic;
    signal x2  : std_logic;
    signal x3  : std_logic;
    signal x4  : std_logic;
    signal x5  : std_logic;
    signal x6  : std_logic;
    signal x7  : std_logic;
    signal x8  : std_logic;
    signal x9  : std_logic;
    signal x10 : std_logic;
    signal x11 : std_logic;
    signal x12 : std_logic;
    signal x13 : std_logic;
    signal x14 : std_logic;
    signal x15 : std_logic;
    signal z0  : std_logic;
    signal z1  : std_logic;
    signal z2  : std_logic;
    signal z3  : std_logic;
    signal z4  : std_logic;
    signal z5  : std_logic;
    signal w0  : std_logic;
    signal w1  : std_logic;
    signal w2  : std_logic;
    signal w3  : std_logic;

begin

    dut : rete16_4
```

```
port map (x0  => x0,
          x1  => x1,
          x2  => x2,
          x3  => x3,
          x4  => x4,
          x5  => x5,
          x6  => x6,
          x7  => x7,
          x8  => x8,
          x9  => x9,
          x10 => x10,
          x11 => x11,
          x12 => x12,
          x13 => x13,
          x14 => x14,
          x15 => x15,
          z0  => z0,
          z1  => z1,
          z2  => z2,
          z3  => z3,
          z4  => z4,
          z5  => z5,
          w0  => w0,
          w1  => w1,
          w2  => w2,
          w3  => w3);

stimuli : process
begin
    -- EDIT Adapt initialization as needed
    x0 <= '0';
    x1 <= '0';
    x2 <= '0';
    x3 <= '0';
    x4 <= '0';
    x5 <= '0';
    x6 <= '0';
    x7 <= '0';
    x8 <= '0';
    x9 <= '0';
    x10 <= '0';
    x11 <= '0';
    x12 <= '0';
    x13 <= '0';
    x14 <= '0';
    x15 <= '0';
    z0 <= '0';
    z1 <= '0';
    z2 <= '0';
    z3 <= '0';
    z4 <= '0';
    z5 <= '0';

    -- EDIT Add stimuli here

    wait for 100 ns;
```

```
        x0 <= '1';

        wait for 100 ns;

        z5 <= '1';

        wait;
    end process;

end tb;

-- Configuration block below is required by some simulators. Usually no need to edit.

configuration cfg_tb_mux16_4 of tb_mux16_4 is
    for tb
    end for;
end cfg_tb_mux16_4;
```

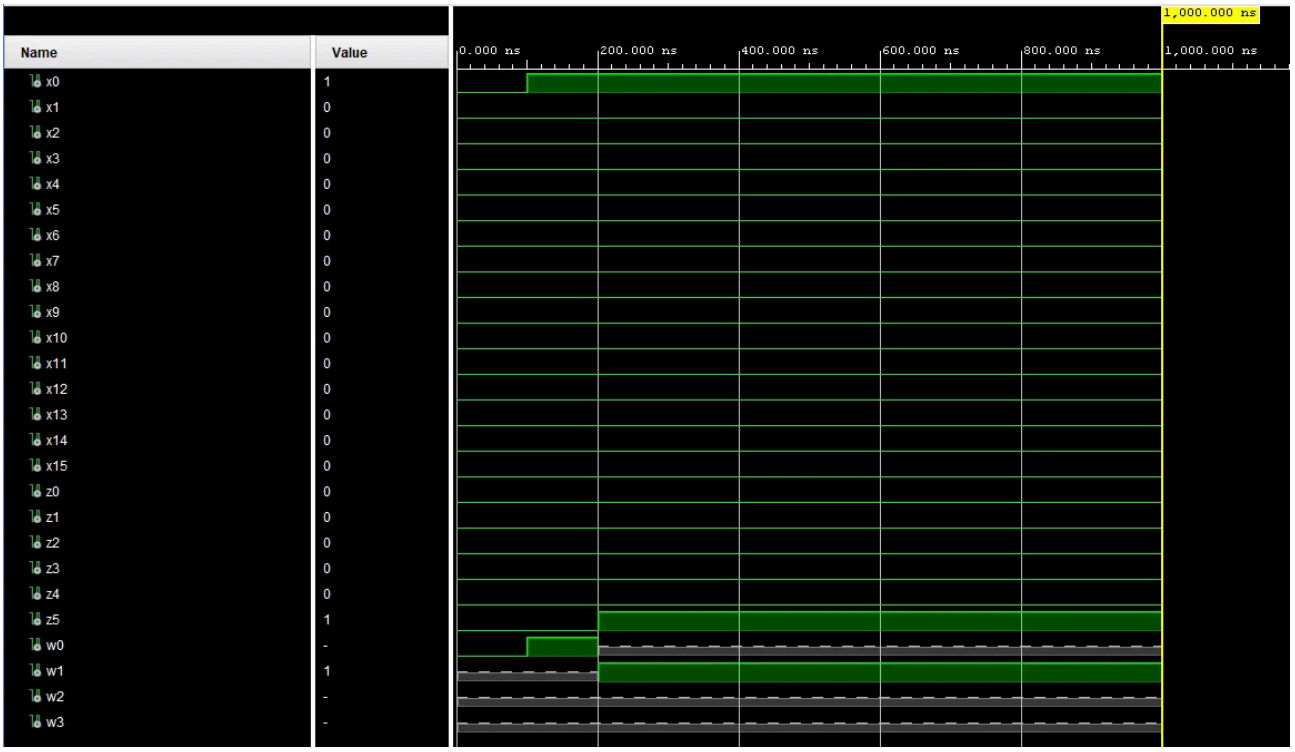



*Figura 1.7 Simulazione*

## Esercizio 1.3

Sintetizzare ed implementare su board il progetto della rete di interconnessione sviluppato al punto 1.2, utilizzando gli switch per fornire gli input di selezione e i led per visualizzare i 4 bit di uscita. Per quanto riguarda i 16 bit dato in input, essi devono essere immessi mediante switch, 8 bit alla volta, sviluppando un'apposita "rete di controllo" per l'acquisizione che utilizzi due bottoni della board per caricare rispettivamente la prima e la seconda metà del dato in ingresso.

### *Progetto e architettura*

Per implementare il progetto della rete di interconnessione su board, abbiamo aggiunto al design una **rete di controllo** che prende in ingresso i segnali di *reset* e di *load* delle due parti del dato di input (*load1* e *load2*) e si occupa semplicemente di comporre il dato di ingresso della rete.

Il nuovo sistema sarà quindi il seguente:

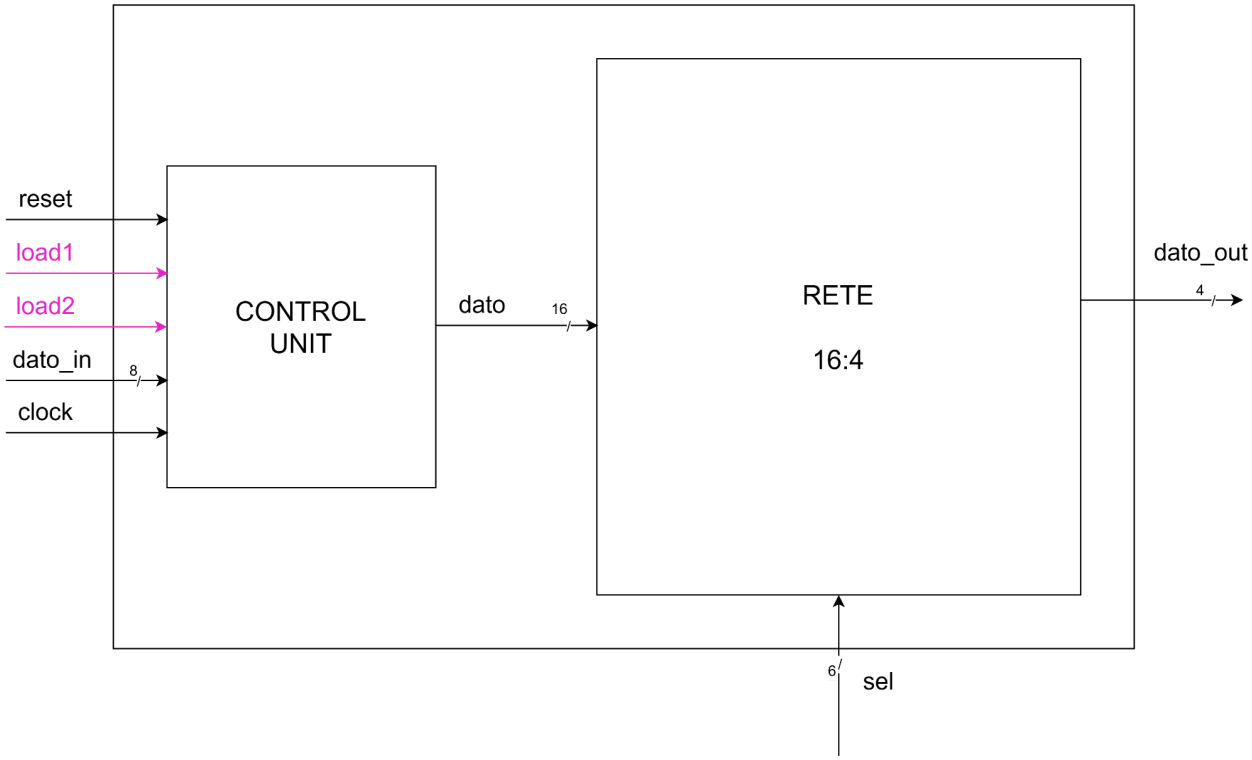



*Figura 1.8 Architettura della rete 16:4 implementata su board*

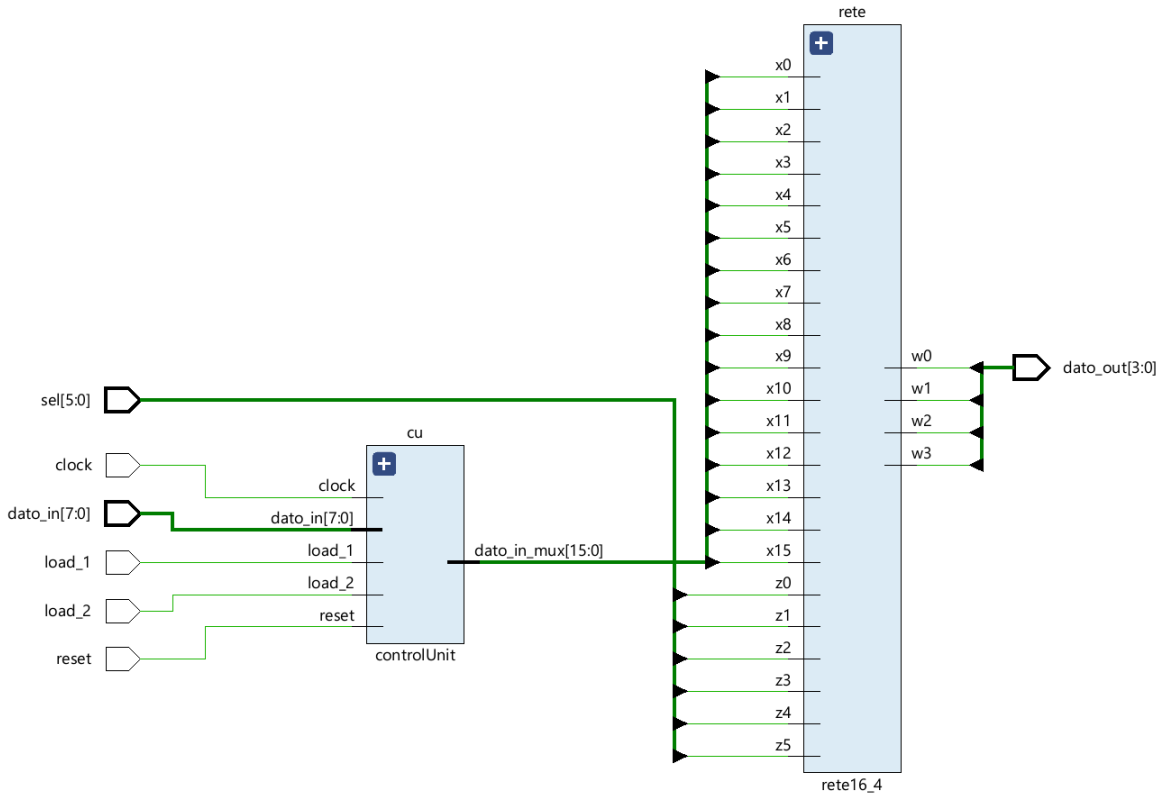



*Figura 1.9 Schematic del sistema complessivo implementato in Vivado*

### *Implementazione*

Il **top module** (**sis_tot**) è implementato nel modo seguente:

```
entity sis_tot is
    Port ( reset : in STD_LOGIC;
           clock : in STD_LOGIC;
           load_1 : in STD_LOGIC;
           load_2 : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           sel : in STD_LOGIC_VECTOR (5 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end sis_tot;

architecture Behavioral of sis_tot is

component rete16_4 is
    Port ( x0 : in STD_LOGIC;
           x1 : in STD_LOGIC;
           x2 : in STD_LOGIC;
           x3 : in STD_LOGIC;
           x4 : in STD_LOGIC;
           x5 : in STD_LOGIC;
           x6 : in STD_LOGIC;
           x7 : in STD_LOGIC;
           x8 : in STD_LOGIC;
           x9 : in STD_LOGIC;
           x10 : in STD_LOGIC;
```

```
            x11 : in STD_LOGIC;
            x12 : in STD_LOGIC;
            x13 : in STD_LOGIC;
            x14 : in STD_LOGIC;
            x15 : in STD_LOGIC;
            z0 : in STD_LOGIC;
            z1 : in STD_LOGIC;
            z2 : in STD_LOGIC;
            z3 : in STD_LOGIC;
            z4 : in STD_LOGIC;
            z5 : in STD_LOGIC;
            w0 : out STD_LOGIC;
            w1 : out STD_LOGIC;
            w2 : out STD_LOGIC;
            w3 : out STD_LOGIC);
end component rete16_4;

component controlUnit is
    Port ( reset : in STD_LOGIC;
           clock : in STD_LOGIC;
           load_1 : in STD_LOGIC;
           load_2 : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           dato_in_mux : out STD_LOGIC_VECTOR (15 downto 0));
end component controlUnit;

-- definiamo un segnale di appoggio per memorizzare l'input ricevuto dagli switch
signal tmp : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');

begin

    rete: rete16_4
    port map ( x0 => tmp(0),
               x1 => tmp(1),
               x2 => tmp(2),
               x3 => tmp(3),
               x4 => tmp(4),
               x5 => tmp(5),
               x6 => tmp(6),
               x7 => tmp(7),
               x8 => tmp(8),
               x9 => tmp(9),
               x10 => tmp(10),
               x11 => tmp(11),
               x12 => tmp(12),
               x13 => tmp(13),
               x14 => tmp(14),
               x15 => tmp(15),
               z0 => sel(0),
               z1 => sel(1),
               z2 => sel(2),
               z3 => sel(3),
               z4 => sel(4),
               z5 => sel(5),
               w0 => dato_out(0),
               w1 => dato_out(1),
```

```
                w2 => dato_out(2),
                w3 => dato_out(3));

    cu: controlUnit
    port map ( reset => reset,
                clock => clock,
                load_1 => load_1,
                load_2 => load_2,
                dato_in => dato_in,
                dato_in_mux => tmp);

end Behavioral;
```

La **control unit** è la seguente:

```
entity controlUnit is
    Port ( reset : in STD_LOGIC;
           clock : in STD_LOGIC;
           load_1 : in STD_LOGIC;
           load_2 : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           dato_in_mux : out STD_LOGIC_VECTOR (15 downto 0));
end controlUnit;

architecture Behavioural of controlUnit is

-- definiamo un segnale di appoggio per memorizzare l'input ricevuto dagli switch
signal tmp : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');

begin

    dato_in_mux <= tmp;

    process(clock)
    begin
        if rising_edge(clock) then
            -- se reset = 1 poniamo tmp = 0
            if reset = '1' then
                tmp <= (others => '0');
            else
                -- se premiamo il bottone load_1 carichiamo la prima parte
dell'ingresso dato
                if load_1 = '1' then
                    tmp(7 downto 0) <= dato_in;
                -- se premiamo il bottone load_2 carichiamo la seconda parte
dell'ingresso dato
                elsif load_2 = '1' then
                    tmp(15 downto 8) <= dato_in;
                end if;
            end if;
        end if;
    end process;

end Behavioural;
```

La **rete 16:4** è la stessa dell'esercizio precedente.

*Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint a cui sono state apportate delle modifiche, cioè quelle relative a switch, led e bottoni.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];

##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { dato_in[0]
}]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { dato_in[1]
}]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { dato_in[2]
}]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { dato_in[3]
}]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { dato_in[4]
}]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { dato_in[5]
}]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { dato_in[6]
}]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { dato_in[7]
}]; #IO_L5N_T0_D07_14 Sch=sw[7]
#set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];
#IO_L24N_T3_34 Sch=sw[8]
#set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }];
#IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { sel[0] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { sel[1] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { sel[2] }];
#IO_L24P_T3_35 Sch=sw[12]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sel[3] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sel[4] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { sel[5] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { dato_out[3]
}]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { dato_out[2]
}]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { dato_out[1]
}]; #IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { dato_out[0]
}]; #IO_L8P_T1_D11_14 Sch=led[3]
#set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
#IO_L7P_T1_D09_14 Sch=led[4]
```

```
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
#IO_L18N_T2_A11_D27_14 Sch=led[5]
#set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
#IO_L17P_T2_A14_D30_14 Sch=led[6]
#set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
#IO_L18P_T2_A12_D28_14 Sch=led[7]
#set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
#set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { LED[9] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
#set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
#set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
#set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { LED[12] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
#set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { LED[13] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
#set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { LED[14] }];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
#set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { LED[15] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]

##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L9P_T1_DQS_14 Sch=btnc
#set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { load_enable
}]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { load_1 }];
#IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { load_2 }];
#IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

# Esercizio 2: Sistema ROM+M

## Esercizio 2.1

Progettare, implementare in VHDL e testare mediante simulazione un sistema S composto da una ROM puramente combinatoria di 16 locazioni da 8 bit ciascuna e da una macchina combinatoria M che opera come segue: fornito al sistema un indirizzo A di 4 bit, il sistema restituisce il valore contenuto nella ROM all'indirizzo A opportunamente "trasformato" attraverso la macchina M. Il comportamento della macchina M è totalmente a scelta dello studente, l'unico vincolo è che essa prenda in ingresso 8 bit e ne fornisca in uscita 4.

### *Progetto e architettura*

Il sistema complessivo comprende al suo interno:

- una **ROM combinatoria**
- una macchina M implementata secondo un approccio strutturale come un **encoder 8:4** (a sua volta costituito quindi da due encoder 4:2)

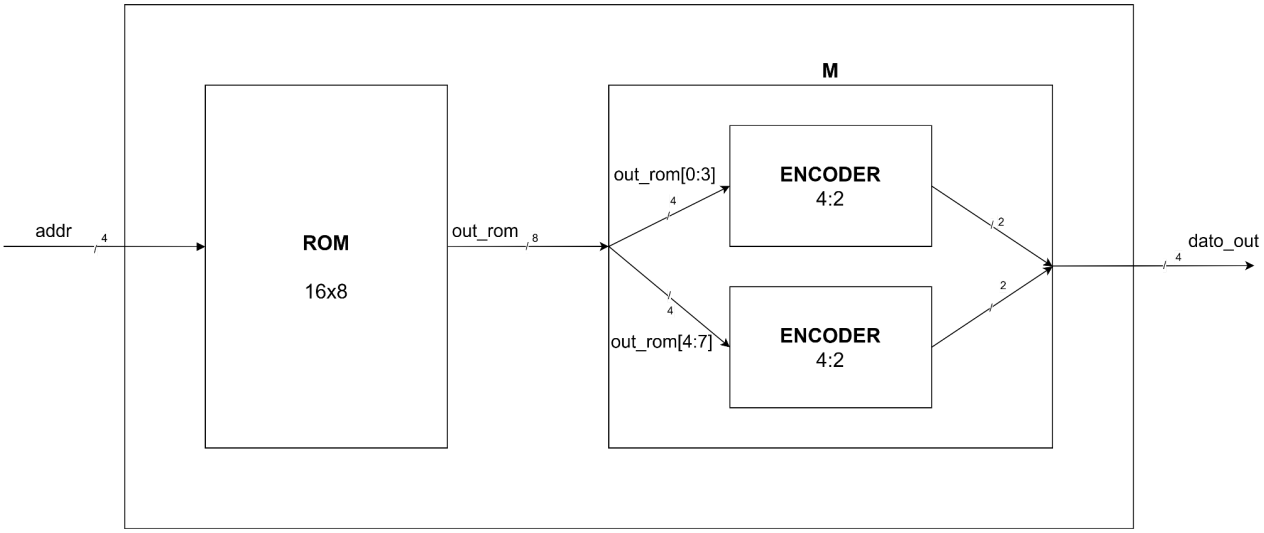



*Figura 2.1 Architettura del sistema complessivo*

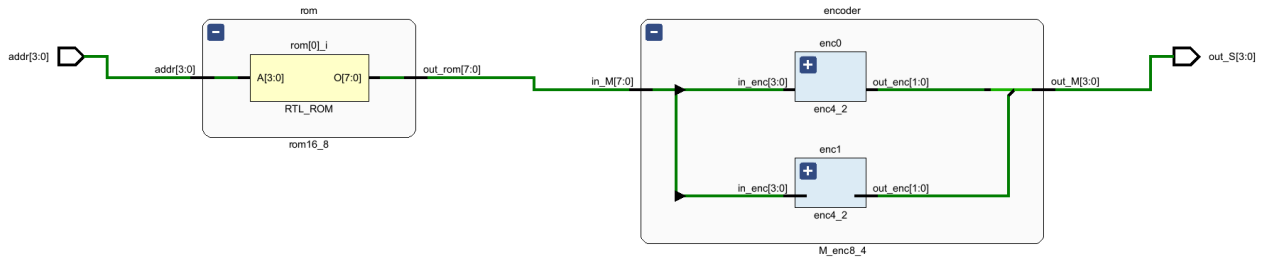



*Figura 2.2 Schematic del sistema complessivo*

### *Implementazione*

L'implementazione del **top module** è la seguente:

```
entity Sistema_Compl is
    Port ( addr : in STD_LOGIC_VECTOR (3 downto 0);
           out_S : out STD_LOGIC_VECTOR (3 downto 0));
end Sistema_Compl;
```

```
architecture structural of Sistema_Compl is

signal tmp : std_logic_vector (7 downto 0) := "00000000";

    component rom16_8
        Port ( addr : in STD_LOGIC_VECTOR (3 downto 0);
               out_rom : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component M_enc8_4
        Port ( in_M : in STD_LOGIC_VECTOR (7 downto 0);
               out_M : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

begin

    rom : rom16_8
        Port map(   addr => addr,
                    out_rom => tmp
                );

    encoder : M_enc8_4
        Port map(   in_M => tmp,
                    out_M => out_S
                );

end structural;
```

La **rom combinatoria 16x8** è stata implementata nel modo seguente:

```
entity rom16_8 is
    Port ( addr : in STD_LOGIC_VECTOR (3 downto 0);
           out_rom : out STD_LOGIC_VECTOR (7 downto 0));
end rom16_8;

architecture Behavioral of rom16_8 is

-- definisco il tipo memory come un array costituito da 16 array ciascuno da 8 elementi
type memory is array (0 to 15) of std_logic_vector (7 downto 0);

-- definisco la costante rom16_8 che è di tipo memory -> ogni cella è inizializzata con
-- un valore esadecimale
constant rom : memory := (
    x"11",   -- 00010001
    x"12",   -- 00010010
    x"14",   -- 00010100
    x"18",   -- 00011000
    x"21",   -- 00100001
    x"22",   -- 00100010
    x"24",   -- 00100100
    x"28",   -- 00101000
    x"41",   -- 01000001
    x"42",   -- 01000010
    x"44",   -- 01000100
    x"48",   -- 01001000
```

```
    x"81",   -- 10000001
    x"82",   -- 10000010
    x"84",   -- 10000100
    x"88"    -- 10001000
);

begin

    out_rom <= rom(to_integer(unsigned(addr))); -- Restituisce immediatamente il dato
                                                   corrispondente all'indirizzo

end Behavioral;
```

La **macchina M** è stata implementata tramite un approccio strutturale come un **encoder 8:4**.

```
entity M_enc8_4 is
    Port ( in_M : in STD_LOGIC_VECTOR (7 downto 0);
           out_M : out STD_LOGIC_VECTOR (3 downto 0));
end M_enc8_4;

architecture structural of M_enc8_4 is

    component enc4_2
        Port ( in_enc : in STD_LOGIC_VECTOR (3 downto 0);
               out_enc : out STD_LOGIC_VECTOR (1 downto 0));
    end component;

begin

    enc0 : enc4_2
        Port map(   in_enc(0) => in_M(0),
                    in_enc(1) => in_M(1),
                    in_enc(2) => in_M(2),
                    in_enc(3) => in_M(3),
                    out_enc(0) => out_M(0),
                    out_enc(1) => out_M(1)
                );

    enc1 : enc4_2
        Port map(   in_enc(0) => in_M(4),
                    in_enc(1) => in_M(5),
                    in_enc(2) => in_M(6),
                    in_enc(3) => in_M(7),
                    out_enc(0) => out_M(2),
                    out_enc(1) => out_M(3)
                );


end structural;
```

Il componente **encoder 4:2** è il seguente:

```
entity enc4_2 is
    Port ( in_enc : in STD_LOGIC_VECTOR (3 downto 0);
           out_enc : out STD_LOGIC_VECTOR (1 downto 0));
end enc4_2;
```

```
architecture Behavioral of enc4_2 is

begin

    out_enc <= "00" when in_enc = "0001" else
               "01" when in_enc = "0010" else
               "10" when in_enc = "0100" else
               "11" when in_enc = "1000";

end Behavioral;
```

*Simulazione*

Per la simulazione del sistema è stato utilizzato il seguente **testbench**:

```
entity tb_Sistema_Compl is
end tb_Sistema_Compl;

architecture tb of tb_Sistema_Compl is

    component Sistema_Compl
        port (addr  : in std_logic_vector (3 downto 0);
              out_S : out std_logic_vector (3 downto 0));
    end component;

    signal addr  : std_logic_vector (3 downto 0);
    signal out_S : std_logic_vector (3 downto 0);

begin

    dut : Sistema_Compl
    port map (addr  => addr,
              out_S => out_S);

    stimuli : process
    begin
        -- EDIT Adapt initialization as needed
        addr<= (others => '0');

        -- EDIT Add stimuli here

        wait for 100 ns;
        addr<= "0000";

        wait for 100 ns;
        addr<= "1111";

        wait for 100 ns;
        addr<= "0111";

        wait for 100 ns;
        addr<= "0010";

        wait;
    end process;
```

```
end tb;

-- Configuration block below is required by some simulators. Usually no need to edit.

configuration cfg_tb_Sistema_Compl of tb_Sistema_Compl is
    for tb
    end for;
end cfg_tb_Sistema_Compl;
```

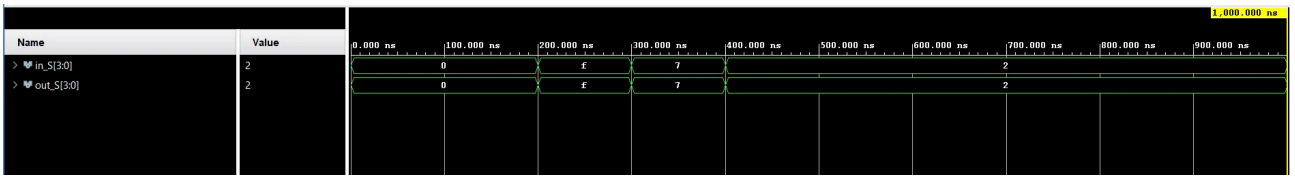



*Figura 2.3 Simulazione*

## Esercizio 2.2

Sintetizzare ed implementare su board il progetto del sistema ROM+M sviluppato al punto 2.1, utilizzando gli switch per fornire l'indirizzo della ROM da cui leggere i valori da trasformare e i led per visualizzare i 4 bit di uscita.

### *Progetto e architettura*

Il progetto del sistema è lo stesso esposto nell'esercizio precedente, con l'unica differenza che questa volta l'input al sistema proviene da 4 switch sulla scheda.

### *Implementazione*

L'implementazione è ancora la stessa rispetto all'esercizio precedente, in quanto il codice VHDL nel progetto non è cambiato.

### *Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint a cui sono state apportate delle modifiche, cioè quelle relative a switch e led.

```
##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { input[0] }];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { input[1] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { input[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { input[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
#set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
#set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }];
#IO_L7N_T1_D10_14 Sch=sw[5]
#set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
#set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
#set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];
#IO_L24N_T3_34 Sch=sw[8]
#set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }];
#IO_25_34 Sch=sw[9]
```

```
#set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
#set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
#set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }];
#IO_L24P_T3_35 Sch=sw[12]
#set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
#set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { output[0] }];
#IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { output[1] }];
#IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { output[2] }];
#IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { output[3] }];
#IO_L8P_T1_D11_14 Sch=led[3]
#set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
#IO_L7P_T1_D09_14 Sch=led[4]
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
#IO_L18N_T2_A11_D27_14 Sch=led[5]
#set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
#IO_L17P_T2_A14_D30_14 Sch=led[6]
#set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
#IO_L18P_T2_A12_D28_14 Sch=led[7]
#set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
#set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { LED[9] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
#set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
#set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
#set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { LED[12] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
#set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { LED[13] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
#set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { LED[14] }];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
#set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { LED[15] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]
```

# Capitolo 2: reti sequenziali elementari

## Esercizio 3: Riconoscitore di sequenze

### Esercizio 3.1

Progettare, implementare in VHDL e testare mediante simulazione una macchina in grado di riconoscere la sequenza **101.** La macchina prende in ingresso un segnale binario *i* che rappresenta il dato, un segnale A di tempificazione e un segnale M di modo, che ne disciplina il funzionamento, e fornisce un'uscita Y alta quando la sequenza viene riconosciuta. In particolare,

- se M=0, la macchina valuta i bit seriali in ingresso a gruppi di 3 (sequenze non sovrapposte),
- se M=1, la macchina valuta i bit seriali in ingresso uno alla volta, tornando allo stato iniziale ogni volta che la sequenza viene correttamente riconosciuta (sequenze parzialmente sovrapposte).

#### *Progetto e architettura*

L'automa del riconoscitore di sequenze parzialmente sovrapposte è rappresentato in Figura 3.1. In questo caso, la macchina valuta i bit seriali in ingresso uno alla volta, tornando allo stato iniziale ogni volta che la sequenza viene correttamente riconosciuta.

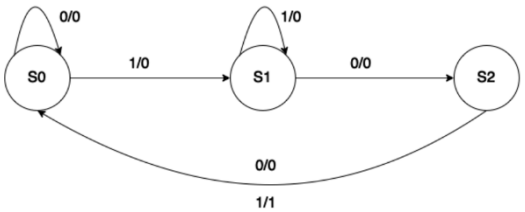



*Figura 3.1. Automa del riconoscitore di sequenze parzialmente sovrapposte*

L'automa del riconoscitore di sequenze non sovrapposte è rappresentato in Figura 3.2. In questo caso, invece, la macchina valuta i bit seriali in ingresso a gruppi di 3.

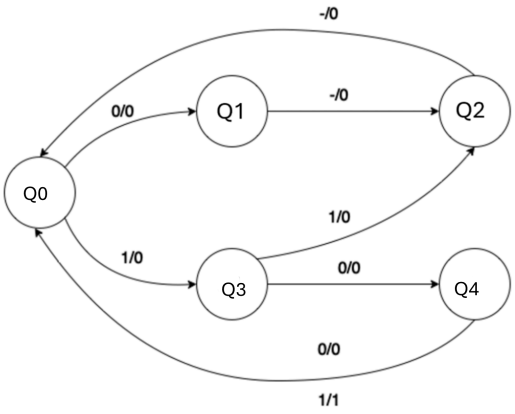



*Figura 3.2. Automa del riconoscitore di sequenze non sovrapposte*

L'automa complessivo presenta uno stato di IDLE, che è lo stato da cui parte la macchina. Se M=1 si va nello stato S0, se invece M=0 si va nello stato Q0.

Il cambio di modalità è stato gestito nel seguente modo: il segnale di modo M può cambiare quando la macchina si trova in un qualsiasi stato, ma a seguito di un cambiamento di modalità la macchina torna in IDLE. Pertanto, ad ogni fronte attivo del clock si verifica se M è cambiato.

L'architettura può essere vista come un sistema che ha in ingresso il segnale M, il segnale di input, il clock e un reset, e in uscita il segnale y.

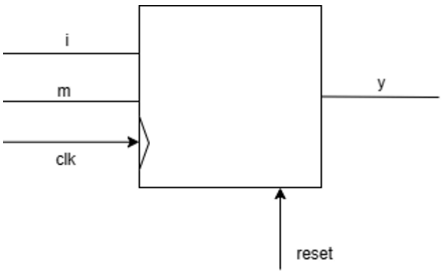



*Figura 3.3 Disegno architetturale del sistema*

### *Implementazione*

L'automa della macchina è stato implementato con un pattern con due *process*, uno puramente combinatorio e uno sequenziale, che ricalca parzialmente il modello di Huffman.

Il processo combinatorio implementa la funzione di uscita e di stato prossimo. Essendo combinatorio, questo process include nella *sensitivity list* tutti i segnali letti: *stato_corrente*, *i*, *m*. È stato utilizzato il costrutto *case…when* per definire cosa fa la macchina in ogni stato: ogni valore degli ingressi *i* e *m* fa evolvere la macchina verso un nuovo stato e produce una specifica uscita.

Il processo sequenziale realizza la memoria di stato, e consente di retroazionare lo stato prossimo nello stato corrente. Nel caso, però, in cui il valore di *m* sia cambiato, la macchina viene forzata a tornare in IDLE. Questo process è sensibile solo al segnale di clock, e la memorizzazione avviene sul fronte di salita del clock. L'uscita Y viene aggiornata da questo processo, pertanto è sincrona con il clock.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity Riconoscitore is
       port( i: in std_logic;
             RST, CLK: in std_logic;
             Y: out std_logic;
             M: in std_logic
       );
end Riconoscitore;

architecture Behavioral_2proc of Riconoscitore is

type stato is (IDLE, S0, S1, S2, Q0, Q1, Q2, Q3, Q4);

signal stato_corrente : stato := IDLE;
signal stato_prossimo : stato;
signal last_mode : std_logic := '0';

signal Ytemp : std_logic;

begin


stato_uscita: process(stato_corrente, i,m)

begin
    case stato_corrente is
        WHEN IDLE =>
            if m='1' then
                stato_prossimo <= S0;
            elsif m='0' then
                stato_prossimo <= Q0;
            else
                stato_prossimo<=IDLE;
           end if;

         WHEN S0 =>
            if i='0' then
                stato_prossimo<=S0;
                Ytemp<='0';
            elsif i='1'then
                stato_prossimo<=S1;
                Ytemp<='0';
            end if;

        WHEN S1 =>
            if i='0' then
                stato_prossimo<=S2;
                Ytemp<='0';
            elsif i='1' then
                stato_prossimo<=S1;
                Ytemp<='0';
            end if;
```

```
        WHEN S2 =>
            if i='0' then
                stato_prossimo<=S0;
                Ytemp<='0';
            elsif i='1' then
                stato_prossimo<=S0;
                Ytemp<='1';
             end if;

         WHEN Q0 =>
            if i='0' then
                stato_prossimo<=Q1;
                Ytemp<='0';
            elsif i='1' then
                stato_prossimo<=Q3;
                Ytemp<='0';
            end if;


         WHEN Q1 =>
            stato_prossimo <= Q2;
            Ytemp<='0';

         WHEN Q2 =>
            stato_prossimo <= Q0;
            Ytemp<='0';

         WHEN Q3 =>
            if i='1' then
                stato_prossimo <= Q2;
                Ytemp<='0';
            elsif i='0' then
                stato_prossimo <= Q4;
                Ytemp<='0';
            end if;

          WHEN Q4 =>
            if i='1' then
                stato_prossimo<=Q0;
                Ytemp<='1';
            elsif i='0' then
                stato_prossimo<=Q0;
                Ytemp<='0';
            end if;



         end case;


end process;

-- questo processo rappresenta gli elementi di memoria
mem: process (CLK)
begin
       if( CLK'event and CLK = '1' ) then
```

```
          if( RST = '1') then
             stato_corrente <= IDLE;
             last_mode<=m;
          else
             if m /= last_mode then
                 stato_corrente<=IDLE;
               else
                 stato_corrente <= stato_prossimo;
                 Y<=Ytemp;

               end if;
             end if;

             last_mode<=m;

   end if;
end process;


end Behavioral_2proc;
```

*Simulazione*

```
library IEEE;
use IEEE.Std_logic_1164.all;
use IEEE.Numeric_Std.all;

entity Riconoscitore_tb is
end;

architecture bench of Riconoscitore_tb is

  component Riconoscitore
       port( i: in std_logic;
                RST, CLK: in std_logic;
                Y: out std_logic;
                M: in std_logic
       );
  end component;

  signal d: std_logic;
  signal RST, CLK: std_logic;
  signal Y: std_logic;
  signal M: std_logic ;

  constant clock_period: time := 10 ns;
  signal stop_the_clock: boolean;

begin

  uut: Riconoscitore port map ( i   => d,
                                RST => RST,
                                CLK => CLK,
                                Y   => Y,
                                M   => M );

  stimulus: process
```

```
begin
  wait for 100 ns;      -- hold reset state for 100 ns.


      m<='1';
           d<='1';
           wait for 10 ns;
           d<='0';
           wait for 10 ns;
           d<='1';
           wait for 10 ns;
           d<='0';
           wait for 10 ns;
           d<='1';
           wait for 10 ns;
           d<='1';
           wait for 10 ns;
           d<='0';
           wait for 10 ns;
           d<='0';

      --CAMBIO M = 0
      wait for 100 ns;
      m <= '0';
      d <= '0';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '0';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '0';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '0';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '0';
      wait for 10 ns;
      d <= '1';
      wait for 10 ns;
      d <= '0';


  stop_the_clock <= true;
  wait;
end process;
```

```
clocking: process
  begin
   while not stop_the_clock loop
      CLK <= '0', '1' after clock_period / 2;
      wait for clock_period;
    end loop;
    wait;
  end process;



end;
```

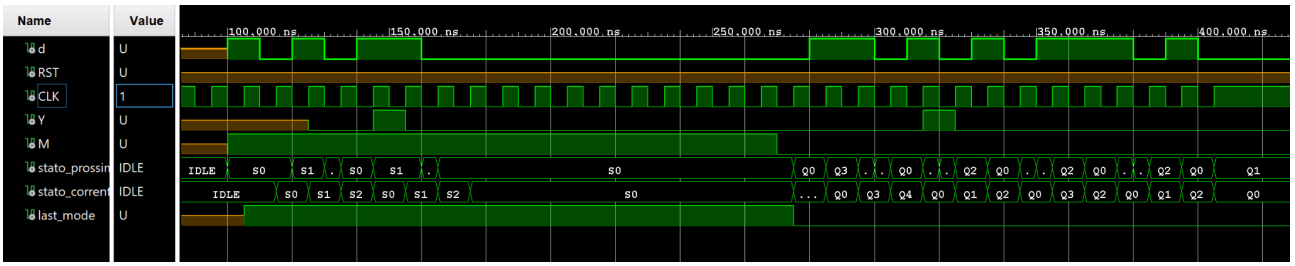



*Figura 3.5 Simulazione*

## Esercizio 3.2

Sintetizzare e implementare su board la rete sviluppata al punto precedente, utilizzando uno switch S1 per codificare l'input *i* e uno switch S2 per codificare il modo *M*, in combinazione con due bottoni *B1* e *B2* utilizzati rispettivamente per acquisire l'input da S1 e S2 in sincronismo con il segnale di tempificazione A, che deve essere ottenuto a partire dal clock della board. Infine, l'uscita Y può essere codificata utilizzando un led.

### *Progetto e architettura*

Per il progetto è stato utilizzato un approccio strutturale. I componenti utilizzati sono due Button Debouncer, la cui implementazione è in Appendice, e la rete implementata nell'esercizio 3.1, opportunamente modificata. L'architettura del sistema realizzato è rappresentata in *Figura 3.6*. Per maggiore chiarezza, gli switch S1 ed S2 sono denominati I ed M, e i bottoni B1 e B2 sono denominati LOAD_I e LOAD_M. Il componente Riconoscitore_2modalità prende in ingresso i segnali LOAD_I_CLEARED e LOAD_M_CLEARED, in uscita rispettivamente dal Button Debouncer che "ripulisce" il segnale I e da quello che "ripulisce" il segnale M.

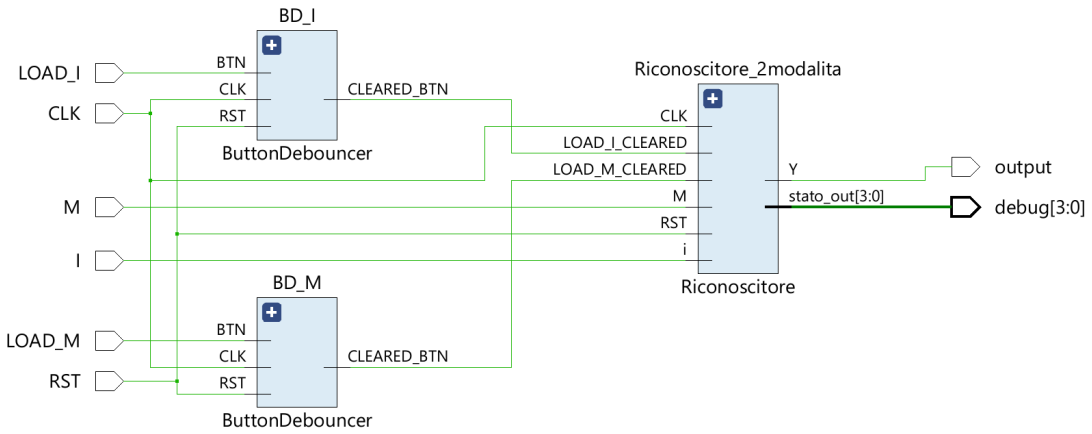



*Figura 3.6 Disegno architetturale del sistema - Schematic*

L'uscita "debug" è stata aggiunta semplicemente per mostrare su dei led lo stato corrente.

### *Implementazione*

L'implementazione del sistema complessivo segue l'approccio strutturale. È necessario dichiarare due segnali, LOAD_I_CLEARED e LOAD_M_CLEARED, per le uscite dei due Button Debouncer.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity riconoscitore_on_board is
     Port (    I: in std_logic;
               RST, CLK: in std_logic;
               output: out std_logic;
               M: in std_logic;
               LOAD_I: in std_logic;
               LOAD_M: in std_logic;
               debug: out std_logic_vector(3 downto 0)
      );
end riconoscitore_on_board;
```

```
architecture Structural of riconoscitore_on_board is
        component ButtonDebouncer is
            generic (
                CLK_period: integer := 10;  -- periodo del clock (della board)
                                            -- in nanosecondi
                btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione
                                                    -- del bottone in nanosecondi
                                                    -- il valore di default è 10 ms
            );
            Port ( RST : in STD_LOGIC;
                   CLK : in STD_LOGIC;
                   BTN : in STD_LOGIC;
                   CLEARED_BTN : out STD_LOGIC);
        end component;

        component Riconoscitore is
            port( i: in std_logic;
                  RST, CLK: in std_logic;
                  Y: out std_logic;
                  M: in std_logic;
                  LOAD_I_CLEARED: in std_logic;
                  LOAD_M_CLEARED: in std_logic;
                  stato_out: out std_logic_vector(3 downto 0)
            );
        end component;




    signal LOAD_I_CLEARED: std_logic;
    signal LOAD_M_CLEARED: std_logic;
    signal stato: std_logic_vector(3 downto 0);



begin

-- Occorrono due button debouncer: uno per I e uno per M
    BD_I: ButtonDebouncer generic map (
            CLK_period => 10,
             btn_noise_time => 10000000
                )
              port map(
                    RST => RST,
                    CLK => CLK,
                    BTN => LOAD_I,
                    CLEARED_BTN => LOAD_I_CLEARED
    );

    BD_M: ButtonDebouncer generic map (
            CLK_period => 10,
             btn_noise_time => 10000000
                )
              port map(
                    RST => RST,
                    CLK => CLK,
```

```
                    BTN => LOAD_M,
                    CLEARED_BTN => LOAD_M_CLEARED
    );


    Riconoscitore_2modalita: Riconoscitore port map(
                i => I,
                RST => RST,
                CLK =>CLK,
                Y => output,
                M => M,
                LOAD_I_CLEARED =>LOAD_I_CLEARED,
                LOAD_M_CLEARED=>LOAD_M_CLEARED,
                stato_out => stato

    );


    debug<=stato;

end Structural;
```

Sono state apportate alcune modifiche sostanziali all'implementazione del riconoscitore di sequenze, rispetto all'Esercizio 3.1.

- Sono stati introdotti i segnali LOAD_I_CLEARED e LOAD_M_CLEARED, provenienti dai button debouncer, per campionare gli ingressi *i* e *m* solo quando vengono premuti i relativi bottoni.
- Sono stati aggiunti i segnali i_temp e m_temp per memorizzare i valori campionati di i e m.
- È stato introdotto il segnale first_m per evitare l'uscita dallo stato IDLE prima del primo caricamento valido di m.
- È stato aggiunto il segnale stato_out, che esporta il valore corrente dello stato come std_logic_vector(3 downto 0), utile per il debug e la visualizzazione tramite LED.

Il process che implementa la memoria di stato (mem) presenta le seguenti modifiche:

- Ad ogni fronte attivo del clock verifica se LOAD_I_CLEARED = 1. In caso affermativo campiona in i_temp il valore letto dallo switch. L'aggiornamento dello stato corrente avviene qui, in quanto è quando viene dato un nuovo ingresso che ci deve essere un avanzamento tra gli stati e il process combinatorio deve attivarsi per calcolare lo stato prossimo in base al valore campionato in i_temp.
- Ad ogni fronte attivo del clock verifica, inoltre, se LOAD_M_CLEARED = 1. In questa fase verifica innanzitutto se è la prima volta che viene premuto il bottone, e in caso affermativo setta first_m a 1. Altrimenti, controlla se il valore di m è cambiato e in caso affermativo forza lo stato corrente allo stato di idle. La modifica dello stato corrente fa attivare il process combinatorio, che imposta lo stato prossimo a S0 o Q0 in base al nuovo valore di m_temp. Il passaggio effettivo a S0 o Q0 avviene però solo quando viene dato un nuovo ingresso, poiché lo stato corrente si aggiorna quando LOAD_I_CLEARED =1. In questo modo, <u>non</u> viene considerato come primo ingresso della nuova sequenza il valore che i_temp assumeva nel momento in cui è stato effettuato il cambio di modalità, ma si aspetta il nuovo valore.

Il process combinatorio (stato_uscita) presenta le seguenti modifiche:

- Nella sensitivity list ci sono i_temp e m_temp invece di i e m, cioè i valori di i e di m impostati sugli switch campionati rispettivamente in corrispondenza dei segnali LOAD_I_CLEARED e LOAD_M_CLEARED.

- Inoltre, nella sensitivity list è stato aggiunto il segnale first_m. In questo modo, all'avvio il riconoscitore resta in idle se first_m è 0, cioè aspetta che m sia stato premuto la prima volta prima di iniziare a riconoscere la sequenza in ingresso nella modalità giusta. Quando first_m diventa 1, infatti, essendo questo segnale nella sensitivity list, il process si attiva e determina lo stato_prossimo (S0 se m_temp=0, Q0 se m_temp=1).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity Riconoscitore is
       port(   i: in std_logic;
               RST, CLK: in std_logic;
               Y: out std_logic;
               M: in std_logic;
               LOAD_I_CLEARED: in std_logic;
               LOAD_M_CLEARED: in std_logic;
               stato_out: out std_logic_vector(3 downto 0)
       );
end Riconoscitore;

architecture Behavioral_2proc of Riconoscitore is

type stato is (IDLE, S0, S1, S2, Q0, Q1, Q2, Q3, Q4);

signal stato_corrente : stato := IDLE;
signal stato_prossimo : stato;
signal last_mode : std_logic := '0';

signal Ytemp : std_logic;
signal m_temp: std_logic;
signal i_temp: std_logic;
signal first_m: std_logic:='0';


begin


stato_uscita: process(stato_corrente, i_temp,m_temp,first_m)

begin
    case stato_corrente is
        WHEN IDLE =>
             -- Se m non è stato ancora dato, la macchina resta in idle.
            if first_m='0' then
                stato_prossimo<=IDLE;
            else
                if m_temp='1' then
                    stato_prossimo <= S0;
                elsif m_temp='0' then
                    stato_prossimo <= Q0;
                else
                    stato_prossimo<=IDLE;
               end if;
```

```
    end if;

 WHEN S0 =>

        if i_temp='0' then
            stato_prossimo<=S0;
            Ytemp<='0';
        elsif i_temp='1'then
            stato_prossimo<=S1;
            Ytemp<='0';
        end if;


WHEN S1 =>
    if i_temp='0' then
        stato_prossimo<=S2;
        Ytemp<='0';
    elsif i_temp='1' then
        stato_prossimo<=S1;
        Ytemp<='0';
    end if;
WHEN S2 =>
    if i_temp='0' then
        stato_prossimo<=S0;
        Ytemp<='0';
    elsif i_temp='1' then
        stato_prossimo<=S0;
        Ytemp<='1';
     end if;

 WHEN Q0 =>

        if i_temp='0' then
            stato_prossimo<=Q1;
            Ytemp<='0';
        elsif i_temp='1' then
            stato_prossimo<=Q3;
            Ytemp<='0';
        end if;

 WHEN Q1 =>
    stato_prossimo <= Q2;
    Ytemp<='0';

 WHEN Q2 =>
    stato_prossimo <= Q0;
    Ytemp<='0';

 WHEN Q3 =>
    if i_temp='1' then
        stato_prossimo <= Q2;
        Ytemp<='0';
    elsif i_temp='0' then
        stato_prossimo <= Q4;
        Ytemp<='0';
    end if;
```

```
          WHEN Q4 =>
            if i_temp='1' then
                stato_prossimo<=Q0;
                Ytemp<='1';
            elsif i_temp='0' then
                stato_prossimo<=Q0;
                Ytemp<='0';
            end if;



         end case;

end process;

mem: process (CLK)
begin
       if( CLK'event and CLK = '1' ) then
              if( RST = '1') then
                   stato_corrente <= IDLE;
                   first_m<='0';
                    Y<='0';
              else

                   if LOAD_I_CLEARED='1' then
                       i_temp<=i;
                       stato_corrente<=stato_prossimo;
                   end if;

                   if LOAD_M_CLEARED='1' then
                       m_temp<=m;
                       if first_m='0' then
                          first_m<='1';
                          last_mode<=m; -- assegno m e non m_temp, perché m_temp viene
                                           aggiornato solo quando si raggiunge
                                           end process
                       elsif m/=last_mode then
                          stato_corrente<=IDLE;
                          last_mode<=m;

                       end if;

                   end if;

                   Y<=Ytemp;

       end if;
   end if;
end process;


with stato_corrente select
    stato_out <= "0000" when IDLE,
                 "0001" when S0,
                 "0010" when S1,
```

```
                "0011" when S2,
                "0100" when Q0,
                "0101" when Q1,
                "0110" when Q2,
                "0111" when Q3,
                "1000" when Q4,
                "1111" when others;


end Behavioral_2proc;
```

*Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint a cui sono state apportate delle modifiche, cioè quelle relative a clock, switch, led e bottoni.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { I}];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { M }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
#set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
#set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
#set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
#set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }];
#IO_L7N_T1_D10_14 Sch=sw[5]
#set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
#set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
#set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];
#IO_L24N_T3_34 Sch=sw[8]
#set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }];
#IO_25_34 Sch=sw[9]
#set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
#set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
#set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }];
#IO_L24P_T3_35 Sch=sw[12]
#set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
```

```
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
#set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { output }];
#IO_L18P_T2_A24_15 Sch=led[0]
#set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
#IO_L24P_T3_RS1_15 Sch=led[1]
#set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { LED[2] }];
#IO_L17N_T2_A25_15 Sch=led[2]
#set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { LED[3] }];
#IO_L8P_T1_D11_14 Sch=led[3]
#set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
#IO_L7P_T1_D09_14 Sch=led[4]
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
#IO_L18N_T2_A11_D27_14 Sch=led[5]
#set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
#IO_L17P_T2_A14_D30_14 Sch=led[6]
#set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
#IO_L18P_T2_A12_D28_14 Sch=led[7]
#set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
#set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { LED[9] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
#set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
#set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { debug[0] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { debug[1] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { debug[2] }];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { debug[3] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]



##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { RST }];
#IO_L9P_T1_DQS_14 Sch=btnc
#set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }];
#IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { LOAD_I }];
#IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { LOAD_M }];
#IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

# Esercizio 4: Shift Register

## Esercizio 4.1

Progettare, implementare in VHDL e testare mediante simulazione un registro a scorrimento di N bit in grado di shiftare a destra o a sinistra di un numero Y variabile di posizioni a seconda di una opportuna selezione. In particolare, i valori possibili di Y sono 1 e 2. L'utente tramite selezione deve scegliere di quante posizioni shiftare. Il componente deve essere realizzato utilizzando sia un a) approccio comportamentale sia un b) approccio strutturale.

Nota: il numero di bit del registro deve essere implementato come un generic, e dall'esterno deve poter essere scelta la modalità di funzionamento mediante opportuni segnali di selezione.

### *Approccio comportamentale*

### Progetto e architettura




*Figura 4.1 Disegno architetturale del sistema*

È stato realizzato uno shift register Serial In - Serial Out. Per realizzare uno shift register in grado di fare shift a destra o a sinistra, e di una o di due posizioni a comando, in ingresso è fornito un vettore di due segnali di selezione (SEL):

- se assume il valore 00 è richiesto lo shift a destra di una posizione;
- se assume il valore 01, è richiesto lo shift a destra di due posizioni;
- se assume il valore 10, è richiesto lo shift a sinistra di una posizione;
- se assume il valore 11, è richiesto lo shift a destra di due posizioni.

Si è scelto di avere due segnali distinti di ingresso:

- Ei è l'ingresso seriale esterno da sinistra (per lo shift a destra)
- Eo è l'ingresso seriale esterno da destra (per lo shift a sinistra)

Inoltre, si è scelto di non usare un ingresso aggiuntivo per il **doppio shift**, ma viene replicato il singolo dato in ingresso.

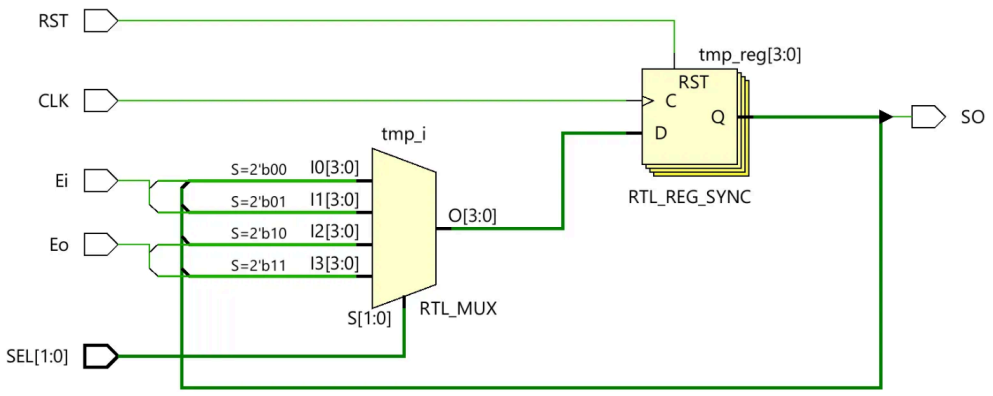



*Figura 4.2 Schematic Approccio Comportamentale*

## Implementazione

L'implementazione dello shift register secondo l'approccio comportamentale è riportata di seguito. È stato utilizzato un process in cui, in corrispondenza di ogni fronte di salita del clock tramite il costrutto case controlla il valore di SEL, e in base alla selezione fa lo shift opportuno.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

library ieee;
use ieee.std_logic_1164.all;

entity shift_register is
    generic(
        N: integer :=4 -- valore di default = 4
        );
    port(CLK, RST: in std_logic;
         Ei : in std_logic; --ingresso seriale esterno da sinistra (per lo shift a
                            destra)
         Eo : in std_logic; --ingresso seriale esterno da destra (per lo shift a
                            sinistra)
         SEL : in std_logic_vector(1 downto 0);
                         -- selezione di direzione e numero di posizioni dello shift
                          -- S= 00 -> shift a destra di una posizione
                          -- S= 01 -> shift a destra di due posizioni
                          -- S= 10 -> shift a sinistra di una posizione
                          -- S= 11 -> shift a sinistra di due posizioni
         SO : out std_logic);
end shift_register;

architecture archi of shift_register is
    signal tmp: std_logic_vector(N-1 downto 0);
begin

    process (CLK)
    begin
        if (CLK'event and CLK='1') then
            if (RST='1') then
                tmp <= (others=>'0');
            else
```

```
          case SEL is
              when "00" => -- shift a destra di una posizione
    --questi segnali vengono aggiornati con i valori che avevano alla fine della
    -- precedente attivazione
                  tmp(0)<=Ei;
                  for i in 1 to (N-1) loop
                      tmp(i)<=tmp(i-1);
                  end loop;

                when "01" => -- shift a destra di due posizioni
                  tmp(0)<=Ei;
                  tmp(1)<=Ei;
                  for i in 2 to (N-1) loop
                      tmp(i)<=tmp(i-2);
                  end loop;

                when "10" => -- shift a sinistra di una posizione

                  tmp(N-1)<=Eo;
                  for i in N-2 downto 0 loop
                      tmp(i)<=tmp(i+1);
                  end loop;

                 when "11" => -- shift a sinistra di due posizioni
                    tmp(N-1)<=Eo;
                    tmp(N-2)<=Eo;
                  for i in N-3 downto 0 loop
                      tmp(i)<=tmp(i+2);
                  end loop;

                  when others =>
                       tmp <= (others => '0');




          -- SO <= tmp(N-1);
          --se inserisco qui questa istruzione invece che fuori dal process,
          --l'uscita SO non viene aggiornata contestualmente a tmp(3)
          --ci vorrà un altro fronte del clock per vedere SO e quindi, per
          --conservare il valore di tmp(3) fino al prossimo clock, verrà inserito
          --un altro flip-flop tra tmp(3) e SO

           end case;
       end if;

  end if;
  --SO <= tmp(N-1);  --se inserisco qui questa istruzione, il valore di SO è
                     --aggiornato alla prossima
                      --attivazione del process, quindi sul fronte di discesa
                     --successivo a quando è stato
                      --aggiornato tmp(3)
end process;
SO <= tmp(N-1);  --se inserisco qui questa istruzione, il valore di SO viene
                 -- aggiornato continuamente in maniera concorrente
```

```
end archi;
```

## Simulazione

```
library ieee;
use ieee.std_logic_1164.all;

entity shift_register_tb is
end shift_register_tb;

architecture tb of shift_register_tb is

    component shift_register is
    generic(
        N: integer :=4 -- valore di default = 8
        );
    port(CLK, RST: in std_logic;
         Ei : in std_logic; --ingresso seriale esterno da sinistra (per lo shift a
                            destra)
         Eo : in std_logic; --ingresso seriale esterno da destra (per lo shift a
                            sinistra)
         SEL : in std_logic_vector(1 downto 0);
         SO : out std_logic);
    end component;

    signal clk_tb : std_logic;
    signal output  : std_logic;
    signal rst : std_logic;
    signal ei : std_logic :='0';
    signal eo : std_logic:= '0';
    signal sel : std_logic_vector(1 downto 0);

    constant clk_period : time := 20 ns;


begin
    -- Clock generation
   clk_process : process
   begin
             clk_tb <= '0';
             wait for clk_period/2;
             clk_tb <= '1';
             wait for clk_period/2;
   end process;



    uut : shift_register
    port map (CLK => clk_tb,
              RST => rst,
```

```
            Ei  => Ei,
            Eo => Eo,
            SEL => sel,
            SO  => output);


    stimuli : process
    begin

        rst <= '1';
        wait for 100ns;  --global reset

        rst <='0';
        sel(0) <= '0';
        sel(1) <= '0';
        Ei <= '1';
        wait for 20ns;
        Ei <= '0';
        wait for 20ns;
        Ei <= '1';
        wait for 100ns;
        Ei <= '0';
        wait for 100ns;
        Ei <= '1';
        wait for 100ns;
        sel(0)<='1';
        Ei <= '0';
        wait for 100ns;
        Ei <= '1';
        wait for 100ns;
        Ei <= '0';
        wait for 100ns;
        sel(0)<='0';
        sel(1)<='1';
        Eo<='1';
        wait for 100ns;
        Eo <= '0';

        wait for 100ns;
        sel(0)<='1';
        sel(1)<='1';
        Eo<='1';
        wait for 20ns;
        Eo<='0';
        wait for 100ns;

        wait;
    end process;

end tb;
```

In *Figura 4.3* è possibile osservare il risultato della simulazione.

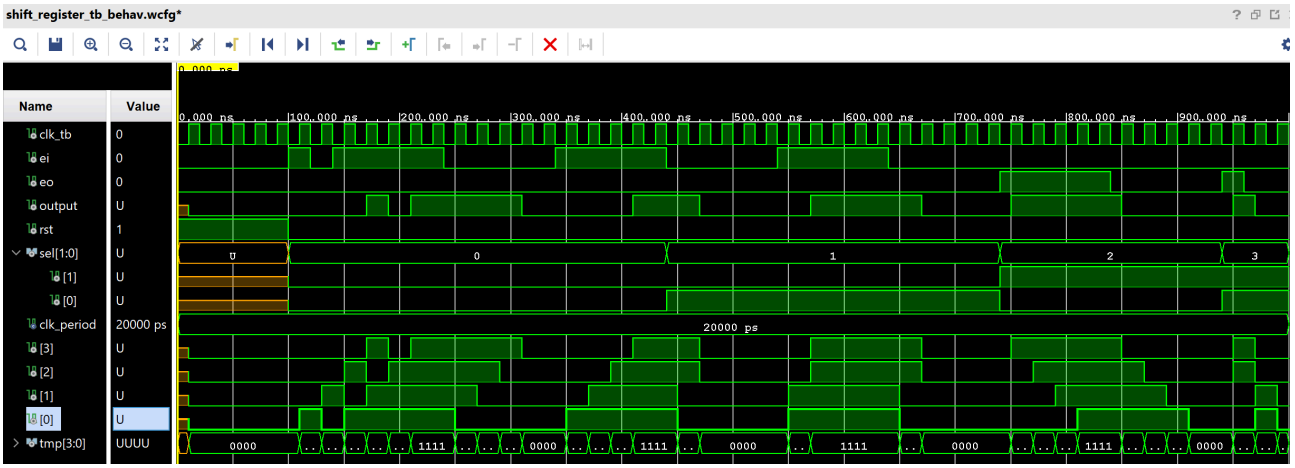



*Figura 4.3 Simulazione*

## *Approccio strutturale*

### Progetto e architettura

A livello strutturale, è necessario inserire un MUX in ingresso ad ogni flip-flop: ogni MUX ha due ingressi di selezione: SEL(0) seleziona il numero di posizioni e SEL(1) seleziona la direzione

- se SEL(1)=0 e SEL(0)=0 è richiesto lo shift a destra di una posizione. Pertanto, l'ingresso di ogni MUX deve essere preso dall'uscita del flip flop a destra di quello considerato;
- se SEL(1)=0 e SEL(0)=1, è richiesto lo shift a destra di due posizioni. Pertanto, l'ingresso di ogni MUX deve essere preso dall'uscita del flip-flop che si trova due posizioni a destra rispetto a quello considerato;
- se SEL(1)=1 e SEL(0)=0, è richiesto lo shift a sinistra di una posizione. Pertanto, l'ingresso di ogni MUX deve essere preso dall'uscita del flip flop a sinistra di quello considerato;
- se SEL(1)=1 e SEL(0)=1, è richiesto lo shift a destra di due posizioni. Pertanto, l'ingresso di ogni MUX deve essere preso dall'uscita del flip-flop che si trova due posizioni a sinistra rispetto a quello considerato.

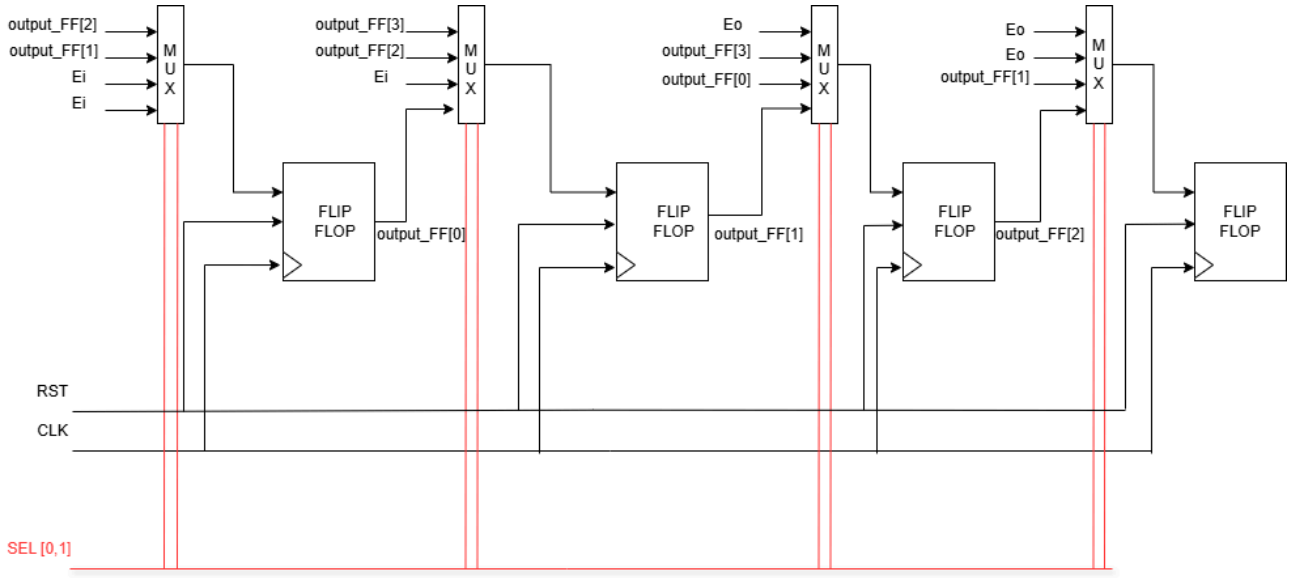



*Figura 4.4 Disegno architetturale del sistema – Approccio strutturale*

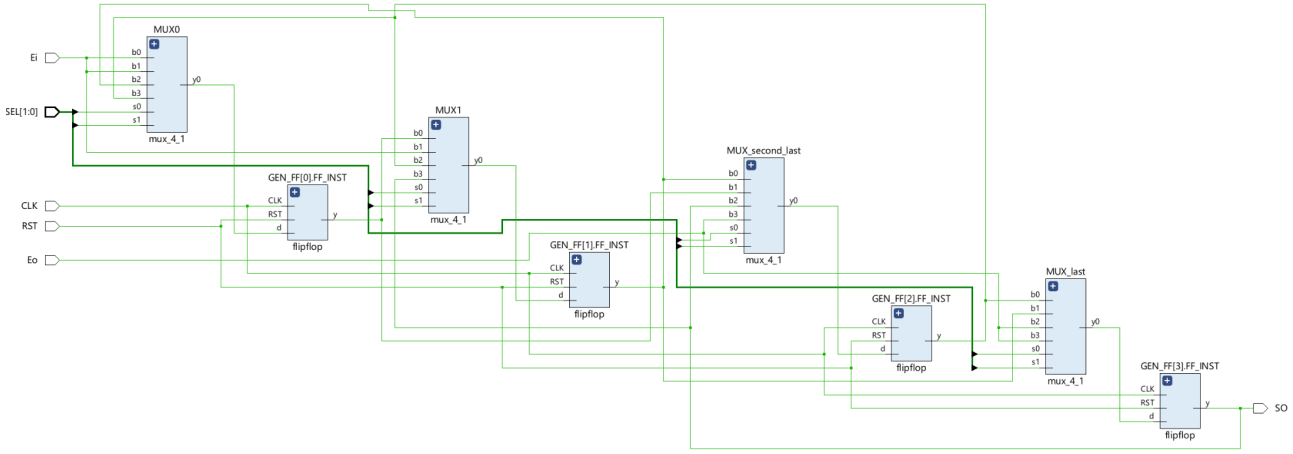



*Figura 4.5 Schematic Approccio Strutturale*

## Implementazione

Di seguito è riportata l'implementazione dello shift register secondo l'approccio strutturale.

Si può notare che il numero di bit, quindi di flip flop, dello shift register è implementato come un generic. Tutti i multiplexer sono istanziati con il costrutto **for generate**, fatta eccezione per i multiplexer in ingresso al primo, al secondo, al penultimo e all'ultimo flip flop, che sono istanziati separatamente, poiché per tali tra i possibili ingressi ci sono anche Ei e Eo, nel modo illustrato in *Figura 4.4*.

```
library ieee;
use ieee.std_logic_1164.all;


entity shift_register is
    generic(
        N: integer :=4 -- valore di default = 4
        );
    port(CLK, RST: in std_logic;
         Ei : in std_logic; --ingresso seriale esterno da sinistra (per lo shift a
                            --destra)
         Eo : in std_logic; --ingresso seriale esterno da destra (per lo shift a
                           -- sinistra)
         SEL : in std_logic_vector(1 downto 0);
                          -- selezione di direzione e numero di posizioni dello shift
                          -- S= 00 -> shift a destra di una posizione
                          -- S= 01 -> shift a destra di due posizioni
                          -- S= 10 -> shift a sinistra di una posizione
                          -- S= 11 -> shift a sinistra di due posizioni
         SO : out std_logic);
end shift_register;


architecture Structural of shift_register is

     signal output_ff: std_logic_vector(N-1 downto 0); -- uscita di ciascun flip flop
     signal input_ff: std_logic_vector(N-1 downto 0);  --ingresso di ciascun flip flop
```

```
    component mux_4_1 is
       port(  b0 : in STD_LOGIC;
                    b1 : in STD_LOGIC;
                    b2 : in STD_LOGIC;
                    b3 : in STD_LOGIC;
                    s0 : in STD_LOGIC;
                    s1 : in STD_LOGIC;
                    y0 : out STD_LOGIC
              );
       end component;

       component flipflop
        port(
                CLK, RST, d: in std_logic;
                y: out std_logic
        );
     end component;

BEGIN

    MUX0: mux_4_1 port map(            -- multiplexer in ingresso al primo FF
        b0 => Ei,
        b1 => Ei,
        b2 => output_ff(1),
        b3 => output_ff(2),
        s0 => SEL(0),
        s1 => SEL(1),
        y0 => input_ff(0)
    );

    MUX1: mux_4_1 port map(                  -- multiplexer in ingresso al 2° FF
        b0 => output_ff(0),
        b1 => Ei,
        b2 => output_ff(2),
        b3 => output_ff(3),
        s0 => SEL(0),
        s1 => SEL(1),
        y0 => input_ff(1)
    );

    MUX_second_last: mux_4_1 port map(  -- multiplexer in ingresso al penultimo FF
        b0 => output_ff(N-3),
        b1 => output_ff(N-4),
        b2 => output_ff(N-1),
        b3 => Eo,
        s0 => SEL(0),
        s1 => SEL(1),
        y0 => input_ff(N-2)
    );


    MUX_last: mux_4_1 port map(        -- multiplexer in ingresso all'ultimo FF
        b0 => output_ff(N-2),
        b1 => output_ff(N-3),
        b2 => Eo,
        b3 => Eo,
```

```
        s0 => SEL(0),
        s1 => SEL(1),
        y0 => input_ff(N-1)
    );


      GEN_MUX: for i in 2 to N-3 generate  -- gli altri mux vengono istanziati con un
                                             costrutto for generate
        MUX_INST: mux_4_1 port map (
            b0 => output_ff(i-1),
            b1 => output_ff(i-2),
            b2 => output_ff(i+1),
            b3 => output_ff(i+2),
            s0 => SEL(0),
            s1 => SEL(1),
            y0 => input_ff(i)
        );
    end generate GEN_MUX;




    GEN_FF: for i in 0 to N-1 generate
        FF_INST: flipflop port map (
            CLK => CLK,
            RST => RST,
            d => input_ff(i),
            y => output_ff(i)
        );
    end generate GEN_FF;



    SO <= output_ff(N-1);



END Structural;
```

Il flip flop di tipo D è implementato come segue:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity flipflop is
    Port (
        CLK : in STD_LOGIC;       -- Segnale di clock
        RST : in STD_LOGIC;       -- Segnale di reset sincrono
        d   : in STD_LOGIC;       -- Ingresso D
        y   : out STD_LOGIC       -- Uscita Q
    );
end flipflop;

architecture Behavioral of flipflop is
begin
```

```
    process(CLK)
    begin
        if rising_edge(CLK) then
            if RST = '1' then
                y <= '0';
            else
                y <= d;
            end if;
        end if;
    end process;
end Behavioral;
```

## Simulazione

Il test bench è identico a quello utilizzato per simulare lo shift register realizzato con approccio comportamentale.

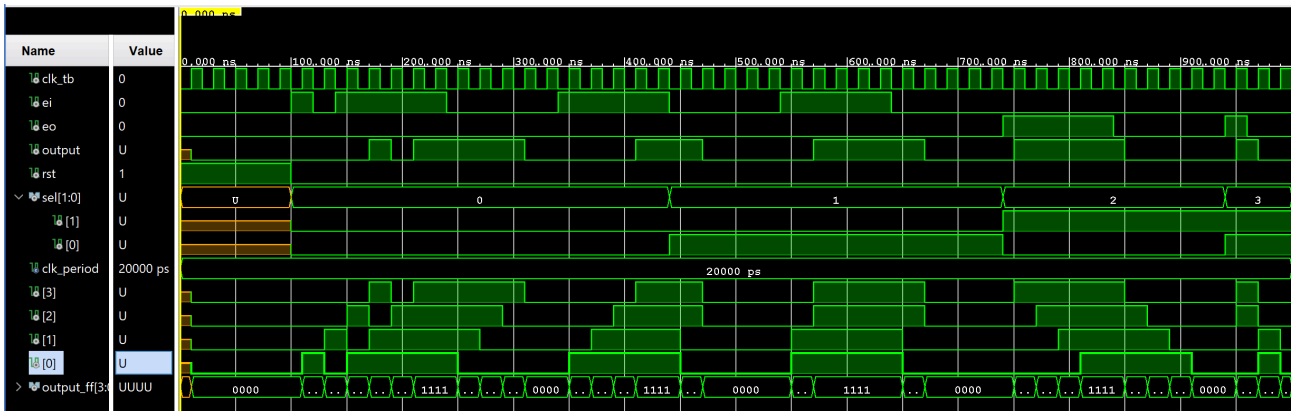



*Figura 4.6 Risultato della simulazione*

# Esercizio 5: Cronometro

## Esercizio 5.1

Progettare, implementare in VHDL e testare mediante simulazione un cronometro, in grado di scandire secondi, minuti e ore a partire da una base dei tempi prefissata (es. si consideri il clock a disposizione sulla board). Il progetto deve prevedere la possibilità di inizializzare il cronometro con un valore iniziale, sempre espresso in termini di ore, minuti e secondi, mediante un opportuno ingresso di *set*, e deve prevedere un ingresso di *reset* per azzerare il tempo.

Il componente deve essere realizzato utilizzando un approccio strutturale, collegando opportunamente dei contatori secondo uno schema a scelta.

### *Progetto e architettura*

Per realizzare il cronometro è stato utilizzato un approccio strutturale. Sono stati utilizzati tre contatori: uno modulo 60 per i secondi, uno modulo 60 per i minuti e uno modulo 24 per le ore. Inoltre è stato usato un divisore di frequenza che ha come ingresso il clock e come uscita un impulso a frequenza inferiore, che è alto per un solo periodo di clock e che quindi non è un vero clock, ma un impulso che viene usato come enable.

A ciascun contatore viene dato in ingresso sia il clock che un segnale di abilitazione differente per ciascuno di essi. Per ottenere i segnali di abilitazione sono necessarie delle porte AND:

- il contatore dei secondi è sempre abilitato, quindi il segnale abilitazione è il segnale in uscita dal divisore di frequenza;
- il segnale di abilitazione per il contatore dei minuti è dato dal risultato dell'operazione di AND tra il segnale che indica che il contatore dei secondi ha raggiunto il massimo e il segnale in uscita dal divisore di frequenza;
- il segnale di abilitazione per il contatore delle ore è dato dall'operazione di AND tra il segnale che indica che il contatore dei minuti ha raggiunto il massimo, il segnale che indica che il contatore dei secondi ha raggiunto il massimo e il segnale in uscita dal divisore di frequenza.

Ciascun contatore, ad ogni fronte di discesa del clock:

- se il relativo segnale di abilitazione è alto, incrementa il proprio valore;
- se il segnale di reset è alto, azzera il proprio valore;
- se il segnale di set è alto, inizializza il contatore con un valore specifico.

Inoltre, ciascun contatore monitora il raggiungimento del valore massimo (59 per i secondi e per i minuti, 23 per le ore) e segnala questo evento alzando l'uscita umax.

Nota: È stato utilizzato questo sistema perché usare semplicemente l'uscita umax del contatore in AND con il segnale come segnale di clock per il contatore successivo (ad esempio, l'uscita **umax** del contatore dei secondi in AND con il clock come segnale di clock per il contatore dei minuti) non sarebbe corretto, in quanto si tratta di segnali di logica e nell'FPGA non è buona norma utilizzare segnali di logica come clock, poiché l'FPGA dispone di una rete di distribuzione dei segnali di clock molto efficiente. Il tool di sintesi utilizza questa rete per riconoscere e gestire correttamente i segnali di clock, e un segnale di logica non viene riconosciuto come tale.

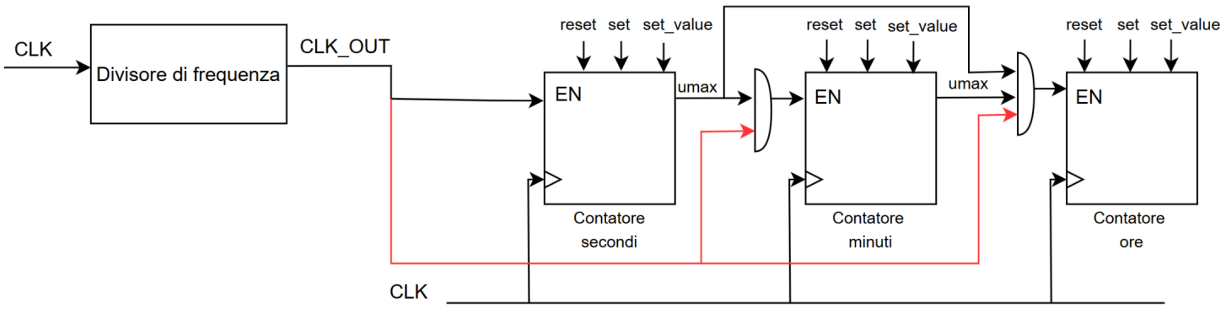



*Figura 5.1 Disegno architetturale del sistema*

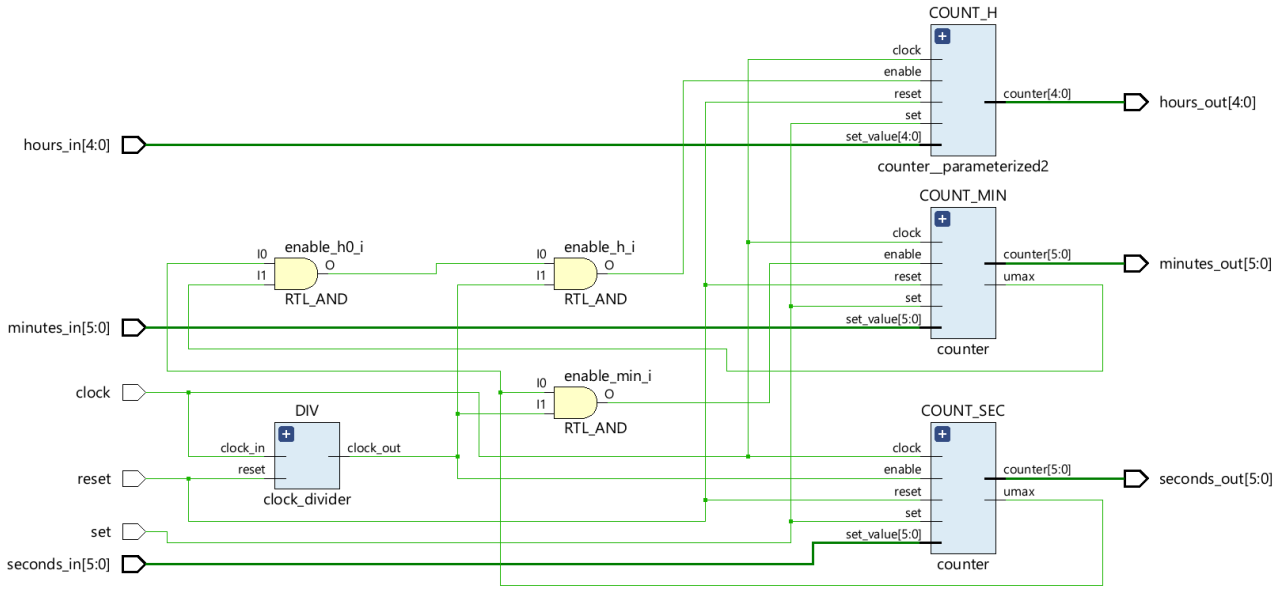



*Figura 5.2 Schematic Cronometro implementato*

## *Implementazione*

Di seguito è riportata l'implementazione del cronometro. Si può notare che è stato istanziato il divisore di frequenza con frequenza di uscita (clock_frequency_out) pari a 50 MHz per poter apprezzare il risultato in simulazione. Nell'esercizio successivo (Esercizio 5.2) sarà posta pari a 1 Hz, in modo da scandire effettivamente i secondi.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cronometro is
    Port (
        clock: in std_logic;
        reset: in std_logic;
        set: in std_logic;
        minutes_in: in std_logic_vector(5 downto 0);
```

```
        seconds_in: in std_logic_vector(5 downto 0);
        hours_in: in std_logic_vector(4 downto 0);

        seconds_out: out std_logic_vector(5 downto 0);
        minutes_out: out std_logic_vector(5 downto 0);
        hours_out: out std_logic_vector(4 downto 0)

    );
end cronometro;

architecture Structural of cronometro is
    signal enable_sec: std_logic :='0';
    signal enable_min: std_logic := '0';
    signal enable_h: std_logic := '0';
    signal umax_sec: std_logic :='0';
    signal umax_min: std_logic :='0';
    signal umax_h: std_logic :='0';


    component counter is

        generic(

            n: integer := 3 ;--numero di bit necessari
            modulo: integer := 8

            );

        Port ( clock : in  STD_LOGIC;
               reset : in  STD_LOGIC; --ingresso di reset per azzerare il tempo
               set : in STD_LOGIC; --ingresso di set
               set_value : in STD_LOGIC_VECTOR (n-1 downto 0); -- ingresso di set per
                                                                  inizializzare il
                                                                  cronometro a un valore
                                                                  iniziale
               enable : in STD_LOGIC; -- questo è l'enable del clock, insieme danno
                                         l'impulso di conteggio
               umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
                                       conteggio
               counter : out  STD_LOGIC_VECTOR (n-1 downto 0)
               );


        end component;


      component clock_divider is
         generic(
                    clock_frequency_in : integer := 100000000;--100MHz
                    clock_frequency_out : integer := 50000000 --frequenza desiderata
                                                          --> il periodo è di 20 ns
                    );
        Port ( clock_in : in  STD_LOGIC;
               reset : in STD_LOGIC;
               clock_out : out  STD_LOGIC); -- nb: è un segnale di logica e non un
clock -> viene usato come abilitazione
```

```
        end component;



begin
        DIV: clock_divider
            generic map(
            clock_frequency_in => 100000000,--100MHz
            clock_frequency_out => 50000000
            )
            port map(
                clock_in=>clock,
                reset=>reset,
                clock_out=>enable_sec
            );

        COUNT_SEC: counter
           generic map(
            n => 6,
            modulo=> 60
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => seconds_in,
               enable => enable_sec,
               umax => umax_sec,
               counter => seconds_out
        );

        enable_min <= umax_sec and enable_sec;


        COUNT_MIN: counter
           generic map(
            n => 6,
            modulo=> 60
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => minutes_in,
               enable => enable_min,
               umax => umax_min,
               counter => minutes_out
        );

        enable_h <=  umax_sec and umax_min and enable_sec;

        COUNT_H: counter
           generic map(
            n => 5,
```

```
            modulo=> 24
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => hours_in,
               enable => enable_h,
               umax => umax_h,
               counter => hours_out
        );

end Structural;
```

Il codice del componente **contatore** è il seguente.
Notiamo che sono stati utilizzati i *generics* per rendere il contatore flessibile, permettendo di configurare il numero di bit e il modulo in base al tipo di contatore (secondi, minuti o ore) nel top module.
Il contatore è realizzato utilizzando due processi separati, entrambi con il clock nella sensitivity list.
Il primo processo si occupa del conteggio vero e proprio: ad ogni fronte di discesa del clock, verifica se il segnale di **reset** è attivo (in tal caso azzera il contatore), se il segnale **set** è attivo (inizializza il contatore con un valore specificato), o se il segnale **enable** è attivo (in tal caso incrementa il contatore, o, se il contatore raggiunge il valore massimo, lo azzera).
Il secondo processo gestisce il segnale **umax**, che indica quando il contatore ha raggiunto il valore massimo. Quando si verifica un fronte di salita del clock aggiorna **umax** a '1' quando il contatore raggiunge il valore massimo, altrimenti lo mantiene a '0'.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;



entity counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il tempo
           set : in STD_LOGIC; --ingresso di set
           set_value : in STD_LOGIC_VECTOR (n-1 downto 0); -- ingresso di set per
                                                           inizializzare il
                                                           cronometro a un valore
                                                           iniziale
          enable : in STD_LOGIC; -- questo è l'enable del clock, insieme danno
                                  l'impulso di conteggio
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
```

```
                                    conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0)
           );

end counter;

architecture Behavioral of counter is

signal c : std_logic_vector (n-1 downto 0) := (others => '0');

signal u : std_logic := '0';

begin

counter <= c; --valore attuale del contatore


counter_process: process(clock)
begin

         if falling_edge(clock) then
             if (reset = '1') then  --sia reset che set sono sincroni con il clock.
                    c <= (others => '0');
             elsif (set='1') then
                    c <= set_value;
             elsif (enable='1') then

                    if( c=std_logic_vector(TO_UNSIGNED(modulo-1,n))) then
                           c <= (others => '0');
                    else
                           c <= std_logic_vector(unsigned(c) + 1);

                    end if;
             end if;
          end if;
    end process;



     proc_count_max: process(clock)
     begin
        if rising_edge(clock) then -- lo attivo sul rising edge per leggere il valore
                                      corrente
            if (reset = '1') then
                u <= '0'; --reset del segnale
            elsif (c= std_logic_vector(TO_UNSIGNED(modulo-1,n))) then
                u <= '1'; --attivo quando raggiungo il massimo
            else
                u <= '0'; --se no rimane basso
            end if;
        end if;
     end process;

    umax<=u;

end Behavioral;
```

Infine, di seguito è riportato il codice del **divisore di frequenza**.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



-- il clock_divider genera un impulso a frequenza inferiore rispetto a quella del clock
-- il segnale generato è alto per un solo periodo di clock

entity clock_divider is
        generic(
                            clock_frequency_in : integer := 100000000;--100MHz
                            clock_frequency_out : integer := 500 --500 Hz
                            );
    Port ( clock_in : in  STD_LOGIC;
                reset : in STD_LOGIC;
           clock_out : out  STD_LOGIC);
end clock_divider;

architecture Behavioral of clock_divider is

signal clockfx: std_logic := '0';

constant count_max_value : integer := (clock_frequency_in/clock_frequency_out) -1;

begin

clock_out <= clockfx;

count_for_division: process(clock_in)
variable counter : integer range 0 to count_max_value := 0;
begin


       if rising_edge(clock_in) then
        if reset = '1' then
              counter := 0;
              clockfx <= '0';
        else
              if counter = count_max_value then
                    clockfx <=  '1';
                    counter := 0;
              else
                    clockfx <=  '0';
                    counter := counter + 1;
              end if;
        end if;
       end if;

end process;


end Behavioral;
```

*Simulazione*

Come anticipato, come base dei tempi è stato utilizzato il clock generato in simulazione. In particolare, nel testbench il clock ha un periodo di 10 ns.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity cronometro_tb is
end cronometro_tb;

architecture testbench of cronometro_tb is
    -- Dichiarazione segnali
    signal clock       : std_logic := '0';
    signal reset       : std_logic := '0';
    signal set         : std_logic := '0';
    signal minutes_in  : std_logic_vector(5 downto 0) := "000000";
    signal seconds_in  : std_logic_vector(5 downto 0) := "000000";
    signal hours_in    : std_logic_vector(4 downto 0) := "00000";
    signal minutes_out : std_logic_vector(5 downto 0);
    signal seconds_out : std_logic_vector(5 downto 0);
    signal hours_out   : std_logic_vector(4 downto 0);

    -- Istanziazione del cronometro
    component cronometro
        Port (
            clock       : in std_logic;
            reset       : in std_logic;
            set         : in std_logic;
            minutes_in  : in std_logic_vector(5 downto 0);
            seconds_in  : in std_logic_vector(5 downto 0);
            hours_in    : in std_logic_vector(4 downto 0);
            minutes_out : out std_logic_vector(5 downto 0);
            seconds_out : out std_logic_vector(5 downto 0);
            hours_out   : out std_logic_vector(4 downto 0)
        );
    end component;

begin
    -- Collegamento segnali
    UUT: cronometro
        port map(
            clock       => clock,
            reset       => reset,
            set         => set,
            minutes_in  => minutes_in,
            seconds_in  => seconds_in,
            hours_in    => hours_in,
            minutes_out => minutes_out,
            seconds_out => seconds_out,
            hours_out   => hours_out
        );

    -- Generazione del clock 100 MHz (10 ns periodo)
```

```
    process
    begin
        while true loop
            clock <= '0';
            wait for 5 ns;
            clock <= '1';
            wait for 5 ns;
        end loop;
    end process;


    -- Stimolo di test
    process
    begin
        -- Reset iniziale
        reset <= '1';
        wait for 10 ns;
        reset <= '0';
        -- Lascia contare per un po'


        -- Lascia contare per un po'
        wait for 500 ns;

        -- TEST 2: Cambia il valore con il set durante il conteggio
        set <= '1';
        hours_in <= "10111";   -- 23 ore
        minutes_in <= "111011"; -- 59 minuti
        seconds_in <= "111000"; -- 56 secondi
        wait for 10 ns;
        set <= '0';

        -- Lascia contare per un po'
        wait for 1 us;

        -- Fine simulazione
        wait;
    end process;

end testbench;
```

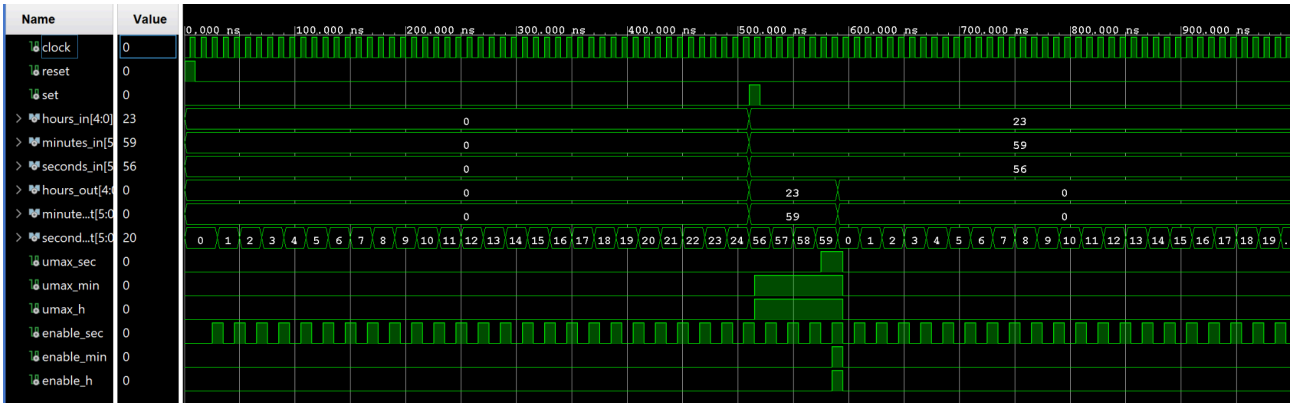



*Figura 5.3 Risultato della simulazione*

Come esempio, in *Figura 5.4* è riportato il risultato della simulazione anche con un valore diverso di frequenza di uscita dal divisore di frequenza. In particolare, il caso di 10 MHz.

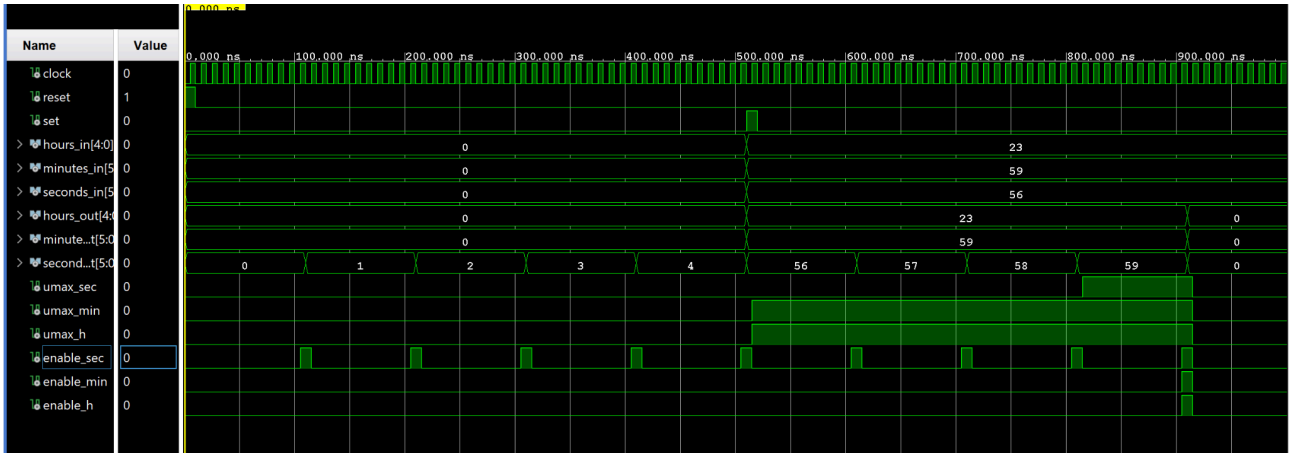



*Figura 5.4 Risultato della simulazione - Secondo esempio*

## Esercizio 5.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando i display a 7 segmenti per la visualizzazione dell'orario (o una combinazione di display e led nel caso in cui i display a disposizione siano in numero inferiore a quello necessario), gli switch per l'immissione dell'orario iniziale e due bottoni, uno per il set dell'orario e uno per il reset. Si utilizzi una codifica a scelta dello studente per la visualizzazione dell'orario sui display (esadecimale o decimale).

### *Progetto e architettura*

Per poter sintetizzare e implementare il cronometro su board è necessario aggiungere alcuni componenti:

- un'**unità di controllo**, che si occupa dell'acquisizione dagli switch della board del tempo iniziale in termini di secondi, minuti e ore, che costituiscono gli ingressi seconds_in, minutes_in e hours_in del cronometro, che saranno impostati dal cronometro stesso come valori iniziali quando l'ingresso di set diventa alto;
- il blocco che gestisce i due display a sette segmenti (**display_seven_segment**);
- un **encoder**, che prende in ingresso i tre segnali binari che rappresentano secondi, minuti e ore e li codifica in un'unica uscita a 32 bit che costituisce l'ingresso del blocco display_seven_segment.

L'architettura complessiva, che mostra le interconnessioni tra tutti i componenti, è rappresentata dallo schematic in *Figura 6.5*.

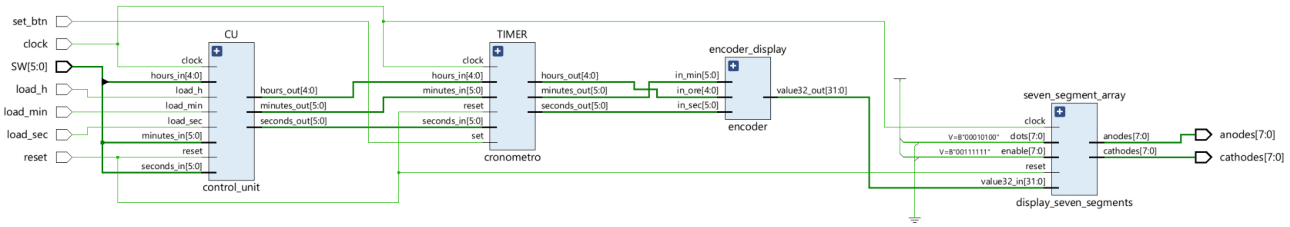



*Figura 5.5 Schematic del sistema complessivo implementato*

### *Implementazione*

L'implementazione strutturale del sistema complessivo è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cronometro_on_board is
  Port (
        clock: in std_logic;
        reset: in std_logic; -- bottone N17
        set_btn: in std_logic; -- bottone M18
        SW: in std_logic_vector(5 downto 0);
        load_sec: in std_logic; -- bottone P17
        load_min: in std_logic; -- bottone M17
        load_h: in std_logic; -- bottone P18
        anodes : out  STD_LOGIC_VECTOR (7 downto 0); -- anodi e catodi delle cifre,
                                                        sono un output del topmodule
         cathodes : out  STD_LOGIC_VECTOR (7 downto 0)
       );
end cronometro_on_board;
```

```
architecture Structural of cronometro_on_board is

        signal value_temp : std_logic_vector(16 downto 0);
        signal value_encoded : std_logic_vector(31 downto 0);


        signal sec_set: std_logic_vector(5 downto 0);
        signal min_set: std_logic_vector(5 downto 0);
        signal h_set: std_logic_vector(4 downto 0);



        component cronometro is
        Port (
                  clock: in std_logic;
                  reset: in std_logic;
                  set: in std_logic;
                  minutes_in: in std_logic_vector(5 downto 0);
                  seconds_in: in std_logic_vector(5 downto 0);
                  hours_in: in std_logic_vector(4 downto 0);
                  seconds_out: out std_logic_vector(5 downto 0);
                  minutes_out: out std_logic_vector(5 downto 0);
                  hours_out: out std_logic_vector(4 downto 0)
           );
        end component;


        component display_seven_segments is
        Generic(
                    clock_frequency_in : integer := 50000000;
                    clock_frequency_out : integer := 5000000
                    );
        Port ( clock : in  STD_LOGIC;
               reset : in  STD_LOGIC;
               value32_in : in  STD_LOGIC_VECTOR (31 downto 0);
               enable : in  STD_LOGIC_VECTOR (7 downto 0);
               dots : in  STD_LOGIC_VECTOR (7 downto 0);
               anodes : out  STD_LOGIC_VECTOR (7 downto 0);
               cathodes : out  STD_LOGIC_VECTOR (7 downto 0));
        end component;


        component control_unit is -- gestisce l'acquisizione del tempo settato
        Port (
                clock : in  STD_LOGIC;
                reset : in  STD_LOGIC;
                load_sec : in  STD_LOGIC;
                 load_min : in  STD_LOGIC;
                 load_h : in  STD_LOGIC;
                seconds_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                                switch per i secondi
                minutes_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                                switch per i minuti
                hours_in : in STD_LOGIC_VECTOR(4 downto 0); --valore acquisito da 6
                                                                switch per le ore
```

```
            seconds_out: out STD_LOGIC_VECTOR(5 downto 0); -- valori da mostrare in
                                                              decimale sul display
            minutes_out : out STD_LOGIC_VECTOR(5 downto 0);
            hours_out : out STD_LOGIC_VECTOR(4 downto 0)
                 );
    end component;

    component encoder is
    Port(
           in_sec: in std_logic_vector(5 downto 0);
           in_min: in std_logic_vector(5 downto 0);
           in_ore: in std_logic_vector(4 downto 0);
           value32_out: out  STD_LOGIC_VECTOR (31 downto 0)

        );
     end component;

begin


    TIMER: cronometro
        port map(
            clock => clock,
            reset => reset,
            set => set_btn,
            minutes_in =>  min_set,
            seconds_in =>sec_set,
            hours_in => h_set,
            seconds_out => value_temp(5 downto 0),
            minutes_out =>value_temp(11 downto 6),
            hours_out => value_temp(16 downto 12)
        );

      CU: control_unit
        Port map(
               clock => clock,
               reset => reset,
               load_sec => load_sec,
               load_min => load_min,
               load_h => load_h,
               seconds_in => SW, --valore acquisito da 6 switch per i secondi
               minutes_in => SW, --valore acquisito da 6 switch per i minuti
               hours_in => SW(4 downto 0), --valore acquisito da 6 switch per le ore
               seconds_out => sec_set,
               minutes_out => min_set,
               hours_out => h_set

               );

         seven_segment_array: display_seven_segments GENERIC MAP(
            clock_frequency_in => 100000000, --qui inserisco i parametri effettivi
                                    (clock della board e clock in uscita desiderato)
            clock_frequency_out => 500 --inserendo un valore inferiore si vedranno le
                                    cifre illuminarsi in sequenza
            )
            PORT MAP(
```

```
            clock => clock,
            reset => reset,
            value32_in => value_encoded,
            enable => "00111111", --stabilisco che tutti i display siano accesi
            dots =>"00010100",
            anodes => anodes,
            cathodes => cathodes
        );



        encoder_display: encoder
        Port map(
               in_sec => value_temp(5 downto 0),
               in_min => value_temp(11 downto 6),
               in_ore => value_temp(16 downto 12),
               value32_out => value_encoded
            );



end Structural;
```

Il codice del **cronometro** presenta, rispetto all'implementazione riportata nell'Esercizio 5.1, come unica differenza l'assegnazione `clock_frequency_out => 1` che imposta la frequenza desiderata pari a 1 Hz, e quindi genera un impulso di durata pari al periodo del clock ogni secondo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cronometro is
    Port (
        clock: in std_logic;
        reset: in std_logic;
        set: in std_logic;
        minutes_in: in std_logic_vector(5 downto 0);
        seconds_in: in std_logic_vector(5 downto 0);
        hours_in: in std_logic_vector(4 downto 0);

        seconds_out: out std_logic_vector(5 downto 0);
        minutes_out: out std_logic_vector(5 downto 0);
        hours_out: out std_logic_vector(4 downto 0)

    );
end cronometro;

architecture Structural of cronometro is
    signal enable_sec: std_logic :='0';
    signal enable_min: std_logic := '0';
    signal enable_h: std_logic := '0';
    signal umax_sec: std_logic :='0';
    signal umax_min: std_logic :='0';
    signal umax_h: std_logic :='0';
```

```
    component counter is

        generic(

            n: integer := 3 ;--numero di bit necessari
            modulo: integer := 8

            );

        Port ( clock : in  STD_LOGIC;
               reset : in  STD_LOGIC; --ingresso di reset per azzerare il tempo
               set : in STD_LOGIC; --ingresso di set
               set_value : in STD_LOGIC_VECTOR (n-1 downto 0); -- ingresso di set per
                                                                  inizializzare il
                                                                  cronometro a un valore
                                                                  iniziale
               enable : in STD_LOGIC; -- questo è l'enable del clock, insieme danno
                                         l'impulso di conteggio
               umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
                                       conteggio
               counter : out  STD_LOGIC_VECTOR (n-1 downto 0)
               );


        end component;


      component clock_divider is
         generic(
                    clock_frequency_in : integer := 100000000;--100MHz
                    clock_frequency_out : integer := 1 --1 Hz : frequenza desiderata
                    );
        Port ( clock_in : in  STD_LOGIC;
               reset : in STD_LOGIC;
               clock_out : out  STD_LOGIC); -- nb: è un segnale di logica e non un
                                               clock -> viene usato come abilitazione
       end component;



begin
        DIV: clock_divider
            generic map(
            clock_frequency_in => 100000000,--100MHz
            clock_frequency_out => 1
            )
            port map(
                clock_in=>clock,
                reset=>reset,
                clock_out=>enable_sec
            );

        COUNT_SEC: counter
           generic map(
            n => 6,
```

```
            modulo=> 60
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => seconds_in,
               enable => enable_sec,
               umax => umax_sec,
               counter => seconds_out
        );

        enable_min <= umax_sec and enable_sec;


        COUNT_MIN: counter
           generic map(
            n => 6,
            modulo=> 60
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => minutes_in,
               enable => enable_min,
               umax => umax_min,
               counter => minutes_out
        );

        enable_h <=  umax_sec and umax_min and enable_sec;

        COUNT_H: counter
           generic map(
            n => 5,
            modulo=> 24
            )

            port map(
               clock => clock,
               reset => reset,
               set =>set,
               set_value => hours_in,
               enable => enable_h,
               umax => umax_h,
               counter => hours_out
        );


end Structural;
```

L'**unità di controllo**, che gestisce l'acquisizione tramite gli switch della board dei valori da settare quando l'ingresso di set è alto, è implementata nel seguente modo. Su ciascun fronte di salita del clock:

- se l’ingresso di reset è alto, allora tutti i registri interni (*reg_value*, *reg_dots*, *reg_enable*) vengono azzerati;
- se, invece, reset è basso, l'unità di controllo verifica quale bottone è stato premuto (*load_sec*, *load_min*, *load_h*). Se *load_sec* è alto, il valore presente sugli switch *seconds_in* viene memorizzato nei bit 5-0 del registro *reg_value*. Se *load_min* è alto, il valore presente sugli switch *minutes_in* viene memorizzato nei bit 11-6 del registro *reg_value*. Se *load_h* è alto, il valore presente sugli switch *hours_in* viene memorizzato nei bit 16-12 del registro *reg_value*.

Alle uscite (*seconds_out*, *minutes_out* e *hours_out*) sono assegnati i tre gruppi di bit del registro *reg_value*: 5-0 per i secondi, 11-6 per i minuti, 16-12 per le ore. Queste uscite costituiscono gli ingressi del cronometro, che li imposta come valori iniziali quando si alza l’ingresso di set.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity control_unit is
    Port (
           clock : in  STD_LOGIC;
           reset : in  STD_LOGIC;
            load_sec : in  STD_LOGIC;
           load_min : in  STD_LOGIC;
           load_h : in  STD_LOGIC;
           seconds_in : in STD_LOGIC_VECTOR(5 downto 0);
                    --valore acquisito da 6 switch per i secondi
            minutes_in : in STD_LOGIC_VECTOR(5 downto 0);
                    --valore acquisito da 6 switch per i minuti
           hours_in : in STD_LOGIC_VECTOR(4 downto 0);
                    --valore acquisito da 6 switch per le ore

           seconds_out: out STD_LOGIC_VECTOR(5 downto 0);
                    --valori da mostrare in decimale sul display
           minutes_out : out STD_LOGIC_VECTOR(5 downto 0);
           hours_out : out STD_LOGIC_VECTOR(4 downto 0)
                     );
end control_unit;

architecture Behavioral of control_unit is

signal reg_value : STD_LOGIC_VECTOR(16 downto 0) := (others => '0');
signal reg_dots, reg_enable : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');

begin



seconds_out <= reg_value(5 downto 0);
minutes_out <= reg_value(11 downto 6);
hours_out <= reg_value(16 downto 12);


main: process(clock)
begin

       if clock'event and clock = '1' then
```

```
          if reset = '1' then
                reg_value <= (others => '0');
                reg_dots <= (others => '0');
                reg_enable <= (others => '0');
          else
                if load_sec = '1' then
                     reg_value(5 downto 0) <= seconds_in;
                elsif load_min = '1' then
                     reg_value(11 downto 6) <= minutes_in;
                elsif load_h = '1' then
                  reg_value(16 downto 12) <=hours_in;

                end if;
        end if;
       end if;

end process;


end Behavioral;
```

L'implementazione del blocco **seven_segment_display** è in Appendice.

Infine, di seguito è riportata l'implementazione dell'**encoder**, che prende in ingresso il tempo in secondi, minuti e ore, rappresentato come vettori di bit. L'encoder produce in uscita un vettore di 32 bit, denominato value32_out, che contiene la codifica finale del tempo. Questo vettore viene poi fornito in ingresso al blocco di gestione del display a sette segmenti, che si occupa di visualizzare il valore codificato sul display. La codifica viene eseguita separatamente per i secondi, i minuti e le ore, combinando i valori in un'unica uscita attraverso una concatenazione di segnali.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.all;



entity encoder is
    Port(
           in_sec: in std_logic_vector(5 downto 0); --secondi
           in_min: in std_logic_vector(5 downto 0); --minuti
           in_ore: in std_logic_vector(4 downto 0); --ore
           value32_out: out  STD_LOGIC_VECTOR (31 downto 0) --vettore di 32 bit che
                                                              contiene la codifica finale
                                                              da dare in ingresso al
                                                              blocco seven_segment_display
        );
end encoder;

architecture DataFlow of encoder is

    signal sec_temp : std_logic_vector (7 downto 0) := (others => '0');
    signal min_temp :  std_logic_vector (7 downto 0) := (others => '0');
    signal ore_temp :  std_logic_vector (7 downto 0) := (others => '0');

begin
```

```
    value32_out <= "00000000" & ore_temp & min_temp & sec_temp; --concatenazione per
                                                                  formare il vettore di
                                                                  32 bit in uscita

-- CODIFICA DEI SECONDI

--Cifra meno significativa dei secondi
    with conv_integer(unsigned(in_sec)) select
            sec_temp (3 downto 0) <= "0000" when 0|10|20|30|40|50,
                                     "0001" when 1|11|21|31|41|51,
                                     "0010" when 2|12|22|32|42|52,
                                     "0011" when 3|13|23|33|43|53,
                                     "0100" when 4|14|24|34|44|54,
                                     "0101" when 5|15|25|35|45|55,
                                     "0110" when 6|16|26|36|46|56,
                                     "0111" when 7|17|27|37|47|57,
                                     "1000" when 8|18|28|38|48|58,
                                     "1001" when 9|19|29|39|49|59,
                                     "1111" when others;

    -- Cifra più significativa dei secondi
     with conv_integer(unsigned(in_sec)) select
            sec_temp (7 downto 4) <= "0000" when 0 to 9,
                                     "0001" when 10 to 19,
                                     "0010" when 20 to 29,
                                     "0011" when 30 to 39,
                                     "0100" when 40 to 49,
                                     "0101" when 50 to 59,
                                     "1111" when others;

 -- CODIFICA DEI MINUTI

 -- Cifra meno significativa dei minuti
     with conv_integer(unsigned(in_min)) select
            min_temp (3 downto 0) <= "0000" when 0|10|20|30|40|50,
                                     "0001" when 1|11|21|31|41|51,
                                     "0010" when 2|12|22|32|42|52,
                                     "0011" when 3|13|23|33|43|53,
                                     "0100" when 4|14|24|34|44|54,
                                     "0101" when 5|15|25|35|45|55,
                                     "0110" when 6|16|26|36|46|56,
                                     "0111" when 7|17|27|37|47|57,
                                     "1000" when 8|18|28|38|48|58,
                                     "1001" when 9|19|29|39|49|59,
                                     "1111" when others;

    -- Cifra più significativa dei minuti
     with conv_integer(unsigned(in_min)) select
            min_temp (7 downto 4) <= "0000" when 0 to 9,
                                     "0001" when 10 to 19,
                                     "0010" when 20 to 29,
                                     "0011" when 30 to 39,
                                     "0100" when 40 to 49,
                                     "0101" when 50 to 59,
                                     "1111" when others;
```

```
-- CODIFICA DELLE ORE

-- Cifra meno significativa delle ore
     with conv_integer(unsigned(in_ore)) select
            ore_temp (3 downto 0) <= "0000" when 0|10|20,
                                     "0001" when 1|11|21,
                                     "0010" when 2|12|22,
                                     "0011" when 3|13|23,
                                     "0100" when 4|14,
                                     "0101" when 5|15,
                                     "0110" when 6|16,
                                     "0111" when 7|17,
                                     "1000" when 8|18,
                                     "1001" when 9|19,
                                     "1111" when others;

    -- Cifra più significativa delle ore
    with conv_integer(unsigned(in_ore)) select
             ore_temp (7 downto 4) <= "0000" when 0 to 9,
                                     "0001" when 10 to 19,
                                     "0010" when 20 to 23,
                                     "1111" when others;


end DataFlow;
```

*Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint a cui sono state apportate delle modifiche, cioè quelle relative a clock, switch, bottoni e display a 7 segmenti.

```
## This file is a general .xdc for the Nexys A7-100T
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to the top level
signal names in the project

## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
```

```
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }];
#IO_L7N_T1_D10_14 Sch=sw[5]
#set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
#set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
#set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];
#IO_L24N_T3_34 Sch=sw[8]
#set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }];
#IO_25_34 Sch=sw[9]
#set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
#set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
#set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }];
#IO_L24P_T3_35 Sch=sw[12]
#set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
#set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

##7 segment display
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { cathodes[0]
}]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { cathodes[1]
}]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { cathodes[2]
}]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { cathodes[3]
}]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { cathodes[4]
}]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { cathodes[5]
}]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { cathodes[6]
}]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { cathodes[7]
}]; #IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { anodes[0] }];
#IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { anodes[1] }];
#IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { anodes[2] }];
#IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { anodes[3] }];
#IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { anodes[4] }];
#IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { anodes[5] }];
#IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { anodes[6] }];
#IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { anodes[7] }];
#IO_L23N_T3_A02_D18_14 Sch=an[7]
```

```
##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { set_btn }];
#IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { load_sec }];
#IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { load_min }];
#IO_L10N_T1_D15_14 Sch=btnr
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { load_h }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

## Esercizio 5.3

Estendere il componente sviluppato ai punti precedenti in modo che sia in grado di acquisire e memorizzare internamente fino ad un numero N di intertempi in corrispondenza di un ingresso di *stop*. Opzionalmente, il componente può prevedere una modalità di visualizzazione in cui, alla pressione di un bottone, vengano visualizzati sui display gli intertempi memorizzati (uno per ogni pressione).

### *Progetto e architettura*

Il sistema è stato realizzato trascurando inizialmente la modalità di visualizzazione opzionale, e verificandone il corretto funzionamento esclusivamente in simulazione.

In questa fase, è stato impiegato un approccio strutturale, e i componenti usati sono il **cronometro** realizzato nell'Esercizio 5.1, una **memoria**, un **contatore** che incrementa l'indirizzo della memoria in cui scrivere, e un "**gestore della memoria**" che implementa l'automa illustrato in *Figura 5.8* e che fornisce alla memoria il segnale di write e al contatore il segnale di enable.

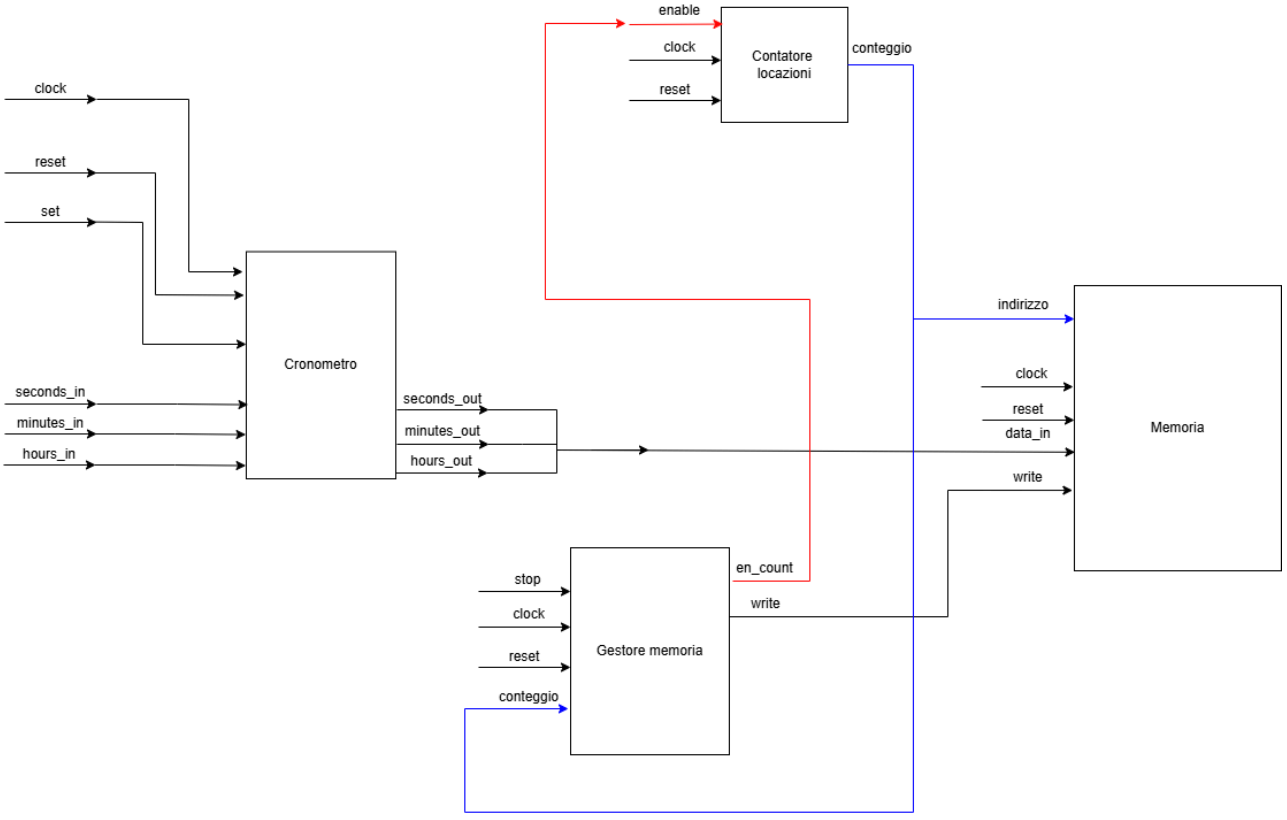



*Figura 5.6 Disegno architetturale del sistema*

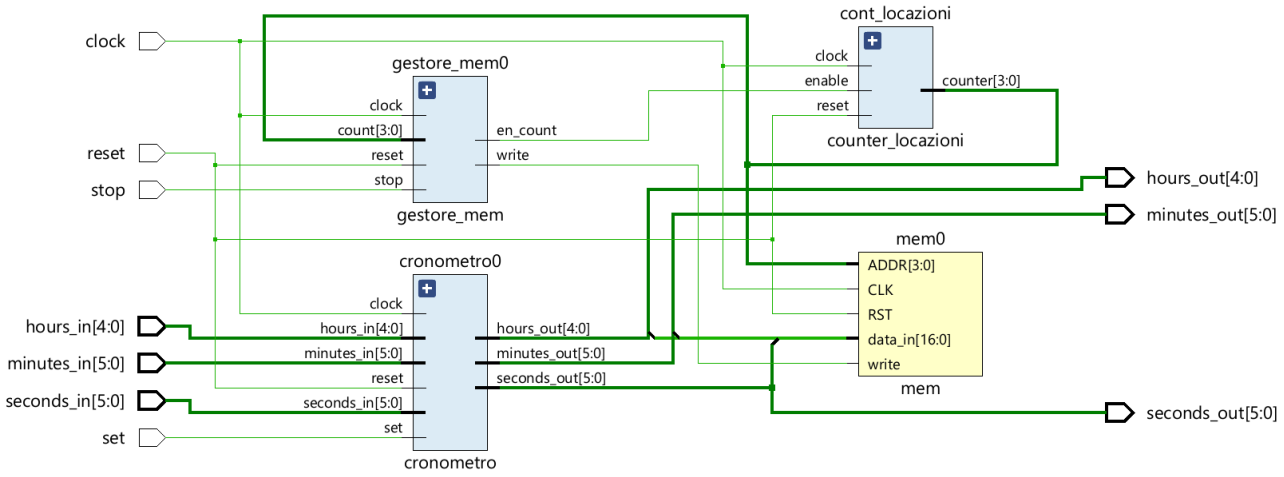



*Figura 5.7 Schematic del sistema complessivo implementato*

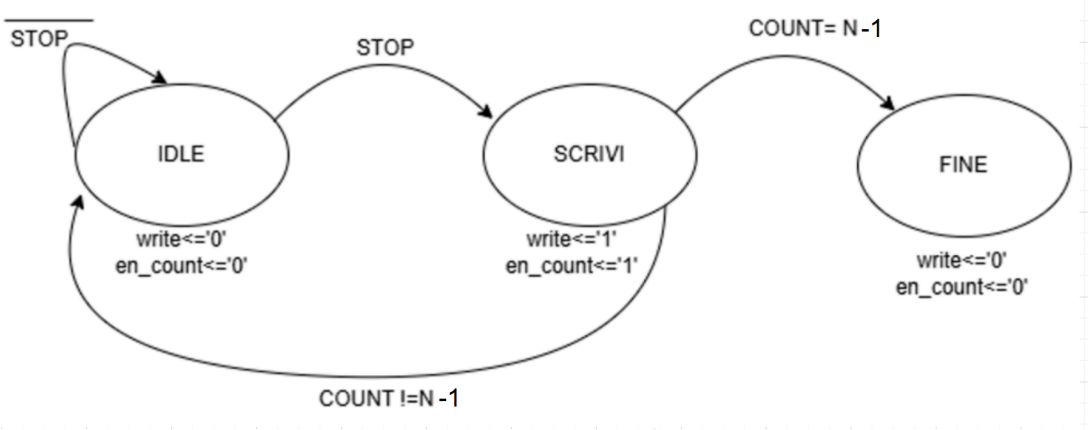



*Figura 5.8 Automa del componente gestore_mem*

### *Implementazione*

L'implementazione del cronometro è quella riportata nell'Esercizio 5.1.

L'implementazione del sistema complessivo, che include il cronometro, la memoria, il contatore delle locazioni e il gestore della memoria, è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity sistema_complessivo is
  Port (
        stop: in std_logic;
        clock: in std_logic;
        reset: in std_logic;
        set: in std_logic;
        minutes_in: in std_logic_vector(5 downto 0);
        seconds_in: in std_logic_vector(5 downto 0);
        hours_in: in std_logic_vector(4 downto 0);
        seconds_out: out std_logic_vector(5 downto 0);
```

```
        minutes_out: out std_logic_vector(5 downto 0);
        hours_out: out std_logic_vector(4 downto 0)
        );

end sistema_complessivo;

architecture Structural of sistema_complessivo is

          signal en_write: std_logic; -- segnale in uscita dal gestore della memoria
                                         che dice alla memoria quando scrivere
          signal addr_temp: std_logic_vector(3 downto 0); -- indirizzo in uscita dal
                                                             contatore che indica
                                                             l'indirizzo di memoria in
                                                             cui scrivere
          signal output_timer: std_logic_vector(16 downto 0);
          constant N_locazioni: integer := 16;
          constant N_bit: integer := 4;
          signal umax: std_logic;
          signal en_count: std_logic; -- abilitazione del contatore, in uscita da
                                         gestore memoria


          component cronometro is
            Port (
                clock: in std_logic;
                reset: in std_logic;
                set: in std_logic;
                minutes_in: in std_logic_vector(5 downto 0);
                seconds_in: in std_logic_vector(5 downto 0);
                hours_in: in std_logic_vector(4 downto 0);

                seconds_out: out std_logic_vector(5 downto 0);
                minutes_out: out std_logic_vector(5 downto 0);
                hours_out: out std_logic_vector(4 downto 0)

            );
        end component;



           component mem is
            port(
                CLK: in std_logic;
                RST: in std_logic;
                write: in std_logic; -- Segnale di scrittura
                ADDR : in std_logic_vector(3 downto 0);
                data_in: in std_logic_vector(16 downto 0)
                --data_out: out std_logic_vector(16 downto 0)
            );
        end component;


        component counter_locazioni is
        generic(

            n: integer := N_bit ;--numero di bit necessari
```

```
            modulo: integer := N_locazioni

            );

        Port ( clock : in  STD_LOGIC;
               reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
               enable : in STD_LOGIC;
               umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
                                    conteggio
               counter : out  STD_LOGIC_VECTOR (N_bit-1 downto 0));

    end component;



    component gestore_mem is
      Port (
            stop: in std_logic;
            clock: in std_logic;
            reset: in std_logic;
            write: out std_logic;
            en_count: out std_logic;
            count: STD_LOGIC_VECTOR (N_bit-1 downto 0)
        );
    end component;

begin


           cronometro0: cronometro
          Port map (
                clock=>clock,
                reset=> reset,
                set=> set,
                minutes_in => minutes_in,
                seconds_in=> seconds_in,
                hours_in=> hours_in,

                seconds_out=>output_timer(5 downto 0),
                minutes_out=> output_timer(11 downto 6),
                hours_out => output_timer(16 downto 12)

           );

            seconds_out<=output_timer(5 downto 0);
            minutes_out<= output_timer(11 downto 6);
            hours_out <= output_timer(16 downto 12);


           mem0: mem
            port map(
                CLK => clock,
                RST => reset,
                write => en_write,
                ADDR => addr_temp,
                data_in => output_timer -- INGRESSO CHE SCRIVO SOLO QUANDO MI ARRIVA IL
```

```
                                            SEGNALE DI WRITE
            );

        cont_locazioni: counter_locazioni
        generic map(

            n => N_bit ,--numero di bit necessari
            modulo=>  N_locazioni

            )

        Port map
              ( clock=> clock,
               reset=> reset, --ingresso di reset per azzerare il contatore
               enable=> en_count,
               umax=>  umax, -- si alza quando si raggiunge il valore massimo di
                               conteggio
               counter => addr_temp
               );


        gestore_mem0: gestore_mem
        port map(
            stop => stop,
            reset=> reset,
            clock=> clock,
            write=>en_write,
            en_count=> en_count,
            count=>addr_temp
        );



end Structural;
```

Di seguito è riportata l'implementazione della **memoria**.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mem is
    port(
        CLK: in std_logic;
        RST: in std_logic;
        write: in std_logic; -- Segnale di scrittura
        ADDR : in std_logic_vector(3 downto 0);
        data_in: in std_logic_vector(16 downto 0)
        --data_out: out std_logic_vector(16 downto 0)
    );
end mem;

architecture Behavioral of mem is
    type mem_type is array (0 to 15) of std_logic_vector(16 downto 0);
    signal MEM : mem_type := (others => (others => '0')); -- Inizializza tutta la
                                                             memoria a 0
```

```
begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if RST = '1' then  -- Reset sincrono: azzera tutta la memoria
                for i in 0 to 15 loop
                    MEM(i) <= (others => '0');
                end loop;
            elsif write = '1' then  -- Scrittura sincrona
                MEM(to_integer(unsigned(ADDR))) <= data_in;
            end if;
        end if;
    end process;

   -- data_out <= MEM(to_integer(unsigned(ADDR)));

end Behavioral;
```

L'implementazione del contatore delle locazioni è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;



entity counter_locazioni is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
            enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
                                   conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));


end counter_locazioni;

architecture Behavioral of counter_locazioni is

signal c : std_logic_vector (n-1 downto 0) := (others => '0');


signal u : std_logic := '0';

begin
```

```
counter_process: process(clock)
begin

         if rising_edge(clock) then
          if (reset = '1') then  -- reset sincrono con il clock
            c <= (others => '0');
          elsif (enable='1') then
              if( c=std_logic_vector(TO_UNSIGNED(modulo-1,n))) then
                    c <= (others => '0');

               else
                    c <= std_logic_vector(unsigned(c) + 1);

              end if;
              end if;
          end if;
    end process;



     proc_count_max: process(clock)
     begin
        if rising_edge(clock) then -- lo attivo sul rising edge in modo da essere
                                      sicura di leggere il valore corrente e non del
                                      ciclo precedente
           if (reset = '1') then
               u <= '0'; – reset del segnale
           elsif (c= std_logic_vector(TO_UNSIGNED(modulo-1,n))) then -- cast per type
                                                                        mismatch
               u <= '1'; --attivo quando raggiungo il massimo
           else
               u <= '0'; --se no rimane basso
           end if;
        end if;
     end process;


   counter<=c;
   umax<=u;

end Behavioral;
```

Infine, di seguito è riportata l'implementazione del **gestore della memoria**.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity gestore_mem is
  Port (
        stop: in std_logic;
        clock: in std_logic;
        reset: in std_logic;
        write: out std_logic;
        en_count: out std_logic;
```

```
        count: STD_LOGIC_VECTOR (3 downto 0)
     );
end gestore_mem;

architecture Behavioral of gestore_mem is
    type state is (idle, scrivi, fine);

    signal current_state,next_state: state;

      begin


      reg_stato: process(clock)
            begin
                    if(clock'event and clock='1') then
                       if(reset='1') then
                            current_state <=idle;
                       else
                            current_state <=next_state;
                       end if;
                    end if;
            end process;


    comb: process(current_state,stop)
               begin


           CASE current_state is

               WHEN idle =>
                   write<='0';
                   en_count<='0';
                   if stop = '1' then
                          next_state<=scrivi;
                    else
                          next_state<=idle;
                   end if;

               WHEN scrivi =>
                   write<='1';
                   en_count<='1';
                   if count="1111" then
                       next_state<=fine;
                   else
                       next_state<=idle;
               end if;



        when fine =>
                    write <='0';
                    en_count<='0';

             end CASE;
      end process;
```

*Simulazione*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity tb_sistema_complessivo is
end tb_sistema_complessivo;

architecture behavior of tb_sistema_complessivo is

    -- Component declaration
    component sistema_complessivo
        Port (
            stop: in std_logic;
            clock: in std_logic;
            reset: in std_logic;
            set: in std_logic;
            minutes_in: in std_logic_vector(5 downto 0);
            seconds_in: in std_logic_vector(5 downto 0);
            hours_in: in std_logic_vector(4 downto 0);
            seconds_out: out std_logic_vector(5 downto 0);
            minutes_out: out std_logic_vector(5 downto 0);
            hours_out: out std_logic_vector(4 downto 0)
        );
    end component;

    -- Signal declaration
    signal stop       : std_logic := '0';
    signal clock      : std_logic := '0';
    signal reset      : std_logic := '0';
    signal set        : std_logic := '0';
    signal minutes_in : std_logic_vector(5 downto 0) := (others => '0');
    signal seconds_in : std_logic_vector(5 downto 0) := (others => '0');
    signal hours_in   : std_logic_vector(4 downto 0) := (others => '0');
    signal seconds_out: std_logic_vector(5 downto 0);
    signal minutes_out: std_logic_vector(5 downto 0);
    signal hours_out  : std_logic_vector(4 downto 0);

    -- Clock period
    constant clk_period : time := 10 ns;

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: sistema_complessivo Port Map (
        stop => stop,
        clock => clock,
        reset => reset,
        set => set,
        minutes_in => minutes_in,
        seconds_in => seconds_in,
        hours_in => hours_in,
        seconds_out => seconds_out,
        minutes_out => minutes_out,
        hours_out => hours_out
```

```
    );

    -- Clock process
    clk_process :process
    begin
        clock <= '0';
        wait for clk_period/2;
        clock <= '1';
        wait for clk_period/2;
    end process;

    -- Stimulus process
    stim_proc: process
    begin
        -- Reset
        reset <= '1';
        wait for 20 ns;
        reset <= '0';

        -- Wait and observe normal operation
        wait for 200 ns;

        -- primo stop
        stop <= '1';
        wait for 10 ns;
        stop <= '0';


        -- Attendi un altro po' per vedere il comportamento
        wait for 100 ns;

       -- TEST 2: Cambia il valore con il set durante il conteggio
        set <= '1';
        hours_in <= "10111";   -- 23 ore
        minutes_in <= "111011"; -- 59 minuti
        seconds_in <= "111000"; -- 56 secondi
        wait for 10 ns;
        set <= '0';

      --secondo stop
        wait for 10 ns;
        stop<='1';
        wait for 10 ns;
        stop<='0';
      --terzo stop
        wait for 105 ns;
        stop<='1';
        wait for 10 ns;
        stop<='0';

        -- Termina simulazione
        wait;
    end process;

end behavior;
```

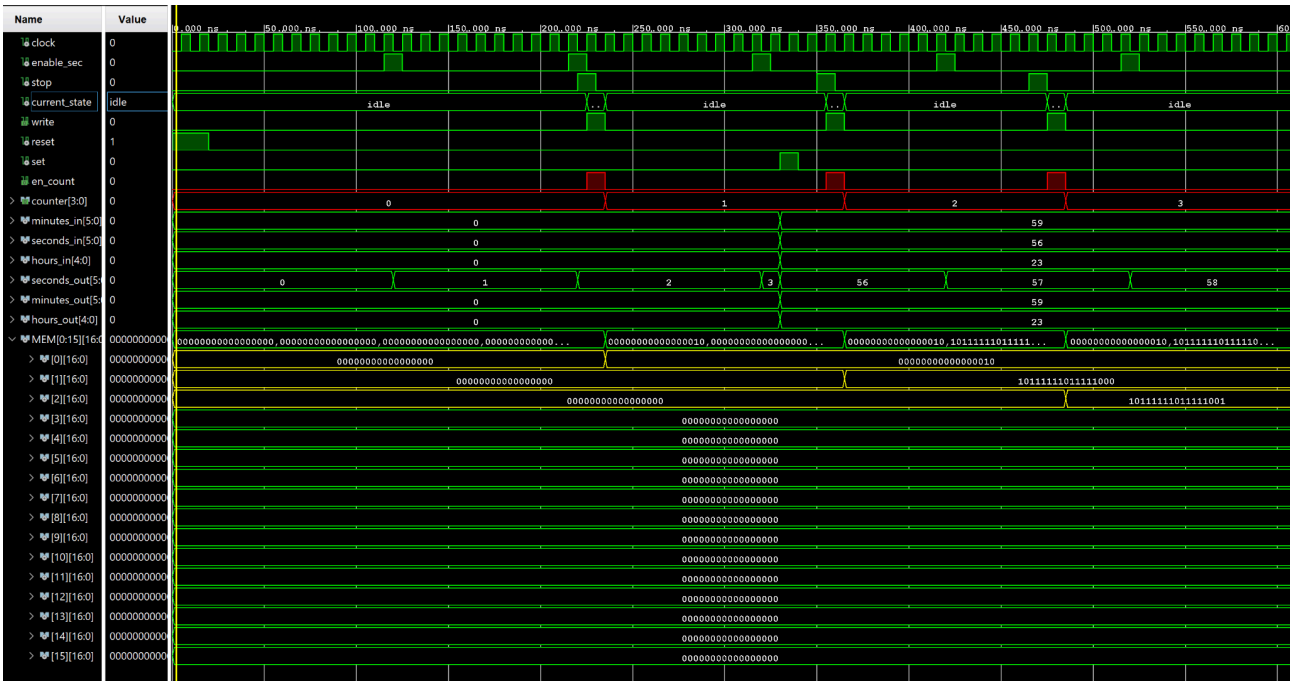



*Figura 5.9 Risultato della simulazione*

In *Figura 5.8* è possibile osservare il risultato della simulazione. In rosso sono evidenziati il segnale che abilita il contatore e il valore in uscita dal contatore, cioè l'indirizzo in cui sarà scritto in memoria l'intertempo.

In giallo ci sono le prime tre locazioni della memoria, cioè quelle che vengono modificate in questa simulazione. Le locazioni di memoria contengono un numero binario di 17 bit: i 6 meno significativi sono i secondi, i precedenti 6 sono i minuti  e i primi 5 sono le ore.

## *Progetto e architettura - Parte Opzionale*

Per aggiungere la modalità opzionale di visualizzazione sulla board, è stato innanzitutto necessario modificare il componente realizzato nell'Esercizio 5.2, in quanto i bottoni presenti sulla board non sono più sufficienti. Pertanto, è stato usato un unico bottone per fare il load sia di secondi, che di minuti, che di ore, ed è stata modificata l'unità di controllo affinché implementi un automa vero e proprio per l'acquisizione degli input. A tal fine è stato aggiunto un Button Debouncer (la cui implementazione è in Appendice) che "ripulisce" il segnale di load.

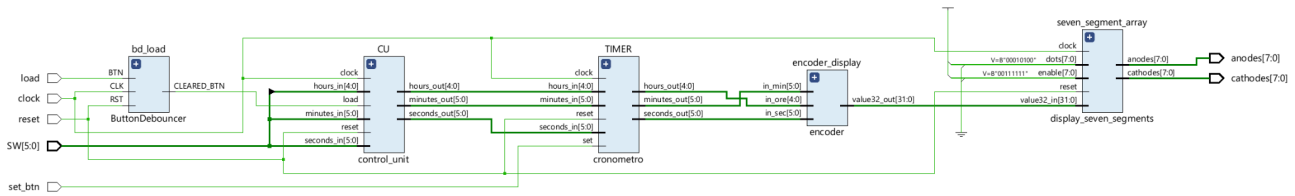



*Figura 5.9 Schematic del sistema cronometro_on_board aggiornato*

Una volta apportate queste modifiche all'unità di controllo, è necessario estendere l'architettura realizzata nella parte non opzionale dell'esercizio, cioè quella rappresentata in *Figura 5.7*. Di seguito sono riportate le principali aggiunte e modifiche.

- Il segnale di *stop* è stato associato a un bottone. Di conseguenza, è stato aggiunto un Button Debouncer che "ripulisce" il segnale di stop.
- Dal momento che è necessario visualizzare il contenuto della memoria sul display, alla memoria è stato aggiunto un ulteriore ingresso che rappresenta l'indirizzo della locazione da cui leggere, e un'uscita che rappresenta il contenuto della locazione selezionata.
- Inoltre, è stato aggiunto un ulteriore contatore delle locazioni, che serve a scorrere le locazioni della memoria, ma a differenza di quello già presente viene utilizzato per determinare le locazioni di memoria da cui leggere. L'abilitazione di questo contatore è data dal componente "gestione visualizza".
- Il componente "gestione visualizza" implementa l'automa (in *Figura 5.11*) che consente di visualizzare il contenuto della memoria, cioè gli intertempi memorizzati. In particolare, quando si preme il bottone denominato "visualizza", di passare alla modalità di visualizzazione degli intertempi. Ad ogni pressione di questo bottone, il componente "gestione visualizza" dà il segnale di read alla memoria, e questo consente di visualizzare sul display uno degli intertempi memorizzati. Inoltre, viene incrementato l'indirizzo della locazione da mostrare alla successiva pressione del bottone. Anche in questo caso, è stato aggiunto un Button Debouncer per "ripulire" questo segnale.
- La modalità di visualizzazione degli intertempi è attivata quando il componente *gestione_visualizza* rileva che il pulsante visualizza è stato premuto e imposta *visualizza_mode* (l'ingresso di selezione del multiplexer) a 1.
- Sono stati aggiunti, inoltre, il codificatore e il blocco che gestisce il display a sette segmenti, già utilizzati nell'Esercizio 5.2.
- Per scegliere se visualizzare sul display l'uscita del cronometro o quella della memoria, è stato introdotto un multiplexer. Quest'ultimo invia in ingresso all'encoder l'uscita della memoria quando si attiva la modalità di visualizzazione degli intertempi, altrimenti invia l'uscita del cronometro.

Il sistema ottenuto è rappresentato in *Figura 5.10*.

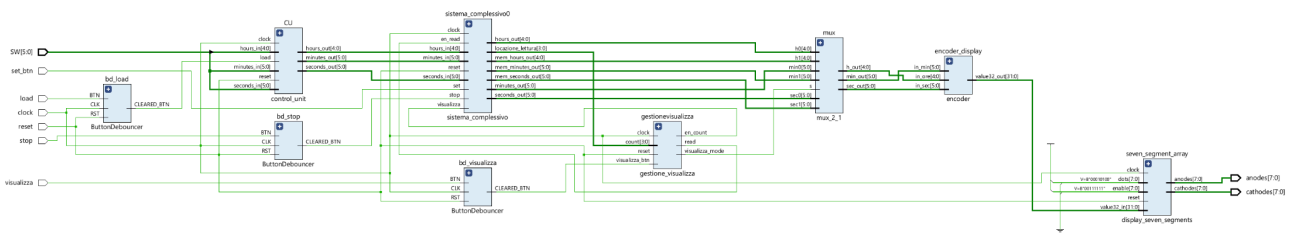



*Figura 5.10 Schematic del sistema*

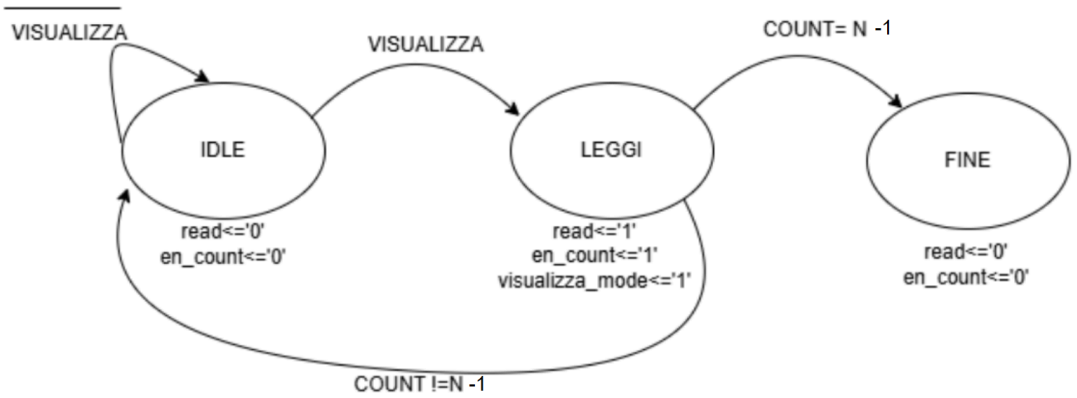



*Figura 5.11 Automa del componente "gestione visualizza"*

### *Implementazione*

Di seguito è riportato il codice a cui sono state apportate delle modifiche (in grassetto) rispetto al componente realizzato nell'Esercizio 5.2: sistema complessivo e unità di controllo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cronometro_on_board is
  Port (
        clock: in std_logic;
        reset: in std_logic; -- bottone N17
        set_btn: in std_logic; -- bottone M18
        SW: in std_logic_vector(5 downto 0);

        load: in std_logic; -- bottone P17
        anodes : out  STD_LOGIC_VECTOR (7 downto 0); --anodi e catodi delle cifre, sono
                                                        un output del topmodule
       cathodes : out  STD_LOGIC_VECTOR (7 downto 0)



   );
end cronometro_on_board;

architecture Structural of cronometro_on_board is

        signal value_temp : std_logic_vector(16 downto 0);
        signal value_encoded : std_logic_vector(31 downto 0);

        signal cleared_load: std_logic;
        signal sec_set: std_logic_vector(5 downto 0);
        signal min_set: std_logic_vector(5 downto 0);
        signal h_set: std_logic_vector(4 downto 0);



        component cronometro is
        Port (
            clock: in std_logic;
            reset: in std_logic;
```

```
    set: in std_logic;
    minutes_in: in std_logic_vector(5 downto 0);
    seconds_in: in std_logic_vector(5 downto 0);
    hours_in: in std_logic_vector(4 downto 0);
    seconds_out: out std_logic_vector(5 downto 0);
    minutes_out: out std_logic_vector(5 downto 0);
    hours_out: out std_logic_vector(4 downto 0)
   );
end component;

component ButtonDebouncer is
generic (
    CLK_period: integer := 10;  -- periodo del clock (della board) in
                                   nanosecondi
    btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del
                                           bottone in nanosecondi
                                       -- il valore di default è 10
                                           millisecondi
);
Port ( RST : in STD_LOGIC;
       CLK : in STD_LOGIC;
       BTN : in STD_LOGIC;
       CLEARED_BTN : out STD_LOGIC);
end component;


component display_seven_segments is
Generic(
           clock_frequency_in : integer := 50000000;
           clock_frequency_out : integer := 5000000
           );
Port ( clock : in  STD_LOGIC;
       reset : in  STD_LOGIC;
       value32_in : in  STD_LOGIC_VECTOR (31 downto 0);
       enable : in  STD_LOGIC_VECTOR (7 downto 0);
       dots : in  STD_LOGIC_VECTOR (7 downto 0);
       anodes : out  STD_LOGIC_VECTOR (7 downto 0);
       cathodes : out  STD_LOGIC_VECTOR (7 downto 0));
end component;


component control_unit is -- gestisce l'acquisizione del tempo settato
Port (
        clock : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        load : in  STD_LOGIC;
        seconds_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                        switch per i secondi
        minutes_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                        switch per i minuti
        hours_in : in STD_LOGIC_VECTOR(4 downto 0); --valore acquisito da 6
                                                        switch per le ore

        seconds_out: out STD_LOGIC_VECTOR(5 downto 0); --valori da mostrare in
                                                        decimale sul display
        minutes_out : out STD_LOGIC_VECTOR(5 downto 0);
```

```
                hours_out : out STD_LOGIC_VECTOR(4 downto 0)
                      );
        end component;

        component encoder is
        Port(
               in_sec: in std_logic_vector(5 downto 0);
               in_min: in std_logic_vector(5 downto 0);
               in_ore: in std_logic_vector(4 downto 0);
               value32_out: out  STD_LOGIC_VECTOR (31 downto 0)

            );
         end component;

begin


    TIMER: cronometro
        port map(
            clock => clock,
            reset => reset,
            set => set_btn,
            minutes_in =>  min_set,
            seconds_in =>sec_set,
            hours_in => h_set,
            seconds_out => value_temp(5 downto 0),
            minutes_out =>value_temp(11 downto 6),
            hours_out => value_temp(16 downto 12)
        );

      CU: control_unit
        Port map(
               clock => clock,
               reset => reset,
               load => cleared_load,
               seconds_in => SW, --valore acquisito da 6 switch per i secondi
               minutes_in => SW, --valore acquisito da 6 switch per i minuti
               hours_in => SW(4 downto 0), --valore acquisito da 6 switch per le ore
               seconds_out => sec_set,
               minutes_out => min_set,
               hours_out => h_set

               );

         seven_segment_array: display_seven_segments GENERIC MAP(
            clock_frequency_in => 100000000, --qui inserisco i parametri effettivi
                                             -- (clock della board e clock in
                                             -- uscita desiderato)
            clock_frequency_out => 500 --inserendo un valore inferiore si vedranno
                                       -- le cifre illuminarsi in sequenza
            )
            PORT MAP(
                clock => clock,
                reset => reset,
                value32_in => value_encoded,
                enable => "00111111", --stabilisco che tutti i display siano accesi
```

```
            dots => "00010100",
            anodes => anodes,
            cathodes => cathodes
        );

        bd_load: ButtonDebouncer
            generic map (
                CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
                btn_noise_time => 10000000
                    -- durata stimata dell'oscillazione del bottone in nanosecondi
                   -- il valore di default è 10 millisecondi
            )
            Port map ( RST => reset,
                   CLK => clock,
                   BTN => load,
                   CLEARED_BTN => cleared_load
             );


        encoder_display: encoder
        Port map(
               in_sec => value_temp(5 downto 0),
               in_min => value_temp(11 downto 6),
               in_ore => value_temp(16 downto 12),
               value32_out => value_encoded
            );

end Structural;
```

La nuova **control unit** è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity control_unit is
    Port (
        clock       : in  STD_LOGIC;
        reset       : in  STD_LOGIC;
        load        : in  STD_LOGIC;  -- già debounced
        seconds_in  : in  STD_LOGIC_VECTOR(5 downto 0);
        minutes_in  : in  STD_LOGIC_VECTOR(5 downto 0);
        hours_in    : in  STD_LOGIC_VECTOR(4 downto 0);
        seconds_out : out STD_LOGIC_VECTOR(5 downto 0);
        minutes_out : out STD_LOGIC_VECTOR(5 downto 0);
        hours_out   : out STD_LOGIC_VECTOR(4 downto 0)
    );
end control_unit;

architecture Behavioral of control_unit is


    type stato is (S_LOAD_SEC, S_LOAD_MIN, S_LOAD_HOUR);
    signal stato_corrente, stato_prossimo : stato := S_LOAD_SEC;
    signal reg_value : STD_LOGIC_VECTOR(16 downto 0) := (others => '0');

    -- Segnali per rilevamento fronte
```

```
    signal load_prev_sync : STD_LOGIC := '0';
    signal load_rise      : STD_LOGIC := '0';

begin

    seconds_out <= reg_value(5 downto 0);
    minutes_out <= reg_value(11 downto 6);
    hours_out   <= reg_value(16 downto 12);

    -- Processo rilevamento fronte di salita
    process(clock)
        begin
            if rising_edge(clock) then
                if (load = '1' and load_prev_sync = '0') then
                    load_rise <= '1';
                else
                    load_rise <= '0';
                end if;

                load_prev_sync <= load;
            end if;
    end process;



    process(stato_corrente, load_rise)
    begin
        case stato_corrente is
            when S_LOAD_SEC =>

                if load_rise = '1' then
                    stato_prossimo <= S_LOAD_MIN;
                else
                    stato_prossimo <= S_LOAD_SEC;
                end if;

            when S_LOAD_MIN =>
                if load_rise = '1' then
                    stato_prossimo <= S_LOAD_HOUR;
                else
                    stato_prossimo <= S_LOAD_MIN;
                end if;

            when S_LOAD_HOUR =>
                if load_rise = '1' then
                    stato_prossimo <= S_LOAD_SEC;
                else
                    stato_prossimo <= S_LOAD_HOUR;
                end if;
        end case;
    end process;


    process(clock)
    begin
        if rising_edge(clock) then
```

```
            if reset = '1' then
                stato_corrente <= S_LOAD_SEC;
                reg_value <= (others => '0');
            else
                stato_corrente <= stato_prossimo;


                    case stato_corrente is
                        when S_LOAD_SEC =>
                            if load_rise = '1' then
                                reg_value(5 downto 0) <= seconds_in;
                             end if;

                        when S_LOAD_MIN =>
                             if load_rise = '1' then
                                 reg_value(11 downto 6) <= minutes_in;
                             end if;

                        when S_LOAD_HOUR =>
                             if load_rise = '1' then
                                 reg_value(16 downto 12) <= hours_in;
                              end if;

                    end case;

            end if;
        end if;
    end process;

end Behavioral;
```

Segue l'implementazione del **top module** del sistema.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sistema_on_board is
  Port (
        clock: in std_logic;
        stop: in std_logic; -- bottone M17
        visualizza: in std_logic; -- bottone P18
        reset: in std_logic; -- bottone N17
        set_btn: in std_logic; -- bottone M18
        SW: in std_logic_vector(5 downto 0);
        load: in std_logic; -- bottone P17
        anodes : out  STD_LOGIC_VECTOR (7 downto 0); --anodi e catodi delle cifre, sono
                                                        un output del topmodule
       cathodes : out  STD_LOGIC_VECTOR (7 downto 0)


   );
end sistema_on_board;

architecture Structural of sistema_on_board is

        signal value_temp : std_logic_vector(16 downto 0);
```

```
signal value_temp_mem : std_logic_vector(16 downto 0);

signal value_to_encode: std_logic_vector(16 downto 0);
signal value_encoded : std_logic_vector(31 downto 0);

signal cleared_load: std_logic;
signal sec_set: std_logic_vector(5 downto 0);
signal min_set: std_logic_vector(5 downto 0);
signal h_set: std_logic_vector(4 downto 0);

signal stop_cleared: std_logic;
signal visualizza_cleared: std_logic;
signal locazione_lettura: std_logic_vector(3 downto 0);


signal visualizza_mode : std_logic := '0';

signal en_count_lettura: std_logic :='0';
signal en_read: std_logic :='0';


component sistema_complessivo is
  Port (
        stop: in std_logic;
        visualizza: in std_logic;
        locazione_lettura: out std_logic_vector(3 downto 0);
        clock: in std_logic;
        reset: in std_logic;
        set: in std_logic;
        minutes_in: in std_logic_vector(5 downto 0);
        seconds_in: in std_logic_vector(5 downto 0);
        hours_in: in std_logic_vector(4 downto 0);
        seconds_out: out std_logic_vector(5 downto 0);
        minutes_out: out std_logic_vector(5 downto 0);
        hours_out: out std_logic_vector(4 downto 0);
        mem_seconds_out: out std_logic_vector(5 downto 0);
        mem_minutes_out: out std_logic_vector(5 downto 0);
        mem_hours_out: out std_logic_vector(4 downto 0);
        en_read: in std_logic
        );

end component;

component ButtonDebouncer is
generic (
    CLK_period: integer := 10;  -- periodo del clock (della board) in
                                   nanosecondi
    btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del
                                          bottone in nanosecondi
                                       -- il valore di default è 10
                                           millisecondi
);
Port ( RST : in STD_LOGIC;
       CLK : in STD_LOGIC;
       BTN : in STD_LOGIC;
       CLEARED_BTN : out STD_LOGIC);
```

```
end component;


component display_seven_segments is
Generic(
            clock_frequency_in : integer := 50000000;
            clock_frequency_out : integer := 5000000
            );
Port ( clock : in  STD_LOGIC;
       reset : in  STD_LOGIC;
       value32_in : in  STD_LOGIC_VECTOR (31 downto 0);
       enable : in  STD_LOGIC_VECTOR (7 downto 0);
       dots : in  STD_LOGIC_VECTOR (7 downto 0);
       anodes : out  STD_LOGIC_VECTOR (7 downto 0);
       cathodes : out  STD_LOGIC_VECTOR (7 downto 0));
end component;


component control_unit is -- gestisce l'acquisizione del tempo settato
Port (
        clock : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        load : in  STD_LOGIC;
        seconds_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                        switch per i secondi
        minutes_in : in STD_LOGIC_VECTOR(5 downto 0); --valore acquisito da 6
                                                        switch per i minuti
        hours_in : in STD_LOGIC_VECTOR(4 downto 0); --valore acquisito da 6
                                                        switch per le ore
        seconds_out: out STD_LOGIC_VECTOR(5 downto 0); --valori da mostrare in
                                                        decimale sul display
        minutes_out : out STD_LOGIC_VECTOR(5 downto 0);
        hours_out : out STD_LOGIC_VECTOR(4 downto 0)
     );
end component;

component encoder is
Port(
       in_sec: in std_logic_vector(5 downto 0);
       in_min: in std_logic_vector(5 downto 0);
       in_ore: in std_logic_vector(4 downto 0);
       value32_out: out  STD_LOGIC_VECTOR (31 downto 0)

    );
 end component;


component mux_2_1 is
    Port ( sec0    : in  STD_LOGIC_VECTOR(5 downto 0);
           min0    : in  STD_LOGIC_VECTOR(5 downto 0);
           h0      : in  STD_LOGIC_VECTOR(4 downto 0);
           sec1    : in  STD_LOGIC_VECTOR(5 downto 0);
           min1    : in  STD_LOGIC_VECTOR(5 downto 0);
           h1      : in  STD_LOGIC_VECTOR(4 downto 0);
           s       : in  STD_LOGIC;
           sec_out : out STD_LOGIC_VECTOR(5 downto 0);
```

```
                  min_out : out STD_LOGIC_VECTOR(5 downto 0);
                  h_out   : out STD_LOGIC_VECTOR(4 downto 0)
                );
            end component;


         component gestione_visualizza is
          Port (
            clock : in std_logic;
            reset : in std_logic;
            visualizza_btn : in std_logic;
            en_count: out std_logic; -- abilita il contatore degli indirizzi di memoria
in lettura
            read: out std_logic;
            visualizza_mode : out std_logic;
            count: in std_logic_vector(3 downto 0)
          );
        end component;


begin


    sistema_complessivo0: sistema_complessivo
      Port map (
            stop => stop_cleared,
            visualizza=> en_count_lettura,
            locazione_lettura =>locazione_lettura,
            clock => clock,
            reset=> reset,
            set=> set_btn,
            minutes_in => min_set,
            seconds_in => sec_set,
            hours_in=> h_set,
            seconds_out => value_temp(5 downto 0),
            minutes_out =>value_temp(11 downto 6),
            hours_out => value_temp(16 downto 12),
            mem_seconds_out => value_temp_mem(5 downto 0),
            mem_minutes_out =>value_temp_mem(11 downto 6),
            mem_hours_out => value_temp_mem(16 downto 12),
            en_read=>en_read
            );


      CU: control_unit
        Port map(
              clock => clock,
              reset => reset,
              load => cleared_load,
              seconds_in => SW, --valore acquisito da 6 switch per i secondi
              minutes_in => SW, --valore acquisito da 6 switch per i minuti
              hours_in => SW(4 downto 0), --valore acquisito da 6 switch per le ore
              seconds_out => sec_set,
              minutes_out => min_set,
              hours_out => h_set
             );
```

```
         seven_segment_array: display_seven_segments GENERIC MAP(
            clock_frequency_in => 100000000, --qui inserisco i parametri effettivi
(clock della board e clock in uscita desiderato)
            clock_frequency_out => 500 --inserendo un valore inferiore si vedranno le
cifre illuminarsi in sequenza
            )
            PORT MAP(
                clock => clock,
                reset => reset,
                value32_in => value_encoded,
                enable => "00111111", --stabilisco che tutti i display siano accesi
                dots =>"00010100",
                anodes => anodes,
                cathodes => cathodes
        );

        bd_load: ButtonDebouncer
            generic map (
                CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
                btn_noise_time => 10000000 -- durata stimata dell'oscillazione del
bottone in nanosecondi
                                                        -- il valore di default è 10
millisecondi
            )
            Port map ( RST => reset,
                   CLK => clock,
                   BTN => load,
                   CLEARED_BTN => cleared_load
             );

        bd_stop: ButtonDebouncer
           generic map(
                 CLK_period => 10,
                 btn_noise_time=> 10000000
                )
           port map(
                   RST => reset,
                   CLK => clock,
                   BTN => stop,
                   CLEARED_BTN => stop_cleared
                   );

           bd_visualizza: ButtonDebouncer
           generic map(
                 CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
                 btn_noise_time=> 10000000
                )
           port map(
                   RST => reset,
                   CLK => clock,
                   BTN => visualizza,
                   CLEARED_BTN => visualizza_cleared
                   );

        encoder_display: encoder
```

```
        Port map(
               in_sec => value_to_encode(5 downto 0),
               in_min => value_to_encode(11 downto 6),
               in_ore => value_to_encode(16 downto 12),
               value32_out => value_encoded
            );

                mux: mux_2_1
            Port map(
                sec0  => value_temp(5 downto 0),
                min0  => value_temp(11 downto 6),
                h0    => value_temp(16 downto 12),
                sec1  => value_temp_mem(5 downto 0),
                min1  => value_temp_mem(11 downto 6),
                h1    => value_temp_mem(16 downto 12),
                s     => visualizza_mode,  -- usa il segnale latched
                sec_out => value_to_encode(5 downto 0),
                min_out => value_to_encode(11 downto 6),
                h_out   => value_to_encode(16 downto 12)
            );



        gestionevisualizza: gestione_visualizza
          port map (
            clock => clock,
            reset => reset,
            visualizza_btn => visualizza_cleared,
            visualizza_mode => visualizza_mode,
            count => locazione_lettura,
            en_count=> en_count_lettura,
            read => en_read

          );


end Structural;
```

Di seguito è riportata l'implementazione del sistema che memorizza gli intertempi (realizzato nella parte non opzionale dell'esercizio) con le aggiunte e le modifiche necessarie, evidenziate in grassetto.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity sistema_complessivo is
  Port (
        stop: in std_logic;
        visualizza: in std_logic;
        locazione_lettura: out std_logic_vector(3 downto 0);
        clock: in std_logic;
        reset: in std_logic;
        set: in std_logic;
        minutes_in: in std_logic_vector(5 downto 0);
        seconds_in: in std_logic_vector(5 downto 0);
```

```
        hours_in: in std_logic_vector(4 downto 0);
        seconds_out: out std_logic_vector(5 downto 0);
        minutes_out: out std_logic_vector(5 downto 0);
        hours_out: out std_logic_vector(4 downto 0);
        -- uscita dalla memoria
        mem_seconds_out: out std_logic_vector(5 downto 0);
        mem_minutes_out: out std_logic_vector(5 downto 0);
        mem_hours_out: out std_logic_vector(4 downto 0);

        en_read: in std_logic

        );

end sistema_complessivo;

architecture Structural of sistema_complessivo is

          signal en_write: std_logic; -- segnale in uscita dal gestore della memoria
                                         che dice alla memoria quando scrivere
          signal addr_temp: std_logic_vector(3 downto 0); -- indirizzo in uscita dal
                                                             contatore che indica
                                                             l'indirizzo di memoria in
                                                             cui scrivere
          signal addr_temp_lettura: std_logic_vector(3 downto 0); -- indirizzo in
                                                                     uscita dal secondo
                                                                     contatore che indica
                                                                     l'indirizzo di memoria
                                                                     da cui leggere
          signal output_timer: std_logic_vector(16 downto 0);
          constant N_locazioni: integer := 16;
          constant N_bit: integer := 4;
          signal umax: std_logic;
          signal output_mem: std_logic_vector(16 downto 0);
          signal en_count_scrittura: std_logic; -- abilitazione del contatore, in
                                                   uscita da gestore memoria
          signal en_count_lettura: std_logic; -- abilitazione del contatore per la
                                                 lettura. L'abilitazione è visualizza,
                                                 l'uscita del gestore gestione_visualizza



          component cronometro is
            Port (
                clock: in std_logic;
                reset: in std_logic;
                set: in std_logic;
                minutes_in: in std_logic_vector(5 downto 0);
                seconds_in: in std_logic_vector(5 downto 0);
                hours_in: in std_logic_vector(4 downto 0);

                seconds_out: out std_logic_vector(5 downto 0);
                minutes_out: out std_logic_vector(5 downto 0);
                hours_out: out std_logic_vector(4 downto 0)

            );
        end component;
```

```
          component mem is
           port(
               CLK: in std_logic;
               RST: in std_logic;
               write: in std_logic; -- Segnale di scrittura
               read: in std_logic; -- segnale di lettura
               ADDR : in std_logic_vector(3 downto 0);
               ADDR_lettura: in std_logic_vector(3 downto 0);
               data_in: in std_logic_vector(16 downto 0);
               data_out: out std_logic_vector(16 downto 0)
           );
        end component;


        component counter_locazioni is
        generic(

            n: integer := N_bit ;--numero di bit necessari
            modulo: integer := N_locazioni

            );

        Port ( clock : in  STD_LOGIC;
               reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
               enable : in STD_LOGIC;
               umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
                                    conteggio
               counter : out  STD_LOGIC_VECTOR (N_bit-1 downto 0));

    end component;


    component gestore_mem is
      Port (
            stop: in std_logic;
            clock: in std_logic;
            reset: in std_logic;
            write: out std_logic;
            en_count: out std_logic;
            count: STD_LOGIC_VECTOR (N_bit-1 downto 0)
         );
    end component;

begin

          cronometro0: cronometro
          Port map (
                clock=>clock,
                reset=> reset,
                set=> set,
                minutes_in => minutes_in,
                seconds_in=> seconds_in,
                hours_in=> hours_in,
```

```
            seconds_out=>output_timer(5 downto 0),
            minutes_out=> output_timer(11 downto 6),
            hours_out => output_timer(16 downto 12)

        );

         seconds_out<=output_timer(5 downto 0);
         minutes_out<= output_timer(11 downto 6);
         hours_out <= output_timer(16 downto 12);


        mem0: mem
         port map(
            CLK => clock,
            RST => reset,
            write => en_write,
            read =>en_read,
            ADDR => addr_temp,
            ADDR_lettura => addr_temp_lettura,
            data_in => output_timer, -- INGRESSO CHE SCRIVO SOLO QUANDO
                                        -- MI ARRIVA IL SEGNALE DI WRITE
            data_out => output_mem
         );


         mem_seconds_out<=output_mem(5 downto 0);
         mem_minutes_out<= output_mem(11 downto 6);
         mem_hours_out <= output_mem(16 downto 12);


    cont_locazioni_scrittura: counter_locazioni
    generic map(

        n => N_bit ,--numero di bit necessari
        modulo=>  N_locazioni

        )

    Port map
          ( clock=> clock,
           reset=> reset, --ingresso di reset per azzerare il contatore
           enable=> en_count_scrittura,
           umax=>  umax, -- si alza quando si raggiunge il valore massimo di
conteggio
           counter => addr_temp
           );

    cont_locazioni_lettura: counter_locazioni
    generic map(

        n => N_bit ,--numero di bit necessari
        modulo=>  N_locazioni

        )
```

```
        Port map
              ( clock=> clock,
               reset=> reset, --ingresso di reset per azzerare il contatore
               enable=> visualizza,
               umax=>  umax, -- si alza quando si raggiunge il valore massimo di
conteggio
               counter => addr_temp_lettura
               );


       gestore_mem0: gestore_mem
        port map(
            stop => stop,
            reset=> reset,
            clock=> clock,
            write=>en_write,
            en_count=> en_count_scrittura,
            count=>addr_temp
        );


        locazione_lettura<=addr_temp_lettura;


end Structural;
```

Come anticipato, è stata modificata la memoria per permettere anche la lettura. L'implementazione è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mem is
    port(
        CLK: in std_logic;
        RST: in std_logic;
        write: in std_logic; -- Segnale di scrittura
        read: in std_logic;  -- Segnale di lettura

        ADDR : in std_logic_vector(3 downto 0);
        ADDR_lettura: in std_logic_vector(3 downto 0);
        data_in: in std_logic_vector(16 downto 0);
        data_out: out std_logic_vector(16 downto 0)
    );
end mem;

architecture Behavioral of mem is
    type mem_type is array (0 to 15) of std_logic_vector(16 downto 0);
    signal MEM : mem_type := (others => (others => '0')); -- Inizializza tutta la
memoria a 0
    signal data_reg : std_logic_vector(16 downto 0);

begin
```

```
    process(CLK)
    begin
        if rising_edge(CLK) then
            if RST = '1' then  -- Reset sincrono: azzera tutta la memoria
                for i in 0 to 15 loop
                    MEM(i) <= (others => '0');
                end loop;
            elsif write = '1' then  -- Scrittura sincrona
                MEM(to_integer(unsigned(ADDR))) <= data_in;
            end if;
            if read = '1' then
                    data_reg <= MEM(to_integer(unsigned(ADDR_lettura)));
                end if;
        end if;
    end process;


    data_out <= data_reg;

end Behavioral;
```

Il codice del multiplexer utilizzato è il seguente. Si tratta di una sorta di multiplexer 2:1, ma che seleziona insieme tre valori (ore, minuti e secondi) invece di uno solo. In base al segnale di selezione, decide se inviare in uscita i dati del cronometro oppure quelli memorizzati.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_2_1 is
    Port ( sec0    : in  STD_LOGIC_VECTOR(5 downto 0);
           min0    : in  STD_LOGIC_VECTOR(5 downto 0);
           h0      : in  STD_LOGIC_VECTOR(4 downto 0);
           sec1    : in  STD_LOGIC_VECTOR(5 downto 0);
           min1    : in  STD_LOGIC_VECTOR(5 downto 0);
           h1      : in  STD_LOGIC_VECTOR(4 downto 0);
           s       : in  STD_LOGIC;
           sec_out : out STD_LOGIC_VECTOR(5 downto 0);
           min_out : out STD_LOGIC_VECTOR(5 downto 0);
           h_out   : out STD_LOGIC_VECTOR(4 downto 0)
         );
end mux_2_1;

architecture Behavioral of mux_2_1 is
begin

    sec_out <= sec0 when s = '0' else sec1;
    min_out <= min0 when s = '0' else min1;
    h_out <= h0 when s = '0' else h1;

end Behavioral;
```

Infine, è riportata di seguito la parte che gestisce il segnale visualizza_mode, che implementa l'automa in *Figura 5.11*. Nell'implementazione si può osservare che il segnale mode non viene aggiornato nel processo combinatorio che implementa la funzione di stato prossimo, bensì in un processo sequenziale sincrono al

clock. Questa scelta è motivata dalla necessità di memorizzare mode in un registro, in modo da mantenere attiva la modalità di visualizzazione (mode = '1') fino a una disattivazione esplicita, con il segnale di reset.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity gestione_visualizza is
  Port (
    clock : in std_logic;
    reset : in std_logic;
    visualizza_btn : in std_logic;
    en_count: out std_logic;                  -- abilita il contatore degli indirizzi
                                              -- di memoria in lettura
    read: out std_logic;
    visualizza_mode : out std_logic;
    count: in std_logic_vector(3 downto 0)
  );
end gestione_visualizza;

architecture Behavioral of gestione_visualizza is
 signal mode : std_logic := '0';

     type state is (idle, leggi, fine);


signal current_state,next_state: state;

       begin


       reg_stato: process(clock)
                      begin
                      if(clock'event and clock='1') then
                       if(reset='1') then
                               current_state <=idle;
                         else
                               current_state <=next_state;
                          end if;
                      end if;
                      end process;


    comb: process(current_state,visualizza_btn)
                begin


            CASE current_state is

                WHEN idle =>
                    read<='0';
                    en_count<='0';
                    if visualizza_btn = '1' then
```

```
                        next_state<=leggi;

                else
                    next_state<=idle;
                 end if;

               WHEN leggi =>
                   read<='1';
                   en_count<='1';
                   if count="1111" then
                       next_state<=fine;
                   else
                       next_state<=idle;
               end if;


        when fine =>
                    read <='0';
                    en_count<='0';



             end CASE;
      end process;


  visualizza_mode <= mode;



-- Il segnale 'mode' viene aggiornato in un processo sincrono
-- per garantire che rimanga attivo (a '1') durante l'intera
-- fase di visualizzazione, fino a disattivazione esplicita (reset).
-- Questa scelta implica l'uso di un registro per memorizzare l'output.

  process(clock)
  begin
    if rising_edge(clock) then
      if reset = '1' then
        mode <= '0';
      elsif current_state = leggi then
        mode <= '1';
      end if;
    end if;
  end process;



end Behavioral;
```

*Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint a cui sono state apportate delle modifiche, cioè quelle relative a clock, switch, bottoni e display a 7 segmenti.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }];
#IO_L7N_T1_D10_14 Sch=sw[5]
#set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
#set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
#set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];
#IO_L24N_T3_34 Sch=sw[8]
#set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }];
#IO_25_34 Sch=sw[9]
#set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
#set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
#set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }];
#IO_L24P_T3_35 Sch=sw[12]
#set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
#set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

##7 segment display
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { cathodes[0]
}]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { cathodes[1]
}]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { cathodes[2]
}]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { cathodes[3]
}]; #IO_L17P_T2_A26_15 Sch=cd
```

```
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { cathodes[4]
}]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { cathodes[5]
}]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { cathodes[6]
}]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { cathodes[7]
}]; #IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { anodes[0] }];
#IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { anodes[1] }];
#IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { anodes[2] }];
#IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { anodes[3] }];
#IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { anodes[4] }];
#IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { anodes[5] }];
#IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { anodes[6] }];
#IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { anodes[7] }];
#IO_L23N_T3_A02_D18_14 Sch=an[7]

##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { set_btn }];
#IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { load }];
#IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { stop }];
#IO_L10N_T1_D15_14 Sch=btnr
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { visualizza
}]; #IO_L9N_T1_DQS_D13_14 Sch=btnd
```

## Esercizio 6: Sistema di lettura-elaborazione-scrittura PO_PC

### Esercizio 6.1

Progettare, implementare in VHDL e verificare mediante simulazione un sistema dotato di una memoria ROM di N locazioni da 8 bit ciascuna, una macchina combinatoria M in grado di trasformare (secondo una funzione a scelta dello studente) la stringa di 8 bit letta dalla ROM in una stringa di 4 bit, e una memoria MEM di N locazioni che memorizza la stringa in output da M.

Il sistema si avvia in corrispondenza di un segnale di START che viene fornito esternamente. Una volta avviato, tramite un'apposita unità di controllo che gestisce la tempificazione del sistema, viene scandita una locazione alla volta della ROM e viene scritta la corrispondente locazione di MEM. Gli indirizzi di memoria sono forniti da un contatore. Le memorie ROM e MEM hanno rispettivamente un read e un write sincrono.

#### *Progetto e architettura*

Per realizzare il sistema PO-PC, è stato utilizzato un approccio strutturale. I componenti necessari sono: una memoria **ROM**, una macchina **M** (implementata come un **encoder 8:4**), una memoria **MEM** in cui memorizzare la stringa in uscita dalla macchina M, un'**unità di controllo** per la gestione della tempificazione del sistema e un **contatore** che fornisce gli indirizzi di memoria, sia per il componente ROM che per il componente MEM.

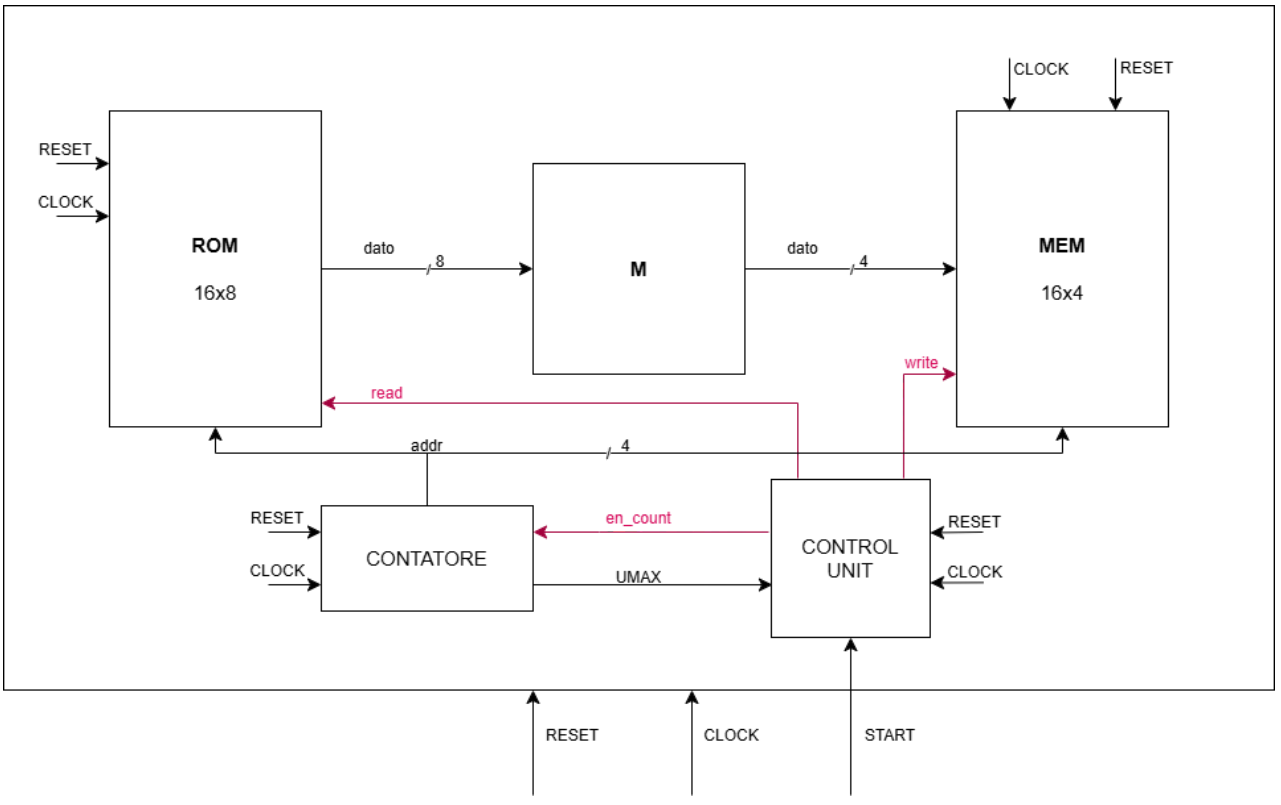



*Figura 6.1 Disegno architetturale del sistema*

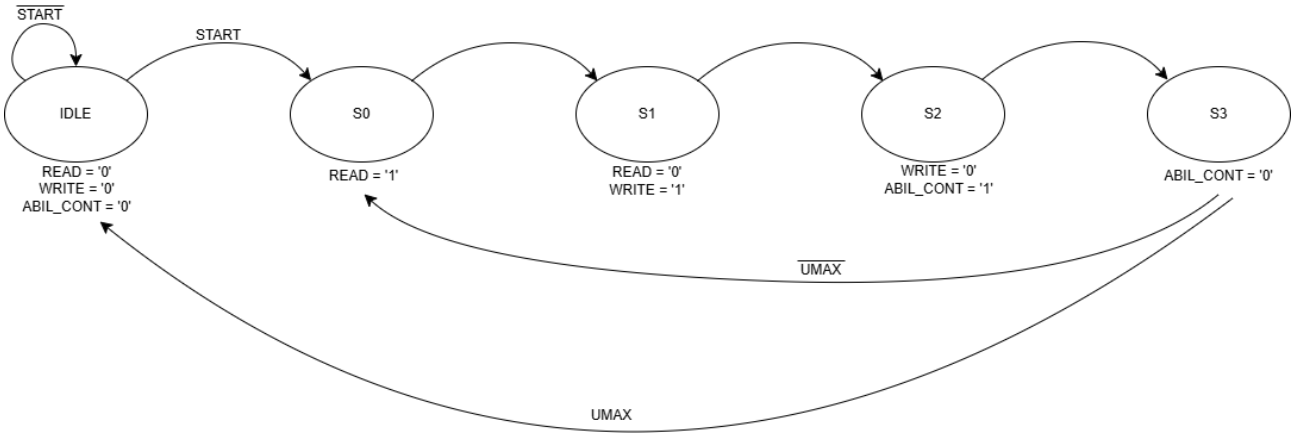



*Figura 6.2 Automa dell'unità di controllo*

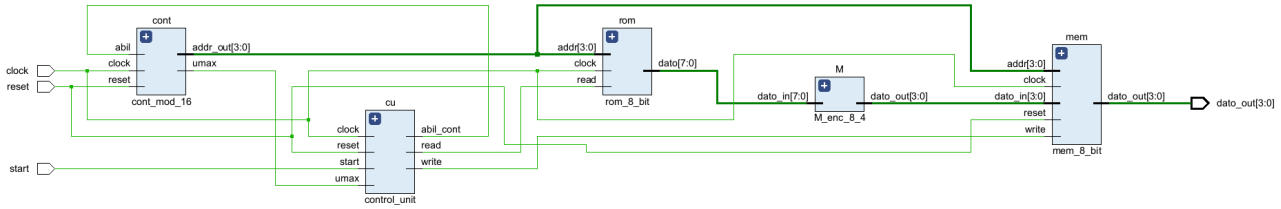



*Figura 6.3 Schematic del sistema implementato*

### *Implementazione*

Il sistema complessivo è stato implementato con approccio strutturale, come riportato di seguito.

**Sistema Totale:**

```
entity sis_tot is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           start : in STD_LOGIC;
           dato_out : out STD_LOGIC_VECTOR(3 downto 0));
end sis_tot;

architecture Structural of sis_tot is

component rom_8_bit is
    Port ( clock : in STD_LOGIC;
           read : in STD_LOGIC;
           addr : in STD_LOGIC_VECTOR(3 downto 0);
           dato : out STD_LOGIC_VECTOR(7 downto 0));
end component rom_8_bit;

component mem_8_bit is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           write : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (3 downto 0);
           addr : in STD_LOGIC_VECTOR (3 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end component mem_8_bit;

component M_enc_8_4 is
```

```
    Port ( dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end component M_enc_8_4;

component cont_mod_16 is
    Port ( abil : in STD_LOGIC;
           clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           addr_out : out STD_LOGIC_VECTOR (3 downto 0);
           umax : out STD_LOGIC);
end component cont_mod_16;

component control_unit is
    Port ( start : in STD_LOGIC;
           reset : in STD_LOGIC;
           umax : in STD_LOGIC;
           clock : in STD_LOGIC;
           read : out STD_LOGIC;
           write : out STD_LOGIC;
           abil_cont : out STD_LOGIC);
end component control_unit;

signal dato_out_rom : std_logic_vector(7 downto 0);
signal dato_in_mem : std_logic_vector(3 downto 0);
signal dato_out_mem : std_logic_vector(3 downto 0);
signal addr_tmp : std_logic_vector(3 downto 0);
signal read_tmp : std_logic := '0';
signal write_tmp : std_logic := '0';
signal abil_cont_tmp : std_logic := '0';
signal umax_tmp : std_logic := '0';

begin

    rom : rom_8_bit
    port map( clock => clock,
              read => read_tmp,
              addr => addr_tmp,
              dato => dato_out_rom
    );

    mem : mem_8_bit
    port map( clock => clock,
              reset => reset,
              write => write_tmp,
              dato_in => dato_in_mem,
              addr => addr_tmp,
              dato_out => dato_out_mem
    );

    M : M_enc_8_4
    port map( dato_in => dato_out_rom,
              dato_out => dato_in_mem
    );

    cont : cont_mod_16
    port map( abil => abil_cont_tmp,
```

```
            clock => clock,
            reset => reset,
            addr_out => addr_tmp,
            umax => umax_tmp
    );

    cu : control_unit
    port map( start => start,
            reset => reset,
            umax => umax_tmp,
            clock => clock,
            read => read_tmp,
            write => write_tmp,
            abil_cont => abil_cont_tmp
    );

    dato_out <= dato_out_mem;

end Structural;
```

L'**unità di controllo** realizza l'automa rappresentato in *Figura 6.2*. L'implementazione è la seguente.

```
entity control_unit is
    Port ( start : in STD_LOGIC;
           reset : in STD_LOGIC;
           umax : in STD_LOGIC;
           clock : in STD_LOGIC;
           read : out STD_LOGIC;
           write : out STD_LOGIC;
           abil_cont : out STD_LOGIC);
end control_unit;

architecture Behavioral of control_unit is

type stato is (IDLE, S0, S1, S2, S3); -- S1 -> read, S2 -> write, S3 -> count
signal stato_attuale: stato := IDLE;
signal stato_prossimo: stato := IDLE;


begin

    control : process(stato_attuale, start, umax)
    begin
            case stato_attuale is

                when IDLE =>
                    read <= '0';
                    write <= '0';
                    abil_cont <= '0';

                    if start = '1' then
                        stato_prossimo <= S0;
                    else
                        stato_prossimo <= IDLE;
```

```
                    end if;

                when S0 =>
                    read <= '1';
                    stato_prossimo <= S1;

                when S1 =>
                    read <= '0';
                    write <= '1';
                    stato_prossimo <= S2;

                when S2 =>
                    write <= '0';
                    abil_cont <= '1';
                    stato_prossimo <= S3;

                when S3 =>
                    abil_cont <= '0';

                    if umax = '1' then
                        stato_prossimo <= IDLE;
                    else
                        stato_prossimo <= S0;
                    end if;

            end case;

    end process;

    cambio_stato : process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                stato_attuale <= IDLE;
            else
                stato_attuale <= stato_prossimo;
            end if;
        end if;
    end process;

end Behavioral;
```

La **macchina M** è implementata come segue, ed è la medesima dell'esercizio 2.1.

```
entity M_enc_8_4 is
    Port ( dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end M_enc_8_4;

architecture Structural of M_enc_8_4 is

    component encoder4_2
        Port ( in_enc : in STD_LOGIC_VECTOR (3 downto 0);
               out_enc : out STD_LOGIC_VECTOR (1 downto 0));
    end component;
```

```
begin

    enc0 : encoder4_2
        Port map(in_enc(0) => dato_in(0),
                 in_enc(1) => dato_in(1),
                 in_enc(2) => dato_in(2),
                 in_enc(3) => dato_in(3),
                 out_enc(0) => dato_out(0),
                 out_enc(1) => dato_out(1));

    enc1 : encoder4_2
        Port map(in_enc(0) => dato_in(4),
                 in_enc(1) => dato_in(5),
                 in_enc(2) => dato_in(6),
                 in_enc(3) => dato_in(7),
                 out_enc(0) => dato_out(2),
                 out_enc(1) => dato_out(3));

end Structural;
```

La **ROM** è stata modificata rispetto all'esercizio 2.1, al fine di rendere la **lettura sincrona** con il **fronte di salita del clock**. L'implementazione è la seguente.

```
entity rom_8_bit is
    Port ( clock : in STD_LOGIC;
           read : in STD_LOGIC;
           addr : in STD_LOGIC_VECTOR(3 downto 0);
           dato : out STD_LOGIC_VECTOR(7 downto 0));
end rom_8_bit;

architecture Behavioral of rom_8_bit is

-- definisco il tipo memory come un array costituito da 16 array ciascuno da 8 elementi
type memory is array (0 to 15) of std_logic_vector (7 downto 0);

-- definisco la costante rom16_8 che è di tipo memory -> ogni cella è inizializzata con
-- un valore esadecimale
constant rom : memory := (
    x"11",   -- 00010001
    x"12",   -- 00010010
    x"14",   -- 00010100
    x"18",   -- 00011000
    x"21",   -- 00100001
    x"22",   -- 00100010
    x"24",   -- 00100100
    x"28",   -- 00101000
    x"41",   -- 01000001
    x"42",   -- 01000010
    x"44",   -- 01000100
    x"48",   -- 01001000
    x"81",   -- 10000001
    x"82",   -- 10000010
    x"84",   -- 10000100
    x"88"    -- 10001000
);
```

```
begin

    process(clock)
    begin
        if rising_edge(clock) then
            if read <= '1' then
                dato <= rom(to_integer(unsigned(addr)));
            end if;
        end if;
    end process;

end Behavioral;
```

L'implementazione della memoria **MEM** è, invece, la seguente.

```
entity mem_8_bit is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           write : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (3 downto 0);
           addr : in STD_LOGIC_VECTOR (3 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end mem_8_bit;

architecture Behavioral of mem_8_bit is

-- definisco il tipo memory come un array costituito da 16 array ciascuno da 4 elementi
type memory is array (0 to 15) of std_logic_vector (3 downto 0);

-- definisco il segnale mem che è di tipo memory -> ogni cella è inizializzata a 0
signal mem : memory := (others => (others => '0'));

begin

    process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                mem <= (others => (others => '0'));
            elsif write = '1' then
                mem(to_integer(unsigned(addr))) <= dato_in;
            end if;
        end if;

    end process;

    dato_out <= mem(to_integer(unsigned(addr)));


end Behavioral;
```

Infine, l'implementazione del contatore, necessario per incrementare gli indirizzi, è in Appendice.

*Simulazione*

```
entity tb_sis_tot is
end tb_sis_tot;

architecture tb of tb_sis_tot is

    component sis_tot
        port (clock    : in std_logic;
              reset    : in std_logic;
              start    : in std_logic;
              dato_out : out std_logic_vector (3 downto 0));
    end component;

    signal clock    : std_logic;
    signal reset    : std_logic;
    signal start    : std_logic;
    signal dato_out : std_logic_vector (3 downto 0);

    constant clk_period : time := 10 ns;
    signal stop_the_clock: boolean;

begin

    dut : sis_tot
    port map (clock    => clock,
              reset    => reset,
              start    => start,
              dato_out => dato_out);

    stimuli : process
    begin
        -- ***EDIT*** Adapt initialization as needed
        reset <= '1';
        start <= '0';

        -- ***EDIT*** Add stimuli here
        wait for 100 ns;
        reset <= '0';
        start <= '1';

        wait for 10 ns;
        start <= '0';

        wait;
    end process;

    -- Clock Generation Process
    clocking: process
     begin
        while not stop_the_clock loop
            clock <= '0', '1' after clk_period / 2;
            wait for clk_period;
        end loop;
        wait for 10 ns;
     end process;
```

```
end tb;

configuration cfg_tb_sis_tot of tb_sis_tot is
    for tb
    end for;
end cfg_tb_sis_tot;
```

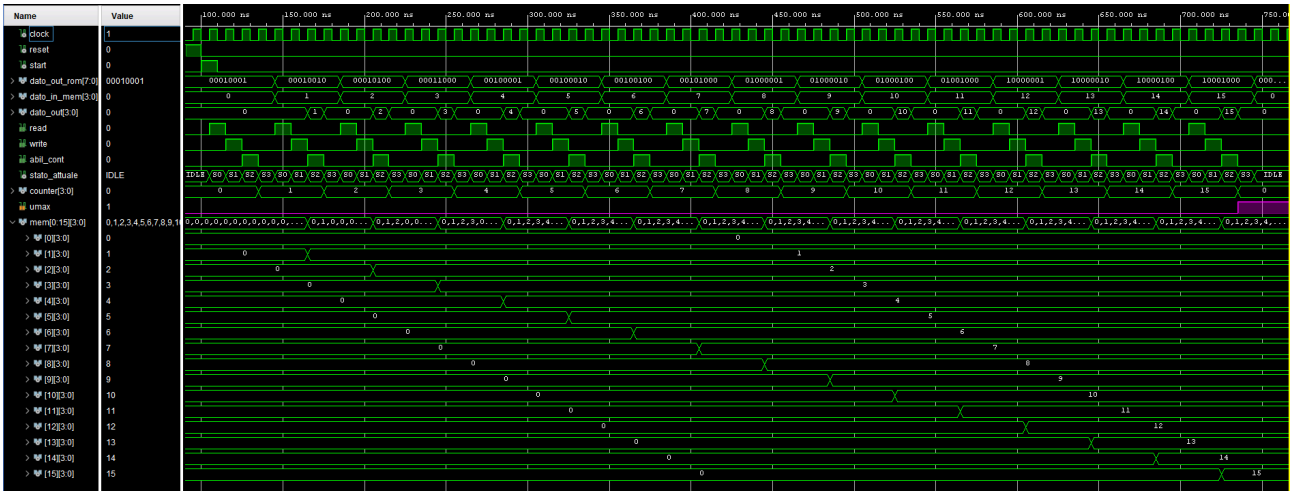



*Figura 6.4 Risultato complessivo della simulazione*

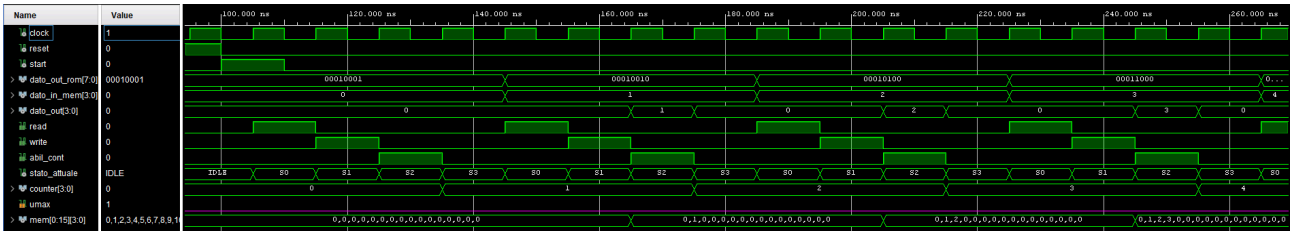



*Figura 6.5 Risultato della simulazione - Zoom*

È possibile osservare, in *Figura 6.5*, che:

- In corrispondenza del primo fronte alto del clock successivo all'arrivo del segnale di START, lo *stato_attuale* passa da IDLE a S0 e il segnale di read diventa alto. L'indirizzo (*counter* nella figura) da cui leggere in ROM e su cui scrivere in memoria è 0000 (in binario).
- Al successivo fronte di salita del clock viene effettivamente letto il contenuto della prima locazione della ROM (*dato_out_rom*), ovvero il valore binario 00010001. L'uscita dalla macchina M, in ingresso alla memoria MEM, (*data_in_mem*) è il valore deciamale 0. Contestualmente, lo *stato_attuale* passa da S0 a S1. Il segnale read torna basso e write diventa alto.
- Al successivo fronte alto del clock, avviene la scrittura in memoria, e *stato_attuale* passa da S1 a S2. Inoltre, il segnale di write torna basso e viene alzato il segnale *abil_count*.
- Al successivo fronte alto del clock, *stato_attuale* passa da S2 a S3. In S3 abbassiamo il segnale *abil_count* e valutiamo l'uscita UMAX del contatore, che in questo caso vale ancora 0.
- Dal momento che UMAX è ancora pari a 0, cioè non sono terminate le locazioni di memoria, al successivo fronte alto del clock, lo *stato_attuale* è nuovamente S0.

L'andamento è lo stesso fino a quando UMAX non diventa pari a 1. A questo punto, le operazioni di lettura e scrittura sono terminate e si torna nello stato di *idle*.

## Esercizio 6.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando due bottoni per i segnali di read e reset rispettivamente e i led per la visualizzazione delle uscite della macchina istante per istante.

### *Progetto e architettura*

Per sintetizzare e implementare su board il componente sviluppato nell'Esercizio 6.1, sono state apportate alcune modifiche al progetto. In primo luogo, è stato aggiunto un **Button Debouncer**, necessario per "ripulire" i segnali di *en_read* e di *start*, che provengono da due bottoni.

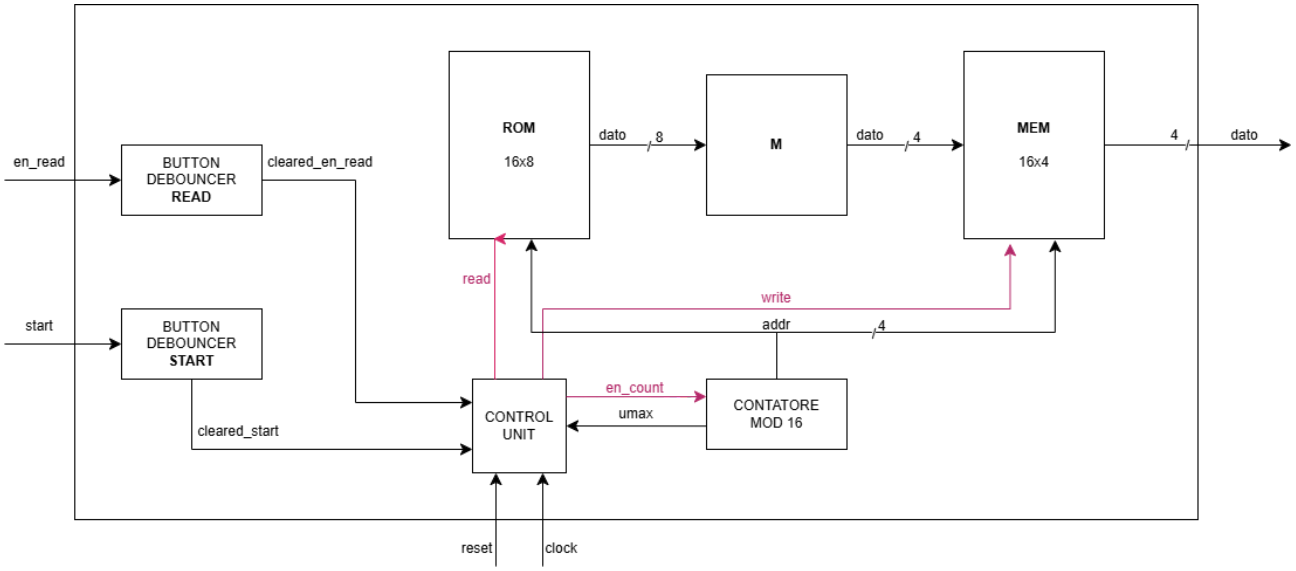



*Figura 6.6 Disegno architetturale del sistema*

Inoltre, l'**unità di controllo** è stata modificata per implementare un automa leggermente diverso. In particolare, dopo il segnale di *start*, rimaniamo nello stato S0 in attesa dell'abilitazione alla lettura (*en_read*), fornita dall'esterno tramite un bottone. Quando arriva il segnale *en_read*, allora l'unità di controllo fornisce il segnale di *read* alla ROM e avanza nello stato S1.

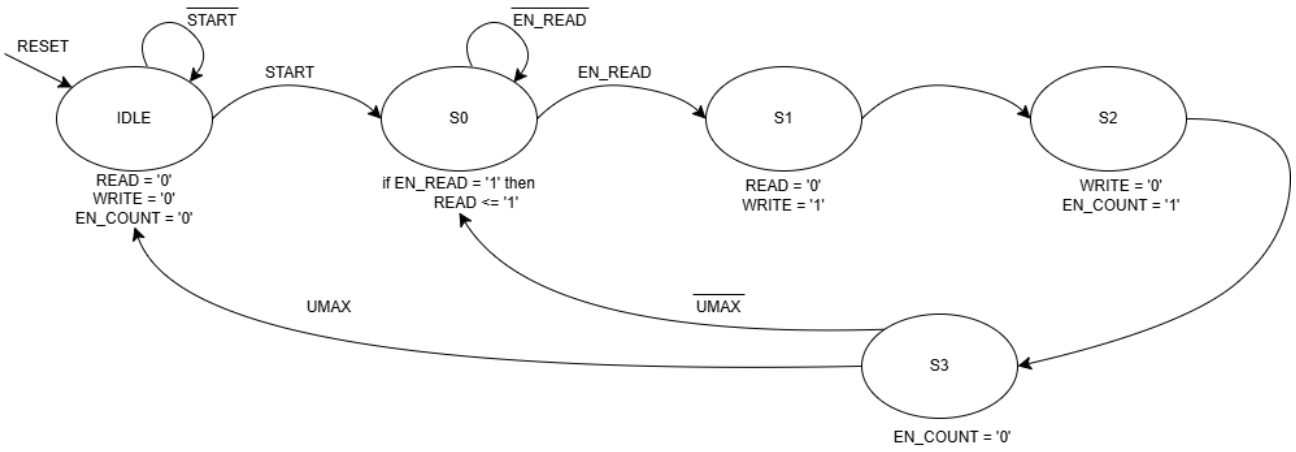



*Figura 6.7 Automa dell'unità di controllo*

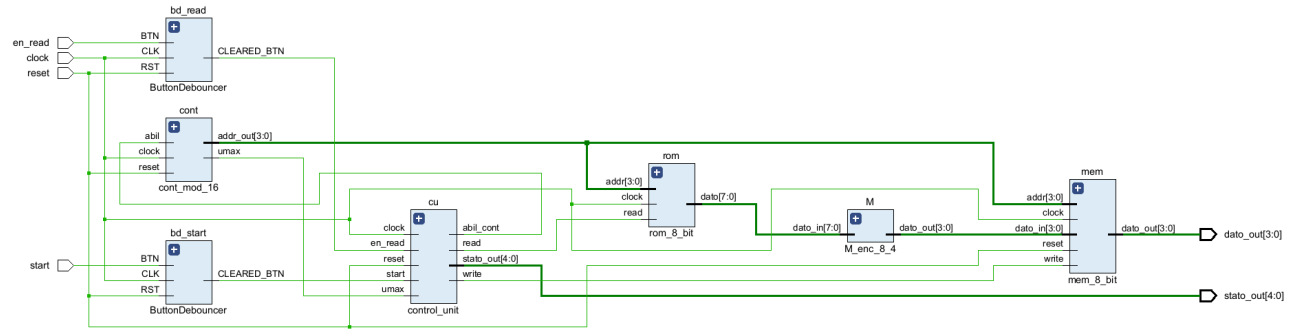



*Figura 6.8 Schematic del sistema implementato*

### *Implementazione*

L'implementazione del **top module** è la seguente. Le modifiche apportate rispetto all'Esercizio 6.1 sono evidenziate in grassetto.

```
entity sis_tot is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           start : in STD_LOGIC;
           en_read : in STD_LOGIC;
           dato_out : out STD_LOGIC_VECTOR(3 downto 0));
end sis_tot;

architecture Structural of sis_tot is

component rom_8_bit is
    Port ( clock : in STD_LOGIC;
           read : in STD_LOGIC;
           addr : in STD_LOGIC_VECTOR(3 downto 0);
           dato : out STD_LOGIC_VECTOR(7 downto 0));
end component rom_8_bit;

component mem_8_bit is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           write : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (3 downto 0);
           addr : in STD_LOGIC_VECTOR (3 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end component mem_8_bit;

component M_enc_8_4 is
    Port ( dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           dato_out : out STD_LOGIC_VECTOR (3 downto 0));
end component M_enc_8_4;

component cont_mod_16 is
    Port ( abil : in STD_LOGIC;
           clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           addr_out : out STD_LOGIC_VECTOR (3 downto 0);
           umax : out STD_LOGIC);
end component cont_mod_16;
```

```
component control_unit is
    Port ( start : in STD_LOGIC;
           reset : in STD_LOGIC;
           umax : in STD_LOGIC;
           clock : in STD_LOGIC;
           en_read : in STD_LOGIC;
           read : out STD_LOGIC;
           write : out STD_LOGIC;
           abil_cont : out STD_LOGIC);
end component control_unit;

component ButtonDebouncer is
    generic (
        CLK_period: integer := 10;  -- periodo del clock (della board) in nanosecondi
        btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del
                                               bottone in nanosecondi
                                            -- il valore di default è 10 millisecondi
    );
    Port ( RST : in STD_LOGIC;
           CLK : in STD_LOGIC;
           BTN : in STD_LOGIC;
           CLEARED_BTN : out STD_LOGIC);
end component ButtonDebouncer;


signal dato_out_rom : std_logic_vector(7 downto 0);
signal dato_in_mem : std_logic_vector(3 downto 0);
signal dato_out_mem : std_logic_vector(3 downto 0);
signal addr_tmp : std_logic_vector(3 downto 0);
signal read_tmp : std_logic := '0';
signal write_tmp : std_logic := '0';
signal abil_cont_tmp : std_logic := '0';
signal umax_tmp : std_logic := '0';
signal cleared_en_read : std_logic := '0';
signal cleared_start : std_logic := '0';

begin

    rom : rom_8_bit
    port map( clock => clock,
              read => read_tmp,
              addr => addr_tmp,
              dato => dato_out_rom
    );

    mem : mem_8_bit
    port map( clock => clock,
              reset => reset,
              write => write_tmp,
              dato_in => dato_in_mem,
              addr => addr_tmp,
              dato_out => dato_out_mem
    );

    M : M_enc_8_4
```

```
    port map( dato_in => dato_out_rom,
              dato_out => dato_in_mem
    );

    cont : cont_mod_16
    port map( abil => abil_cont_tmp,
              clock => clock,
              reset => reset,
              addr_out => addr_tmp,
              umax => umax_tmp
    );

    cu : control_unit
    port map( start => cleared_start,
              reset => reset,
              umax => umax_tmp,
              clock => clock,
              en_read => cleared_en_read,
              read => read_tmp,
              write => write_tmp,
              abil_cont => abil_cont_tmp
    );

    bd_read : ButtonDebouncer
    generic map ( CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
                  btn_noise_time => 10000000 -- durata stimata dell'oscillazione del
                                             bottone in nanosecondi
                                           -- il valore di default è 10 millisecondi
                )
    port map ( RST => reset,
               CLK => clock,
               BTN => en_read,
               CLEARED_BTN => cleared_en_read);

    bd_start : ButtonDebouncer
    generic map ( CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
                  btn_noise_time => 10000000 -- durata stimata dell'oscillazione del
                                             bottone in nanosecondi
                                           -- il valore di default è 10 millisecondi
                )
    port map ( RST => reset,
               CLK => clock,
               BTN => start,
               CLEARED_BTN => cleared_start);

    dato_out <= dato_out_mem;

end Structural;
```

L'**unità di controllo** è la seguente. L'unica modifica effettuata è evidenziata in grassetto.

```
entity control_unit is
    Port ( start : in STD_LOGIC;
           reset : in STD_LOGIC;
```

```
            umax : in STD_LOGIC;
            clock : in STD_LOGIC;
            en_read : in STD_LOGIC;
            read : out STD_LOGIC;
            write : out STD_LOGIC;
            abil_cont : out STD_LOGIC);
end control_unit;

architecture Behavioral of control_unit is

type stato is (IDLE, S0, S1, S2, S3); -- S1 -> read, S2 -> write, S3 -> count
signal stato_attuale: stato := IDLE;
signal stato_prossimo: stato := IDLE;


begin

    control : process(stato_attuale, start, en_read, umax)
    begin

        case stato_attuale is
            when IDLE =>
                read <= '0';
                write <= '0';
                abil_cont <= '0';

                if start = '1' then
                    stato_prossimo <= S0;
                else
                    stato_prossimo <= IDLE;
                end if;

            when S0 =>
                if en_read = '1' then
                    read <= '1';
                    stato_prossimo <= S1;
                else
                    stato_prossimo <= S0;
                end if;

            when S1 =>
                read <= '0';
                write <= '1';
                stato_prossimo <= S2;

            when S2 =>
                write <= '0';
                abil_cont <= '1';
                stato_prossimo <= S3;

            when S3 =>
                if umax = '1' then
                    stato_prossimo <= IDLE;
                else
                    stato_prossimo <= S0;
                end if;
```

```
        end case;

    end process;


    cambio_stato : process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                stato_prossimo <= IDLE;
            else
                stato_attuale <= stato_prossimo;
            end if;

        end if;
    end process;

end Behavioral;
```

L'implementazione del **Button Debouncer** è in Appendice.

### *Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del **file di constraint** a cui sono state apportate delle modifiche, cioè quelle relative a clock, bottoni e led.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { dato_out[0]
}]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { dato_out[1]
}]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { dato_out[2]
}]; #IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { dato_out[3]
}]; #IO_L8P_T1_D11_14 Sch=led[3]
#set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
#IO_L7P_T1_D09_14 Sch=led[4]
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
#IO_L18N_T2_A11_D27_14 Sch=led[5]
#set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
#IO_L17P_T2_A14_D30_14 Sch=led[6]
#set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
#IO_L18P_T2_A12_D28_14 Sch=led[7]
#set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
#set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { LED[9] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
#set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
```

```
#set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
#set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { LED[12] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
#set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { LED[13] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
#set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { LED[14]}];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
#set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { LED[15] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15


##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { start }];
#IO_L9P_T1_DQS_14 Sch=btnc
#set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }];
#IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { en_read }];
#IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

# Capitolo 3: Macchine Aritmetiche

## Esercizio 7: Moltiplicatore di Booth

### Esercizio 7.1

Progettare, implementare in VHDL e simulare una macchina moltiplicatore di Booth in grado di effettuare il prodotto di 2 stringhe A e B da 8 bit ciascuna.

#### *Progetto e architettura*

Per realizzare il moltiplicatore di Booth, il progetto è stato suddiviso in un'unità operativa e un'unità di controllo. L'unità di controllo implementa l'automa che gestisce l'esecuzione dell'algoritmo di Booth, mentre l'unità operativa contiene tutti i componenti necessari a realizzare l'architettura dell'algoritmo.

L'architettura del moltiplicatore di Booth che si è inteso realizzare è rappresentata in *Figura 7.1*.

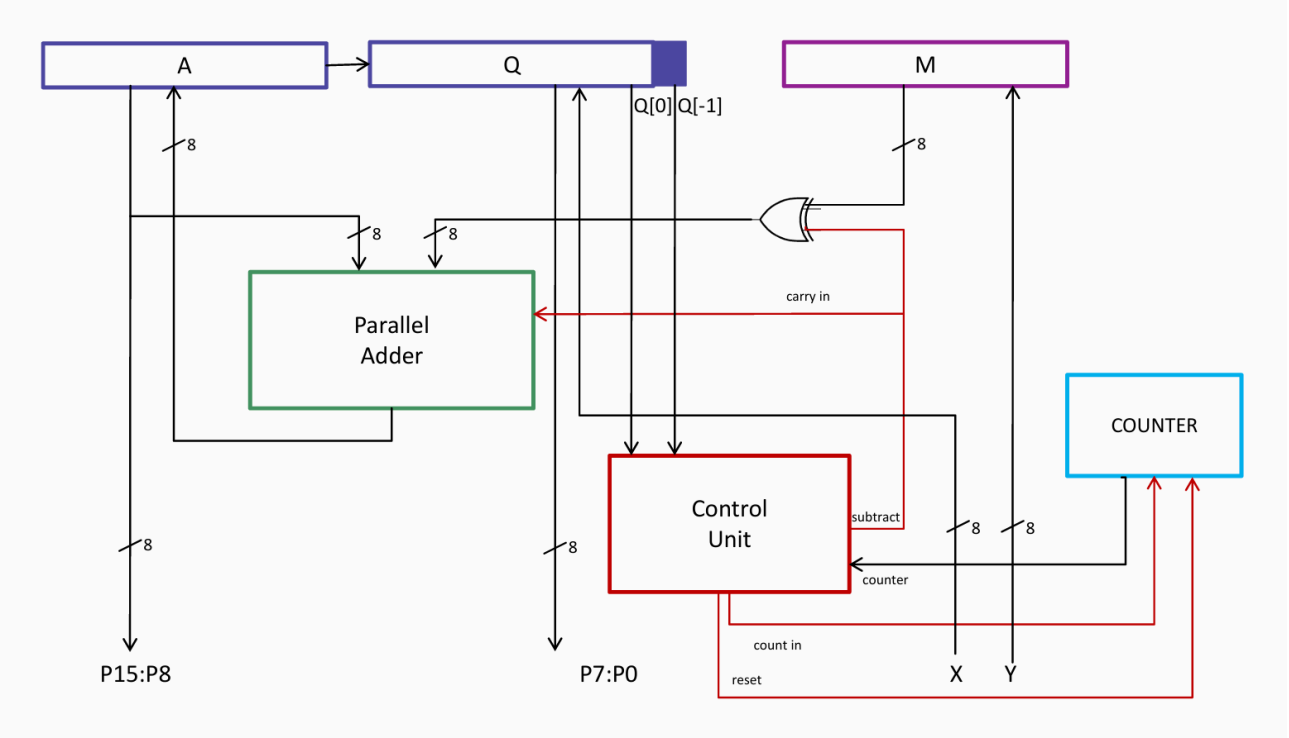



*Figura 7.1 Architettura del Moltiplicatore di Booth*

I componenti necessari per realizzare questa architettura sono:

- un adder/subtractor, cioè una macchina in grado di effettuare sia somme che sottrazioni;
- un registro da 8 bit per contenere il moltiplicando;
- uno shift register che contiene A e Q concatenati (quindi con 17 bit in tutto, considerando anche il bit Q[-1]);
- un mux che permette di selezionare l'ingresso parallelo dello shift register A.Q. per inizializzare il registro con il valore di partenza o per aggiornare il registro con il risultato della somma/sottrazione dopo ogni ciclo del moltiplicatore di Booth;
- un contatore che tiene traccia del numero di iterazioni, permettendo alla macchina di fermarsi quando viene raggiunto il numero di cicli necessari per completare l'operazione;

- la control unit, che prende in ingresso i due bit meno significativi di Q (Q[0]Q[-1]) e, in base al loro valore, determina quale operazione deve essere eseguita. Se Q[0]Q[-1]= 11 o 00 non viene effettuata né la somma né la sottrazione, ma solo lo shift. Se Q[0]Q[-1]=01 il moltiplicando Y viene aggiunto al prodotto parziale corrente. Se Q[0]Q[-1]=10 il moltiplicando Y viene sottratto al prodotto parziale corrente. Per eseguire la sottrazione, la control unit fornisce un segnale di subtract, che viene inviato sia all'adder/subtractor per abilitare l'operazione di sottrazione, sia a una porta XOR con il moltiplicando per invertire il segno durante la sottrazione. Inoltre, la control unit gestisce l'abilitazione del contatore ad ogni iterazione e si occupa di monitorare l'uscita del contatore per determinare quando l'operazione è terminata, interrompendo il processo al termine delle iterazioni previste. L'automa implementato dall'unità di controllo è mostrato in *Figura 7.4*.

La divisione tra unità operativa e unità di controllo è mostrata in *Figura 7.2*, mentre in *Figura 7.3* è possibile vedere nel dettaglio tutti i componenti dell'unità operativa.

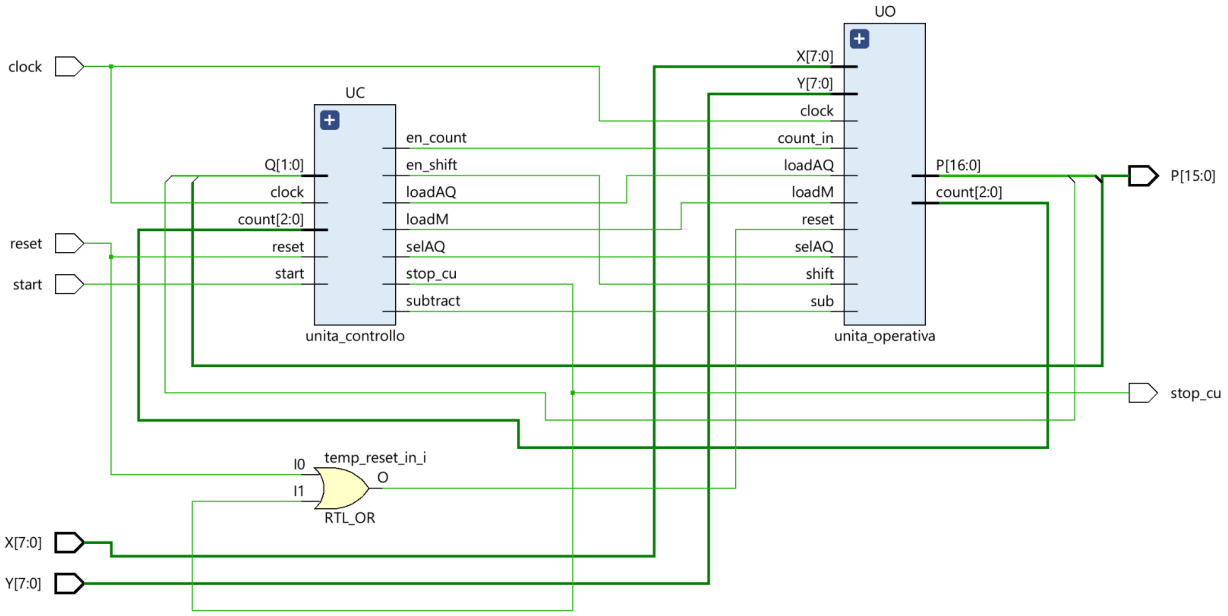



*Figura 7.2 Schematic del sistema implementato*

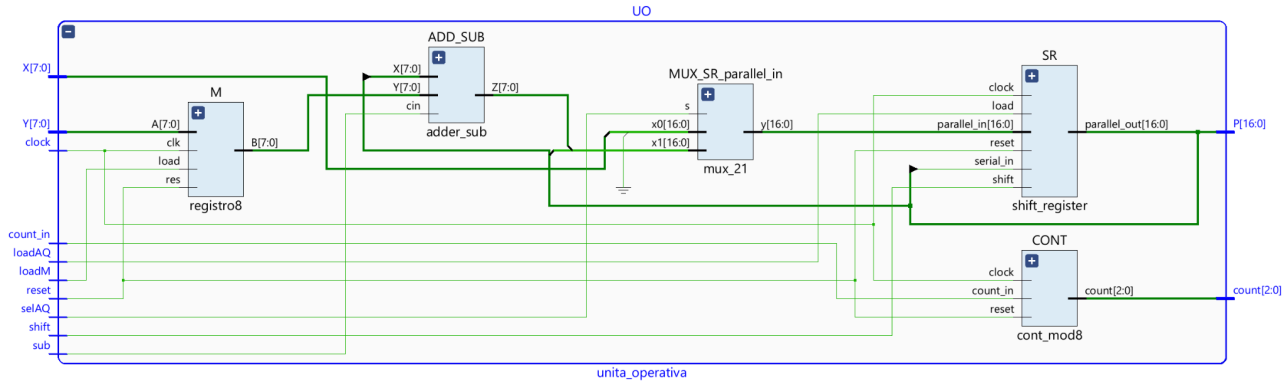



*Figura 7.3 Schematic del sistema implementato: unità operativa*

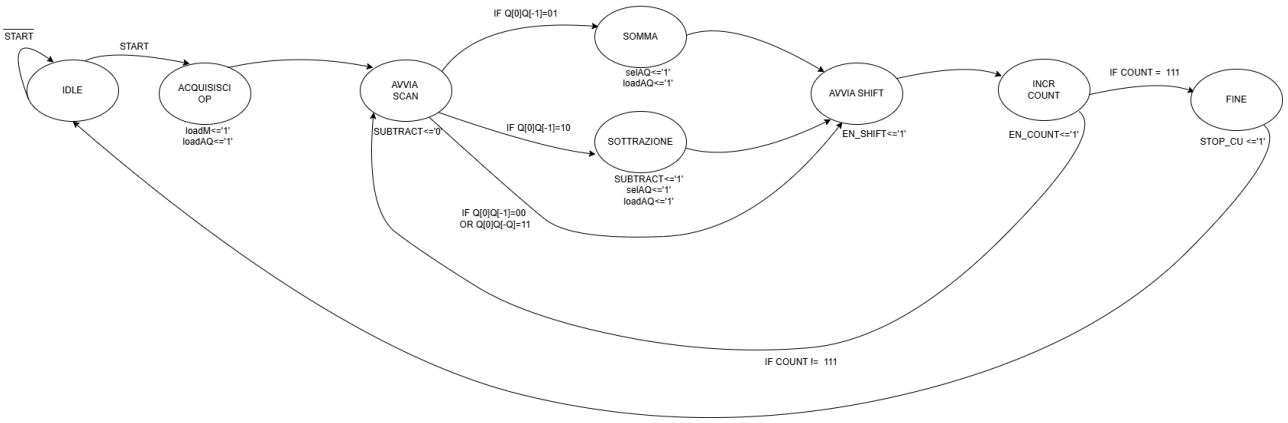



*Figura 7.4 Automa della control unit*

Nota: quando si esegue l'operazione -128 x -128 il risultato è errato (il segno è negativo invece che positivo), perché nell'ultima iterazione la sottrazione ha come risultato un numero non rappresentabile su 8 bit. L'ultimo shift copia 1 nel bit più significativo di AQ, e invece ci vorrebbe uno 0. Il risultato errato che si ottiene è mostrato in *Figura 7.9*.

### *Implementazione*

L'implementazione del top module, composto da unità operativa e unità di controllo, è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity molt_booth is
        port( clock, reset, start: in std_logic;
              X, Y: in std_logic_vector(7 downto 0);
              P: out std_logic_vector(15 downto 0);
              stop_cu: out std_logic);
end molt_booth;

architecture structural of molt_booth is
       component unita_controllo is
              port( Q: in std_logic_vector(1 downto 0);
                    clock, reset, start: in std_logic;
                    count: in std_logic_vector(2 downto 0);
                    loadM, en_count, loadAQ, en_shift: out std_logic;
                    selAQ, subtract, stop_cu: out std_logic);
       end component;

       component unita_operativa is
       port( X, Y: in std_logic_vector(7 downto 0);--moltiplicatore e moltiplicando
             clock, reset: in std_logic;
             loadAQ, shift, loadM, sub, selAQ,  count_in: in std_logic;
             count: out std_logic_vector(2 downto 0);
             P: out std_logic_vector(16 downto 0));
       end component;


       signal temp_selAQ, temp_clock, temp_sub,temp_loadAQ: std_logic;
```

```
	signal temp_Q: std_logic_vector(1 downto 0); -- booth non ha bisogno solo
                                                  dell'ultimo bit ma degli ultimi due
	signal temp_count: std_logic_vector(2 downto 0);
	signal temp_p: std_logic_vector(16 downto 0);
	signal temp_count_in, t_load_add: std_logic;
	signal fine_conteggio: std_logic;
	signal temp_shift, temp_fshift: std_logic;
	signal temp_loadM: std_logic;
	signal temp_stop_cu: std_logic; -- segnale di reset generato dalla UC
	signal temp_reset_in: std_logic; -- segnale di reset in ingresso alla UO
	signal temp_selF: std_logic;

	begin

	UC: unita_controllo port map
	(temp_Q, clock, reset, start,
	temp_count,
	temp_loadM, temp_count_in, temp_loadAQ, temp_shift,
	temp_selAQ,  temp_sub, temp_stop_cu);



	UO: unita_operativa port map
	(X, Y, clock, temp_reset_in, temp_loadAQ, temp_shift, temp_loadM,
	temp_sub, temp_selAQ,  temp_count_in, temp_count, temp_p);




	temp_Q(0)<=temp_p(0); -- invio all'unità di controllo i due bit meno
                          significativi del registro A.Q
	temp_Q(1)<=temp_p(1);
	P<=temp_p(16 downto 1);

	-- la UO viene resettata sia se arriva un reset dall'esterno sia se l'operazione
	di moltiplicazione termina
	temp_reset_in <= reset or temp_stop_cu;

	stop_cu <= temp_stop_cu;
	end structural;
```

L'unità di controllo, che implementa l'automa illustrato in *Figura 7.4*, è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity unita_controllo is
	port( Q: in std_logic_vector(1 downto 0);
	      clock, reset, start: in std_logic;--clock è il clock della board, clock_div
viene dal divisore di freq
		count: in std_logic_vector(2 downto 0);
		loadM, en_count, loadAQ, en_shift: out std_logic;
		selAQ, subtract, stop_cu: out std_logic);
end unita_controllo;
```

```
architecture structural of unita_controllo is
	type state is (idle, acquisisci_op, aspetta_caricamento, avvia_scan, somma,
sottrazione,solo_shift, incr_count,  fine);
	signal current_state,next_state: state;

	begin


	reg_stato: process(clock)
			begin
			if(clock'event and clock='1') then
			 if(reset='1') then
				current_state <=idle;
			  else
				current_state <=next_state;
			   end if;
			end if;
			end process;

	comb: process(current_state, start, count)
		begin

		-- Attenzione! questo process si attiva ogni volta che c'è una
		--  variazione nei segnali della sensitivity list
		-- current_state e count per loro natura variano sempre in
		-- corrispondenza del fronte di salita del clock
		-- start viene dall'esterno: se non varia (sale e scende) col fronte del
		-- clock, si potrebbe avere una situazione
		-- in cui il next_state varia ma non ha modo da stabilizzarsi
		-- (perchè current_state non è ancora variato)
		-- quando il moltiplicatore sarà messo su board, START dovrà essere
		-- generato come uscita del button debouncer

	   en_count <='0';
	  subtract <='0';
	  selAQ <= '0';
	  loadAQ <='0';  --carica nello shift register
	  loadM <='0';   --carica il moltiplicando nel registro M
	  stop_cu <='0';
	  en_shift <='0'; --segnale che abilita lo shift durante le prime N-1 iterazioni


	    CASE current_state is

		WHEN idle =>


		   if(start='1') then
				next_state <= acquisisci_op;
			else
				next_state <= idle;
			  end if;

		--fornisce i segnali di caricamento operandi
```

```
      WHEN acquisisci_op =>


                loadM <='1'; --abilita il caricamento del moltiplicando
                             -- nel registro M

                 loadAQ <='1'; --abilita il caricamento del moltiplicatore
                             --e degli 8 zeri in testa
                             -- nello shift register A.Q (perchè selAQ=0)

                 next_state <= aspetta_caricamento;

     WHEN aspetta_caricamento => --per aspettare che il caricamento nei
                                 -- registri sia completato
                       next_state <= avvia_scan;

    -- acquisisce gli operandi, su cui il sommatore inizia a lavorare
       immediatamente
     WHEN avvia_scan =>
           subtract<='0';

           if (Q = "11") or (Q = "00") then
               next_state <= avvia_shift;
           elsif Q = "10" then
               next_state <= sottrazione;
           elsif Q = "01" then
               next_state <= somma;
           end if;

when somma =>
           selAQ<='1';
           loadAQ<='1';
           next_state<=avvia_shift;

when sottrazione =>
           selAQ<='1';
           loadAQ<='1';
           subtract<='1';
           next_state<=avvia_shift;



     --carica il risultato della somma in A e da fornisce il segnale di shift
     WHEN avvia_shift =>

                       en_shift <='1';

                       next_state <= incr_count;

    -- esegue lo shift e abilita il contatore. controlla il valore corrente di
    -- count (quindi non ancora incrementato), se è minore di 7 predispone
    -- per nuova iterazione, altrimenti termina

     WHEN incr_count =>
               en_count <= '1';
                if count="111" then
```

```
                              next_state<=fine;
                        else
                               next_state<=avvia_scan;
                                 end if;

              WHEN fine =>

                       stop_cu <='1';

                       next_state <= idle;
             WHEN others =>
                    next_state <= idle;
             end CASE;

    end process;
 end structural;
```

L'unità operativa è implementata come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity unita_operativa is
       port(    X, Y: in std_logic_vector(7 downto 0);--moltiplicatore e moltiplicando
                clock, reset: in std_logic;
                loadAQ, shift, loadM, sub, selAQ, count_in: in std_logic;
                count: out std_logic_vector(2 downto 0);
                P: out std_logic_vector(16 downto 0));
end unita_operativa;

architecture structural of unita_operativa is

       component adder_sub is
       port(   X, Y: in std_logic_vector(7 downto 0);
                cin: in std_logic;
                Z: out std_logic_vector(7 downto 0);
                cout: out std_logic);
       end component;

       component registro8 is
              port( A: in std_logic_vector(7 downto 0);
                   clk, res, load: in std_logic;
                   B: out std_logic_vector(7 downto 0));
       end component;


       component mux_21 is
       generic (width : integer range 0 to 32 := 17);
       port(   x0, x1: in std_logic_vector(width-1 downto 0);
                s: in std_logic;
                y: out std_logic_vector(width-1 downto 0));
       end component;


       component shift_register is
```

```
      port(   parallel_in: in std_logic_vector(16 downto 0);
              serial_in: in std_logic;
              clock, reset, load, shift: in std_logic;
              parallel_out: out std_logic_vector(16 downto 0));
      end component;


      component cont_mod8 is
      port( clock,  reset: in std_logic;
              count_in: in std_logic;
              count: out std_logic_vector(2 downto 0));
      end component;


      signal Mreg: std_logic_vector(7 downto 0); -- segnale temporaneao tra reg8 M e
                                                    mux 21
      signal op2: std_logic_vector(7 downto 0); -- segnale temporaneo di uscita dal
                                                    multiplexer tra mux e adder
      signal AQ_init: std_logic_vector(16 downto 0); --segnale in input all'SR
      signal AQ_in: std_logic_vector(16 downto 0); --segnale in input all'SR
      signal AQ_out: std_logic_vector(16 downto 0); --segnale temporaneo uscita dell'SR
      signal sum: std_logic_vector(7 downto 0); --uscita del parallel adder
      signal AQ_sum_in : std_logic_vector(16 downto 0);
      signal riporto: std_logic; -- riporto in uscita dell'adder che non utilizziamo
      signal  SRserialIn: std_logic;

begin

      -- 1) predisposizione del secondo operando della somma:

      -- registro moltiplicando
      M: registro8 port map(Y, clock, reset, loadM, Mreg);



    -- 2) predisposizione del primo operando della somma:

    --stringa da 17 bit da inserire nello shift register A.Q durante la fase di
      inizializzazione:
    --è ottenuta concatenando 00000000 con il moltiplicatore X e Q[-1] che vale 0.
    AQ_init <= "00000000" & X & "0";  -- valore da inserire all'inizio nello shift
                                         register

    --stringa di 17 bit da inserire nello shift register A.Q durante la fase operativa
      dopo aver effettuato la somma
    AQ_sum_in <=sum & AQ_out(8 downto 0);

    -- mux per selezionare l'ingresso parallelo dello shift register: valore iniziale
      AQ_init
    -- oppure uscita dell'adder AQ_sum_in
    MUX_SR_parallel_in : mux_21
    generic map (width => 17)
    port map(AQ_init, AQ_sum_in, selAQ, AQ_in);

        -- 3) predisposizione dell'ingresso seriale dello shift register: sarebbe
              A[7]:=A[7]
```

```
            SRserialIn <= AQ_out(16);

      -- 4) shift register A.Q
      SR: shift_register
      port map(AQ_in, SRserialIn, clock, reset, loadAQ, shift, AQ_out);


      -- 5) sommatore
      ADD_SUB: adder_sub port map(AQ_out(16 downto 9), Mreg, sub, sum, riporto);

      -- 6) contatore
      CONT: cont_mod8 port map(clock, reset, count_in, count);

      --7) uscita del moltiplicatore, corrispondente al valore contenuto nello shift
      register

      P<=AQ_out;



end structural;
```

L'implementazione del registro da 8 bit che contiene il moltiplicando è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
--registro parallelo-parallelo che mantiene il valore del moltiplicando Y
entity registro8 is
      port( A: in std_logic_vector(7 downto 0);
              clk, res, load: in std_logic;
              B: out std_logic_vector(7 downto 0));
end registro8;

architecture behavioural of registro8 is
      signal temp_b: std_logic_vector(7 downto 0);
      begin

      R_PP: process(clk)
            begin
            if(clk'event and clk='1') then
              if(res='1') then
                    temp_b<= (others=>'0');
              else
                if(load='1') then
                      temp_b<=A;
              end if;
            end if;
          end if;
      end process;
      B<=temp_b;
end behavioural;
```

Il mux 2:1 è implementato in Appendice.

Lo shift register è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

---shift register che contiene inizialmente una stringa di 8 zeri e il moltiplicatore X
---al termine dell'operazione di moltiplicazione conterrà il risultato A+Q

entity shift_register is
       port( parallel_in: in std_logic_vector(16 downto 0);
                serial_in: in std_logic;
                clock, reset, load, shift: in std_logic;
                parallel_out: out std_logic_vector(16 downto 0));
end shift_register;

architecture behavioural of shift_register is

       signal temp: std_logic_vector(16 downto 0);

       begin

       SR: process(clock)
              begin
                 if(clock'event and clock='1') then
                      if(reset='1') then
                        temp <=(others=>'0');
                      else
                         if(load='1') then --caricamento iniziale del moltiplicatore
                           temp <= parallel_in;
                      elsif(shift='1') then
                            temp(15 downto 0) <= temp(16 downto 1);
                            temp(16) <= serial_in;
                      end if;
                end if;

                 end if;
              end process;

       parallel_out <= temp;
       end behavioural;
```

L'implementazione dell'adder/subtractor è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder_sub is
       port( X, Y: in std_logic_vector(7 downto 0);
                cin: in std_logic;
                Z: out std_logic_vector(7 downto 0);
                cout: out std_logic);
end adder_sub;

architecture structural of adder_sub is
       component ripple_carry is
       port( X, Y: in std_logic_vector(7 downto 0);
                c_in: in std_logic;
```

```
                c_out: out std_logic;
                Z: out std_logic_vector(7 downto 0));
        end component;

        signal complementoy: std_logic_vector(7 downto 0);

        begin

        complemento_y: FOR i IN 0 TO 7 GENERATE
                              complementoy(i)<=Y(i) xor cin;
                              END GENERATE;

        RA: ripple_carry port map(X, complementoy, cin, cout, Z);

        end structural;
```

Dove l'implementazione del ripple carry adder è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ripple_carry is
        port( X, Y: in std_logic_vector(7 downto 0);
                c_in: in std_logic;
                c_out: out std_logic;
                Z: out std_logic_vector(7 downto 0));
end ripple_carry;

architecture structural of ripple_carry is
        component full_adder is
        port(
        a,b: in std_logic;
        cin: in std_logic;
        cout, s: out std_logic);
        end component;

        signal temp: std_logic_vector(7 downto 0);

        begin

        RA0: full_adder port map(X(0), Y(0), c_in, temp(0), Z(0));

        RA1to6: FOR i IN 1 TO 6 GENERATE
                      RA: full_adder port map(X(i), Y(i), temp(i-1), temp(i), Z(i));
                      END GENERATE;

        RA7: full_adder port map(X(7), Y(7), temp(6), c_out, Z(7));

        end structural;
```

I full adder sono implementati come in Appendice.

Il contatore modulo 8 è implementato come segue.

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
--use IEEE.std_logic_unsigned.ALL;

entity cont_mod8 is
       port( clock, reset: in std_logic;
             count_in: in std_logic;
                count: out std_logic_vector(2 downto 0));
end cont_mod8;

architecture behavioural of cont_mod8 is
       signal c: std_logic_vector(2 downto 0) := (others=>'0');

       begin
       CM8: process(clock)
       begin
              if(clock'event and clock='1') then

                 if(reset='1') then
                        c <= (others=>'0');
                 else
                     if(count_in = '1') then
                           c <= std_logic_vector(unsigned(c) + 1);
                           -- c <= c + "111"; posso fare direttamente questa somma se
                               importo IEEE.std_logic_unisigned.ALL
                           -- è preferibile non farlo perchè sono package non standard
                     end if;
                 end if;
               end if;
       end process;

       count <= c;

       end behavioural;
```

*Simulazione*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.ALL;

entity mrob_tb is
end mrob_tb;

architecture behavioural of mrob_tb is

       component molt_booth is
         port( clock, reset, start: in std_logic;
                X, Y: in std_logic_vector(7 downto 0);
                --stop: out std_logic;
                P: out std_logic_vector(15 downto 0);
                stop_cu: out std_logic);
       end component;
       signal inputx, inputy: std_logic_vector(7 downto 0);
       signal prod: std_logic_vector(15 downto 0);
```

```
    signal clk, res, start: std_logic;
    signal t_stop_cu: std_logic;
  constant clk_period : time := 20 ns;

  signal end_sim : std_logic := '0';


    begin

    uut: molt_booth port map(clk, res, start, inputx, inputy, prod, t_stop_cu);

    clk_process : process
   begin
      while (end_sim = '0') loop
          clk<= '1';
          wait for clk_period/2;
          clk <= '0';
          wait for clk_period/2;
      end loop;
      wait;
   end process;


    sim: process
           begin

           wait for 50 ns;

           res<='1';
           wait for 20 ns;
           res<='0';

--         -- ------------------------------------   operazione numero 1:
           -- 15*3=45
           inputx<="00001111";
           inputy<="00000011";


           wait for 40 ns;
           start<='1';
           wait for 20 ns;
           start<='0';
           -- aspetto fine operazione, ci vogliono circa 700ns
           wait for 700ns;

           res<='1';
           wait for 20 ns;
           res<='0';
           wait for 20 ns;

           -- ------------------------------------------operazione numero 2:
           -- -5*3=-15
           inputy<="11111011";  -- -5
           inputx<="00000011";  -- +3

           wait for 40 ns;
```

```
-- start
start<='1';
wait for 20 ns;
start<='0';
-- aspetto fine operazione
--wait for 700 ns;
wait until t_stop_cu='1'; --alternativa a wait for 700 ns


res<='1';
wait for 20 ns;
res<='0';
wait for 20 ns;

-- -----------------------------------------operazione numero 3:
-- -3*3=-9
inputx<="11111101"; -- -3
inputy<="00000011"; -- +3

wait for 40 ns;
-- start
start<='1';
wait for 20 ns;
start<='0';
-- aspetto fine operazione
wait for 700 ns;


res<='1';
wait for 20 ns;
res<='0';
wait for 20 ns;

-- -----------------------------------------operazione numero 4:
-- -3*-5= 15
inputx<="11111101";
inputy<="11111011";

wait for 40 ns;
-- start
start<='1';
wait for 20 ns;
start<='0';
wait for 700 ns;


res<='1';
wait for 20 ns;
res<='0';
wait for 20 ns;

-- -----------------------------------------operazione numero 5:
-- -60*0=0
inputx<="11000100";
inputy<="00000000";
```

```
        wait for 40 ns;
        -- start
        start<='1';
        wait for 20 ns;
        start<='0';
        wait for 700 ns;
        -- ----------------------------------------operazione numero 6:
       -- -128*-128 -> IL RISULTATO E' ERRATO
        inputx<="10000000";
        inputy<="10000000";

        wait for 40 ns;
        -- start
        start<='1';
        wait for 20 ns;
        start<='0';
        wait for 100000 ns;




        end_sim <= '1';
        wait;
        end process;


end behavioural;
```

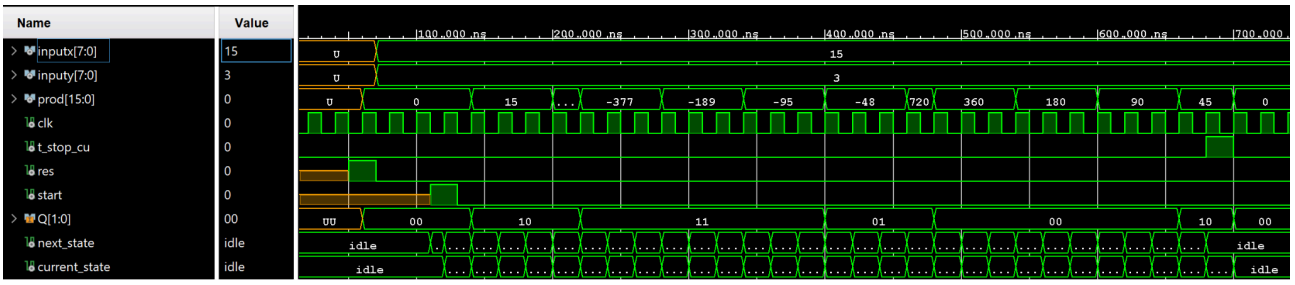



*Figura 7.4 Operazione 1: 15 x 3*

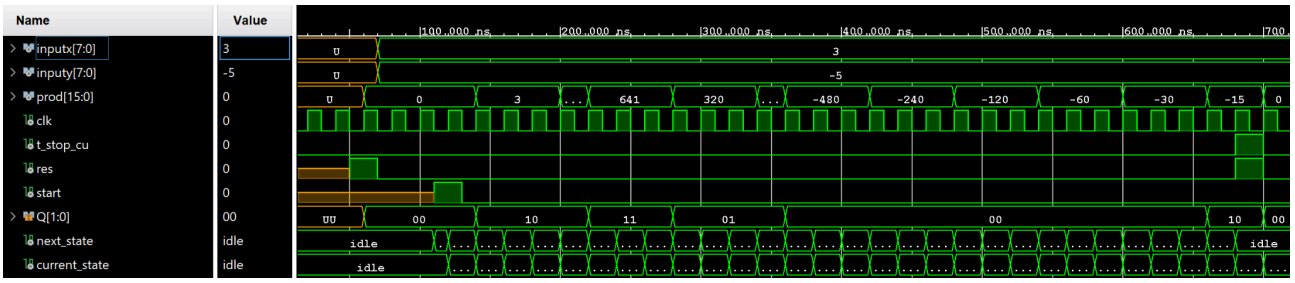



*Figura 7.5 Operazione 2: 3 x -5*

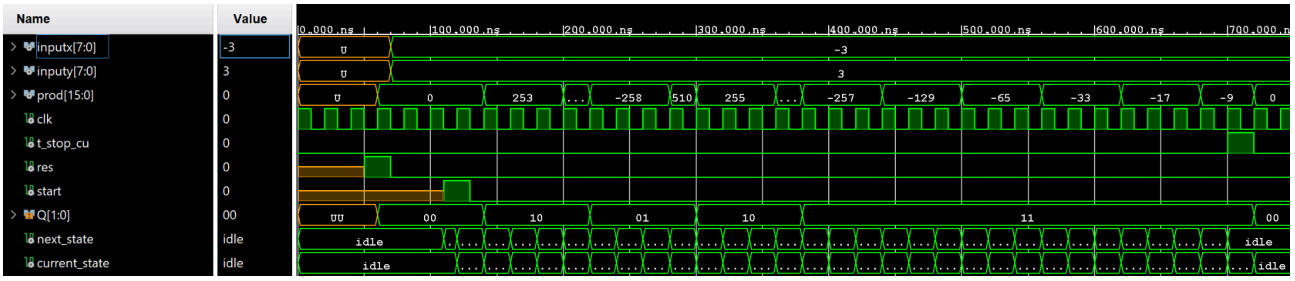



*Figura 7.6 Operazione 3: -3 x 3*

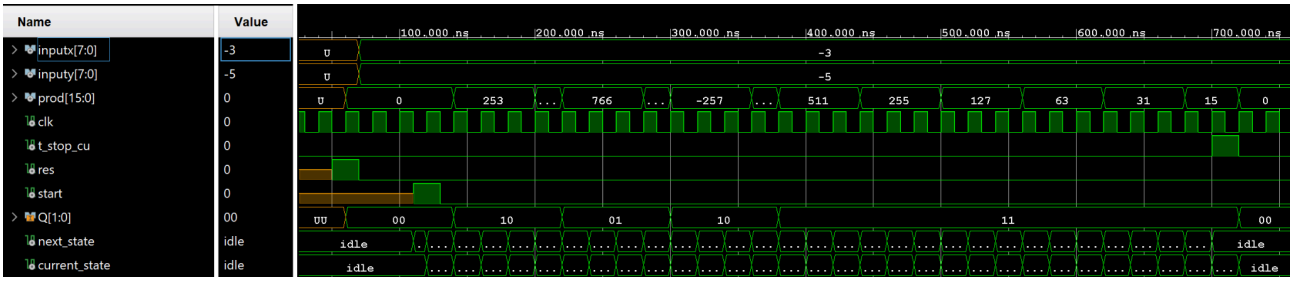



*Figura 7.7 Operazione 4: -3 x -5*

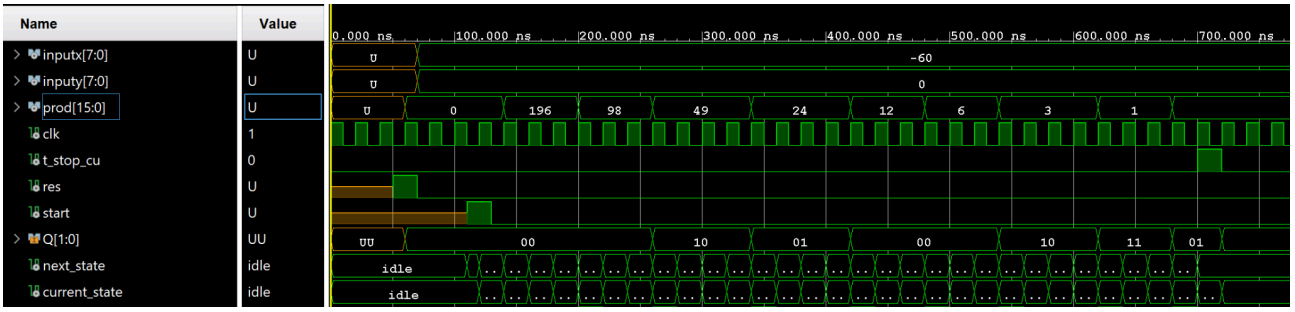



*Figura 7.8 Operazione 5: -60 x 0*

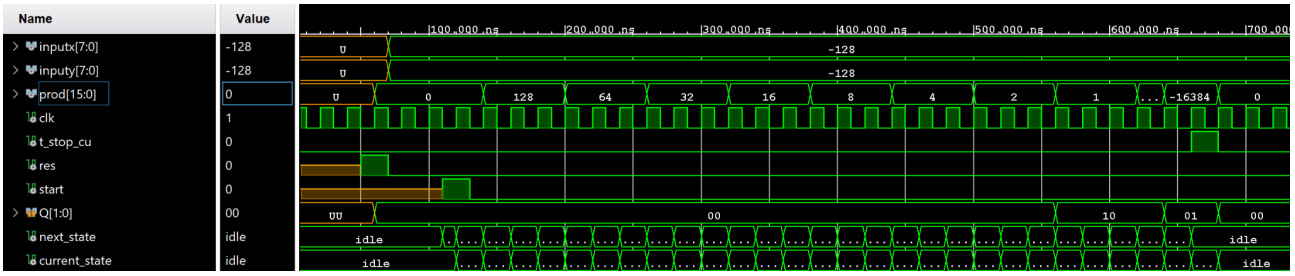



*Figura 7.9 Operazione 6: -128 x -128 - RISULTATO ERRATO*

## Esercizio 7.2

Sintetizzare il moltiplicatore implementato al punto 7.1 su FPGA e testarlo mediante l'utilizzo dei dispositivi di input/output (switch, bottoni, led, display) presenti sulla board di sviluppo in dotazione. La modalità di utilizzo degli stessi è a completa discrezione degli studenti.

### *Progetto e architettura*

Per sintetizzare il moltiplicatore implementato al punto 7.1 su FPGA, sono stati utilizzati gli 8 bit meno significativi per il moltiplicando, e gli 8 bit più significativi per il moltiplicatore. Una volta impostati tutti gli switch, viene premuto il bottone di start che avvia la moltiplicazione.

Poiché il processo dell'unità di controllo si attiva ogni volta che c'è una variazione nei segnali presenti nella sensitivity list (current_state, count e start), è importante considerare che current_state e count variano sempre in corrispondenza del fronte di salita del clock, mentre start proviene da un segnale esterno. Se start non è sincronizzato con il clock (cioè non varia in corrispondenza del fronte di salita), si potrebbe verificare una situazione in cui next_state cambia in modo non stabile, perché current_state non è stato ancora aggiornato. Per evitare questo problema, è necessario che il segnale START venga generato come uscita di un button debouncer, in modo da essere stabilizzato e sincronizzato con il clock del sistema.

Per mostrare il risultato della moltiplicazione sono stati utilizzati i 16 led disponibili sulla board.

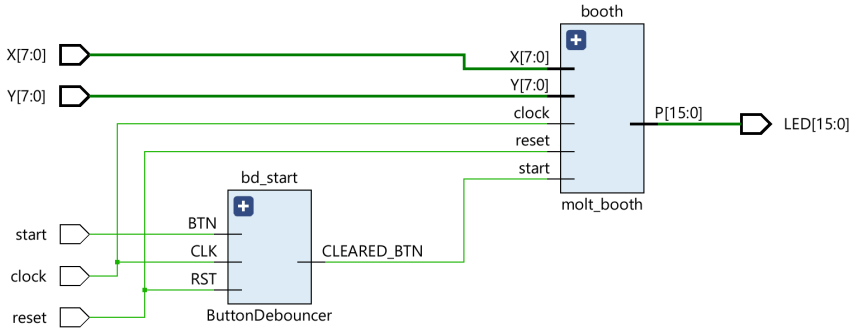



*Figura 7.10 Schematic del sistema realizzato*

Per rendere visibile il risultato sui LED, è sufficiente modificare il componente molt_booth in modo che l'unità operativa venga resettata solo in presenza di un segnale di reset esterno, e non anche al termine dell'operazione di moltiplicazione. Pertanto la riga `temp_reset_in <= reset or temp_stop_cu;` è stata modificata in `temp_reset_in <= reset;`

### *Implementazione*

L'implementazione del top module è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity booth_on_board is
```

```
  Port (
        X, Y: in std_logic_vector(7 downto 0);  -- switch
        clock: in std_logic;
        reset: in std_logic; -- bottone M18
        start: in std_logic; -- bottone P17
        LED: out std_logic_vector(15 downto 0)

    );
end booth_on_board;

architecture Structural of booth_on_board is


    signal P_temp: std_logic_vector(15 downto 0);
    signal stop_temp: std_logic;
    signal start_cleared: std_logic;


    component molt_booth is
        port( clock, reset,start: in std_logic;
                X, Y: in std_logic_vector(7 downto 0);
                P: out std_logic_vector(15 downto 0);
                stop_cu: out std_logic);
    end component;


   COMPONENT ButtonDebouncer is
    generic (
        CLK_period: integer := 10;  -- periodo del clock (della board) in nanosecondi
        btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del
bottone in nanosecondi. il valore di default e' 10 millisecondi
    );
    Port ( RST : in STD_LOGIC;
           CLK : in STD_LOGIC;
           BTN : in STD_LOGIC;
           CLEARED_BTN : out STD_LOGIC);
    end component;


begin


    bd_start: ButtonDebouncer

    Port map( RST => reset,
           CLK => clock,
           BTN => start,
           CLEARED_BTN => start_cleared
           );




    booth : molt_booth
        port map(
              clock => clock,
```

```
            reset => reset,
            start => start_cleared,
              X=> X,
              Y=> Y,
              P => P_temp,
              stop_cu => stop_temp
              );

    LED<=P_temp;


end Structural;
```

L'implementazione del Button Debouncer è in Appendice.

L'implementazione del componente molt_booth, come anticipato, differisce solo per la riga `temp_reset_in <= reset or temp_stop_cu;` che è stata modificata in `temp_reset_in <= reset;`.

### *Sintesi su board di sviluppo*

Di seguito sono riportate le porzioni del file di constraint che sono state utilizzate.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { X[0] }];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { X[1] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { X[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { X[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { X[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { X[5] }];
#IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { X[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { X[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { Y[0] }];
#IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { Y[1] }];
#IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { Y[2] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { Y[3] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { Y[4] }];
#IO_L24P_T3_35 Sch=sw[12]
```

```
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { Y[5] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { Y[6] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { Y[7] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }];
#IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
#IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { LED[2] }];
#IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { LED[3] }];
#IO_L8P_T1_D11_14 Sch=led[3]
set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
#IO_L7P_T1_D09_14 Sch=led[4]
set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
#IO_L18N_T2_A11_D27_14 Sch=led[5]
set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
#IO_L17P_T2_A14_D30_14 Sch=led[6]
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
#IO_L18P_T2_A12_D28_14 Sch=led[7]
set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
set_property -dict { PACKAGE_PIN T15   IOSTANDARD LVCMOS33 } [get_ports { LED[9] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports { LED[10] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { LED[11] }];
#IO_L15N_T2_DQS_DOUT_CSO_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports { LED[12] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports { LED[13] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [get_ports { LED[14] }];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { LED[15] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]

##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN
}]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
#set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports {  BTNC }];
#IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { start }];
#IO_L12P_T1_MRCC_14 Sch=btnl
#set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { BTNR }];
#IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

### *Timing analysis*

Come riportato nel paragrafo *Sintesi su board di sviluppo*, nel file dei constraints è stato dichiarato un primary clock, che è un board clock che entra nel design attraverso un pin di input e deve essere pertanto collegato a un elemento della netlist. Rappresenta un riferimento temporale per il progetto e viene utilizzato dalla timing engine per derivare i requisiti del timing path e la relazione di fase con altri clock. In questo caso il clock della board entra nel device attraverso il porto di input sys_clk_pin e si propaga attraverso un input buffer e un clock buffer prima di raggiungere i requisiti del path. Il clock primario generato in questo caso ha periodo di 10 ns (-period 10.00)  e duty cycle del 50% (-waveform {0 5}), ed è collegato al PIN E3 della board.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clock }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clock}];
```

L'analisi temporale è stata eseguita dopo l'implementazione, poiché in questo modo si ottengono valori più accurati rispetto a un'analisi temporale effettuata dopo la sintesi, la quale non considera i ritardi dovuti al routing, che solitamente rappresentano tra il 45% e il 65% del tempo totale.

I parametri di configurazione della timing analysis sono stati opportunamente settati nel pannello Report Timing Analysis.

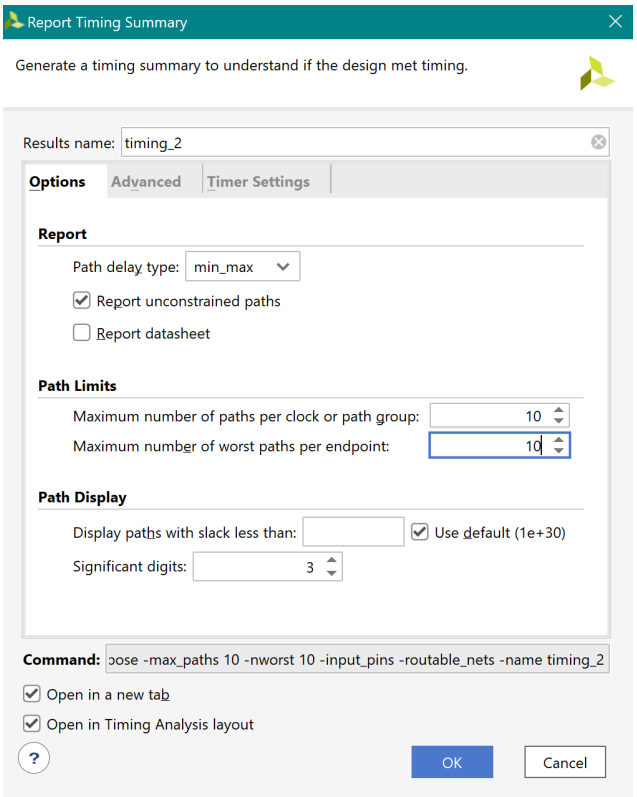



*Figura 7.11 Configurazione parametri timing analysis*

I risultati sono visibili in *Figura 7.12*. È possibile osservare che il tempo di setup, cioè il tempo che impiega il segnale di input per stabilizzarsi prima del prossimo fronte attivo del clock, tale che le uscite raggiungano il valore desiderato, è pari a 5,113 ns.

**Design Timing Summary**

| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 5,113 ns | Worst Hold Slack (WHS): | 0,204 ns | Worst Pulse Width Slack (WPWS): | 4,500 ns |
| Total Negative Slack (TNS): | 0,000 ns | Total Hold Slack (THS): | 0,000 ns | Total Pulse Width Negative Slack (TPWS): | 0,000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 142 | Total Number of Endpoints: | 142 | Total Number of Endpoints: | 73 |

**All user specified timing constraints are met.**

*Figura 7.12 Design Timing Summary*

Per poter ridurre tale parametro è stata ricercata la frequenza massima di funzionamento (FMAX). Per farlo, è stata aumentata progressivamente la frequenza del clock del design e eseguendo una timing analysis finché non si ottiene uno slack negativo (worst negative slack).

Impostando il periodo pari a 4 ns otteniamo un WNS negativo (*Figura 7.13*). Per cui FMAX=1/(T-WNS) ≈ 249 Mhz. Oltre questa frequenza i vincoli temporali non sono rispettati.

**Design Timing Summary**

| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | -0,014 ns | Worst Hold Slack (WHS): | 0,216 ns | Worst Pulse Width Slack (WPWS): | 1,500 ns |
| Total Negative Slack (TNS): | -0,045 ns | Total Hold Slack (THS): | 0,000 ns | Total Pulse Width Negative Slack (TPWS): | 0,000 ns |
| Number of Failing Endpoints: | 4 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 142 | Total Number of Endpoints: | 142 | Total Number of Endpoints: | 73 |

**Timing constraints are not met.**

*Figura 7.13 Design Timing Summary - WNS negativo*

**Design Timing Summary**

| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 0,144 ns | Worst Hold Slack (WHS): | 0,191 ns | Worst Pulse Width Slack (WPWS): | 1,508 ns |
| Total Negative Slack (TNS): | 0,000 ns | Total Hold Slack (THS): | 0,000 ns | Total Pulse Width Negative Slack (TPWS): | 0,000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 145 | Total Number of Endpoints: | 145 | Total Number of Endpoints: | 74 |

**All user specified timing constraints are met.**

*Figura 7.14 Design Timing Summary - FMAX*

# Capitolo 4: Comunicazione con handshaking

## Esercizio 8

### Esercizio 8.1

Progettare, implementare in VHDL e testare mediante simulazione un sistema composto da 2 nodi, A e B, che comunicano mediante un protocollo di handshaking. Il nodo A e il nodo B possiedono entrambi una memoria interna in cui sono memorizzate N stringhe di M bit, denominate X(i) e Y(i) rispettivamente (i=0,..,N-1). Il nodo A trasmette a B ciascuna stringa X(i) utilizzando un protocollo di handshaking; B, ricevuta la stringa X(i), calcola S(i)=X(i)+Y(i) e immagazzina la somma in opportune locazioni della propria memoria interna.

Per il progetto è possibile considerare una implementazione di tipo comportamentale per effettuare la somma, mentre è necessario prevedere esplicitamente un componente contatore sia nel sistema A sia nel sistema B per scandire la trasmissione/ricezione delle stringhe e per terminare la comunicazione.

#### *Progetto e architettura*

Il sistema realizzato è illustrato in *Figura 8.1*. Il nodo A contiene una *control Unit,* un *contatore* modulo 4, una ROM. Il nodo B contiene una *control Unit,* un *contatore modulo 4,* una memoria e un *sommatore*, che permette di sommare i dati inviati da A e quelli prelevati dalla prima metà della memoria in B rispettivamente.

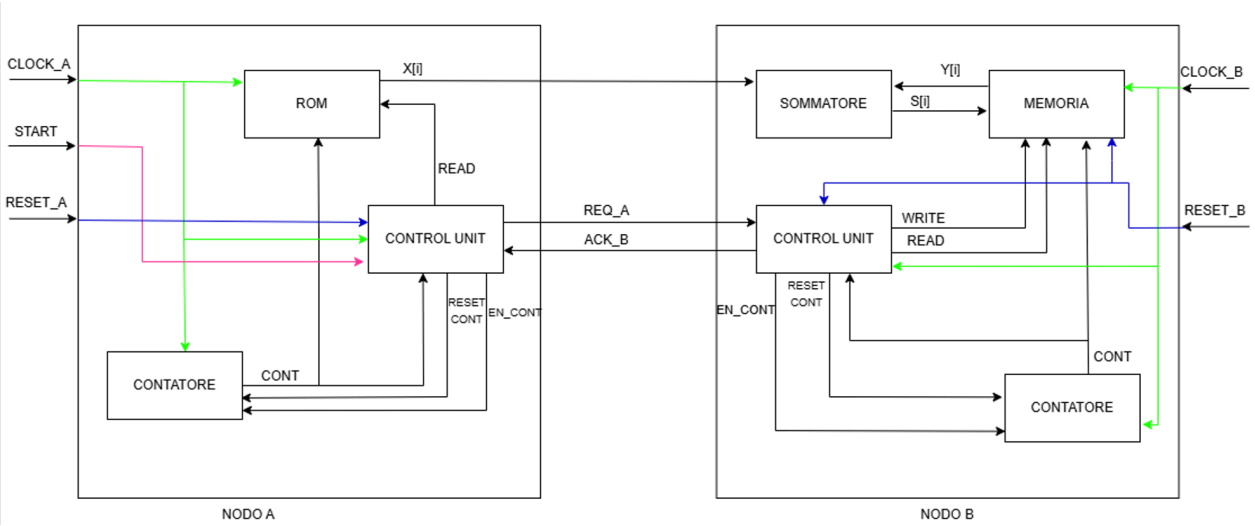



*Figura 8.1 Disegno architetturale del sistema*

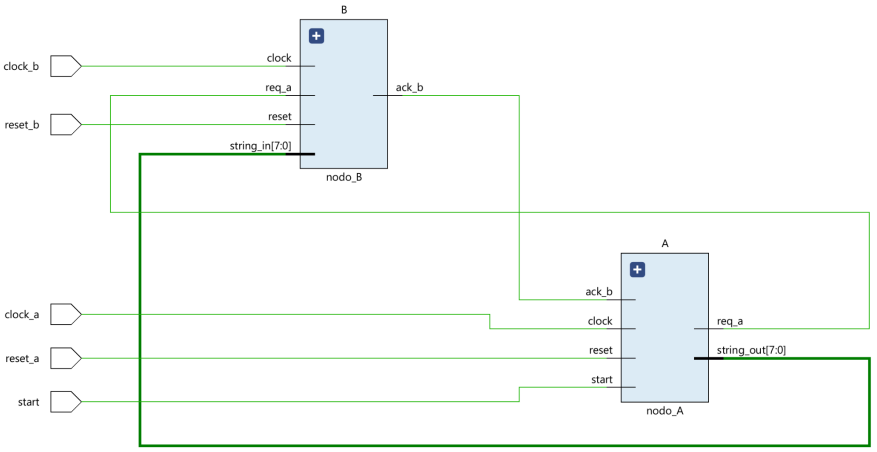



*Figura 8.2 Schematic del sistema realizzato*

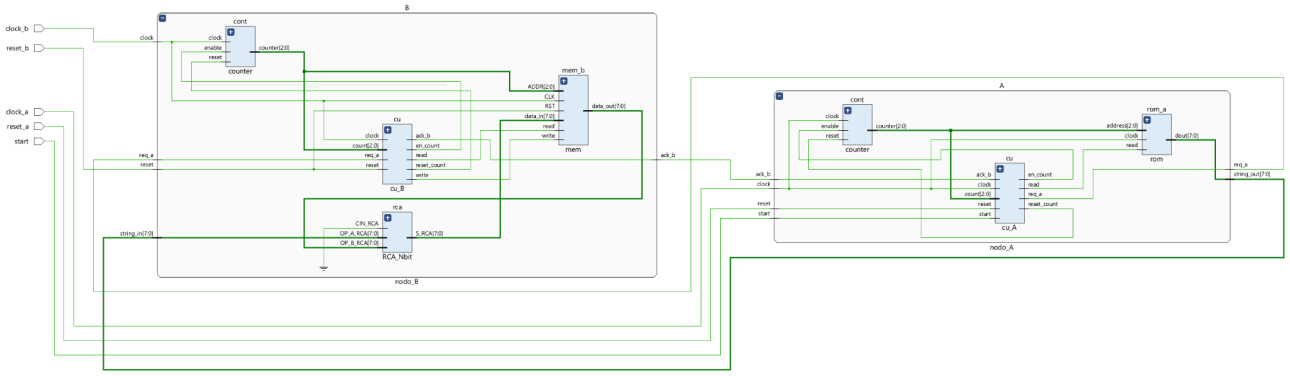


*Figura 8.3 Schematic del sistema realizzato: nodo A e nodo B*

Gli automi delle unità di controllo del nodo A e del nodo B sono illustrate rispettivamente in *Figura 8.4* e in *Figura 8.5*. Si può osservare che da un lato B mette a 0 l'ACK_B quando ha terminato l'operazione, dall'altro che finché A non rileva che ACK_B si è abbassato rimane nello stato S2 e non abilita il conteggio per proseguire l'eventuale invio della stringa successiva. In questo modo, A prosegue solo quando B ha finito l'elaborazione del dato ricevuto. Questo costituisce, quindi, un alternativa all'invio di un'ulteriore segnale di done.

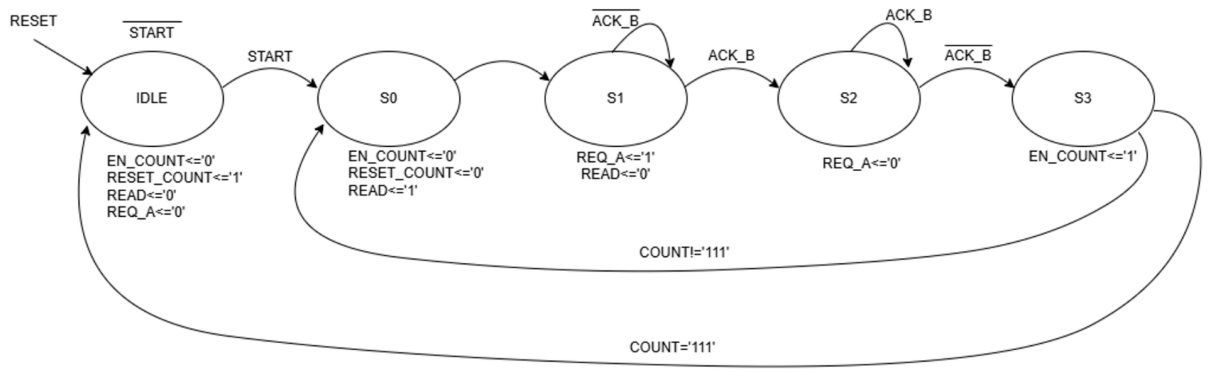



*Figura 8.4 Automa dell'unità di controllo del nodo A*

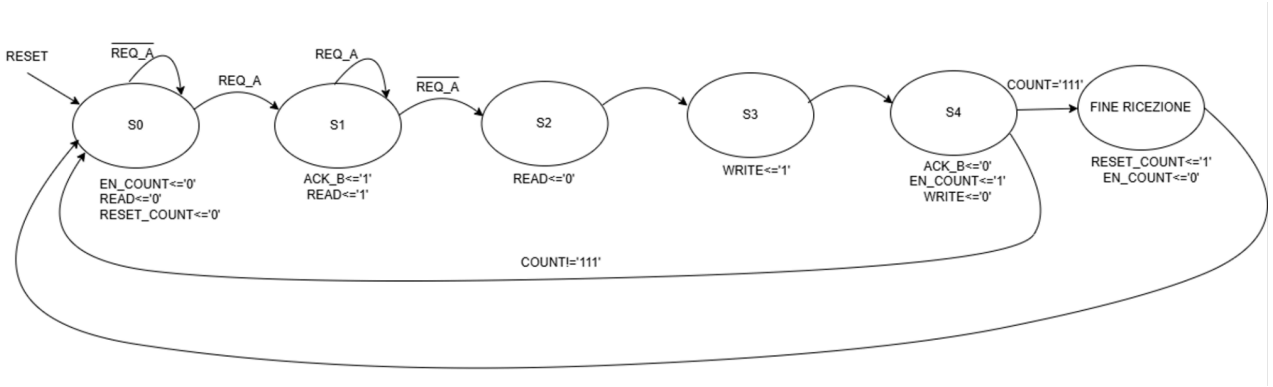



*Figura 8.5 Automa dell'unità di controllo del nodo B*

### *Implementazione*

Il top module è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity com_handshaking is
  Port (
        start: in std_logic;
        reset_a: in std_logic;
        reset_b: in std_logic;
        clock_a: in std_logic;
        clock_b: in std_logic

        );
end com_handshaking;

architecture Structural of com_handshaking is

  signal req_a: std_logic;
  signal ack_b: std_logic;
  signal string_a:  std_logic_vector(7 downto 0);

  component nodo_B is
  Port (
        clock: in std_logic;
        reset: in std_logic;
        string_in: in std_logic_vector(7 downto 0);
        req_a: in std_logic;
        ack_b: out std_logic
  );
   end component;

  component nodo_A is
  Port (
        start: in std_logic;
        clock: in std_logic;
        reset: in std_logic;
        string_out: out std_logic_vector(7 downto 0);
        req_a: out std_logic;
        ack_b: in std_logic
   );
```

```
   end component;


begin

  A: nodo_A
  Port map(

        start => start,
        clock => clock_a,
        reset => reset_a,
        string_out => string_a,
        req_a => req_a,
        ack_b => ack_b
   );

  B: nodo_B
  Port map(
        clock => clock_b,
        reset => reset_b,
        string_in => string_a,
        req_a => req_a,
        ack_b => ack_b
  );



end Structural;
```

Il **Nodo A** è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity nodo_A is
  Port (
        start: in std_logic;
        clock: in std_logic;
        reset: in std_logic;
        string_out: out std_logic_vector(7 downto 0);
        req_a: out std_logic;
        ack_b: in std_logic
   );
end nodo_A;

architecture Structural of nodo_A is

    signal en_count: std_logic;
    signal count: std_logic_vector(2 downto 0);
    signal read: std_logic;
    signal umax: std_logic;
    signal reset_count: std_logic;


    component rom is
```

```
    port(
        clock: in std_logic;
        read: in std_logic;
        address : in  std_logic_vector(2 downto 0);
        dout    : out std_logic_vector(7 downto 0)
    );
    end component;


    component counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
                enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));

    end component;


    component cu_A is
      port(
            clock, reset, start: in std_logic;
            count: in std_logic_vector (2 downto 0);
            en_count: out std_logic;
            reset_count: out std_logic;
            read: out std_logic;
            req_a: out std_logic;
            ack_b: in std_logic
            );
    end component;


begin
    rom_a: rom port map(
        clock=>clock,
        read=> read,
        address => count,
        dout  => string_out
    );

    cont: counter
     generic map(

        n => 3 ,--numero di bit necessari
        modulo => 8

        )
```

```
    Port map ( clock => clock,
           reset => reset_count, --ingresso di reset per azzerare il contatore
                enable => en_count,
           umax => umax, -- si alza quando si raggiunge il valore massimo di conteggio
           counter => count
    );


   cu: cu_A port map(
            clock => clock,
            reset=> reset,
            start=> start,
            count=> count,
            en_count => en_count,
            reset_count => reset_count,
            read => read,
            req_a => req_a,
            ack_b=> ack_b
            );




end Structural;
```

La **Rom di A** è la seguente.

```
library ieee;
 use ieee.std_logic_1164.all;
 use ieee.numeric_std.all;
 entity rom is
    port(
        clock: in std_logic;
        read: in std_logic;
        address : in  std_logic_vector(2 downto 0);
        dout    : out std_logic_vector(7 downto 0)
    );
 end entity rom;
 architecture dataflow of rom is
    type MEMORY_8_8 is array (0 to 7) of std_logic_vector(7 downto 0);
    constant ROM_8_8 : MEMORY_8_8 := (
        x"00",
        x"01",
        x"02",
        x"03",
        x"04",
        x"05",
        x"06",
        x"07"
    );
 begin
    main : process(clock)
    begin
       if rising_edge(clock) then
        if read='1' then
            dout <= ROM_8_8(to_integer(unsigned(address)));
```

```
        end if;
      end if;
    end process main;
 end architecture dataflow;
```

La **Control unit di A,** che implementa l'automa in *Figura 8.4,* è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity cu_A is
       port(
             clock, reset, start: in std_logic;
             count: in std_logic_vector (2 downto 0);
             en_count: out std_logic;
             reset_count: out std_logic;
             read: out std_logic;
             req_a: out std_logic;
             ack_b: in std_logic
             );
end cu_A;


architecture structural of cu_A is
       type state is (idle, S0, S1, S2,S3);
       -- idle: aspetta lo start
       -- S0 : dà il segnale di read
       -- S1: dà il segnale di req e aspetta l'ACK
       -- S2 : quando arriva l'ack, abbassa il segnale di req
       -- S3 : quando B abbassa l'ack, abilita il conteggio
       signal current_state,next_state: state;

       begin


       reg_stato: process(clock)
                      begin
                      if(clock'event and clock='1') then
                       if(reset='1') then
                               current_state <=idle;
                         else
                               current_state <=next_state;
                          end if;
                      end if;
                      end process;

       comb: process(current_state, start, ack_b)
                begin


            CASE current_state is

                WHEN idle =>
```

```
                    en_count<='0';
                    reset_count<='1';
                    read<='0';
                    req_a<='0';

                if(start='1') then
                                next_state <= S0;
                            else
                                next_state <= idle;
                              end if;


             WHEN S0 =>

                        read<='1';
                        reset_count<='0';
                                en_count<='0';

                                next_state <= S1;

            WHEN S1 =>
                        req_a<='1';
                        read<='0';
                        if ack_b='0' then
                          next_state<=S1;
                        elsif ack_b='1' then
                          next_state <= S2;
                        end if;


            WHEN S2 =>
                  req_a<='0';

                                if ack_b='1' then
                                   next_state<=S2;
                                elsif ack_b='0' then
                                   next_state<=S3;
                                 end if;

         when S3 =>
                    en_count<='1';
                    if count="111" then
                       next_state<=IDLE;
                    else
                        next_state<=S0;

                     end if;


        WHEN others =>
                   next_state <= idle;
            end CASE;

   end process;
end structural;
```

Il **Nodo B** è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nodo_B is
  Port (
        clock: in std_logic;
        reset: in std_logic;
        string_in: in std_logic_vector(7 downto 0);
        req_a: in std_logic;
        ack_b: out std_logic
  );
end nodo_B;

architecture Structural of nodo_B is

    signal write: std_logic;
    signal read: std_logic;
    signal reset_count: std_logic;
    signal en_count: std_logic;
    signal umax: std_logic;
    signal count: std_logic_vector(2 downto 0);
    signal somma: std_logic_vector(7 downto 0);
    signal string_b: std_logic_vector(7 downto 0);
    signal ov: std_logic;
    signal c: std_logic;

    component mem is
    port(
        CLK: in std_logic;
        RST: in std_logic;
        write: in std_logic; -- Segnale di scrittura
        read: in std_logic; -- Segnale di lettura
        ADDR : in std_logic_vector(2 downto 0);
        data_in: in std_logic_vector(7 downto 0);
        data_out: out std_logic_vector(7 downto 0)
    );
    end component;

    component counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
                enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));
```

```
    end component;

    component RCA_Nbit is
      generic (N: natural range 0 to 32 := 8);
      port(
            OP_A_RCA: in std_logic_vector(N-1 downto 0);
            OP_B_RCA: in std_logic_vector(N-1 downto 0);
            CIN_RCA: in std_logic;

            S_RCA: out std_logic_vector(N-1 downto 0);
            COUT_RCA: out std_logic;
            OV: out std_logic
      );
    end component;


    component cu_B is
      port(
            clock, reset: in std_logic;
            count: in std_logic_vector (2 downto 0);
            en_count: out std_logic;
            reset_count: out std_logic;
            read: out std_logic;
            write: out std_logic;
            req_a: in std_logic;
            ack_b: out std_logic
            );
    end component;

begin


    mem_b:  mem port map (
        CLK => clock,
        RST => reset,
        write => write,
        read=> read,
        ADDR  => count,
        data_in => somma,
        data_out => string_b
    );

     cont: counter
     generic map(

        n => 3 ,--numero di bit necessari
        modulo => 8

        )

    Port map ( clock => clock,
           reset => reset_count, --ingresso di reset per azzerare il contatore
                enable => en_count,
           umax => umax, -- si alza quando si raggiunge il valore massimo di conteggio
           counter => count
    );
```

```
    cu: cu_B port map(
             clock => clock,
              reset => reset,
             count => count,
             en_count => en_count,
             reset_count => reset_count,
             read => read,
             write => write,
             req_a => req_a,
             ack_b => ack_b
             );


       rca: RCA_Nbit
       port map(
              OP_A_RCA =>string_in,
              OP_B_RCA => string_b,
              CIN_RCA => '0',

              S_RCA =>somma,
              COUT_RCA => c,
              OV => ov
       );


end Structural;
```

La **memoria di B** è la seguente**:**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mem is
    port(
        CLK: in std_logic;
        RST: in std_logic;
        write: in std_logic; -- Segnale di scrittura
        read: in std_logic;  -- Segnale di lettura
        ADDR : in std_logic_vector(2 downto 0);
        data_in: in std_logic_vector(7 downto 0);
        data_out: out std_logic_vector(7 downto 0) -- Uscita dati
    );
end mem;

architecture Behavioral of mem is
    type mem_type is array (0 to 7) of std_logic_vector(7 downto 0);
    signal MEM : mem_type := (
        0 => "00000000", -- 0
        1 => "00000001", -- 1
        2 => "00000010", -- 2
        3 => "00000011", -- 3
```

```
        4 => "00000100", -- 4
        5 => "00000101", -- 5
        6 => "00000110", -- 6
        7 => "00000111"  -- 7
    );
    signal temp_out : std_logic_vector(7 downto 0) := (others => '0');
begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if RST = '1' then  -- Reset sincrono: azzera tutta la memoria
                for i in 0 to 7 loop
                     MEM(i) <= std_logic_vector(to_unsigned(i, 8));
                end loop;
                temp_out <= (others => '0');
            else
                if write = '1' then  -- Scrittura
                    MEM(conv_integer(ADDR)) <= data_in;
                end if;
                if read = '1' then   -- Lettura
                    temp_out <= MEM(conv_integer(ADDR));
                end if;
            end if;
        end if;
    end process;

    data_out <= temp_out;

end Behavioral;
```

La **control unit di B,** che implementa l'automa in *Figura 8.5*, è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity cu_B is
       port(
             clock, reset: in std_logic;
             count: in std_logic_vector (2 downto 0);
             en_count: out std_logic;
             reset_count: out std_logic;
             read: out std_logic;
             write: out std_logic;
             req_a: in std_logic;
             ack_b: out std_logic
             );
end cu_B;


architecture structural of cu_B is
       type state is (S0, S1, S2,S3,S4, fine_ricezione);
```

```
signal current_state,next_state: state;

begin


reg_stato: process(clock)
                begin
                if(clock'event and clock='1') then
                 if(reset='1') then
                         current_state <=S0;
                  else
                         current_state <=next_state;
                   end if;
                end if;
                end process;

comb: process(current_state, req_a)
        begin


     CASE current_state is

         WHEN S0 =>

                   read<='0';
                          en_count<='0';
                          reset_count<='0';
                          if req_a='1' then
                             next_state <= S1;
                          elsif req_a='0' then
                             next_state<=S0;
                          end if;

        WHEN S1 =>
                    ack_b<='1';
                    read<='1';
                    if req_a='1' then
                      next_state<=S1;
                    elsif req_a='0' then
                      next_state <= S2;
                    end if;

       WHEN S2 =>
                  read<='0';
                  next_state<=S3;

        WHEN S3 =>
                    -- i dati sono pronti: avvio la scrittura
              write<='1';
                    next_state<=S4;

    when S4 =>
               ack_b<='0';         -- INVECE DI USARE UN ULTERIORE SEGNALE DI
DONE, SI MANTIENE ALTO L'ACK FINO A CHE L'ELABORAZIONE NON E' FINITA
```

```
                    en_count<='1';
                    write<='0';
                    if count = "111" then
                        next_state<=fine_ricezione;
                    else
                         next_state<=S0;
                    end if;

          when fine_ricezione =>
                    reset_count<='1';
                    en_count<='0';
                    next_state<=S0;



             end CASE;

    end process;
 end structural;
```

Le implementazioni del ripple carry adder e del contatore sono riportate in Appendice.

*Simulazione*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_com_handshaking is
end tb_com_handshaking;

architecture Behavioral of tb_com_handshaking is

    signal reset_a_tb    : std_logic := '0';
    signal reset_b_tb    : std_logic := '0';
    signal start_tb    : std_logic := '0';
    signal clock_a_tb  : std_logic := '0';
    signal clock_b_tb  : std_logic := '0';

    constant PERIOD_A_EQUAL   : time := 10 ns;
    constant PERIOD_B_EQUAL   : time := 10 ns;

    constant PERIOD_A_FAST    : time := 6 ns;
    constant PERIOD_B_SLOW    : time := 12 ns;

    constant PERIOD_A_SLOW    : time := 14 ns;
    constant PERIOD_B_FAST    : time := 7 ns;

   -- constant PHASE_SHIFT_B : time := 3 ns; -- sfasamento di clock_b rispetto a
clock_a


    signal test_case : integer := 0;

    component com_handshaking is
        Port (
```

```
            start   : in std_logic;
            reset_a   : in std_logic;
            reset_b   : in std_logic;
            clock_a : in std_logic;
            clock_b : in std_logic
        );
    end component;

begin

    uut: com_handshaking
        port map (
            start   => start_tb,
            reset_a   => reset_a_tb,
            reset_b   => reset_b_tb,
            clock_a => clock_a_tb,
            clock_b => clock_b_tb
        );

    -- Processo clock A dinamico
    clk_a_process: process
    begin
        while true loop
            case test_case is
                when 0 => wait for PERIOD_A_EQUAL / 2;
                when 1 => wait for PERIOD_A_FAST / 2;
                when 2 => wait for PERIOD_A_SLOW / 2;
                when others => wait for 10 ns;
            end case;
            clock_a_tb <= not clock_a_tb;
        end loop;
    end process;

    -- Processo clock B dinamico
    clk_b_process: process
    begin
        --wait for PHASE_SHIFT_B;  -- sfasamento iniziale
        while true loop
            case test_case is
                when 0 => wait for PERIOD_B_EQUAL / 2;
                when 1 => wait for PERIOD_B_SLOW / 2;
                when 2 => wait for PERIOD_B_FAST / 2;
                when others => wait for 10 ns;
            end case;
            clock_b_tb <= not clock_b_tb;
        end loop;
    end process;

    -- Stimolo e sequenza dei test
    stim_proc: process
    begin
        -- Reset iniziale
        reset_a_tb <= '1';
        reset_b_tb <= '1';
        wait for 20 ns;
        reset_a_tb <= '0';
```

```
        reset_b_tb <= '0';

        --------------------------------
        -- Test 1: clock_a = clock_b
        test_case <= 0;
        start_tb <= '1';
        wait for 10 ns;
        start_tb <= '0';
        wait for 700 ns;

        --------------------------------
        -- Test 2: clock_a > clock_b
        test_case <= 1;
        reset_a_tb <= '1';
        reset_b_tb <= '1';
        wait for 20 ns;
        reset_a_tb <= '0';
        reset_b_tb <= '0';
        start_tb <= '1';
        wait for 10 ns;
        start_tb <= '0';
        wait for 700 ns;

        -------------------------------
        -- Test 3: clock_b > clock_a
        test_case <= 2;
        reset_a_tb <= '1';
        reset_b_tb <= '1';
        wait for 20 ns;
        reset_a_tb <= '0';
        reset_b_tb <= '0';
        start_tb <= '1';
        wait for 14 ns;
        start_tb <= '0';
        wait for 500 ns;

       -- Fine simulazione
        wait;
    end process;

end Behavioral;
```

La simulazione è stata effettuata in tre casi:

- il clock di A è lo stesso di B

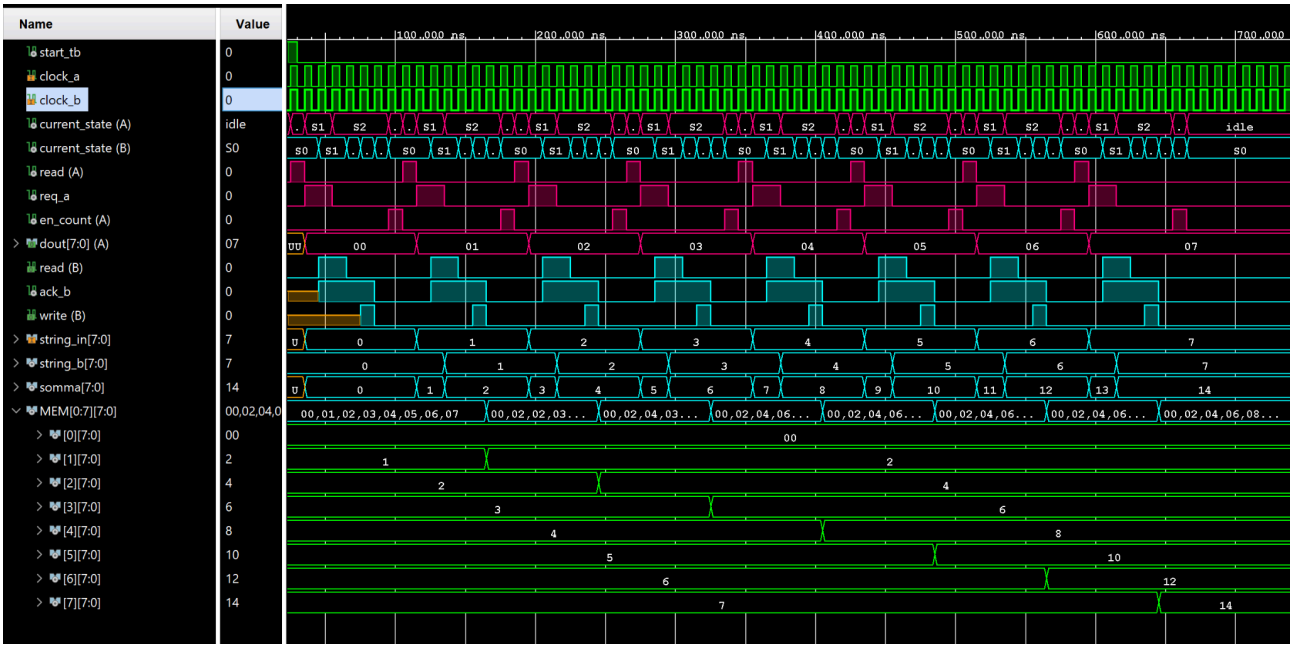



*Figura 8.6 Risultato della simulazione: caso clock_a = clock_b*

- il clock di A è più veloce di quello di B

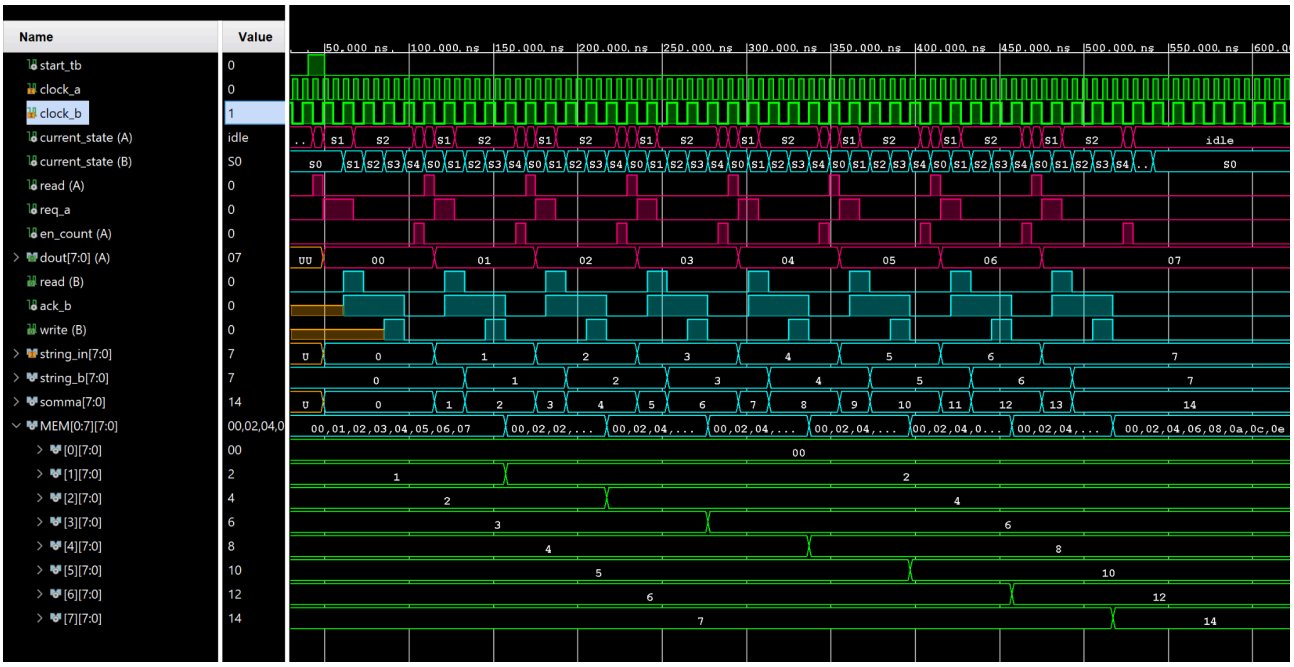



*Figura 8.7 Risultato della simulazione: caso clock_a > clock_b*

- il clock di A è più lento di quello di B

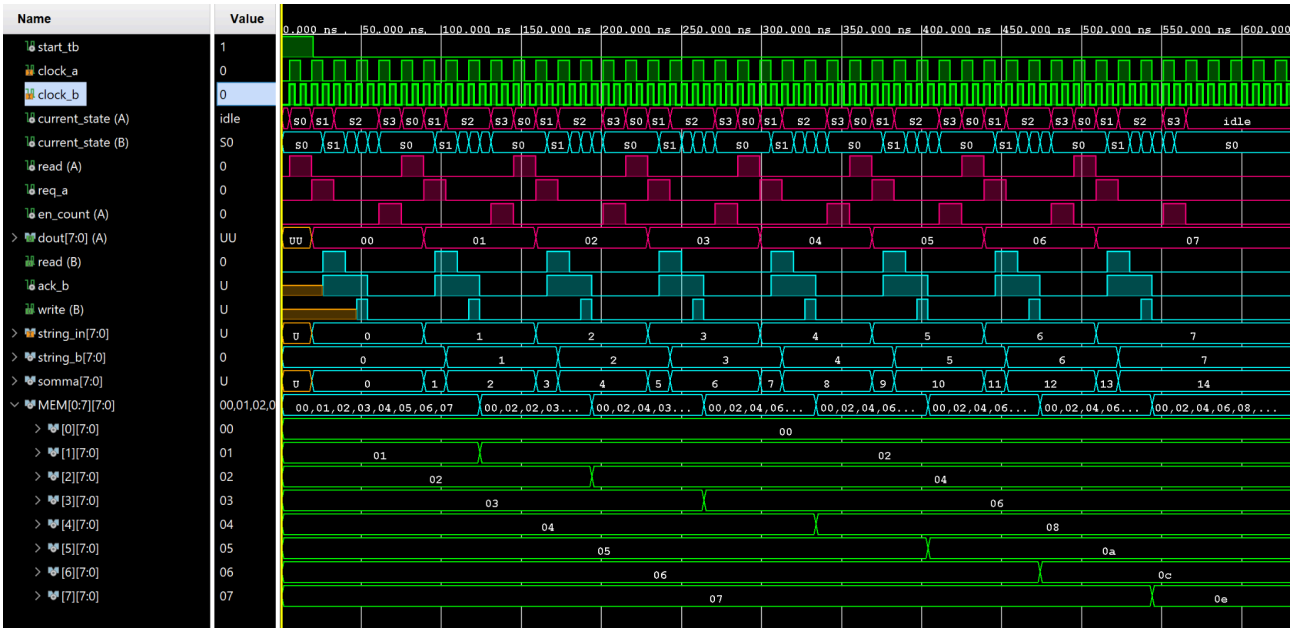



*Figura 8.7 Risultato della simulazione: caso clock_a < clock_b*

# Capitolo 5: Processore

## Esercizio 9

A partire dall'implementazione fornita del processore operante secondo il modello IJVM,

a) si proceda all'analisi dell'architettura mediante simulazione e si approfondisca lo studio del suo funzionamento per due istruzioni a scelta,

b) si modifichi un codice operativo a scelta, documentando tutte le modifiche effettuate

### *Analisi dell'architettura*

Il processore Mic-1 è una macchina a stack, e non dispone di registri generali. Il set di istruzioni implementato dal Mic-1 è un sottoinsieme di quello della Java Virtual Machine, denominato IJVM in quanto opera unicamente sugli interi. Ogni istruzione della IJVM viene implementata attraverso una sequenza di microistruzioni gestite secondo l'approccio microprogrammato. L'architettura del MIC-1 è illustrata in *Figura 9.1*.

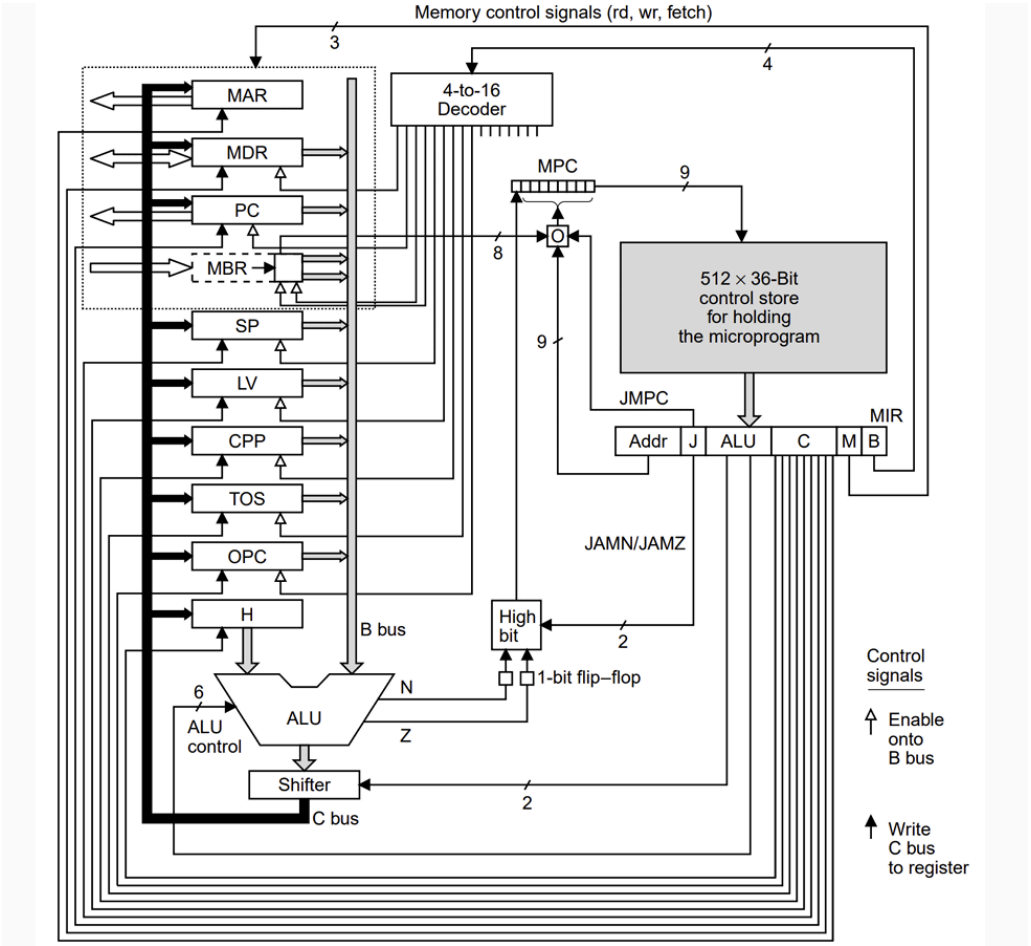



*Figura 9.1 Architettura del MIC-1*

Il data path del MIC-1, che consente di implementare le istruzioni IJVM, è costituito da una ALU, da una serie di registri a 32 bit accessibili solo a livello microarchitetturale, e da un BUS B per la lettura dai registri e un BUS C per la scrittura sui registri.

La ALU  ha 6 linee di controllo (teoricamente ci sono 64 combinazioni possibili, ma non tutte sono utilizzate). Le principali combinazioni sono illustrate in *Figura 9.2*. L'uscita della ALU entra in uno shifter che ha due ulteriori linee di controllo: SLL8 shifta di 1 byte a sinistra e SRA1 shifta di 1 bit a destra.

| $F_0$ | $F_1$ | ENA | ENB | INVA | INC | Function |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | A |
| 0 | 1 | 0 | 1 | 0 | 0 | B |
| 0 | 1 | 1 | 0 | 1 | 0 | $\bar{A}$ |
| 1 | 0 | 1 | 1 | 0 | 0 | $\bar{B}$ |
| 1 | 1 | 1 | 1 | 0 | 0 | A + B |
| 1 | 1 | 1 | 1 | 0 | 1 | A + B + 1 |
| 1 | 1 | 1 | 0 | 0 | 1 | A + 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | B + 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | B – A |
| 1 | 1 | 0 | 1 | 1 | 0 | B – 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | –A |
| 0 | 0 | 1 | 1 | 0 | 0 | A AND B |
| 0 | 1 | 1 | 1 | 0 | 0 | A OR B |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | –1 |

*Figura 9.2 Operazioni della ALU*

Nota: nell'implementazione, la struttura dei control bit della ALU è ***SLL8 | SRA1 | F_0 | F_1 | EN_A | EN_B | INV_A | INC***.

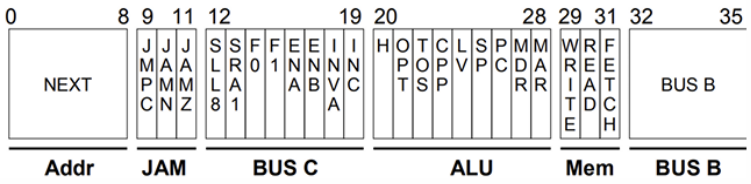
Quasi tutti i registri dispongono di segnali di controllo per la lettura dal bus C e la scrittura sul bus B: 9 registri possono scrivere su B (tutti tranne MAR) e 9 registri possono essere caricati da C (tutti tranne MBR).

I registri sono:

- CPP, LV, SP: usati per la gestione della memoria
- PC: program counter
- MAR (memory address register): indirizzo della word da leggere/scrivere
- MDR (memory data register): word letta/da scrivere in memoria
- TOS: top of stack (valore puntato da SP)
- OPC: registro temporaneo usato dalle istruzioni
- MBR (memory byte register): memorizza negli otto bit meno significativi i byte dell'istruzione man mano che viene interpretata.

Ci sono, dunque, quattro registri utilizzati per comunicare con la memoria: la coppia MAR (Memory Address Register) e MDR (Memory Data Register) per la lettura/scrittura di word di 32 bit di dati e la coppia PC (Program Counter) ed MBR (Memory Byte Register) per leggere il programma eseguibile permettendo il fetch delle istruzioni ISA.

Ogni istruzione della IJVM viene realizzata mediante l'esecuzione di una sequenza di microistruzioni di 36 bit ciascuna.  Le microistruzioni sono memorizzate in una control store formata da 512 parole di 36 bit. Il formato delle microistruzioni è il seguente:



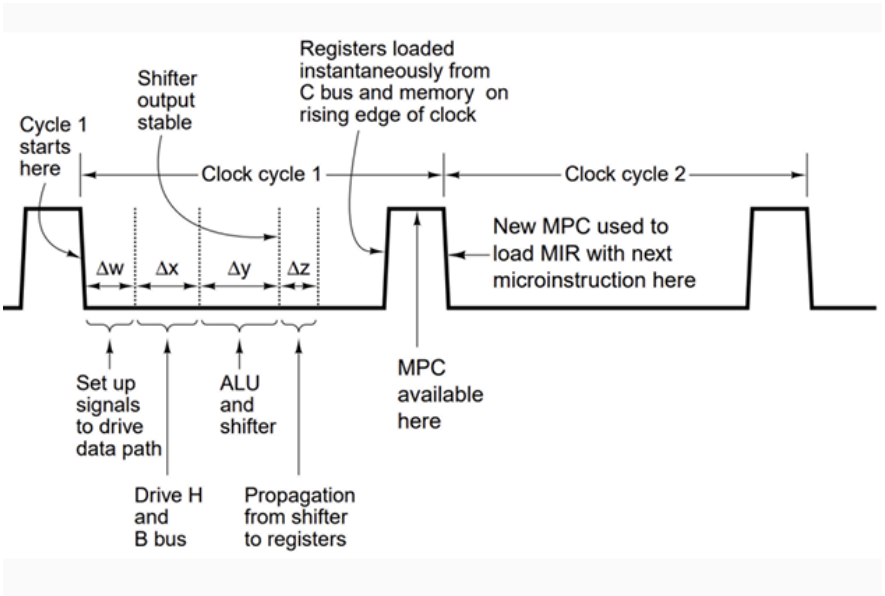

- Next address: indica qual è la successiva microistruzione da eseguire;
- JAM: viene usato per gestire i salti
- ALU: per decidere quali operazioni l'ALU deve effettuare
- C: 9 bit per abilitare i registri in lettura dal bus C
- Mem: per specificare le operazioni verso la memoria
- B: per abilitare i registri in scrittura sul bus B; è su 4 bit e non 9 perché la scrittura su B è mutuamente esclusiva.

La control store utilizza:

- MPC: Micro Program Counter: indica, ad ogni passo di controllo, quale microistruzione leggere (è un registro virtuale).
- MIR: Micro Instruction Register. Contiene la microistruzione letta all'indirizzo puntato da MPC.

L'indirizzo nella control store della prima microistruzione di una istruzione ISA corrisponde al valore binario dell'opcode dell' istruzione stessa.

La tempificazione del datapath si basa sul clock asimmetrico, caratterizzato da un breve impulso, rappresentato di seguito:




L'esecuzione di ogni istruzione ISA consiste in:

- Lettura dell'istruzione dalla memoria
- Salto al microcodice dell'istruzione ISA
- Esecuzione del microcodice
- Ritorno al microcodice per il caricamento della prossima istruzione (PC = PC+1; FETCH, GOTO(MBR)).

*Istruzioni ISUB e ISTORE*

Il file program.ajvm editato per permettere la simulazione delle istruzioni **ISUB** e **ISTORE** è il seguente:

```
1   .main
2   .var
3   a
4   .endvar
5   BIPUSH 0xA
6   BIPUSH 0xE
7   ISUB
8   ISTORE a
9   HALT
10  .endmethod
```

Per rigenerare la RAM è sufficiente lanciare, dalla directory build, il seguente comando:

```
flavia@flavia-VirtualBox:~/esercitazione_mic/amic-0$ cmake --build build --target create_ram
Built target assemble_program
Built target create_ram
```

**ISUB** corrisponde alla locazione di memoria della control store 0x5C, ovvero la novantaduesima parola.

```
isub = 0x5C:
        MAR = SP = SP - 1; rd
        H = TOS
        MDR = TOS = MDR - H; wr; goto main
```

Nella control store è riportata la micro routine da eseguire, composta da tre microistruzioni:

```
92 => "00101110100000110110000001001010010 0",
93 => "00101111000000001010010000000000111",
94 => "00000011000000111111001000010100000 0",
```

In particolare:

| | NEXT | JAM | ALU | BUS C | MEM | BUS B |
|---|---|---|---|---|---|---|
| **92** | 001011101 | 000 | 00110110 | 000001001 | 010 | 0100 |
| **93** | 001011110 | 000 | 00010100 | 100000000 | 000 | 0111 |
| **94** | 000000110 | 000 | 00111111 | 001000010 | 100 | 0000 |

Di seguito è riportata la spiegazione dettagliata di ciascuna delle tre microistruzioni (92, 93 e 94).

La prima cosa che fa la ISUB (*MAR = SP = SP - 1; rd*) è decrementare lo stack pointer, per effettuare una lettura dalla memoria. Il decremento e il successivo caricamento nel registro MAR permette l'acquisizione dell'operando dalla memoria. NB: il segnale di read *rd* diventa alto solo al fronte di clock successivo, poiché è solo allora che l'indirizzo da leggere diventa disponibile nel MAR.

Questo primo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 110110, che equivale al decremento dell'operando B,

- BUS C: 000001001, indicando che il bus C scrive su SP e MAR,
- MEM: 010 (lettura dalla memoria),
- BUS B: 0100, il bus B è controllato da SP,
- Next_addr: 001011101, che equivale a 93, la locazione di memoria successiva,
- JAM: 000, nessun salto

La seconda cosa da fare (*H = TOS*) è far passare l'operando B invariato e scriverlo sul registro H (prendendolo dal registro TOS).

Questo secondo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 010100, che equivale al passaggio dell'operando B invariato;
- BUS C: 100000000, indicando che il bus C scrive su H,
- Mem: 000, nessuna operazione in memoria,
- BUS B: 0111, il bus B viene controllato da TOS,
- Next_addr: 001011110, che equivale a 94, la locazione di memoria successiva,
- JAM: 000, nessun salto

L'ultima cosa da fare è la sottrazione vera e propria (*MDR = TOS = MDR - H; wr; goto main*) tra il contenuto di MDR e il contenuto H e memorizzarlo in MDR stesso e nel TOS.

Questo ultimo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 00111111, che equivale alla sottrazione dei due operandi,
- BUS C: 001000010, il bus C scrive su MDR e TOS,
- Mem: 100 (scrittura in memoria),
- BUS B: 0000, MDR controlla il bus B,
- Next_addr: 000000110, equivale a goto main, il microcodice per il caricamento della prossima istruzione,
- JAM: 000, nessun salto

**ISTORE** è l'istruzione che consente di memorizzare il valore in cima allo stack in una variabile locale. Corrisponde alla locazione di memoria della control store 0x36, ovvero la cinquantaquattresima parola.

```
istore = 0x36:
        H = LV
        MAR = MBRU + H
istore_cont:
        MDR = TOS; wr
        SP = MAR = SP - 1; rd
        PC = PC + 1; fetch
        TOS = MDR; goto main
```

Nella control store è riportata la micro routine da eseguire, composta da sei microistruzioni:

```
54 => "00011011100000010100100000000000101",
55 => "00011100000000111100000000010000011",
56 => "00011100100000010100000000101000111",
57 => "00011101000000110110000010010100100",
58 => "00011101100000110101000001000010001",
59 => "00000011000000010100001000000000000",
```

In particolare:

| | NEXT | JAM | ALU | BUS C | MEM | BUS B |
|---|---|---|---|---|---|---|
| **54** | 000110111 | 000 | 00010100 | 100000000 | 000 | 0101 |
| **55** | 000111000 | 000 | 00111100 | 000000001 | 000 | 0011 |
| **56** | 000111001 | 000 | 00010100 | 000000010 | 100 | 0111 |
| **57** | 000111010 | 000 | 00110110 | 000001001 | 010 | 0100 |
| **58** | 000111011 | 000 | 00110101 | 000000100 | 001 | 0001 |
| **59** | 000000110 | 000 | 00010100 | 001000000 | 000 | 0000 |

Di seguito è riportata la spiegazione dettagliata di ciascuna delle sei microistruzioni (54, 55, 56, 57, 58 e 59).

La prima cosa che fa la ISTORE (*H = LV*) è caricare il contenuto del registro local variable (che punta alla base della Local Variable Frame, l'area allocata nello stack che contiene le variabili locali) nel registro H. Questo primo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 010100, che equivale al passaggio dell'operando B invariato,
- BUS C: 100000000, che indica che il bus C scrive su H,
- MEM: 000, nessuna operazione in memoria,
- BUS B: 0101, il bus B è controllato da LV,
- Next_addr: 000110111, che equivale a 55, la locazione di memoria successiva,
- JAM: 000, nessun salto

La seconda cosa da fare (*MAR = MBRU + H*) è calcolare l'indirizzo della variabile sommando al contenuto di MBRU, l'offset della variabile, il contenuto di H, la base della Local Variable Frame. MBRU è il contenuto di MBR come unsigned. Il contenuto di MBR è il risultato dell'ultimo fetch eseguito dalla microistruzione main.

Questo secondo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 111100, che equivale alla somma degli operandi A e B,
- BUS C: 000000001, che indica che il bus C scrive su MAR,
- Mem: 000, nessuna operazione in memoria,
- BUS B: 0011, il bus B viene controllato da MBRU,
- Next_addr: 000111000, che equivale a 56, la locazione di memoria successiva,
- JAM: 000, nessun salto

La terza cosa da fare ( *MDR = TOS; wr*) è caricare il registro MDR con il contenuto del registro TOS e avviare la scrittura in memoria all'indirizzo contenuto puntato da MAR.

Questo terzo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 010100, che equivale al passaggio dell'operando B invariato,
- BUS C: 000000010, che indica che il bus C scrive su MDR,
- Mem: 100 (scrittura in memoria),

- BUS B: 0111, il bus B viene controllato da TOS,
- Next_addr: 000111001, che equivale a 57, la locazione di memoria successiva,
- JAM: 000, nessun salto

La quarta cosa da fare ( *SP = MAR = SP - 1; rd*) è decrementare il contenuto del registro SP e caricare il valore calcolato sia in MAR che in SP, e avviare la lettura del valore puntato dal nuovo SP. NB: il segnale di read *rd* diventa alto solo al fronte di clock successivo, poiché è solo allora che l'indirizzo da leggere diventa disponibile nel MAR. In questo modo viene rimosso il valore in cima allo stack.

Questo quarto passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 110110, che equivale al decremento dell'operando B,
- BUS C: 000001001, indicando che il bus C scrive su SP e MAR,
- MEM: 010 (lettura dalla memoria),
- BUS B: 0100, il bus B è controllato da SP,
- Next_addr: 000111010, che equivale a 58, la locazione di memoria successiva,
- JAM: 000, nessun salto

La quinta cosa da fare (*PC = PC + 1; fetch*) è incrementare il PC e fare il fetch della prossima istruzione, in modo che main possa eseguire il goto all'istruzione corretta.

Questo quinto passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 110101, che equivale all'incremento dell'operando B,
- C: 000000100, indicando che il bus C scrive su PC,
- Mem: 001, indicando l'operazione di fetch,
- B: 0001, PC controlla il bus B,
- Next_addr: 000111011, che equivale a 59, la locazione di memoria successiva,
- JAM: 000, nessun salto necessario.

L'ultimo passo (TOS = MDR; goto main) prevede la copia del contenuto di MDR in TOS. La parola chiave wr specifica la scrittura in memoria del dato. Infine, il goto main riporta alla micro routine main dove avviene l'incremento di PC, un'operazione di fetch per prelevare un byte dalla memoria istruzioni ed un goto(MBR) per saltare alla microistruzione memorizzata nel buffer.

Questo ultimo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 010100, che equivale al passaggio dell'operando B invariato,
- C: 001000010, il bus C scrive su MDR e TOS,
- Mem: 100, indicando la scrittura di dati,
- B: 0010, MBR controlla il bus B,
- Next_addr: 000000110, un indirizzo che porta ad un idle in cui viene incrementato il pc,
- JAM: 000, nessun salto necessario.

Di seguito sono riportati i risultati della simulazione, e sono evidenziati i principali passi delle micro routine di ISUB e ISTORE.

In particolare, in *Figura 9.3* è possibile osservare che, in primo luogo vengono effettuate le BIPUSH dei due operandi 0xA e 0xE. La micro routine della BIPUSH:

- incrementa il contenuto del registro SP, e carica il risultato sia in MAR che in SP;
- incrementa il PC così da poter leggere (fetch) l'operando contenuto nell'istruzione, che viene memorizzato nel registro MBR;
- scrive il contenuto di MBR sia in TOS che in MDR;
- ritorna al microcodice per il caricamento della prossima istruzione (goto main).

Il risultato delle due BIPUSH è, dunque, che i due operandi della ISUB sono stati posti in cima allo stack.

```
bipush = 0x10:
        SP = MAR = SP + 1
        PC = PC + 1; fetch
        MDR = TOS = MBR; wr; goto main
```

Dopo le due BIPUSH viene caricato in MBR l'indirizzo di partenza della ISUB. Come già detto, la prima microistruzione della routine effettua la sottrazione SP-1. Infatti, in *Figura 9.3* è evidenziato che i bit di controllo della ALU in questa fase sono 00110110, che corrisponde proprio all'operazione B-1 (*Figura 9.2*).

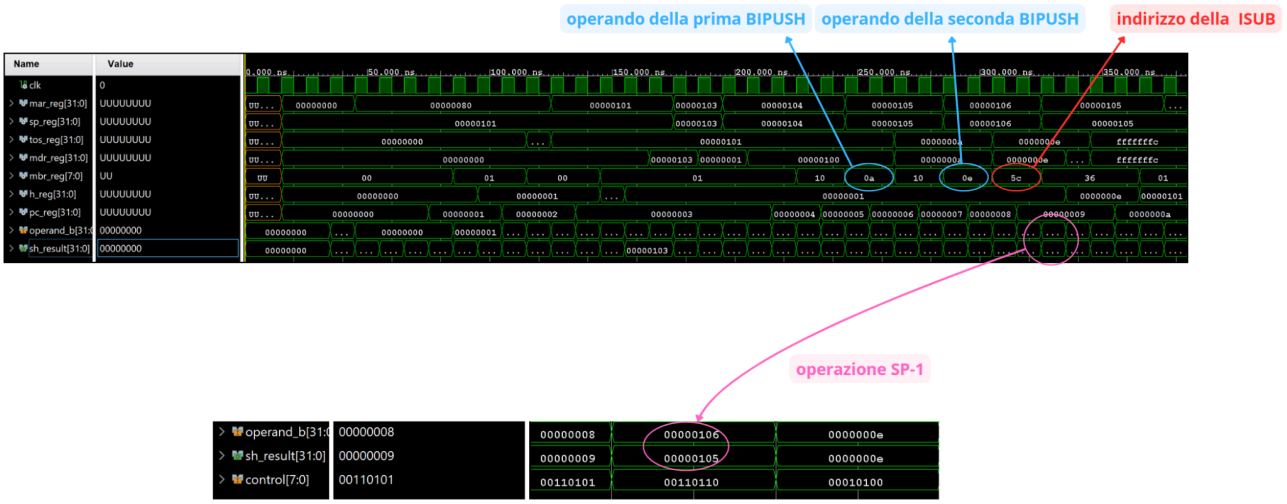



*Figura 9.3 Simulazione - Parte 1*

In *Figura 9.4*, è possibile vedere che MAR e SP, sul fronte di salita del clock, vengono caricati con il valore appena calcolato di SP-1. Viene, inoltre avviata una lettura da memoria, che inizia solo alla fine del ciclo, quando MAR viene caricato sul fronte di salita. Il dato è pronto solo alla fine del prossimo ciclo (sul successivo fronte di salita), quando MDR viene caricato. La figura mostra anche che la microistruzione successiva carica il valore contenuto nel registro TOS (l'ultimo elemento inserito sullo stack, cioè 0xE) nel registro H.

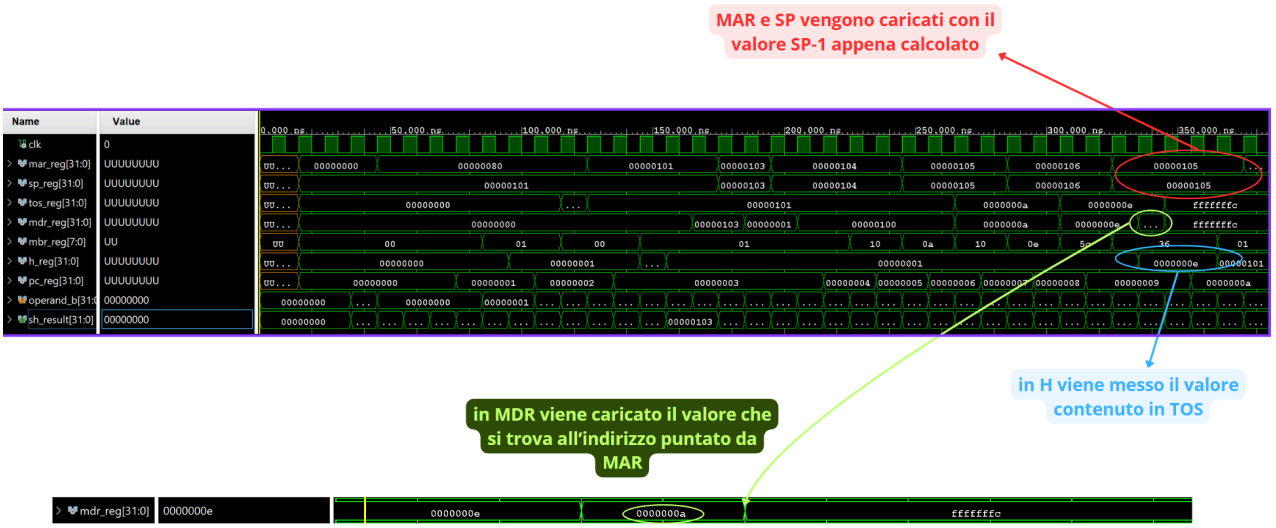



*Figura 9.4 Simulazione - Parte 2*

In *Figura 9.5*, è mostrata l'operazione di sottrazione vera e propria: l'operando A della ALU è 0xE, l'operando B è 0xA, i bit di controllo della ALU sono 00111111 (sottrazione B-A). Il risultato della sottrazione viene caricato nei registri MDR e TOS.

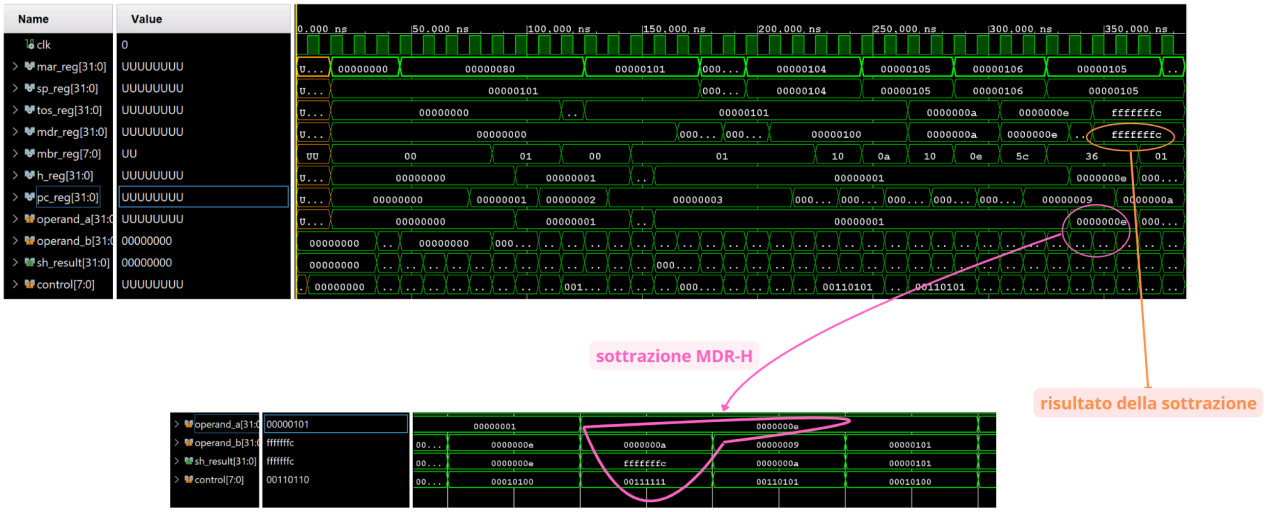



*Figura 9.5 Simulazione - Parte 3*

Infine, viene eseguita l'istruzione ISTORE, che salva il risultato della sottrazione, cioè il valore contenuto nel registro TOS, nella variabile locale **a** e lo rimuove dallo stack.

In *Figura 9.6* è possibile vedere il caricamento in H del contenuto del registro LV, che punta all'indirizzo di base della Local Variable Frame.

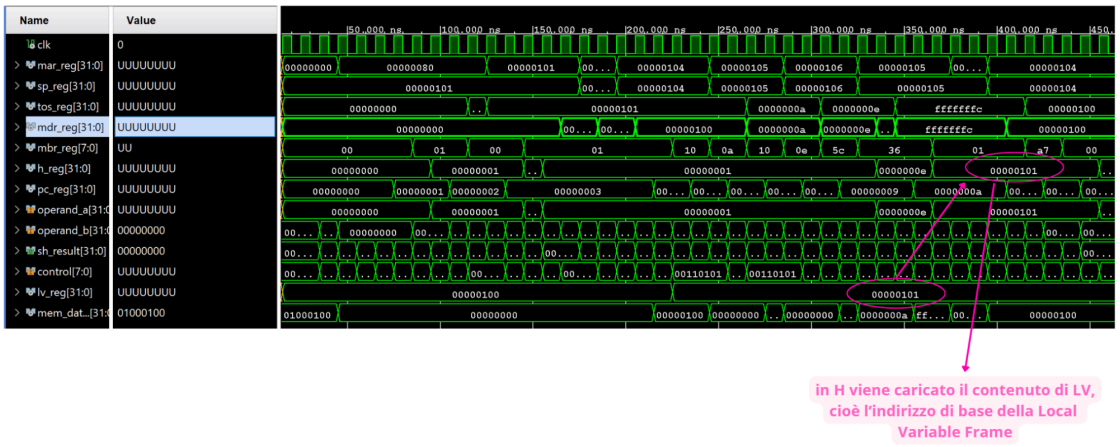



*Figura 9.6 Simulazione - Parte 4*

In *Figura 9.7* è possibile vedere il caricamento nel registro MAR dell'indirizzo della variabile a, dato dalla somma di H, che contiene l'indirizzo di base della Local Variable Frame, e MBRU, che contiene l'indice della variabile a.

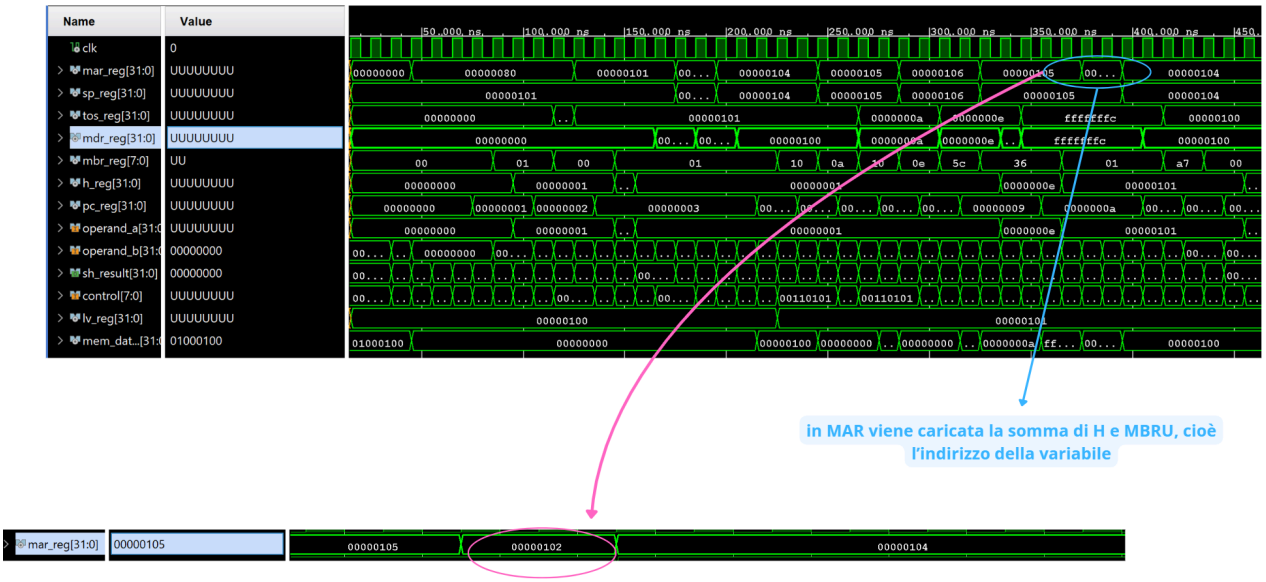



*Figura 9.7 Simulazione - Parte 5*

In *Figura 9.8* è possibile vedere il caricamento nel registro MDR del contenuto di TOS, cioè il risultato della sottrazione. In questo caso, tuttavia, in MDR è già presente il valore, dal momento che l'operazione ISUB ha già fatto il caricamento di TOS in MDR. Alla fine del ciclo, sul fronte di salita, viene anche avviata la scrittura di MDR in memoria, alla locazione puntata da MAR.

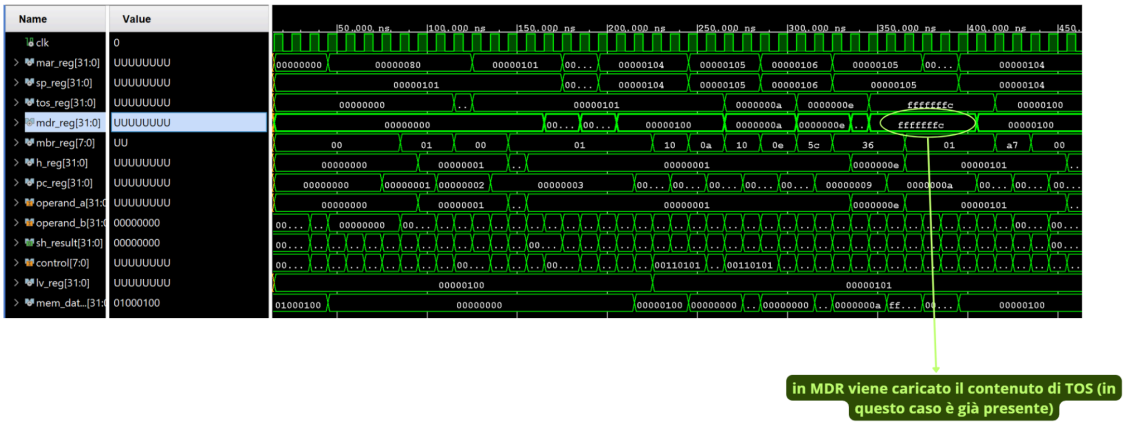



*Figura 9.8 Simulazione - Parte 6*

In *Figura 9.9* è possibile vedere che MAR e SP vengono caricati con il valore di SP-1.

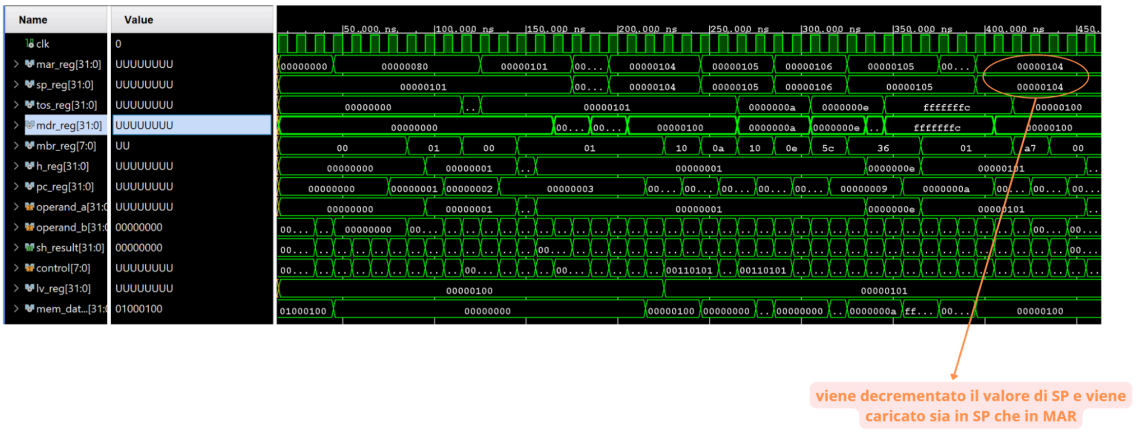



*Figura 9.9 Simulazione - Parte 7*

Infine, l'incremento del PC e l'aggiornamento di TOS è evidenziato in *Figura 9.10*.

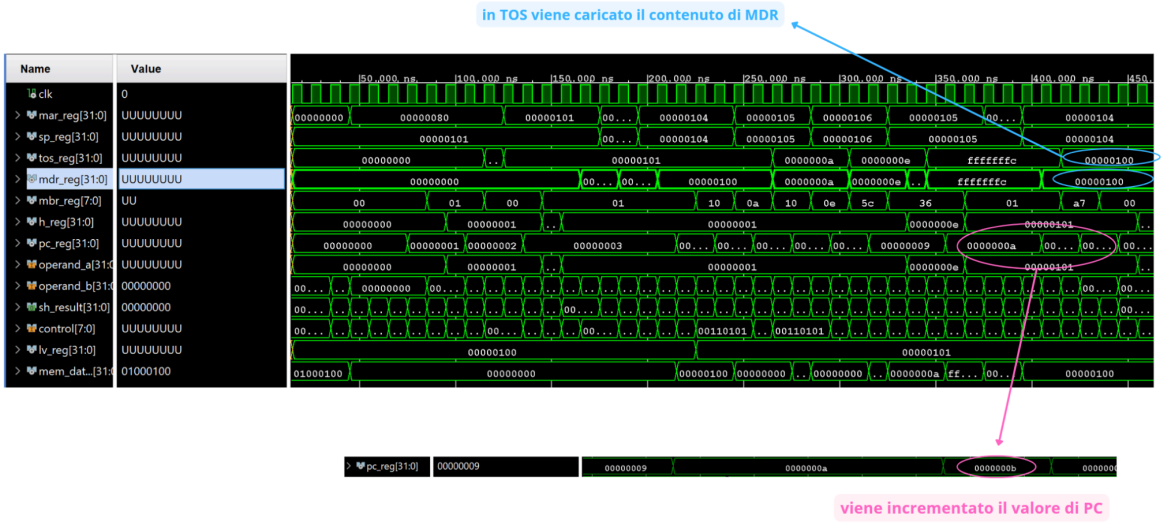



*Figura 9.10 Simulazione - Parte 8*

*Modifica della IAND*

L'implementazione della IAND è la seguente.

```
iand = 0x7E:
        MAR = SP = SP - 1; rd
        H = TOS
        MDR = TOS = MDR AND H; wr; goto main
```

Per modificare l'istruzione, è sufficiente editare il microprogramma in linguaggio MAL (ajvm.mal). La modifica effettuata è la seguente, e fa sì che l'operazione sia di OR piuttosto che di AND.

```
iand = 0x7E:
        MAR = SP = SP - 1; rd
        H = TOS
        TOS = MDR = MDR OR H; wr; goto main
```

Una volta effettuata la modifica, è necessario rigenerare la control store. È possibile farlo lanciando il seguente comando dalla directory build.

```
flavia@flavia-VirtualBox:~/esercitazione_mic/amic-0$ cmake --build build --target create_control_store
Built target assemble_microprogram
Built target create_control_store
```

Il programma in linguaggio ijvm (program.ajvm) è stato modificato nel seguente modo.

```
1   .main
2   .var
3   a
4   .endvar
5   BIPUSH 0xA
6   BIPUSH 0xE
7   IAND
8   ISTORE a
9   HALT
10  .endmethod
```

Per rigenerare la RAM è sufficiente lanciare, dalla directory build, il seguente comando:

```
flavia@flavia-VirtualBox:~/esercitazione_mic/amic-0$ cmake --build build --target create_ram
Built target assemble_program
Built target create_ram
```

Di seguito sono riportate le differenze tra la micro routine della IAND originaria e quella ottenuta in seguito alla modifica.

La IAND corrisponde alla locazione di memoria della control store 0x7E, ovvero alla 126esima parola.

Partendo dalla IAND originaria, la micro routine da eseguire è composta dalle seguenti tre microistruzioni:

```
126 => "00111111100000110110000001001010010 0",
127 => "01000000000000010100100000000000111",
128 => "00000011000000001100001000010100000 0",
```

In particolare:

| | NEXT | JAM | ALU | BUS C | MEM | BUS B |
|---|---|---|---|---|---|---|
| **126** | 001111111 | 000 | 00110110 | 000001001 | 010 | 0100 |
| **127** | 010000000 | 000 | 00010100 | 100000000 | 000 | 0111 |
| **128** | 000000110 | 000 | 00001100 | 001000010 | 100 | 0000 |

Di seguito è riportata la spiegazione dettagliata di ciascuna delle tre microistruzioni (126, 127 e 128).

La prima cosa che fa la ISUB (*MAR = SP = SP - 1; rd*) è decrementare lo stack pointer, per effettuare una lettura dalla memoria.

Questo primo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 110110, che equivale al decremento dell'operando B,
- BUS C: 000001001, indicando che il bus C scrive su SP e MAR,
- MEM: 010 (lettura dalla memoria),
- BUS B: 0100, il bus B è controllato da SP,
- Next_addr: 001111111, che equivale a 127, la locazione di memoria successiva,
- JAM: 000, nessun salto

La seconda cosa da fare (*H = TOS*) è far passare l'operando B invariato e scriverlo sul registro H (prendendolo dal registro TOS).

Questo secondo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 010100, che equivale al passaggio dell'operando B invariato;
- BUS C: 100000000, indicando che il bus C scrive su H,
- Mem: 000, nessuna operazione in memoria,
- BUS B: 0111, il bus B viene controllato da TOS,
- Next_addr: 010000000, che equivale a 128, la locazione di memoria successiva,
- JAM: 000, nessun salto

L'ultima cosa da fare è la AND vera e propria (*MDR = TOS = MDR AND H; wr; goto main*) tra il contenuto di MDR e il contenuto H e memorizzarlo in MDR stesso e nel TOS.

Questo ultimo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 001100, che equivale alla AND tra i due operandi,
- BUS C: 001000010, il bus C scrive su MDR e TOS,
- Mem: 100 (scrittura in memoria),
- BUS B: 0000, MDR controlla il bus B,
- Next_addr: 000000110, equivale a goto main, il microcodice per il caricamento della prossima istruzione,
- JAM: 000, nessun salto

Passando alla **IAND modificata**, la micro routine da eseguire è composta dalle seguenti tre microistruzioni:

```
126 => "00111111100000110110000001001010010 0",
127 => "01000000000000010100100000000000111",
128 => "00000011000000011100001000010100000 0",
```

In particolare: 1000000

| | NEXT | JAM | ALU | BUS C | MEM | BUS B |
|---|---|---|---|---|---|---|
| **126** | 001111111 | 000 | 00110110 | 000001001 | 010 | 0100 |
| **127** | 010000000 | 000 | 00010100 | 100000000 | 000 | 0111 |
| **128** | 000000110 | 000 | 00011100 | 001000010 | 100 | 0000 |

L'unica differenza rispetto all'implementazione originaria è che la microistruzione 128 pone in MDR e in TOS l'OR tra MDR e H. Questo passo si effettua attraverso i campi della microistruzione pari a:

- ALU: 101100, che equivale alla OR tra i due operandi,
- BUS C: 001000010, il bus C scrive su MDR e su TOS,
- Mem: 100 (scrittura in memoria),
- BUS B: 0000, MDR controlla il bus B,
- Next_addr: 000000110, equivale a goto main, il microcodice per il caricamento della prossima istruzione,
- JAM: 000, nessun salto

Nelle *Figure 9.11 - 9.14* è riportato il risultato della simulazione della IAND modificata.

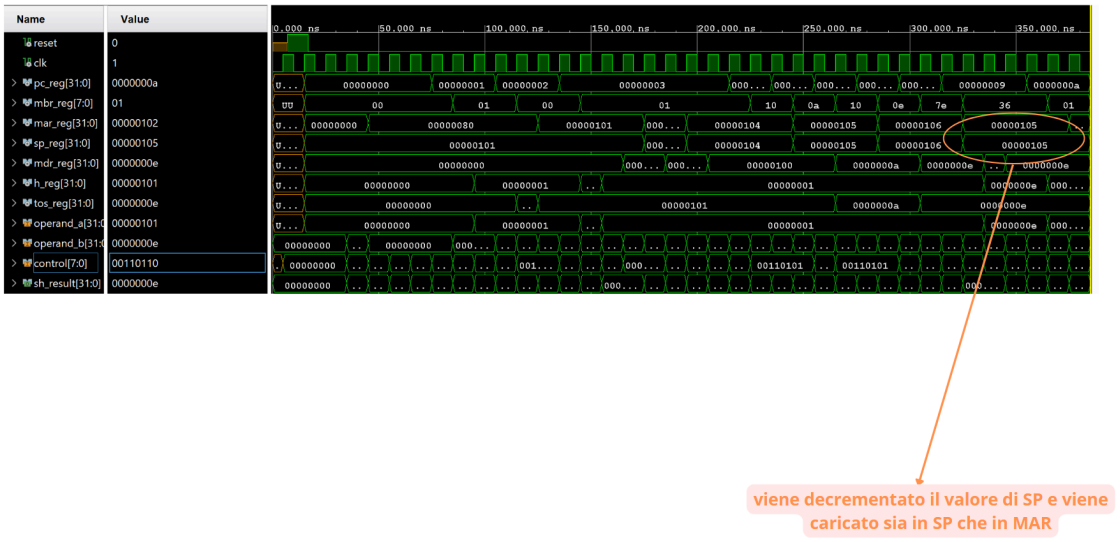



*Figura 9.11 Simulazione IAND modificata - Parte 1*

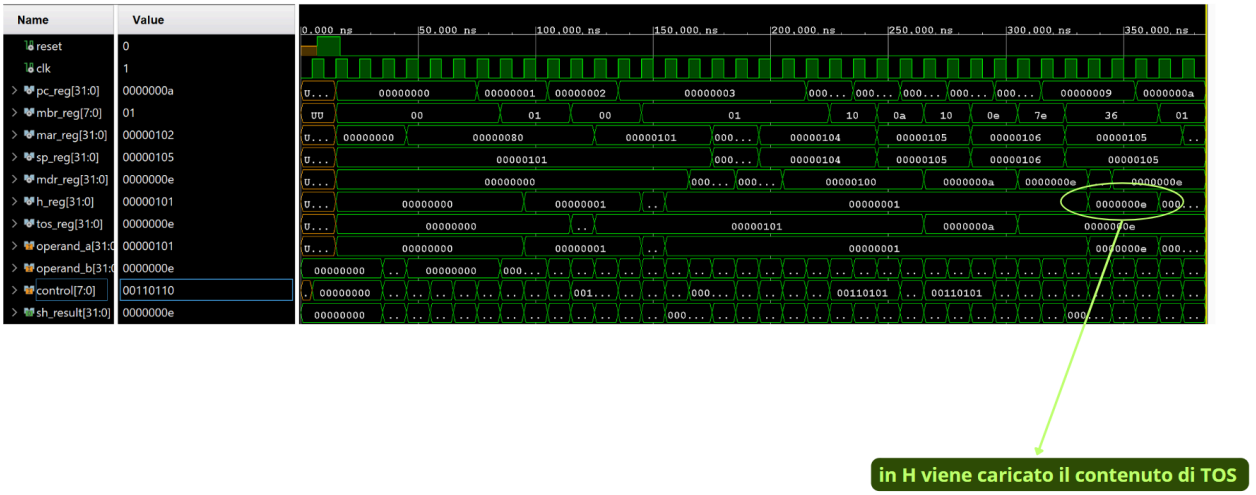



*Figura 9.12 Simulazione IAND modificata - Parte 2*

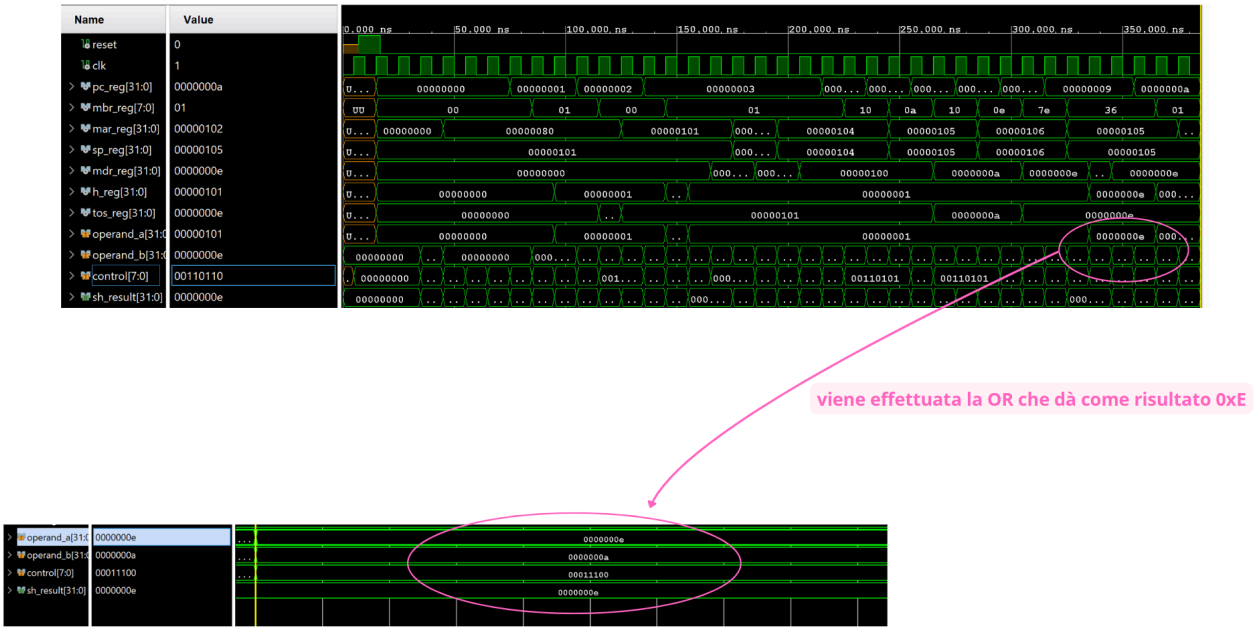



*Figura 9.13 Simulazione IAND modificata - Parte 3*




*Figura 9.14 Simulazione IAND modificata - Parte 4*

# Capitolo 6: Interfaccia seriale

## Esercizio 10

Partendo dall'implementazione fornita dalla Digilent di un dispositivo UART-RS232 (componente RS232RefComp.vhd), progettare, implementare e simulare in VHDL un sistema composto da 2 unità A e B che condividono lo stesso segnale di clock e comunicano tra loro mediante interfaccia seriale. Il sistema A contiene una ROM di 8 locazioni da 1 byte ciascuno, un contatore CONT_A per scandire le locazioni della ROM e una UART_A, mentre il sistema B contiene una memoria MEM di 8 locazioni da 1 byte ciascuno, un contatore CONT_B per scandire le locazioni della MEM e una UART_B. Quando un segnale WR viene asserito nell'unità A, viene prelevato un byte dalla ROM e inviato all'unità B, che dovrà riceverlo e salvarlo in MEM.

### *Progetto e architettura*

Il sistema composto dalle 2 unità è stato realizzato con approccio strutturale. Come illustrato in *Figura 10.1*, le due unità condividono il clock e il reset, e il pin **TXD** della UART di A è stato collegato con il pin **RXD** della UART di B.

Si è scelto inoltre di implementare un **handshaking** prima della trasmissione del dato di 8 bit tramite i due segnali **rts** (*request to send*, inviato dal nodo A al nodo B per indicare che il nodo A è pronto a trasmettere un messaggio) e **cts** (*clear to send*, inviato dal nodo B al nodo A per indicare che il nodo B è pronto a ricevere il messaggio).

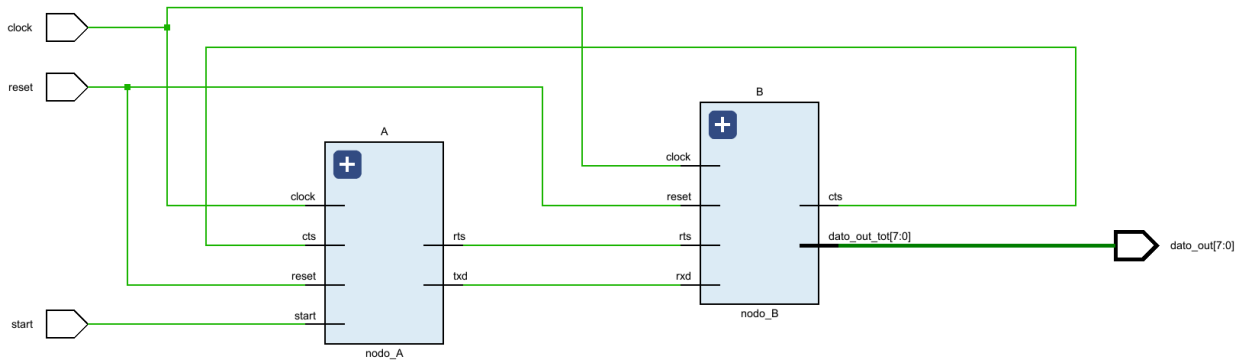



*Figura 10.1 Schematic del sistema complessivo*

Il **disegno architetturale** delle due unità è illustrato in *Figura 10.2* e in *Figura 10.3*.

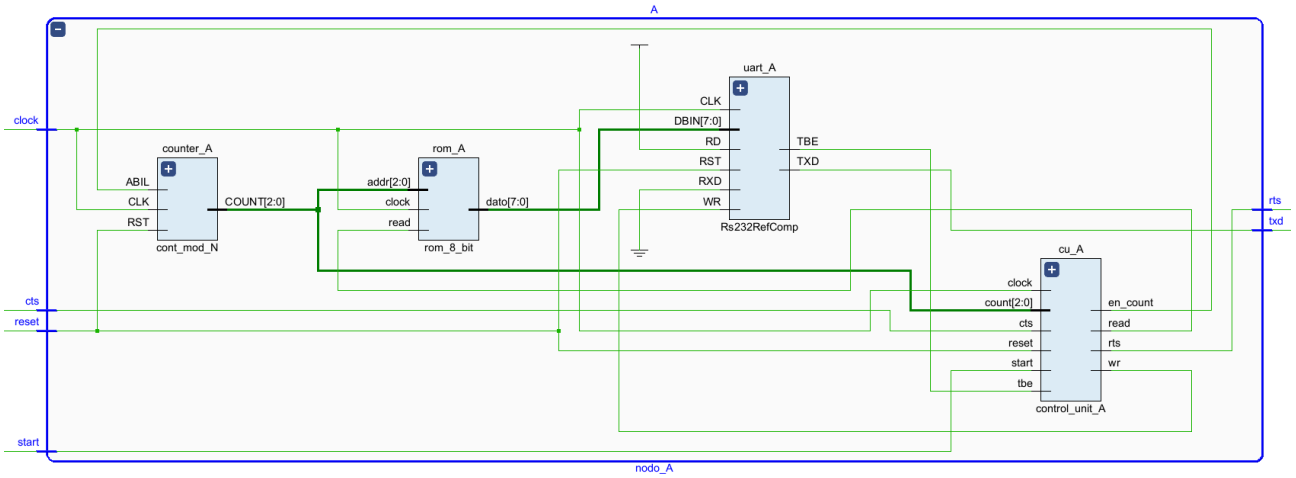



*Figura 10.2 Schematic dell'unità A*

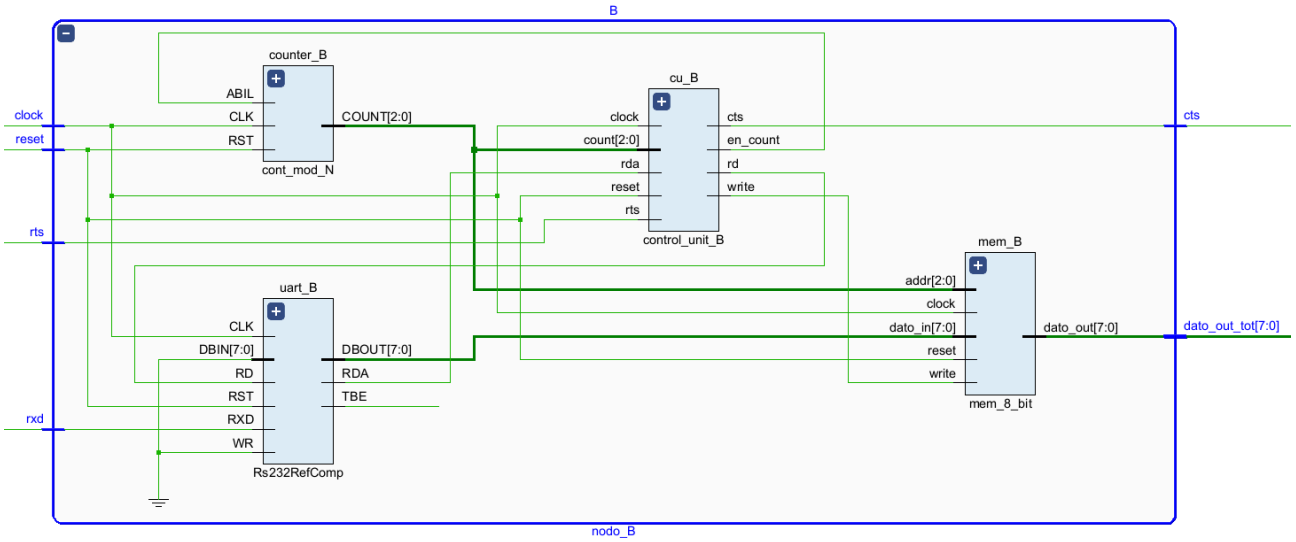



*Figura 10.3 Schematic dell'unità B*

Le **unità di controllo** di A e B sono, invece, rappresentate rispettivamente in *Figura 10.4* e in *Figura 10.5*.

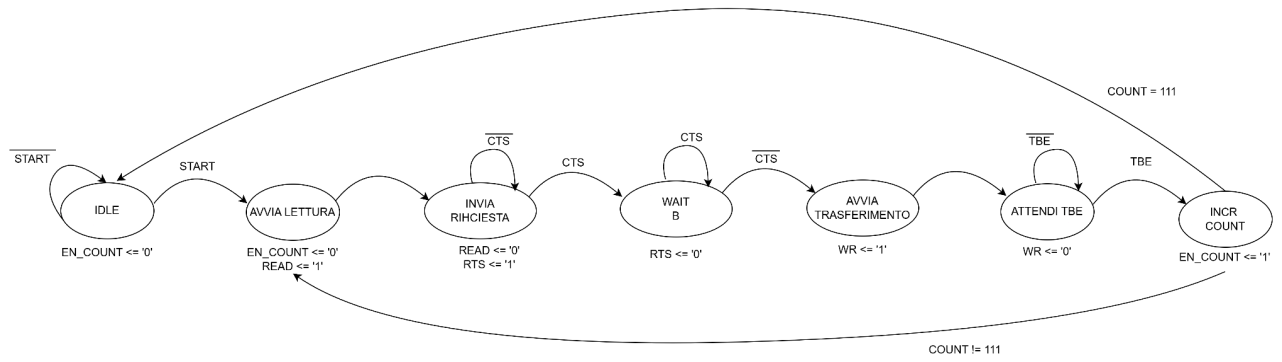



*Figura 10.4 Automa della control unit di A*

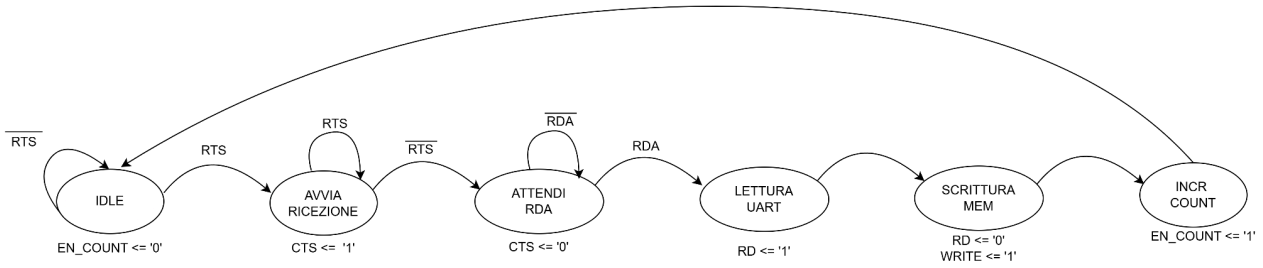



*Figura 10.5 Automa della control unit di B*

### *Implementazione*

L'implementazione del **top module** è la seguente.

```
entity sis_tot is
Port ( reset : in std_logic;
       clock : in std_logic;
       start : in std_logic;
       dato_out : out std_logic_vector(7 downto 0)
    );
end sis_tot;

architecture Behavioral of sis_tot is

component nodo_A is
    Port ( reset : in std_logic;
           clock : in std_logic;
           start : in std_logic;
           cts: in std_logic;
           rts : out std_logic;
           txd : out std_logic
    );
end component nodo_A;

component nodo_B is
Port ( reset : in std_logic;
       clock : in std_logic;
       rxd : in std_logic;
       rts: in std_logic;
       cts : out std_logic;
       dato_out_tot : out std_logic_vector(7 downto 0)
    );
end component nodo_B;

signal txd_rxd : std_logic;
signal cts_tmp : std_logic;
signal rts_tmp : std_logic;

begin

    A: nodo_A
    port map ( reset => reset,
               clock => clock,
               start => start,
               cts => cts_tmp,
               rts => rts_tmp,
```

```
            txd => txd_rxd
          );

    B: nodo_B
    port map ( reset => reset,
               clock => clock,
               rxd => txd_rxd,
               rts => rts_tmp,
               cts => cts_tmp,
               dato_out_tot => dato_out
             );

end Behavioral;
```

Di seguito è riportata l'implementazione del **nodo A**.

```
entity nodo_A is
    Port ( reset : in std_logic;
           clock : in std_logic;
           start : in std_logic;
           cts: in std_logic;
           rts : out std_logic;
           txd : out std_logic
    );
end nodo_A;

architecture Behavioral of nodo_A is

component Rs232RefComp is
    Port (
            TXD     : out std_logic := '1';
            RXD     : in  std_logic;
            CLK     : in  std_logic; -- Master Clock
            DBIN    : in  std_logic_vector (7 downto 0); -- Data Bus in
            DBOUT : out std_logic_vector (7 downto 0);   -- Data Bus out
            RDA     : inout std_logic;              -- Read Data Available(1 quando
                                                       il dato è disponibile nel
                                                       registro rdReg)
            TBE     : inout std_logic   := '1';     -- Transfer Bus Empty(1 quando il
                                                       dato da inviare è stato caricato
                                                       nello shift register)
            RD            : in  std_logic;    -- Read Strobe(se 1 significa "leggi" →
                                                  fa abbassare RDA)
            WR            : in  std_logic;    -- Write Strobe(se 1 significa "scrivi"
                                                  --> fa abbassare TBE)
            PE            : out std_logic;    -- Parity Error Flag
            FE            : out std_logic;    -- Frame Error Flag
            OE            : out std_logic;    -- Overwrite Error Flag
            RST           : in  std_logic     := '0'); --Master Reset
end component Rs232RefComp;

component cont_mod_N is
    Generic ( modulo : integer := 16;
              N : integer := 4 -- numero di bit necessari
             );
    Port ( RST : in STD_LOGIC;
```

```
           CLK : in STD_LOGIC;
           COUNT : out STD_LOGIC_VECTOR (N-1 downto 0);
           UMAX : out STD_LOGIC;
           ABIL : in STD_LOGIC
          );
end component cont_mod_N;

component rom_8_bit is
    Port ( clock : in STD_LOGIC;
           read : in STD_LOGIC;
           addr : in STD_LOGIC_VECTOR(2 downto 0);  -- supponeno di avere N=8 locazioni
                                                    usiamo 3 bit per rappresentare gli
                                                    indirizzi
           dato : out STD_LOGIC_VECTOR(7 downto 0));
end component rom_8_bit;

component control_unit_A is
    Port ( reset : in std_logic;
           clock : in std_logic;
           start : in std_logic;
           count : in std_logic_vector(2 downto 0);
           tbe : in std_logic;
           cts : in std_logic;
           rts : out std_logic;
           wr : out std_logic;
           read : out std_logic;
           en_count : out std_logic
    );
end component control_unit_A;

signal rxd_tmp : std_logic;
signal bus_input : std_logic_vector(7 downto 0);
signal bus_output : std_logic_vector(7 downto 0);
signal rda_tmp : std_logic;
signal tbe_tmp : std_logic;
signal rd_tmp : std_logic;
signal wr_tmp : std_logic;
signal parity : std_logic;
signal frame : std_logic;
signal overwrite : std_logic;
signal count_tmp : std_logic_vector(2 downto 0);
signal umax_tmp : std_logic;
signal en_count_tmp : std_logic;
signal read_tmp : std_logic;

begin

    uart_A: Rs232RefComp
    port map ( TXD  => txd,
               RXD  => '0',
               CLK  => clock,
               DBIN => bus_input,
               --DBOUT => bus_output,
               --RDA      => rda_tmp,
               TBE  => tbe_tmp,
               RD => '1',
```

```
            WR => wr_tmp,
            --PE => parity,
            --FE => frame,
            --OE => overwrite,
            RST => reset );

    counter_A: cont_mod_N
    generic map ( modulo => 8,
                  N => 3 )
    port map ( RST => reset,
               CLK => clock,
               COUNT => count_tmp,
               UMAX => umax_tmp,
               ABIL => en_count_tmp
              );

    rom_A: rom_8_bit
    port map ( clock => clock,
               read => read_tmp,
               addr => count_tmp,
               dato => bus_input);

    cu_A: control_unit_A
    port map ( reset => reset,
               clock => clock,
               start => start,
               count => count_tmp,
               tbe => tbe_tmp,
               cts => cts,
               rts => rts,
               wr => wr_tmp,
               read => read_tmp,
               en_count => en_count_tmp
             );

end Behavioral;
```

La **control unit dell'unità A**, che implementa l'automa in *Figura 10.4*, è la seguente.

```
entity control_unit_A is
    Port ( reset : in std_logic;
           clock : in std_logic;
           start : in std_logic;
           count : in std_logic_vector(2 downto 0);
           tbe : in std_logic;
           cts : in std_logic;
           rts : out std_logic;
           wr : out std_logic;
           read : out std_logic;
           en_count : out std_logic
    );
end control_unit_A;
```

```
architecture Behavioral of control_unit_A is

type state is (IDLE, AVVIA_LETTURA, INVIA_RICHIESTA, WAIT_B, AVVIA_TRASFERIMENTO,
ATTENDI_TBE, INCR_COUNT);
signal current_state, next_state : state := IDLE;

begin

    cambia_stato: process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                current_state <= IDLE;
            else
                current_state <= next_state;
            end if;
        end if;
    end process;

    trasmissione : process(current_state, start, tbe, cts)
    begin

        case current_state is
            when IDLE =>
                en_count <= '0';
                if start = '0' then
                    next_state <= IDLE;
                else
                    next_state <= AVVIA_LETTURA;
                end if;

             when AVVIA_LETTURA =>
                en_count <= '0';
                read <= '1';
                next_state <= INVIA_RICHIESTA;

            when INVIA_RICHIESTA =>
                read <= '0';
                rts <= '1';
                if cts = '0' then
                    next_state <= INVIA_RICHIESTA;
                else
                    next_state <= WAIT_B;
                end if;

            when WAIT_B =>
                rts <= '0';
                if cts = '1' then
                    next_state <= WAIT_B;
                else
                    next_state <= AVVIA_TRASFERIMENTO;
                end if;

            when AVVIA_TRASFERIMENTO =>
                wr <= '1';
                next_state <= ATTENDI_TBE;
```

```
            when ATTENDI_TBE =>
                wr <= '0';
                if tbe = '1' then
                    next_state <= INCR_COUNT;
                else
                    next_state <= ATTENDI_TBE;
                end if;

            when INCR_COUNT =>
                en_count<='1';
                if count="111" then
                    next_state <= IDLE;
                 else
                    next_state <= AVVIA_LETTURA;
                end if;

           end case;
     end process;

end Behavioral;
```

L'implementazione del **contatore** è riportata in Appendice.

La **ROM di A** è implementata come segue.

```
entity rom_8_bit is
    Port ( clock : in STD_LOGIC;
           read : in STD_LOGIC;
           addr : in STD_LOGIC_VECTOR(2 downto 0);
           dato : out STD_LOGIC_VECTOR(7 downto 0));
end rom_8_bit;

architecture Behavioral of rom_8_bit is

-- definisco il tipo memory come un array costituito da 16 array ciascuno da 8 elementi
type memory is array (0 to 7) of std_logic_vector (7 downto 0);

-- definisco la costante rom16_8 che è di tipo memory -> ogni cella è inizializzata con
un valore esadecimale
constant rom : memory := (
    x"11",   -- 00010001
    x"12",   -- 00010010
    x"14",   -- 00010100
    x"18",   -- 00011000
    x"21",   -- 00100001
    x"22",   -- 00100010
    x"24",   -- 00100100
    x"28"    -- 00101000
);

begin

    process(clock)
    begin
        if rising_edge(clock) then
            if read <= '1' then
```

```
                dato <= rom(to_integer(unsigned(addr)));
            else
                dato <= (others => '-');
            end if;
        end if;
    end process;

end Behavioral;
```

Il **nodo B** è implementato come segue.

```
entity nodo_B is
Port ( reset : in std_logic;
       clock : in std_logic;
       rxd : in std_logic;
       rts: in std_logic;
       cts : out std_logic;
       dato_out_tot : out std_logic_vector(7 downto 0)
    );
end nodo_B;

architecture Behavioral of nodo_B is

component Rs232RefComp is
    Port (
            TXD     : out std_logic := '1';
            RXD     : in  std_logic;
            CLK     : in  std_logic; -- Master Clock
            DBIN    : in  std_logic_vector (7 downto 0); -- Data Bus in
            DBOUT : out std_logic_vector (7 downto 0);   -- Data Bus out
            RDA     : inout std_logic;              -- Read Data Available(1 quando
                                                       il dato è disponibile nel
                                                       registro rdReg)
            TBE     : inout std_logic   := '1';     -- Transfer Bus Empty(1 quando il
                                                       dato da inviare è stato caricato
                                                       nello shift register)
            RD            : in  std_logic;    -- Read Strobe(se 1 significa "leggi" →
                                                  fa abbassare RDA)
            WR            : in  std_logic;    -- Write Strobe(se 1 significa "scrivi"
                                                  --> fa abbassare TBE)
            PE            : out std_logic;    -- Parity Error Flag
            FE            : out std_logic;    -- Frame Error Flag
            OE            : out std_logic;    -- Overwrite Error Flag
            RST           : in  std_logic     := '0'); --Master Reset
end component Rs232RefComp;

component cont_mod_N is
    Generic ( modulo : integer := 16;
              N : integer := 4 -- numero di bit necessari
             );
    Port ( RST : in STD_LOGIC;
           CLK : in STD_LOGIC;
           COUNT : out STD_LOGIC_VECTOR (N-1 downto 0);
           UMAX : out STD_LOGIC;
           ABIL : in STD_LOGIC
          );
```

```
end component cont_mod_N;

component mem_8_bit is
Port ( clock : in STD_LOGIC;
       reset : in STD_LOGIC;
       write : in STD_LOGIC;
       dato_in : in STD_LOGIC_VECTOR (7 downto 0);
       addr : in STD_LOGIC_VECTOR (2 downto 0);
       dato_out : out STD_LOGIC_VECTOR (7 downto 0));
end component mem_8_bit;

component control_unit_B is
    Port ( reset : in std_logic;
           clock : in std_logic;
           count : in std_logic_vector(2 downto 0);
           rda : in std_logic;
           rts : in std_logic;
           cts : out std_logic;
           rd : out std_logic;
           write : out std_logic;
           en_count : out std_logic
    );
end component control_unit_B;

signal txd_tmp : std_logic;
signal bus_input : std_logic_vector(7 downto 0);
signal bus_output : std_logic_vector(7 downto 0);
signal rda_tmp : std_logic;
signal tbe_tmp : std_logic;
signal rd_tmp : std_logic;
signal wr_tmp : std_logic;
signal parity : std_logic;
signal frame : std_logic;
signal overwrite : std_logic;
signal count_tmp : std_logic_vector(2 downto 0);
signal umax_tmp : std_logic;
signal en_count_tmp : std_logic;
signal write_tmp : std_logic;

begin

    uart_B: Rs232RefComp
    port map ( --TXD      => txd_tmp,
               RXD  => rxd,
               CLK  => clock,
               DBIN => "00000000",
               DBOUT => bus_output,
               RDA  => rda_tmp,
               --TBE      => tbe_tmp,
               RD => rd_tmp,
               WR => '0',
               --PE => parity,
               --FE => frame,
               --OE => overwrite,
               RST => reset );
```

```
    counter_B: cont_mod_N
    generic map ( modulo => 8,
                  N => 3 )
    port map ( RST => reset,
               CLK => clock,
               COUNT => count_tmp,
               UMAX => umax_tmp,
               ABIL => en_count_tmp
              );

    mem_B: mem_8_bit
    port map ( clock => clock,
               reset => reset,
               write => write_tmp,
               dato_in => bus_output,
               addr => count_tmp,
               dato_out => dato_out_tot);

    cu_B: control_unit_B
    port map ( reset => reset,
               clock => clock,
               count => count_tmp,
               rda => rda_tmp,
               rts => rts,
               cts => cts,
               rd => rd_tmp,
               write => write_tmp,
               en_count => en_count_tmp );

end Behavioral;
```

La **control unit di B**, che implementa l'automa in *Figura 10.5*, è riportata di seguito.

```
entity control_unit_B is
    Port ( reset : in std_logic;
           clock : in std_logic;
           count : in std_logic_vector(2 downto 0);
           rda : in std_logic;
           rts : in std_logic;
           cts : out std_logic;
           rd : out std_logic;
           write : out std_logic;
           en_count : out std_logic
    );
end control_unit_B;

architecture Behavioral of control_unit_B is

type state is (IDLE, AVVIA_RICEZIONE, ATTENDI_RDA, LETTURA_UART, SCRITTURA_MEM,
INCR_COUNT);
signal current_state, next_state : state := IDLE;

begin

    cambia_stato: process(clock)
    begin
```

```
        if rising_edge(clock) then
            if reset = '1' then
                current_state <= IDLE;
            else
                current_state <= next_state;
            end if;
        end if;
    end process;

    ricezione : process(current_state, rda, rts)
    begin

        case current_state is

            when IDLE =>
                en_count <= '0';
                if rts = '1' then
                    next_state <= AVVIA_RICEZIONE;
                else
                    next_state <= IDLE;
                end if;

            when AVVIA_RICEZIONE =>
                cts <= '1';
                if rts = '1' then
                    next_state <= AVVIA_RICEZIONE;
                else
                    next_state <= ATTENDI_RDA;
                end if;

           when ATTENDI_RDA=>
                cts <= '0';
                if rda = '0' then
                    next_state <= ATTENDI_RDA;
                else
                    next_state <= LETTURA_UART;
                end if;

           when LETTURA_UART =>
                rd <= '1';
                next_state <= SCRITTURA_MEM;

           when SCRITTURA_MEM =>
                write <= '1';
                rd <= '0';
                next_state <= INCR_COUNT;

           when INCR_COUNT =>
                write <= '0';
                en_count <= '1';
                next_state <= IDLE;

           end case;
     end process;

end Behavioral;
```

La **memoria** è implementata come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mem_8_bit is
    Port ( clock : in STD_LOGIC;
           reset : in STD_LOGIC;
           write : in STD_LOGIC;
           dato_in : in STD_LOGIC_VECTOR (7 downto 0);
           addr : in STD_LOGIC_VECTOR (2 downto 0);
           dato_out : out STD_LOGIC_VECTOR (7 downto 0));
end mem_8_bit;

architecture Behavioral of mem_8_bit is

-- definisco il tipo memory come un array costituito da 8 array ciascuno da 8 elementi
type memory is array (0 to 7) of std_logic_vector (7 downto 0);

-- definisco il segnale mem che è di tipo memory -> ogni cella è inizializzata a 0
signal mem : memory := (others => (others => '0'));

begin

    process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                mem <= (others => (others => '0'));
            elsif write = '1' then
                mem(to_integer(unsigned(addr))) <= dato_in;
            end if;
        end if;
    end process;

    dato_out <= mem(to_integer(unsigned(addr)));

end Behavioral;
```

### *Simulazione*

Il **testbench** utilizzato per la simulazione del progetto è il seguente:

```
library ieee;
use ieee.std_logic_1164.all;

entity tb_sis_tot is
end tb_sis_tot;

architecture tb of tb_sis_tot is

    component sis_tot
        port (reset    : in std_logic;
              clock    : in std_logic;
              start    : in std_logic;
```

```
            dato_out : out std_logic_vector (7 downto 0));
    end component;

    signal reset    : std_logic;
    signal clock    : std_logic;
    signal start    : std_logic;
    signal dato_out : std_logic_vector (7 downto 0);

    constant clk_period : time := 0.5 ns;
    signal stop_the_clock: boolean;

begin

    dut : sis_tot
    port map (reset    => reset,
              clock    => clock,
              start    => start,
              dato_out => dato_out);

    stimuli : process
    begin
        -- ***EDIT*** Adapt initialization as needed
        reset <= '1';
        start <= '0';

        -- ***EDIT*** Add stimuli here

        wait for 10 ns;
        reset <= '0';

        wait for 10 ns;
        start <= '1';

        wait for 10 ns;
        start <= '0';

        wait;
    end process;

    -- Clock Generation Process
    clocking: process
     begin
        while not stop_the_clock loop
            clock <= '0', '1' after clk_period / 2;
            wait for clk_period;
        end loop;
        wait for 10 ns;
     end process;

end tb;

-- Configuration block below is required by some simulators. Usually no need to edit.

configuration cfg_tb_sis_tot of tb_sis_tot is
    for tb
    end for;
```

```
end cfg_tb_sis_tot;
```

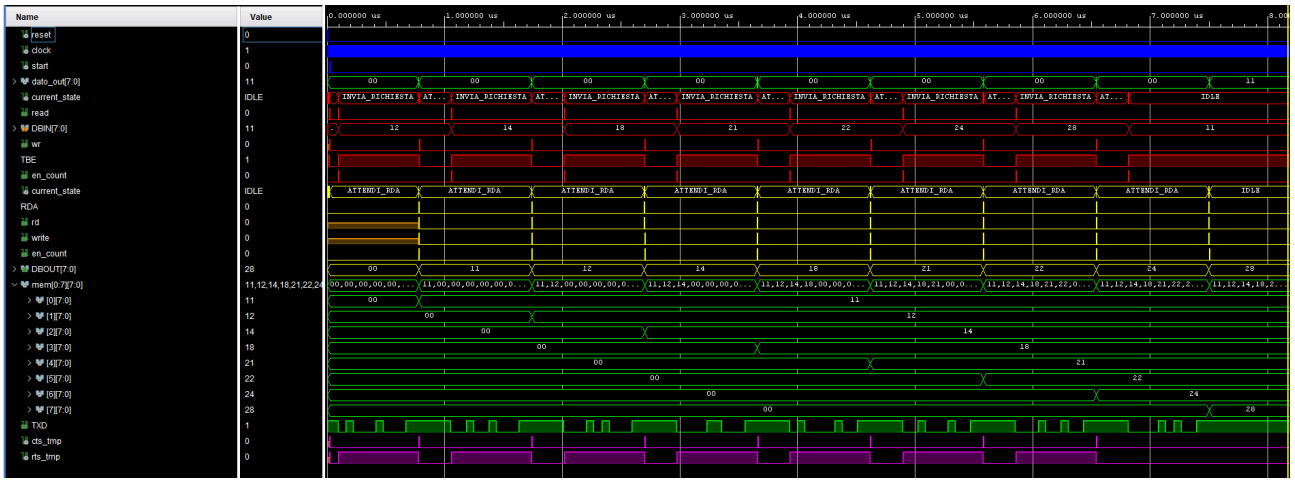


*Figura 10.6 Risultato della simulazione*

All'interno della simulazione i segnali sono raggruppati per colori:

- in **rosso** i segnali relativi al **nodo A**
- in **giallo** i segnali relativi al **nodo B**
- in **verde** è rappresentato il contenuto della memoria **mem** nel nodo B
- in **rosa** i segnali relativi all'**handshaking** (rts e cts)

Osserviamo in particolare cosa avviene durante il trasferimento di un byte.

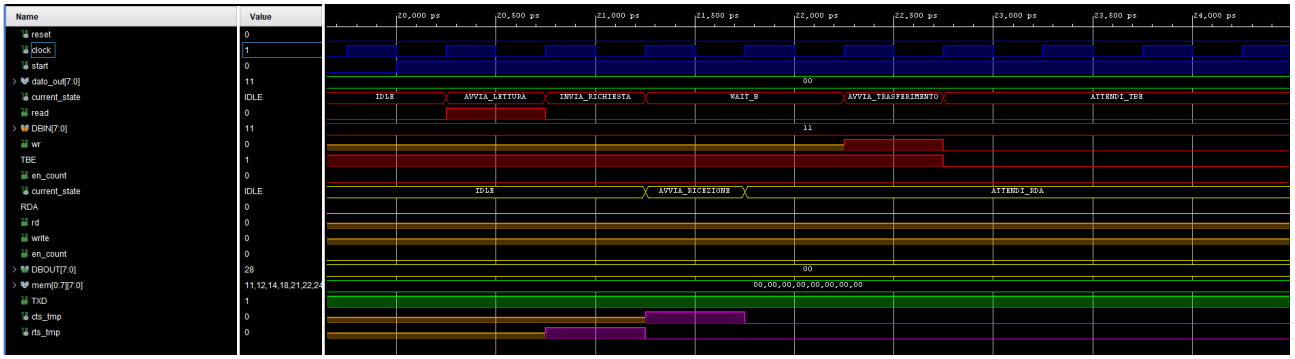


*Figura 10.7 Prima parte della trasmissione*

- A parte dallo stato di *idle* e, con l'arrivo del segnale di *start* si sposta nello stato di *avvia_lettura*.
- In *avvia_lettura* il nodo A dà il segnale di *read* della rom e legge il valore nel primo registro, ovvero il valore esadecimale 11.
- Dopo aver dato il segnale di *read*, A si sposta nello stato *invia_richiesta* in cui alza il segnale *rts* e si pone in attesa che il nodo B alzi il segnale di *rts*.
- Nel frattempo B, che era nello stato di *idle* in attesa del segnale *rts*, vede *rts* alto e si sposta nello stato di *avvia_ricezione*, dove alza il segnale *cts*. Si pone quindi in attesa che *rts* venga abbassato da A.
- Non appena A vede *cts*, abbassa *rts* e si pone in attesa che anche *cts* venga abbassato; quando B abbassa il segnale *cts*, l'handshaking è concluso e A, che può finalmente inviare il byte, si sposta

nello stato *avvia_trasferimento* dove alza il segnale *WR* per avviare la scrittura all'interno del buffer della uart.

- Nel frattempo, il nodo B ha abbassato *cts* e si è posto nello stato *attendi_rda* in attesa che la flag *RDA* venga alzata indicando che il dato nel buffer della UART è pronto per essere letto.
- A questo punto, A si pone in attesa che la flag *TBE* venga rialzata indicando quindi che il byte è stato scritto nello shift register e che si può quindi caricare un nuovo dato nel buffer.

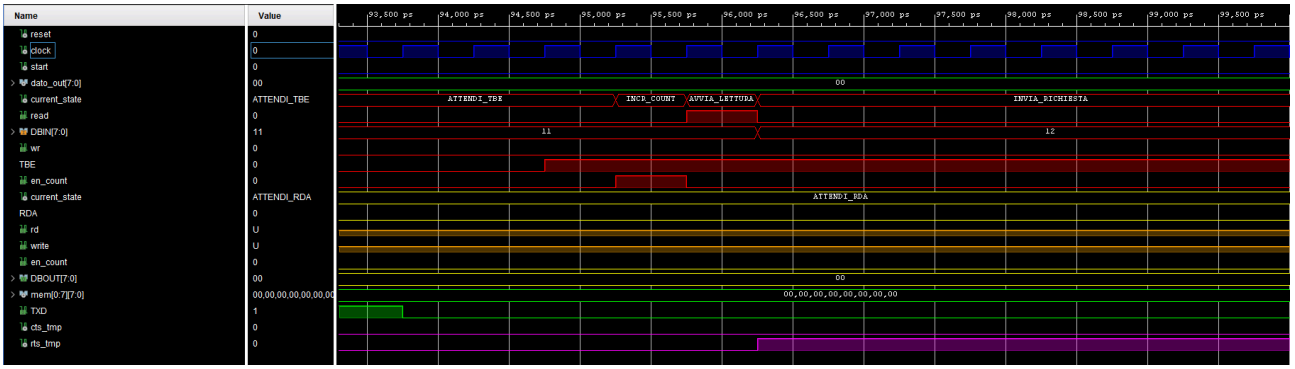



*Figura 10.8 Seconda parte della trasmissione*

- Non appena *TBE* si alza, il nodo A può scrivere nel buffer della UART un nuovo byte, quindi si sposta nello stato *incr_count* dove incrementa il conteggio dei byte inviati e, siccome ci sono ancora altre locazioni da leggere nella rom, ritorna nello stato *avvia_lettura* per cominciare a lavorare di nuovo.
- Osserviamo che nel frattempo il nodo B si trova ancora nello stato *attenda_rda*, in attesa che la ricezione seriale del byte termini. Quindi, seppure il nodo A si è già portato nello stato *invia_richiesta*, dovrà comunque attendere che il nodo B abbia terminato la ricezione e che sia quindi pronto a ricevere un nuovo byte, prima di poterlo scrivere nel buffer della UART.

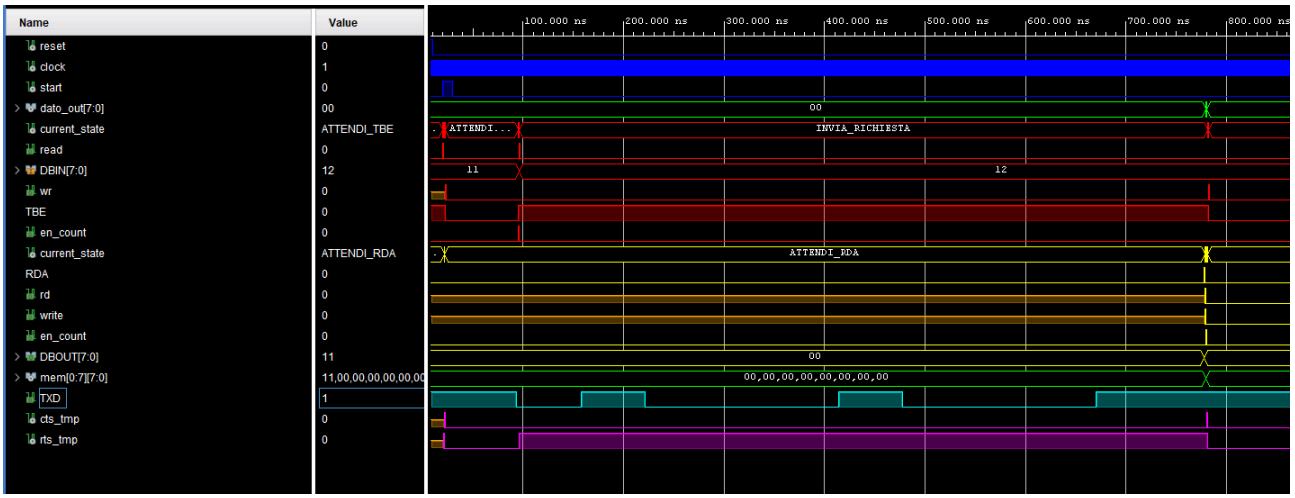



*Figura 10.9 Trasmissione seriale*

Possiamo osservare in questa figura la trasmissione seriale del valore esadecimale 11 inoltrato da A a B. Mentre B si trova nello stato *attendi_rda* sta avvenendo la trasmissione seriale. Vediamo, evidenziata in azzurro, la linea TXD-RXD.

Possiamo notare la linea a riposo inizialmente alta che viene successivamente abbassata per inoltrare il bit di start.

A questo punto viene inviato il dato 11 che in binario corrisponde a 00010001. Il byte viene inoltrato in particolare nell'ordine 10001000.

Alla fine della trasmissione del byte, la linea di trasmissione torna nuovamente a riposo (valore logico alto).




*Figura 10.10 Terza parte della trasmissione*

- Quando il segnale *RDA* si alza ad indicare che il dato è pronto per essere letto nel buffer della UART del nodo B, quest'ultimo si sposta nello stato *lettura_uart* all'interno del quale alza il segnale di *RD* per poter leggere il buffer.
- Successivamente si sposta nello stato *scrittura_mem* dove alza il segnale di *write* della memoria così da poter salvare il dato ricevuto in memoria.
- Infine, si sposta nello stato *incr_count* dove incrementa il conteggio dei byte ricevuti e poi torna nello stato di *idle* in attesa di una nuova richiesta di trasmissione da parte del nodo A.

# Capitolo 7: Switch Multistadio

## Esercizio 11.1

Progettare ed implementare in VHDL uno switch multistadio secondo il modello omega network. Lo switch deve consentire lo scambio di messaggi di 2 bit ciascuno da un nodo sorgente a un nodo destinazione in una rete con 4 nodi, implementando uno schema a priorità fissa fra i nodi (es. nodo 1 più prioritario, con priorità decrescenti fino al nodo 4).

### *Progetto e architettura*

La rete di interconnessione che si vuole realizzare è di questo tipo:

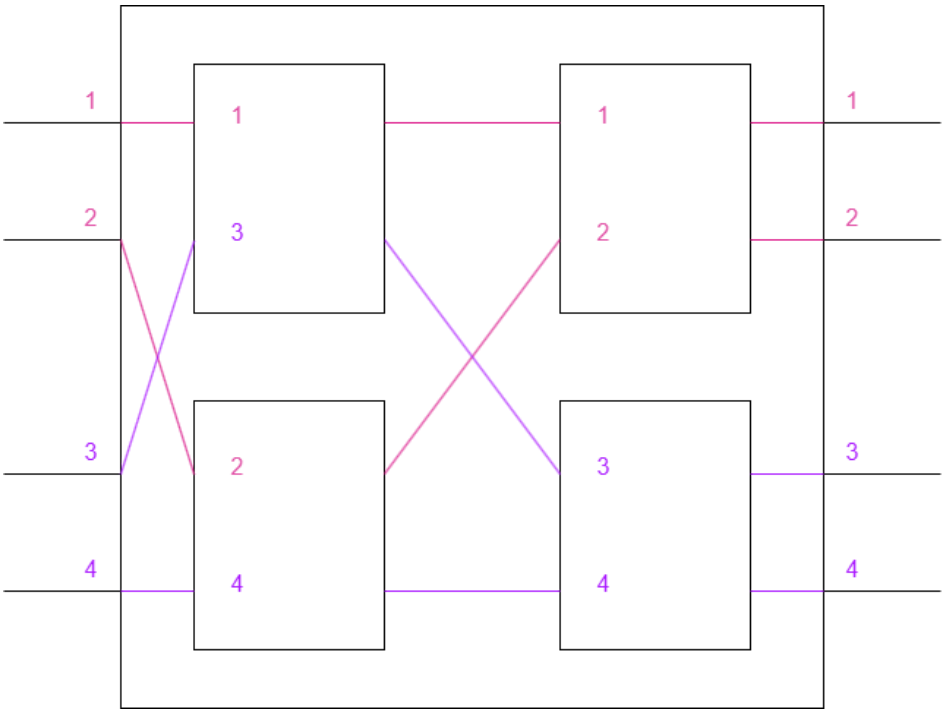



*Figura 11.1 Rete di interconnessione*

Il singolo switch 2x2 nella rete è implementato nel modo seguente:

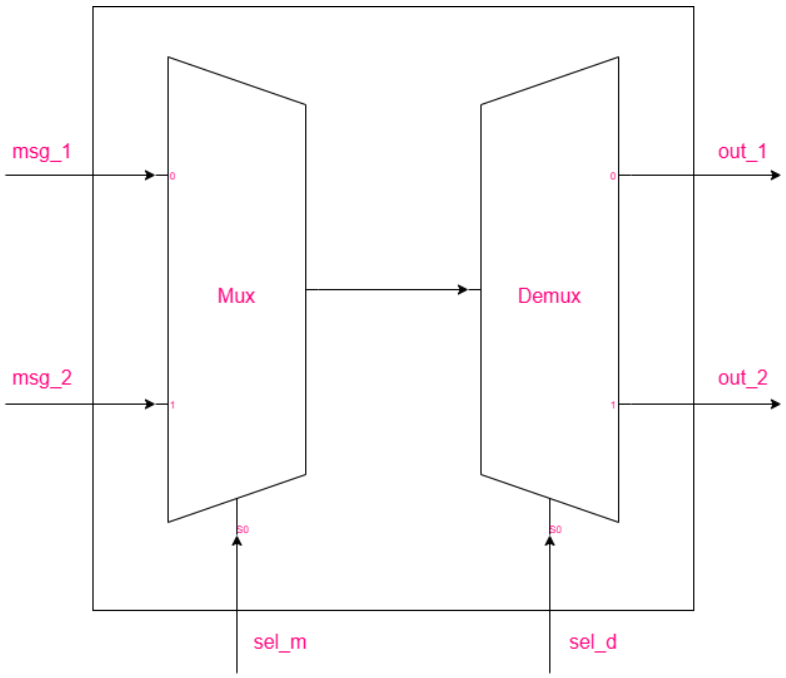



*Figura 11.2 Switch elementare*

In particolare, la rete 4x4 è la seguente, dotata di tutti i segnali di ingresso, uscita e di segnali intermedi:

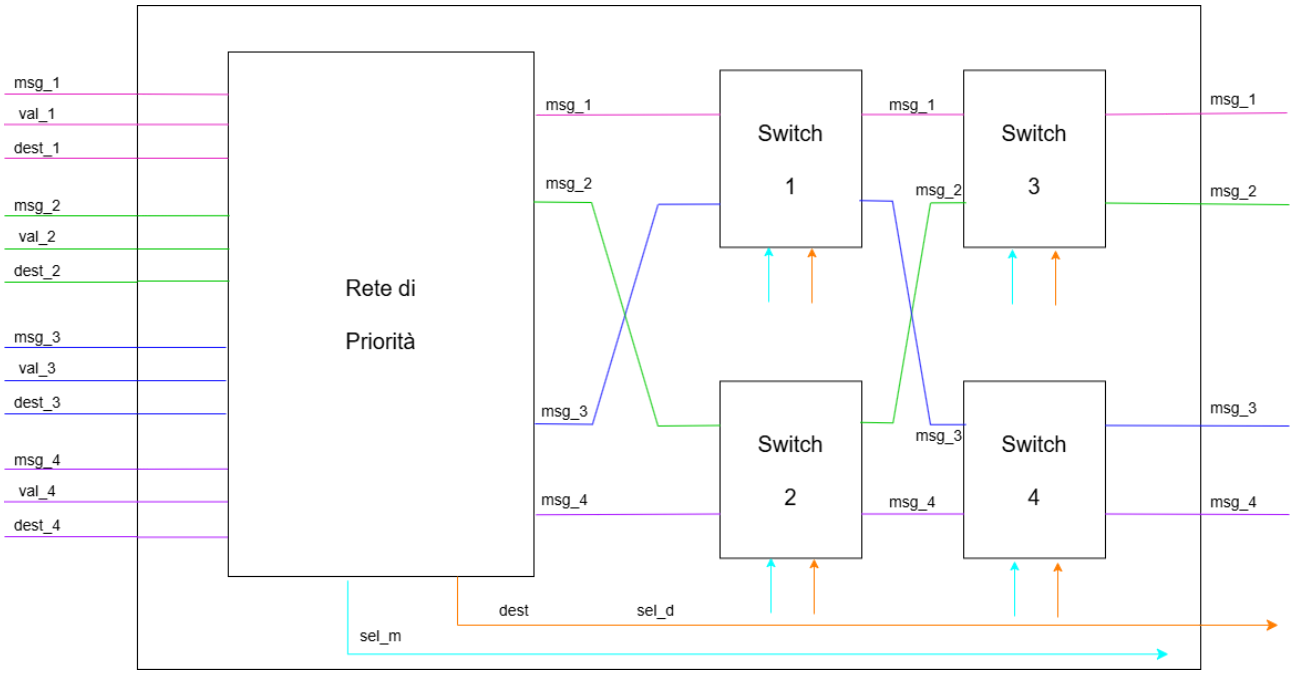



*Figura 11.3 Disegno architetturale del sistema*

Ogni nodo mittente invia nella rete 3 informazioni:

- il messaggio ***msg*** su 2 bit;
- un bit di validità ***val*** per indicare che effettivamente sta cercando di trasmettere qualcosa;
- la destinazione ***dest*** del messaggio su 2 bit.

Queste informazioni vengono elaborate dalla ***rete di priorità*** che tra tutti i messaggi validi in ingresso fa passare soltanto il messaggio più prioritario. La rete di priorità, oltre ai messaggi, restituisce in uscita la

destinazione del messaggio valido, che funge da selezione per i demultiplexer negli switch, e la selezione dei multiplexer.

In uscita dalla rete complessiva, oltre al messaggio, ci sono anche 2 bit di validità (che sono dati in realtà dalla destinazione del messaggio), così da garantire che il messaggio che si sta leggendo è valido.

Vediamo nel dettaglio la rete di priorità:

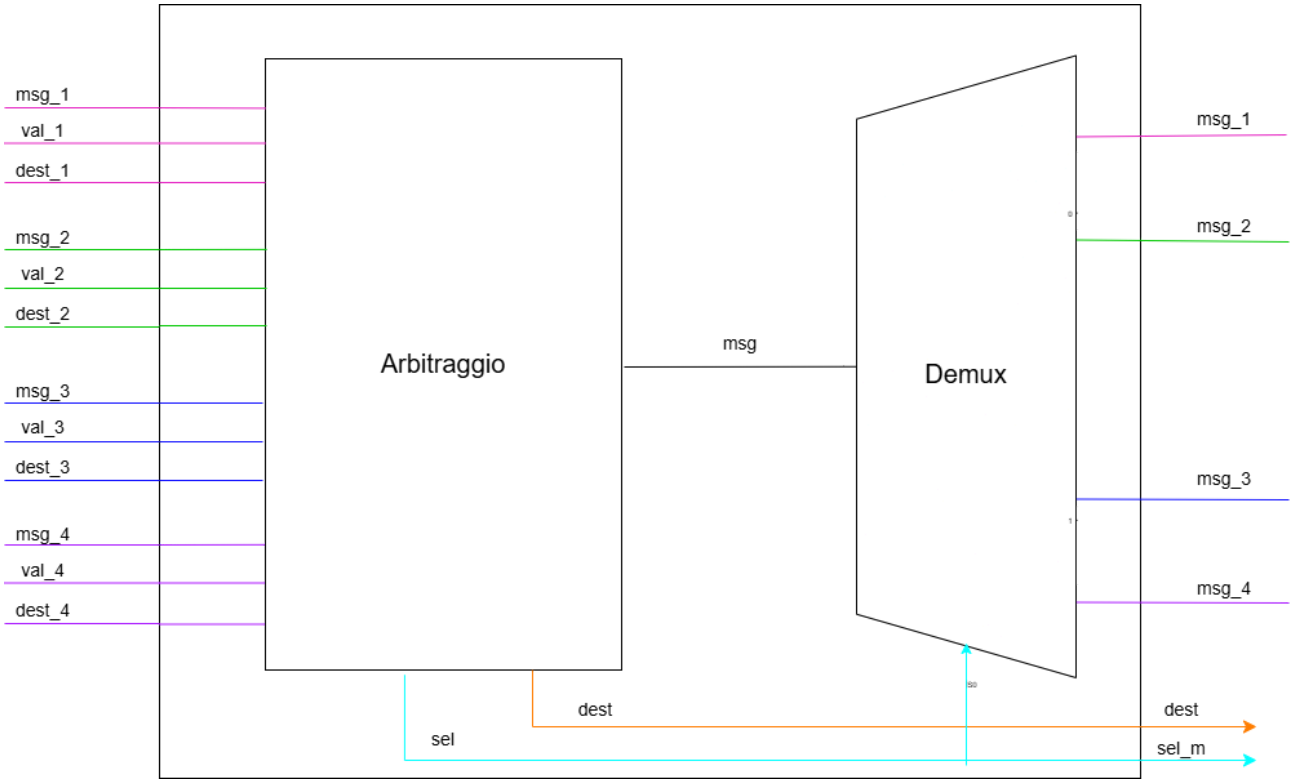



*Figura 11.4 Disegno architetturale della rete di priorità*

La rete di priorità è costituita da 2 componenti:

- **arbitraggio**: prende in ingresso i 4 messaggi con i corrispondenti bit di validità e destinazioni. Implementa uno schema fisso di priorità per cui il mittente con indice più basso ha priorità maggiore (il nodo 1 e il più prioritario).

  Quindi, se arrivano più messaggi contemporaneamente nella rete (ce ne accorgiamo perché abbiamo più bit di validità alti), il componente *arbitraggio* lascerà passare solo quello più prioritario. Restituisce quindi in uscita:

  - **msg** ⟶ il messaggio più prioritario
  - **dest** ⟶ la destinazione del messaggio più prioritario
  - **sel** ⟶ lo utilizziamo come selezione nel demultiplexer della rete di priorità e poi come selezione nei mux degli switch
    - se inoltra il messaggio 1 ⟶ sel=00
    - se inoltra il messaggio 2 ⟶ sel=01
    - se inoltra il messaggio 3 ⟶ sel=10
    - se inoltra il messaggio 4 ⟶ sel=11

- **demultiplexer**: il demultiplexer 1:4 prende in ingresso il messaggio più prioritario da trasmettere nella rete e lo replica sull'uscita corrispondente al mittente in base alla selezione **sel** generata dal componente **arbitraggio**. L'uscita più prioritaria sul demux assume come valore msg, mentre tutte le altre assumono valore "- -".

Concentriamoci sul segnale **sel** restituito dalla rete di priorità che abbiamo detto essere utilizzato anche per la selezione dei mux negli switch. In particolare:

- gli switch 1 e 2 (prima colonna) guardano il bit più significativo di sel
    - lo switch 1 prende in ingresso i messaggi 1 e 3. Se il messaggio valido in un dato istante è 1, allora sel=00 (msb=0); se invece il messaggio valido è 3, allora sel = 10 (msb=1).
    - lo switch 2 prende in ingresso i messaggi 2 e 4. Se il messaggio valido in un dato istante è 2, allora sel=01 (msb=0); se invece il messaggio valido è 4, allora sel = 11 (msb=1).
- gli switch 3 e 4 (seconda colonna) guardano il bit meno significativo di sel
    - lo switch 3 prende in ingresso i messaggi 1 e 2. Se il messaggio valido in un dato istante è 1, allora sel=00 (lsb=0); se invece il messaggio valido è 2, allora sel = 01 (lsb=1).
    - lo switch 4 prende in ingresso i messaggi 3 e 4. Se il messaggio valido in un dato istante è 3, allora sel=10 (lsb=0); se invece il messaggio valido è 4, allora sel = 11 (lsb=1).

La destinazione del messaggio più prioritario viene propagata all'interno della rete dalla rete di priorità e viene utilizzata per la selezione dei demux negli switch. In particolare:

- la prima colonna (switch 1 e 2) guarda il bit più significativo (bit 1)
- la seconda colonna (switch 3 e 4) guarda il bit meno significativo (bit 0)

Il sistema elaborato in Vivado è il seguente:

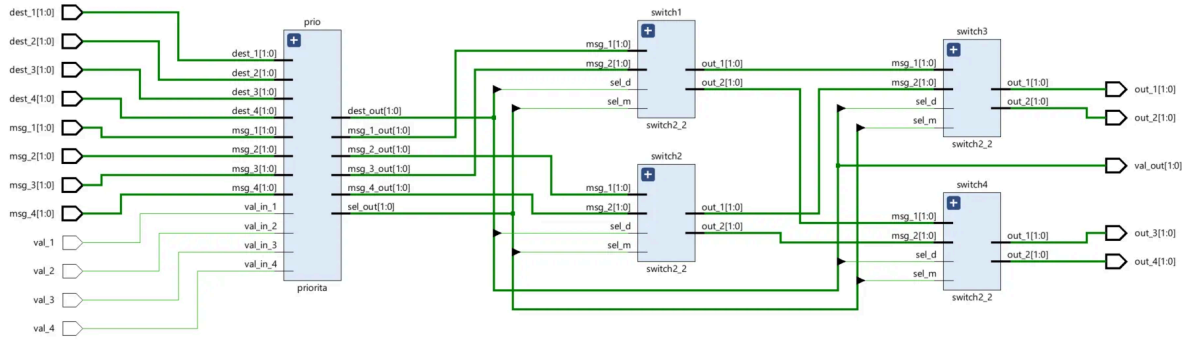


*Figura 11.5 Schematic del sistema*

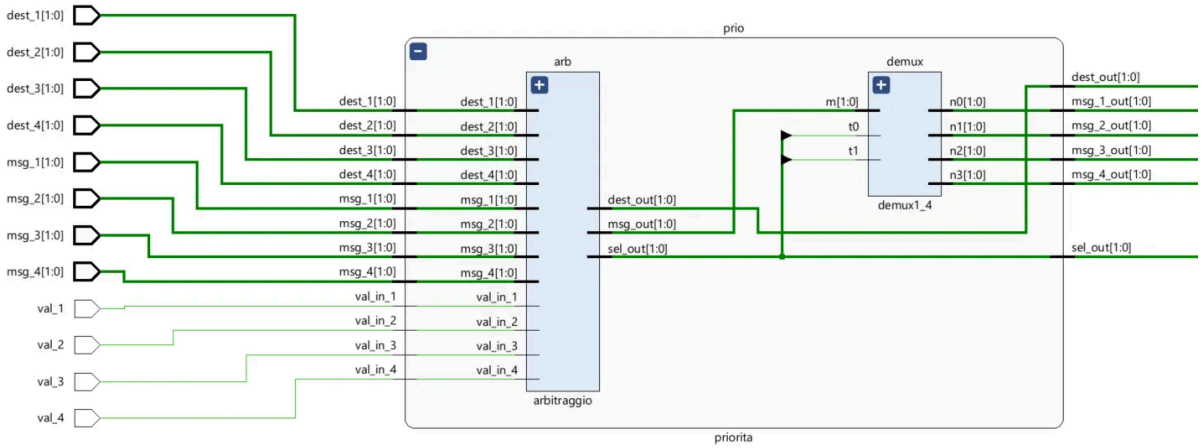



*Figura 11.6 Schematic della rete di priorità*

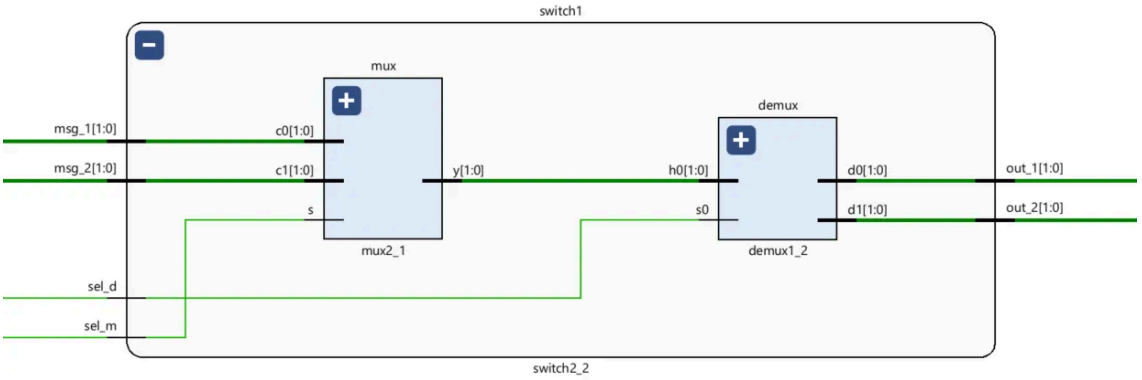



*Figura 11.7 Schematic dello switch*

## *Implementazione*

### Multiplexer 2:1 su 2 bit

```
entity mux2_1 is
    Port ( c0 : in STD_LOGIC_VECTOR(1 downto 0);
           c1 : in STD_LOGIC_VECTOR(1 downto 0);
           s : in STD_LOGIC;
           y : out STD_LOGIC_VECTOR(1 downto 0));
end mux2_1;

architecture Behavioral of mux2_1 is

begin

    y <= c0 when s = '0' else
         c1 when s = '1';

end Behavioral;
```

**Demultiplexer 1:2 su 2 bit**

```
entity demux1_2 is
    Port ( h0 : in STD_LOGIC_VECTOR(1 downto 0);
           d0 : out STD_LOGIC_VECTOR(1 downto 0);
           d1 : out STD_LOGIC_VECTOR(1 downto 0);
           s0 : in STD_LOGIC);
end demux1_2;

architecture Behavioral of demux1_2 is

begin
    d0 <= h0 when s0 = '0';
    d1 <= h0 when s0 = '1';

end Behavioral;
```

**Arbitraggio**

```
entity arbitraggio is
    Port ( val_in_1 : in std_logic;
           val_in_2 : in std_logic;
           val_in_3 : in std_logic;
           val_in_4 : in std_logic;
           msg_1 : in std_logic_vector(1 downto 0);
           msg_2 : in std_logic_vector(1 downto 0);
           msg_3 : in std_logic_vector(1 downto 0);
           msg_4 : in std_logic_vector(1 downto 0);
           dest_1 : in std_logic_vector(1 downto 0);
           dest_2 : in std_logic_vector(1 downto 0);
           dest_3 : in std_logic_vector(1 downto 0);
           dest_4 : in std_logic_vector(1 downto 0);
           sel_out : out std_logic_vector(1 downto 0);
           msg_out : out std_logic_vector(1 downto 0);
           dest_out : out std_logic_vector(1 downto 0)
    );
end arbitraggio;

architecture Behavioral of arbitraggio is

begin

    process(val_in_1, val_in_2, val_in_3, val_in_4)
    begin
        if val_in_1 = '1' then
            sel_out <= "00";
            msg_out <= msg_1;
            dest_out <= dest_1;

        elsif val_in_2 = '1' then
            sel_out <= "01";
            msg_out <= msg_2;
            dest_out <= dest_2;

        elsif val_in_3 = '1' then
```

```
            sel_out <= "10";
            msg_out <= msg_3;
            dest_out <= dest_3;

        elsif val_in_4 = '1' then
            sel_out <= "11";
            msg_out <= msg_4;
            dest_out <= dest_4;

        else
            sel_out <= "--";
            msg_out <= "--";
            dest_out <= "--";
        end if;
    end process;

end Behavioral;
```

**Demux 1:4 su 2 bit**

```
entity demux1_4 is
    Port ( m : in STD_LOGIC_VECTOR(1 downto 0);
           n0 : out STD_LOGIC_VECTOR(1 downto 0);
           n1 : out STD_LOGIC_VECTOR(1 downto 0);
           n2 : out STD_LOGIC_VECTOR(1 downto 0);
           n3 : out STD_LOGIC_VECTOR(1 downto 0);
           t0 : in STD_LOGIC;
           t1 : in STD_LOGIC);
end demux1_4;

architecture Structural of demux1_4 is

    signal v0 : STD_LOGIC_VECTOR(1 downto 0) := "00";
    signal v1 : STD_LOGIC_VECTOR(1 downto 0) := "00";

    component demux1_2
         Port ( h0 : in STD_LOGIC_VECTOR(1 downto 0);
                d0 : out STD_LOGIC_VECTOR(1 downto 0);
                d1 : out STD_LOGIC_VECTOR(1 downto 0);
                s0 : in STD_LOGIC);
    end component;

begin

    demux0: demux1_2
    Port map(   h0 => m,
                d0 => v0,
                d1 => v1,
                s0 => t0);

    demux1: demux1_2
    Port map(   h0 => v0,
                d0 => n0,
                d1 => n1,
                s0 => t1);
```

```
    demux2: demux1_2
    Port map(   h0 => v1,
                d0 => n2,
                d1 => n3,
                s0 => t1);


end Structural;
```

**Priorità**

```
entity priorita is
    Port ( val_in_1 : in std_logic;
           val_in_2 : in std_logic;
           val_in_3 : in std_logic;
           val_in_4 : in std_logic;
           msg_1 : in std_logic_vector(1 downto 0);
           msg_2 : in std_logic_vector(1 downto 0);
           msg_3 : in std_logic_vector(1 downto 0);
           msg_4 : in std_logic_vector(1 downto 0);
           dest_1 : in std_logic_vector(1 downto 0);
           dest_2 : in std_logic_vector(1 downto 0);
           dest_3 : in std_logic_vector(1 downto 0);
           dest_4 : in std_logic_vector(1 downto 0);
           sel_out : out std_logic_vector(1 downto 0);
           msg_1_out : out std_logic_vector(1 downto 0);
           msg_2_out : out std_logic_vector(1 downto 0);
           msg_3_out : out std_logic_vector(1 downto 0);
           msg_4_out : out std_logic_vector(1 downto 0);
           dest_out : out std_logic_vector(1 downto 0)
    );
end priorita;

architecture Structural of priorita is

component arbitraggio is
    Port ( val_in_1 : in std_logic;
           val_in_2 : in std_logic;
           val_in_3 : in std_logic;
           val_in_4 : in std_logic;
           msg_1 : in std_logic_vector(1 downto 0);
           msg_2 : in std_logic_vector(1 downto 0);
           msg_3 : in std_logic_vector(1 downto 0);
           msg_4 : in std_logic_vector(1 downto 0);
           dest_1 : in std_logic_vector(1 downto 0);
           dest_2 : in std_logic_vector(1 downto 0);
           dest_3 : in std_logic_vector(1 downto 0);
           dest_4 : in std_logic_vector(1 downto 0);
           sel_out : out std_logic_vector(1 downto 0);
           msg_out : out std_logic_vector(1 downto 0);
           dest_out : out std_logic_vector(1 downto 0)
    );
end component arbitraggio;

component demux1_4 is
```

```
    Port ( m : in STD_LOGIC_VECTOR(1 downto 0);
           n0 : out STD_LOGIC_VECTOR(1 downto 0);
           n1 : out STD_LOGIC_VECTOR(1 downto 0);
           n2 : out STD_LOGIC_VECTOR(1 downto 0);
           n3 : out STD_LOGIC_VECTOR(1 downto 0);
           t0 : in STD_LOGIC;
           t1 : in STD_LOGIC);
end component demux1_4;

signal msg : std_logic_vector(1 downto 0);
signal sel : std_logic_vector(1 downto 0);

begin

    sel_out <= sel;

    arb: arbitraggio
    port map ( val_in_1 => val_in_1,
               val_in_2 => val_in_2,
               val_in_3 => val_in_3,
               val_in_4 => val_in_4,
               msg_1 => msg_1,
               msg_2 => msg_2,
               msg_3 => msg_3,
               msg_4 => msg_4,
               dest_1 => dest_1,
               dest_2 => dest_2,
               dest_3 => dest_3,
               dest_4 => dest_4,
               sel_out => sel,
               msg_out => msg,
               dest_out => dest_out
        );

    demux: demux1_4
    port map ( m => msg,
               n0 => msg_1_out,
               n1 => msg_2_out,
               n2 => msg_3_out,
               n3 => msg_4_out,
               t0 => sel(1),
               t1 => sel(0));

end Structural;
```

**Switch 2x2**

```
entity switch2_2 is
  Port ( msg_1 : in std_logic_vector(1 downto 0);
         msg_2 : in std_logic_vector(1 downto 0);
         sel_m : in std_logic;
         sel_d : in std_logic;
         out_1 : out std_logic_vector(1 downto 0);
         out_2 : out std_logic_vector(1 downto 0)
        );
end switch2_2;
```

```
architecture Structural of switch2_2 is

component mux2_1 is
    Port ( c0 : in STD_LOGIC_VECTOR(1 downto 0);
           c1 : in STD_LOGIC_VECTOR(1 downto 0);
           s : in STD_LOGIC;
           y : out STD_LOGIC_VECTOR(1 downto 0));
end component mux2_1;

component demux1_2 is
    Port ( h0 : in STD_LOGIC_VECTOR(1 downto 0);
           d0 : out STD_LOGIC_VECTOR(1 downto 0);
           d1 : out STD_LOGIC_VECTOR(1 downto 0);
           s0 : in STD_LOGIC);
end component demux1_2;

signal out_mux : std_logic_vector(1 downto 0);

begin

    mux: mux2_1
    port map ( c0 => msg_1,
               c1 => msg_2,
               s => sel_m,
               y => out_mux);

    demux: demux1_2
    port map ( h0 => out_mux,
               d0 => out_1,
               d1 => out_2,
               s0 => sel_d);


end Structural;
```

**Rete 4x4**

```
entity rete4_4 is
    Port ( msg_1 : in std_logic_vector(1 downto 0);
           msg_2 : in std_logic_vector(1 downto 0);
           msg_3 : in std_logic_vector(1 downto 0);
           msg_4 : in std_logic_vector(1 downto 0);
           dest_1 : in std_logic_vector(1 downto 0);
           dest_2 : in std_logic_vector(1 downto 0);
           dest_3 : in std_logic_vector(1 downto 0);
           dest_4 : in std_logic_vector(1 downto 0);
           val_1 : in std_logic;
           val_2 : in std_logic;
           val_3 : in std_logic;
           val_4 : in std_logic;
           out_1 : out std_logic_vector(1 downto 0);
           out_2 : out std_logic_vector(1 downto 0);
           out_3 : out std_logic_vector(1 downto 0);
           out_4 : out std_logic_vector(1 downto 0);
           val_out : out std_logic_vector(1 downto 0)
```

```
    );
end rete4_4;

architecture Structural of rete4_4 is

component switch2_2 is
  Port ( msg_1 : in std_logic_vector(1 downto 0);
         msg_2 : in std_logic_vector(1 downto 0);
         sel_m : in std_logic;
         sel_d : in std_logic;
         out_1 : out std_logic_vector(1 downto 0);
         out_2 : out std_logic_vector(1 downto 0)
        );
end component switch2_2;

component priorita is
    Port ( val_in_1 : in std_logic;
           val_in_2 : in std_logic;
           val_in_3 : in std_logic;
           val_in_4 : in std_logic;
           msg_1 : in std_logic_vector(1 downto 0);
           msg_2 : in std_logic_vector(1 downto 0);
           msg_3 : in std_logic_vector(1 downto 0);
           msg_4 : in std_logic_vector(1 downto 0);
           dest_1 : in std_logic_vector(1 downto 0);
           dest_2 : in std_logic_vector(1 downto 0);
           dest_3 : in std_logic_vector(1 downto 0);
           dest_4 : in std_logic_vector(1 downto 0);
           sel_out : out std_logic_vector(1 downto 0);
           msg_1_out : out std_logic_vector(1 downto 0);
           msg_2_out : out std_logic_vector(1 downto 0);
           msg_3_out : out std_logic_vector(1 downto 0);
           msg_4_out : out std_logic_vector(1 downto 0);
           dest_out : out std_logic_vector(1 downto 0)
    );
end component priorita;

signal msg_1_tmp : std_logic_vector(1 downto 0);
signal msg_2_tmp : std_logic_vector(1 downto 0);
signal msg_3_tmp : std_logic_vector(1 downto 0);
signal msg_4_tmp : std_logic_vector(1 downto 0);
signal sel_m : std_logic_vector(1 downto 0);
signal sel_d : std_logic_vector(1 downto 0);
signal tmp_1 : std_logic_vector(1 downto 0);
signal tmp_2 : std_logic_vector(1 downto 0);
signal tmp_3 : std_logic_vector(1 downto 0);
signal tmp_4 : std_logic_vector(1 downto 0);

begin

    val_out <= sel_d;

    prio: priorita
    port map ( val_in_1 => val_1,
               val_in_2 => val_2,
               val_in_3 => val_3,
```

```
            val_in_4 => val_4,
            msg_1 => msg_1,
            msg_2 => msg_2,
            msg_3 => msg_3,
            msg_4 => msg_4,
            dest_1 => dest_1,
            dest_2 => dest_2,
            dest_3 => dest_3,
            dest_4 => dest_4,
            sel_out => sel_m,
            msg_1_out => msg_1_tmp,
            msg_2_out => msg_2_tmp,
            msg_3_out => msg_3_tmp,
            msg_4_out => msg_4_tmp,
            dest_out => sel_d
        );

    switch1: switch2_2
    port map ( msg_1 => msg_1_tmp,
               msg_2 => msg_3_tmp,
               sel_m => sel_m(1),
               sel_d => sel_d(1),
               out_1 => tmp_1,
               out_2 => tmp_3
              );

    switch2: switch2_2
    port map ( msg_1 => msg_2_tmp,
               msg_2 => msg_4_tmp,
               sel_m => sel_m(1),
               sel_d => sel_d(1),
               out_1 => tmp_2,
               out_2 => tmp_4
              );

    switch3: switch2_2
    port map ( msg_1 => tmp_1,
               msg_2 => tmp_2,
               sel_m => sel_m(0),
               sel_d => sel_d(0),
               out_1 => out_1,
               out_2 => out_2
              );

    switch4: switch2_2
    port map ( msg_1 => tmp_3,
               msg_2 => tmp_4,
               sel_m => sel_m(0),
               sel_d => sel_d(0),
               out_1 => out_3,
               out_2 => out_4
              );

end Structural;
```

*Simulazione*

```
entity tb_rete4_4 is
end tb_rete4_4;

architecture tb of tb_rete4_4 is

    component rete4_4
        port (msg_1   : in std_logic_vector (1 downto 0);
              msg_2   : in std_logic_vector (1 downto 0);
              msg_3   : in std_logic_vector (1 downto 0);
              msg_4   : in std_logic_vector (1 downto 0);
              dest_1  : in std_logic_vector (1 downto 0);
              dest_2  : in std_logic_vector (1 downto 0);
              dest_3  : in std_logic_vector (1 downto 0);
              dest_4  : in std_logic_vector (1 downto 0);
              val_1   : in std_logic;
              val_2   : in std_logic;
              val_3   : in std_logic;
              val_4   : in std_logic;
              out_1   : out std_logic_vector (1 downto 0);
              out_2   : out std_logic_vector (1 downto 0);
              out_3   : out std_logic_vector (1 downto 0);
              out_4   : out std_logic_vector (1 downto 0);
              val_out : out std_logic_vector (1 downto 0));
    end component;

    signal msg_1   : std_logic_vector (1 downto 0);
    signal msg_2   : std_logic_vector (1 downto 0);
    signal msg_3   : std_logic_vector (1 downto 0);
    signal msg_4   : std_logic_vector (1 downto 0);
    signal dest_1  : std_logic_vector (1 downto 0);
    signal dest_2  : std_logic_vector (1 downto 0);
    signal dest_3  : std_logic_vector (1 downto 0);
    signal dest_4  : std_logic_vector (1 downto 0);
    signal val_1   : std_logic;
    signal val_2   : std_logic;
    signal val_3   : std_logic;
    signal val_4   : std_logic;
    signal out_1   : std_logic_vector (1 downto 0);
    signal out_2   : std_logic_vector (1 downto 0);
    signal out_3   : std_logic_vector (1 downto 0);
    signal out_4   : std_logic_vector (1 downto 0);
    signal val_out : std_logic_vector (1 downto 0);

begin

    dut : rete4_4
    port map (msg_1   => msg_1,
              msg_2   => msg_2,
              msg_3   => msg_3,
              msg_4   => msg_4,
              dest_1  => dest_1,
              dest_2  => dest_2,
              dest_3  => dest_3,
              dest_4  => dest_4,
```

```
            val_1   => val_1,
            val_2   => val_2,
            val_3   => val_3,
            val_4   => val_4,
            out_1   => out_1,
            out_2   => out_2,
            out_3   => out_3,
            out_4   => out_4,
            val_out => val_out);

    stimuli : process
    begin
        -- ***EDIT*** Adapt initialization as needed
        msg_1 <= (others => '0');
        msg_2 <= (others => '0');
        msg_3 <= (others => '0');
        msg_4 <= (others => '0');
        dest_1 <= (others => '0');
        dest_2 <= (others => '0');
        dest_3 <= (others => '0');
        dest_4 <= (others => '0');
        val_1 <= '0';
        val_2 <= '0';
        val_3 <= '0';
        val_4 <= '0';

        -- ***EDIT*** Add stimuli here
        wait for 10 ns;
        msg_1 <= "11";
        dest_1 <= "00";
        msg_2 <= "10";
        dest_2 <= "11";

        wait for 10 ns;
        val_1 <= '1';
        val_2 <= '1';

        wait for 40 ns;
        val_1 <= '0';
        val_2 <= '0';

        wait for 40 ns;
        msg_3 <= "01";
        dest_3 <= "10";
        val_3 <= '1';
        msg_4 <= "11";
        dest_4 <= "11";
        val_4 <= '1';

        wait for 40 ns;
        val_3 <= '0';
        val_4 <= '0';

        wait for 40 ns;
        msg_4 <= "00";
        dest_4 <= "00";
```

```
        val_4 <= '1';

        wait for 40 ns;
        val_4 <= '0';

        wait for 40 ns;
        msg_1 <= "01";
        dest_1 <= "10";
        val_1 <= '1';
        msg_3 <= "11";
        dest_3 <= "11";
        val_3 <= '1';

        wait for 40 ns;
        val_1 <= '0';
        val_3 <= '0';

        wait for 40 ns;
        msg_1 <= "00";
        dest_1 <= "11";
        val_1 <= '1';
        msg_2 <= "01";
        dest_2 <= "10";
        val_2 <= '1';
        msg_3 <= "10";
        dest_3 <= "01";
        val_3 <= '1';
        msg_4 <= "11";
        dest_4 <= "00";
        val_4 <= '1';

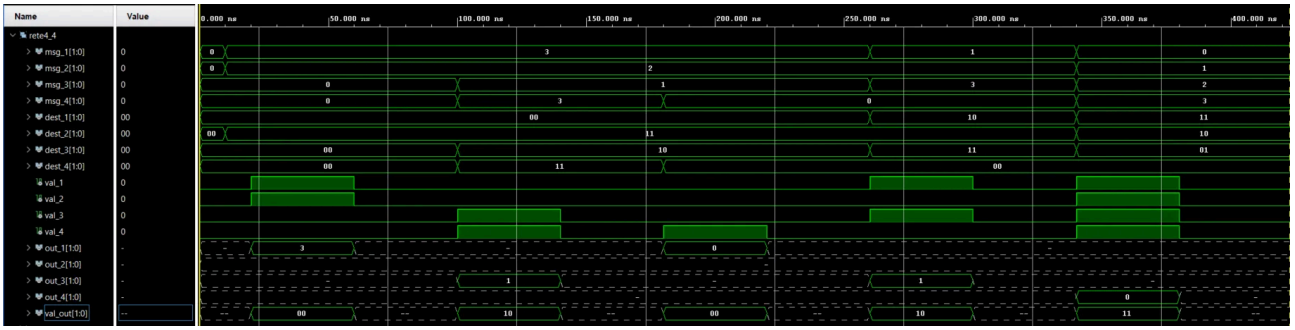
        wait for 40 ns;
        val_1 <= '0';
        val_2 <= '0';
        val_3 <= '0';
        val_4 <= '0';

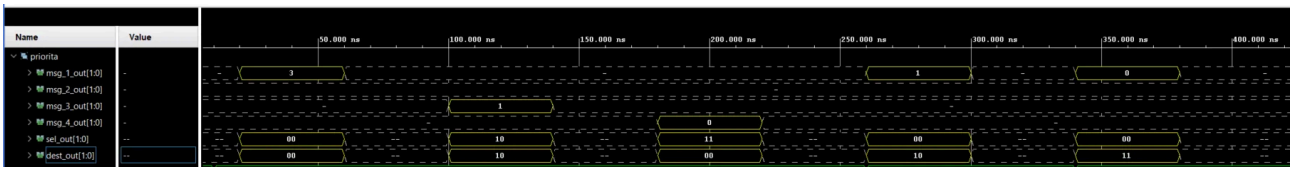
        wait;
    end process;

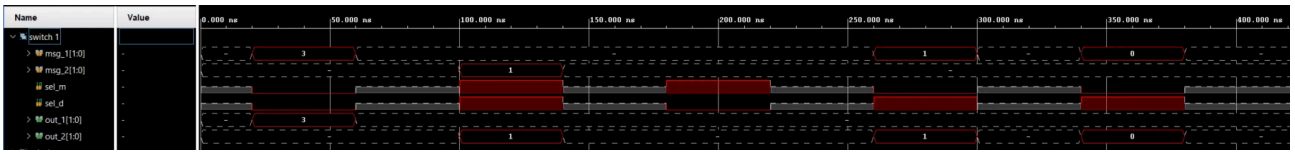
end tb;

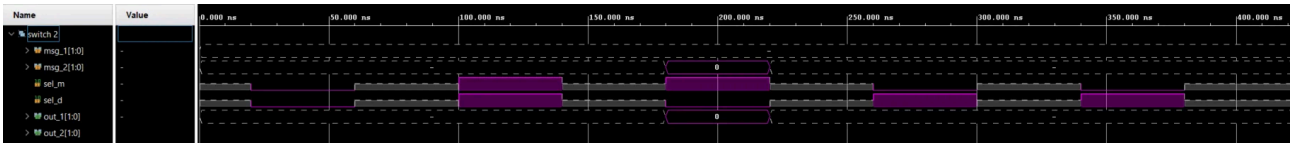
-- Configuration block below is required by some simulators. Usually no need to edit.

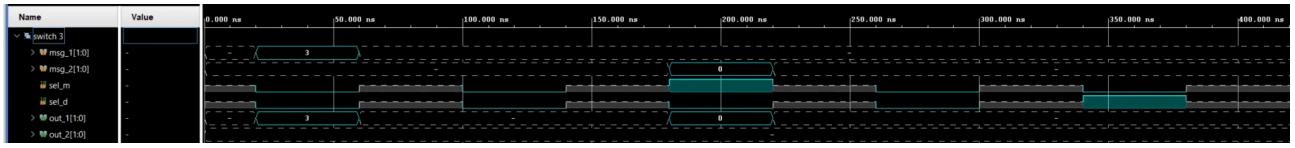
configuration cfg_tb_rete4_4 of tb_rete4_4 is
    for tb
    end for;
end cfg_tb_rete4_4;
```

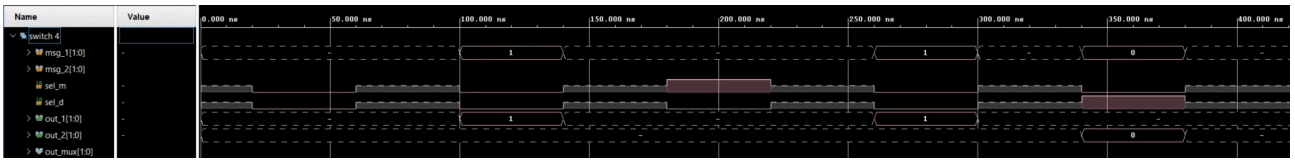

Name
Value
rete4_4
msg_1[1:0]
msg_2[1:0]
msg_3[1:0]
msg_4[1:0]
dest_1[1:0]
dest_2[1:0]
dest_3[1:0]
dest_4[1:0]
val_1
val_2
val_3
val_4
out_1[1:0]
out_2[1:0]
out_3[1:0]
out_4[1:0]
val_out[1:0]
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns


Name
Value
priorita
msg_1_out[1:0]
msg_2_out[1:0]
msg_3_out[1:0]
msg_4_out[1:0]
sel_out[1:0]
dest_out[1:0]
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns


Name
Value
switch 1
msg_1[1:0]
msg_2[1:0]
sel_m
sel_d
out_1[1:0]
out_2[1:0]
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns


Name
Value
switch 2
msg_1[1:0]
msg_2[1:0]
sel_m
sel_d
out_1[1:0]
out_2[1:0]
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns


Name
Value
switch 3
msg_1[1:0]
msg_2[1:0]
sel_m
sel_d
out_1[1:0]
out_2[1:0]
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns


Name
Value
switch 4
msg_1[1:0]
msg_2[1:0]
sel_m
sel_d
out_1[1:0]
out_2[1:0]
out_mux[1:0]
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns

# Capitolo 8: Prova di esame 19 dicembre 2024

## Esercizio 12

Un sistema è composto da 2 nodi, A e B. A include una ROM (progettata come macchina sequenziale con READ sincrono) di 8 locazioni da 4 bit, mentre B include un sommatore parallelo in grado di effettuare la somma di 2 stringhe di 4 bit ciascuna e un registro R di 4 bit. Il sistema opera come segue: all'arrivo di un segnale di start, A inizia a prelevare gli elementi ROM[i] dalla propria memoria e li invia, uno alla volta, a B mediante handshaking. B somma progressivamente le stringhe ricevute utilizzando il sommatore e alla fine inserisce il risultato nel registro R.

1. Si disegni l'architettura complessiva del sistema tramite un diagramma a blocchi, identificando parte operativa e parte di controllo di ciascun nodo. Ogni nodo deve essere progettato seguendo un approccio strutturale, individuando tutti i componenti, le loro interfacce e le loro interconnessioni.
2. Si progettino le unità di controllo di A e B evidenziando gli stati, gli ingressi e le uscite negli automi risultanti. E' obbligatorio specificare la tempificazione che si intende dare alle macchine (fronte attivo del clock, tempificazione dei segnali di READ/WRITE su registri e memorie).
3. Si progetti il sommatore secondo un'architettura di tipo carry look ahead.
4. Si fornisca l'implementazione in VHDL dell'intero sistema e si proceda alla simulazione nel caso in cui il clock del sistema A e del sistema B siano diversi (A più lento e A più veloce).

### *Progetto e architettura*

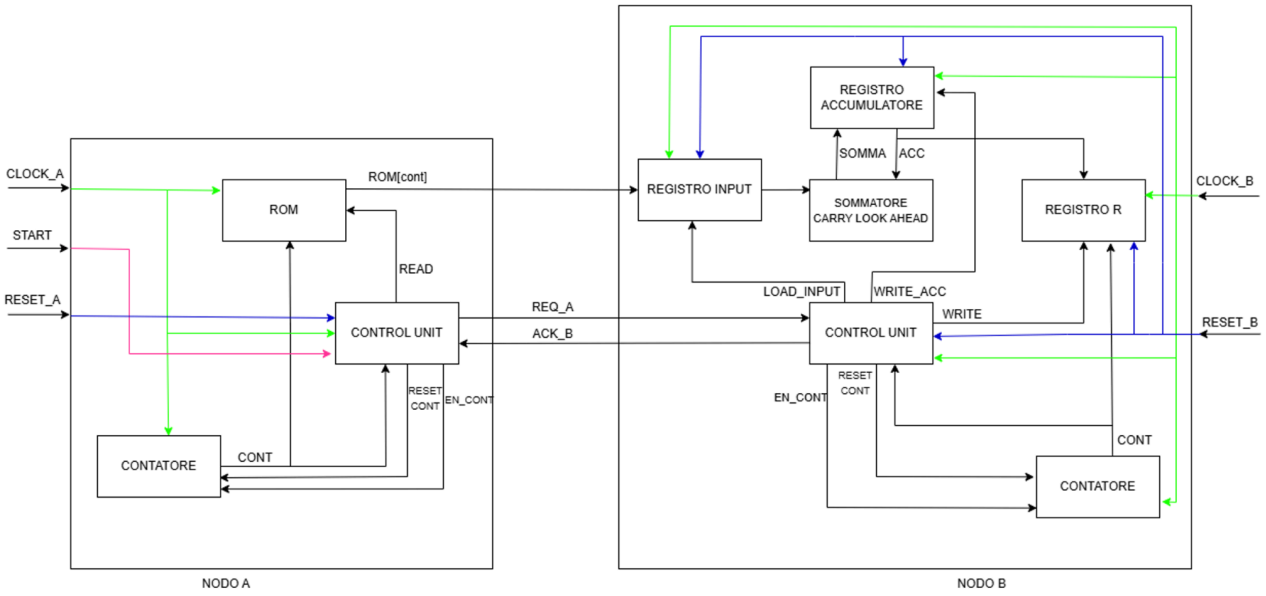



*Figura 12.1 Disegno architetturale del sistema*

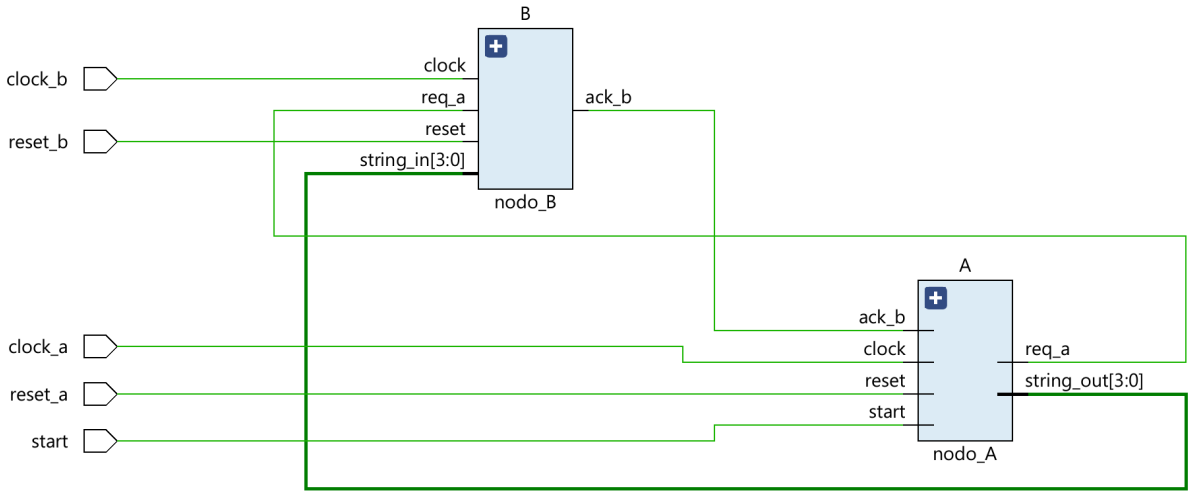



*Figura 12.2 Schematic del sistema realizzato*

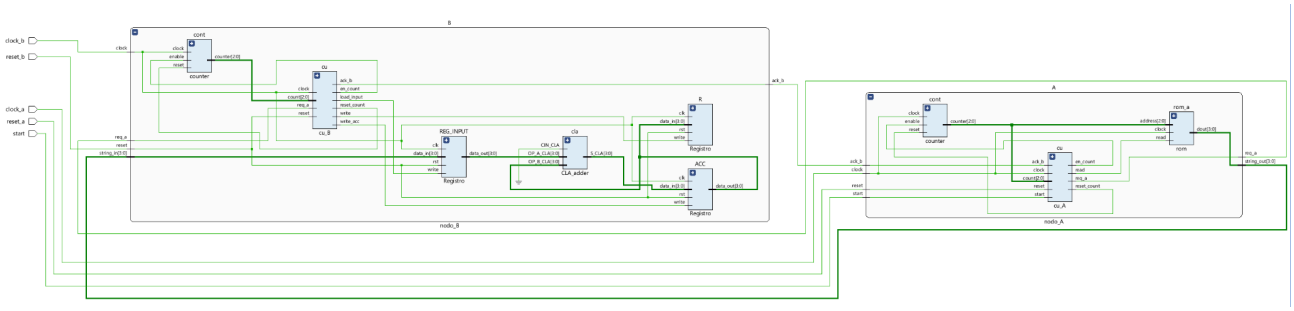


*Figura 12.3 Schematic del sistema realizzato: nodo A e nodo B*

Il sommatore Carry Look Ahead è progettato e implementato come illustrato in Appendice.

In *Figura 12.4* è rappresentato l'automa dell'unità A. Inizialmente e a seguito del RESET A si trova in IDLE. All'arrivo del segnale di START, si passa in S0. In questo stato A avvia la lettura dalla ROM e si passa allo stato S1. In S1 A invia alza il segnale REQ_A per stabilire la comunicazione con B, e attende che B alzi il segnale ACK_B. Quando il segnale ACK_B si alza, A abbassa REQ_A, e attende che B finisca l'elaborazione, cioè abbassi ACK_B. A questo punto, passa in S3 e dà l'abilitazione al contatore. L'uscita del contatore sarà effettivamente incrementata sul fronte di salita successivo del clock, quindi A in S3 può controllare il valore corrente di COUNT, per determinare se ha finito di scorrere la ROM.

In *Figura 12.5* è, invece, rappresentato l'automa dell'unità B. Nello stato S0 B attende che il segnale REQ_A si alzi. Quando rileva REQ_A = 1 passa nello stato S1, dove alza l'ACK_B e dà il load della stringa ricevuta da A nel REGISTRO INPUT e attende che REQ_A torni a zero. Quando REQ_A torna a 0, B passa in S2 e abbassa il LOAD. A questo punto B passa nello stato S3, dove il risultato della somma è pronto e dà il WRITE_ACC per scrivere nel registro in cui si accumula la somma calcolata di volta in volta. Nello stato successivo, S4, viene abbassato l'ACK_B in quanto l'elaborazione del dato ricevuto è terminata, e si incrementa il contatore. L'uscita del contatore sarà effettivamente incrementata sul fronte di salita successivo del clock, quindi B in S4 può controllare il valore corrente di COUNT, per determinare se ha ricevuto tutte le stringhe. In caso affermativo, cioè se COUNT = 111 passa nello stato S5 in cui scrive il risultato finale nel registro R. Altrimenti, torna in S0 e aspetta la prossima stringa.

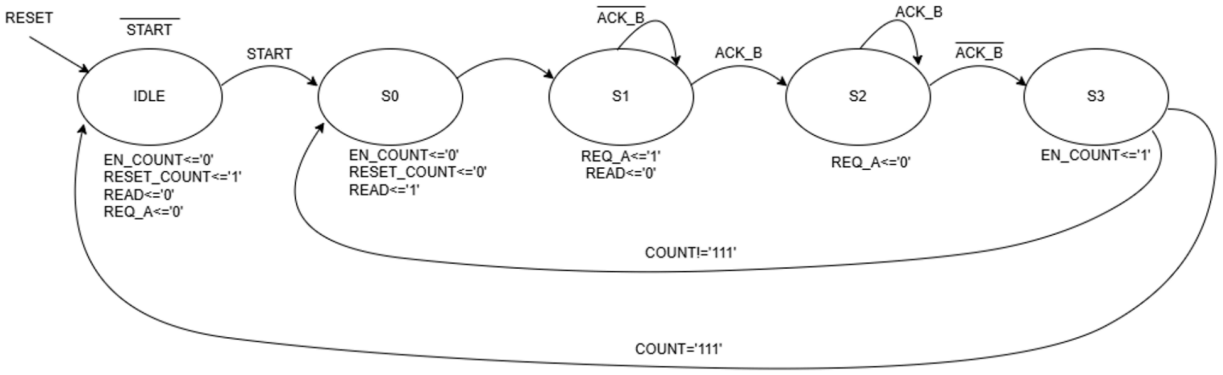



*Figura 12.4 Automa dell'unità di controllo del nodo A*

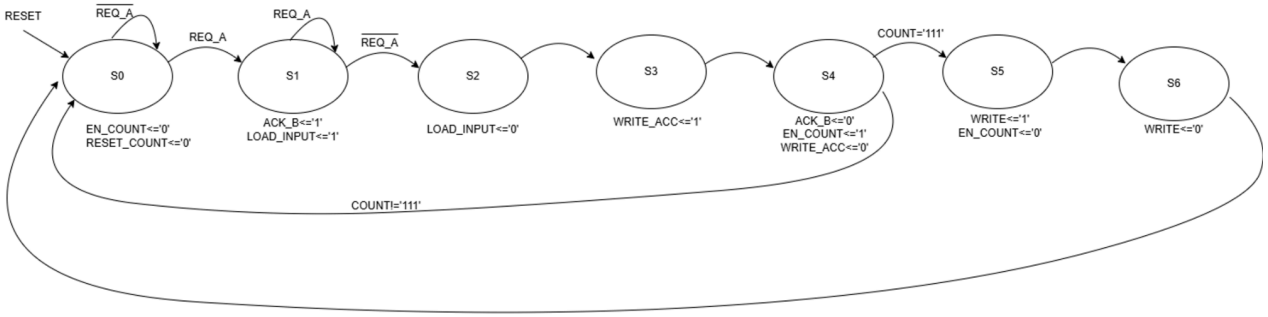



*Figura 12.5 Automa dell'unità di controllo del nodo B*

### *Implementazione*

Il **top module** è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



entity com_handshaking is
  Port (
        start: in std_logic;
        reset_a: in std_logic;
        reset_b: in std_logic;
        clock_a: in std_logic;
        clock_b: in std_logic
  );
end com_handshaking;

architecture Structural of com_handshaking is
    signal req_a: std_logic;
    signal ack_b: std_logic;
    signal string_a: std_logic_vector(3 downto 0);

     component nodo_A is
      Port (
            start: in std_logic;
            clock: in std_logic;
            reset: in std_logic;
```

```
            string_out: out std_logic_vector(3 downto 0);
            req_a: out std_logic;
            ack_b: in std_logic
       );
    end component;

    component nodo_B is
      Port (
            clock: in std_logic;
            reset: in std_logic;
            string_in: in std_logic_vector(3 downto 0);
            req_a: in std_logic;
            ack_b: out std_logic
      );
    end component;

    begin
          A: nodo_A
          Port map(

                start => start,
                clock => clock_a,
                reset => reset_a,
                string_out => string_a,
                req_a => req_a,
                ack_b => ack_b
           );

          B: nodo_B
          Port map(
                clock => clock_b,
                reset => reset_b,
                string_in => string_a,
                req_a => req_a,
                ack_b => ack_b
          );



end Structural;
```

L'implementazione del **nodo A** è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity nodo_A is
  Port (
        start: in std_logic;
        clock: in std_logic;
        reset: in std_logic;
        string_out: out std_logic_vector(3 downto 0);
        req_a: out std_logic;
```

```
        ack_b: in std_logic
    );
end nodo_A;

architecture Structural of nodo_A is

    signal en_count: std_logic;
    signal count: std_logic_vector(2 downto 0);
    signal read: std_logic;
    signal umax: std_logic;
    signal reset_count: std_logic;


    component rom is
    port(
        clock: in std_logic;
        read: in std_logic;
        address : in  std_logic_vector(2 downto 0);
        dout    : out std_logic_vector(3 downto 0)
    );
    end component;


    component counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
                enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));

    end component;


    component cu_A is
      port(
            clock, reset, start: in std_logic;
            count: in std_logic_vector (2 downto 0);
            en_count: out std_logic;
            reset_count: out std_logic;
            read: out std_logic;
            req_a: out std_logic;
            ack_b: in std_logic
            );
    end component;


begin
    rom_a: rom port map(
```

```
        clock=>clock,
        read=> read,
        address => count,
        dout  => string_out
    );

    cont: counter
     generic map(

        n => 3 ,--numero di bit necessari
        modulo => 8

        )

    Port map ( clock => clock,
           reset => reset_count, --ingresso di reset per azzerare il contatore
                enable => en_count,
           umax => umax, -- si alza quando si raggiunge il valore massimo di conteggio
           counter => count
    );


   cu: cu_A port map(
            clock => clock,
            reset=> reset,
            start=> start,
            count=> count,
            en_count => en_count,
            reset_count => reset_count,
            read => read,
            req_a => req_a,
            ack_b=> ack_b
            );




end Structural;
```

La **ROM di A** è la seguente.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity rom is
    port(
        clock   : in std_logic;
        read    : in std_logic;
        address : in std_logic_vector(2 downto 0);
        dout    : out std_logic_vector(3 downto 0)
    );
end entity rom;

architecture dataflow of rom is
    type MEMORY_8_4 is array (0 to 7) of std_logic_vector(3 downto 0);
```

```
    constant ROM_8_4 : MEMORY_8_4 := (
        "0000",
        "0001",
        "0010",
        "0011",
        "0100",
        "0101",
        "0110",
        "0111"
    );
begin
    main : process(clock)
    begin
        if rising_edge(clock) then
            if read = '1' then
                dout <= ROM_8_4(to_integer(unsigned(address)));
            end if;
        end if;
    end process main;
end architecture dataflow;
```

L'**unità di controllo di A**, che implementa l'automa in *Figura 11.4,* è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity cu_A is
       port(
             clock, reset, start: in std_logic;
             count: in std_logic_vector (2 downto 0);
             en_count: out std_logic;
             reset_count: out std_logic;
             read: out std_logic;
             req_a: out std_logic;
             ack_b: in std_logic
             );
end cu_A;


architecture structural of cu_A is
       type state is (idle, S0, S1, S2,S3);
       -- idle: aspetta lo start
       -- S0 : dà il segnale di read
       -- S1: dà il segnale di req e aspetta l'ACK
       -- S2 : quando arriva l'ack, abbassa il segnale di req
       -- S3 : quando B abbassa l'ack, abilita il conteggio
       signal current_state,next_state: state;

       begin


       reg_stato: process(clock)
                     begin
                     if(clock'event and clock='1') then
```

```
                if(reset='1') then
                        current_state <=idle;
                  else
                        current_state <=next_state;
                   end if;
                end if;
                end process;

comb: process(current_state, start, ack_b)
        begin


     CASE current_state is

        WHEN idle =>

                en_count<='0';
                reset_count<='1';
                read<='0';
                req_a<='0';

           if(start='1') then
                           next_state <= S0;
                      else
                           next_state <= idle;
                        end if;


        WHEN S0 =>

                  read<='1';
                  reset_count<='0';
                         en_count<='0';

                         next_state <= S1;

       WHEN S1 =>
                  req_a<='1';
                  read<='0';
                  if ack_b='0' then
                    next_state<=S1;
                  elsif ack_b='1' then
                    next_state <= S2;
                  end if;


       WHEN S2 =>
             req_a<='0';

                         if ack_b='1' then
                            next_state<=S2;
                         elsif ack_b='0' then
                            next_state<=S3;
                          end if;

   when S3 =>
```

```
                    en_count<='1';
                    if count="111" then
                       next_state<=IDLE;
                    else
                        next_state<=S0;

                     end if;


         WHEN others =>
                   next_state <= idle;
              end CASE;

    end process;
 end structural;
```

L'implementazione del **nodo B** è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



entity nodo_B is
  Port (
        clock: in std_logic;
        reset: in std_logic;
        string_in: in std_logic_vector(3 downto 0);
        req_a: in std_logic;
        ack_b: out std_logic
  );
end nodo_B;

architecture Structural of nodo_B is

    signal write: std_logic;
    signal write_acc: std_logic;
    signal load_input: std_logic;
    signal read: std_logic;
    signal reset_count: std_logic;
    signal en_count: std_logic;
    signal umax: std_logic;
    signal count: std_logic_vector(2 downto 0);
    signal somma: std_logic_vector(3 downto 0);
    signal c: std_logic;
    signal uscita_R: std_logic_vector(3 downto 0);
    signal accumulatore: std_logic_vector(3 downto 0):= (others => '0'); -- mantiene la
somma parziale
    signal input_sommatore: std_logic_vector(3 downto 0):= (others => '0');


    component counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8
```

```
        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
                enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));

    end component;


    component CLA_adder is
        generic (N: natural range 0 to 32 := 4);
        port(
            OP_A_CLA: in std_logic_vector(N-1 downto 0);
            OP_B_CLA: in std_logic_vector(N-1 downto 0);
            CIN_CLA: in std_logic; -- riporto entrante

            S_CLA: out std_logic_vector(N-1 downto 0);
            COUT_CLA: out std_logic -- riporto uscente
        );
    end component;


    component cu_B is
       port(
             clock, reset: in std_logic;
             count: in std_logic_vector (2 downto 0);
             en_count: out std_logic;
             reset_count: out std_logic;
             read: out std_logic;
             write: out std_logic;
             write_acc: out std_logic;
             load_input: out std_logic;
             req_a: in std_logic;
             ack_b: out std_logic
             );
    end component;

    component Registro is
    Port (
        clk     : in  std_logic;
        rst     : in  std_logic;
        write    : in  std_logic;
        data_in : in  std_logic_vector(3 downto 0);
        data_out: out std_logic_vector(3 downto 0)
    );
    end component;

begin
```

```
 cont: counter
 generic map(

    n => 3 ,--numero di bit necessari
    modulo => 8

    )

Port map ( clock => clock,
       reset => reset_count, --ingresso di reset per azzerare il contatore
            enable => en_count,
       umax => umax, -- si alza quando si raggiunge il valore massimo di conteggio
       counter => count
);


cu: cu_B port map(
         clock => clock,
          reset => reset,
         count => count,
         en_count => en_count,
         reset_count => reset_count,
         read => read,
         write => write,
         write_acc=>write_acc,
         load_input=>load_input,
         req_a => req_a,
         ack_b => ack_b
         );


   cla: CLA_adder
   port map(
          OP_A_CLA => input_sommatore,
          OP_B_CLA => accumulatore,
          CIN_CLA  => '0',

          S_CLA =>somma,
          COUT_CLA => c
   );

   R: Registro        -- in questo registro si scrive solo il risultato finale
Port map(
    clk  => clock,
    rst  => reset,
    write   => write,
    data_in  => accumulatore,
   data_out => uscita_R
);

ACC: Registro    -- in questo registro si scrivono le somme intermedie
Port map(
    clk  => clock,
    rst  => reset,
    write   => write_acc,
    data_in  => somma,
```

```
        data_out => accumulatore
    );

    REG_INPUT: Registro  -- in questo registro si scrive il valore proveniente da A
    Port map(
        clk  => clock,
        rst  => reset,
        write   => load_input,
        data_in  => string_in,
       data_out => input_sommatore
    );




end Structural;
```

L'**unità di controllo di B**, che implementa l'automa in *Figura 11.5,* è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity cu_B is
       port(
             clock, reset: in std_logic;
             count: in std_logic_vector (2 downto 0);
             en_count: out std_logic;
             reset_count: out std_logic;
             read: out std_logic;
             write: out std_logic;
             load_input: out std_logic;
             write_acc: out std_logic; -- per scrivere nel registro ACC il risultato
della somma corrente (per tutte le somme intermedie)
             req_a: in std_logic;
             ack_b: out std_logic
             );
end cu_B;


architecture structural of cu_B is
       type state is (S0, S1, S2,S3, S4,S5,S6);

       signal current_state,next_state: state;

       begin


       reg_stato: process(clock)
                      begin
                      if(clock'event and clock='1') then
                       if(reset='1') then
                               current_state <=S0;
```

```
                    else
                          current_state <=next_state;
                     end if;
                   end if;
                   end process;

      comb: process(current_state, req_a)
            begin


           CASE current_state is

              WHEN S0 =>
                        write<='0';
                        write_acc<='0';
                        en_count<='0';
                               if req_a='1' then
                                  next_state <= S1;
                               elsif req_a='0' then
                                  next_state<=S0;
                               end if;

              WHEN S1 =>
                        ack_b<='1';
                        load_input<='1';
                        if req_a='1' then
                          next_state<=S1;
                        elsif req_a='0' then
                          next_state <= S2;
                        end if;

              WHEN S2 =>
                        load_input<='0';

                        next_state<=S3;

              WHEN S3 =>
                        write_acc<='1';     -- questo è il write nell'accumulatore dove
memorizzo la somma corrente
                        next_state<=S4;

              WHEN S4 =>
                          write_acc<='0';
                     ack_b<='0'; -- indica "ho finito l'elaborazione di questa stringa"
                     en_count<='1';
                     if count="111" then
                            next_state<=S5;
                     else
                       next_state<=S0;
                     end if;

          when S5 =>
                     en_count<='0';
                     write<='1';     -- questo è il write nel registro R
                     next_state<=S6;
```

```
          when S6 =>
                    write<='0';
                    reset_count<='1';
                    next_state<=S0;




             end CASE;

    end process;
 end structural;
```

L'implementazione del sommatore carry look ahead è in Appendice.

I **registri** (R, ACC e REG_INPUT) sono implementati come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Registro is
    Port (
        clk     : in  std_logic;
        rst     : in  std_logic;
        write    : in  std_logic;
        data_in : in  std_logic_vector(3 downto 0);
        data_out: out std_logic_vector(3 downto 0)
    );
end Registro;

architecture Behavioral of Registro is
    signal reg : std_logic_vector(3 downto 0);
begin
    process(clk, rst)
    begin
        if rst = '1' then
            reg <= (others => '0');
        elsif rising_edge(clk) then
            if write = '1' then
                reg <= data_in;
            end if;
        end if;
    end process;
    data_out <= reg;
end Behavioral;
```

L'implementazione del contatore è riportata in Appendice.

*Simulazione*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_com_handshaking is
```

```
end tb_com_handshaking;

architecture Behavioral of tb_com_handshaking is

    signal reset_a_tb    : std_logic := '0';
    signal reset_b_tb    : std_logic := '0';
    signal start_tb    : std_logic := '0';
    signal clock_a_tb  : std_logic := '0';
    signal clock_b_tb  : std_logic := '0';

    constant PERIOD_A_EQUAL   : time := 10 ns;
    constant PERIOD_B_EQUAL   : time := 10 ns;

    constant PERIOD_A_FAST    : time := 6 ns;
    constant PERIOD_B_SLOW    : time := 12 ns;

    constant PERIOD_A_SLOW    : time := 14 ns;
    constant PERIOD_B_FAST    : time := 7 ns;

   -- constant PHASE_SHIFT_B : time := 3 ns; -- sfasamento di clock_b rispetto a
clock_a


    signal test_case : integer := 0;

    component com_handshaking is
        Port (
            start   : in std_logic;
            reset_a   : in std_logic;
            reset_b   : in std_logic;
            clock_a : in std_logic;
            clock_b : in std_logic
        );
    end component;

begin

    uut: com_handshaking
        port map (
            start   => start_tb,
            reset_a   => reset_a_tb,
            reset_b   => reset_b_tb,
            clock_a => clock_a_tb,
            clock_b => clock_b_tb
        );

    -- Processo clock A dinamico
    clk_a_process: process
    begin
        while true loop
            case test_case is
                when 0 => wait for PERIOD_A_EQUAL / 2;
                when 1 => wait for PERIOD_A_FAST / 2;
                when 2 => wait for PERIOD_A_SLOW / 2;
                when others => wait for 10 ns;
            end case;
```

```
        clock_a_tb <= not clock_a_tb;
    end loop;
end process;

-- Processo clock B dinamico
clk_b_process: process
begin
    --wait for PHASE_SHIFT_B;  -- sfasamento iniziale
    while true loop
        case test_case is
            when 0 => wait for PERIOD_B_EQUAL / 2;
            when 1 => wait for PERIOD_B_SLOW / 2;
            when 2 => wait for PERIOD_B_FAST / 2;
            when others => wait for 10 ns;
        end case;
        clock_b_tb <= not clock_b_tb;
    end loop;
end process;

-- Stimolo e sequenza dei test
stim_proc: process
begin
    -- Reset iniziale
    reset_a_tb <= '1';
    reset_b_tb <= '1';
    wait for 20 ns;
    reset_a_tb <= '0';
    reset_b_tb <= '0';

    --------------------------------
    -- Test 1: clock_a = clock_b
    test_case <= 0;
    start_tb <= '1';
    wait for 10 ns;
    start_tb <= '0';
    wait for 700 ns;

    --------------------------------
    -- Test 2: clock_a > clock_b
    test_case <= 1;
    reset_a_tb <= '1';
    reset_b_tb <= '1';
    wait for 20 ns;
    reset_a_tb <= '0';
    reset_b_tb <= '0';
    start_tb <= '1';
    wait for 10 ns;
    start_tb <= '0';
    wait for 700 ns;

    -------------------------------
    -- Test 3: clock_b > clock_a
    test_case <= 2;
    reset_a_tb <= '1';
    reset_b_tb <= '1';
    wait for 20 ns;
```

```
        reset_a_tb <= '0';
        reset_b_tb <= '0';
        start_tb <= '1';
        wait for 14 ns;
        start_tb <= '0';
        wait for 500 ns;

       -- Fine simulazione
        wait;
    end process;

end Behavioral;
```

La simulazione è stata effettuata in tre casi:

- il clock di A è lo stesso di B

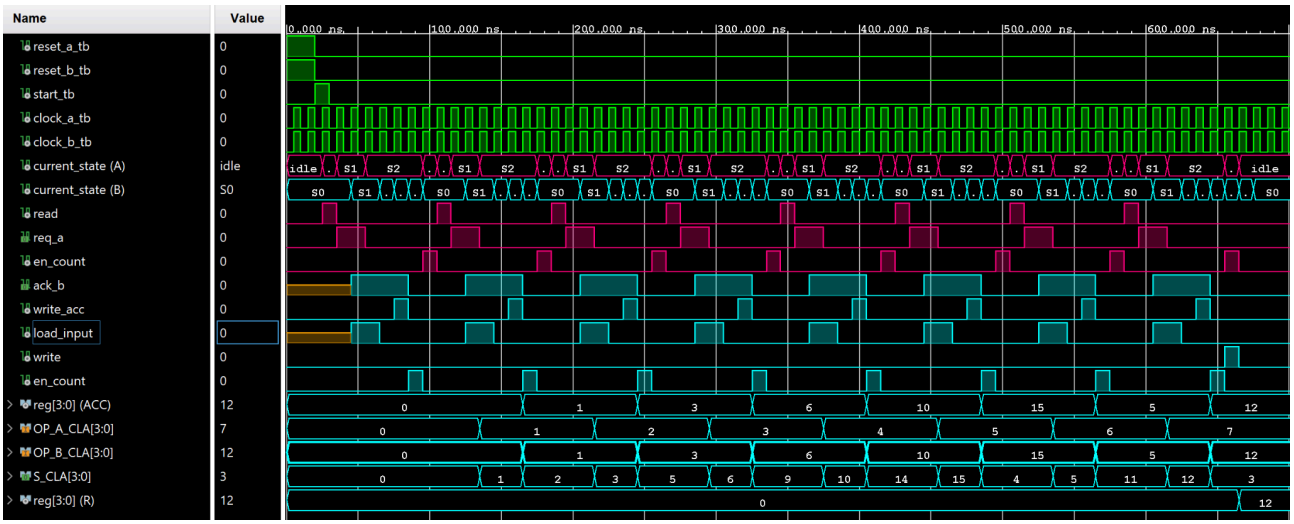



*Figura 12.6 Risultato della simulazione: caso clock_a = clock_b*

- il clock di A è più veloce del clock di B

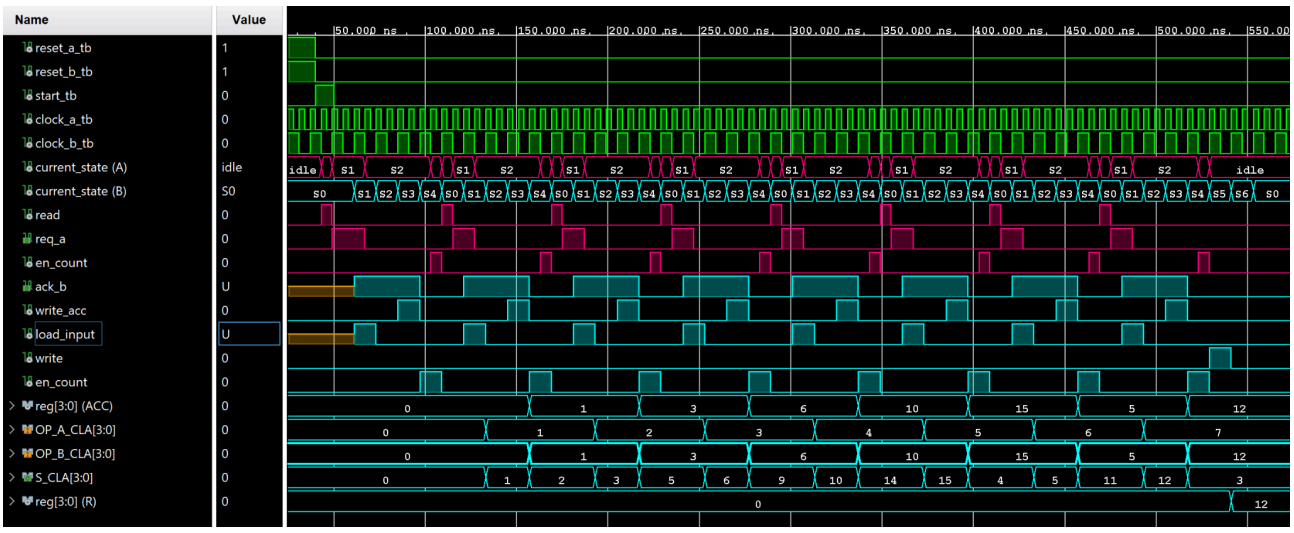



*Figura 10.7 Risultato della simulazione: caso clock_a > clock_b*

- il clock di A è più lento del clock di B

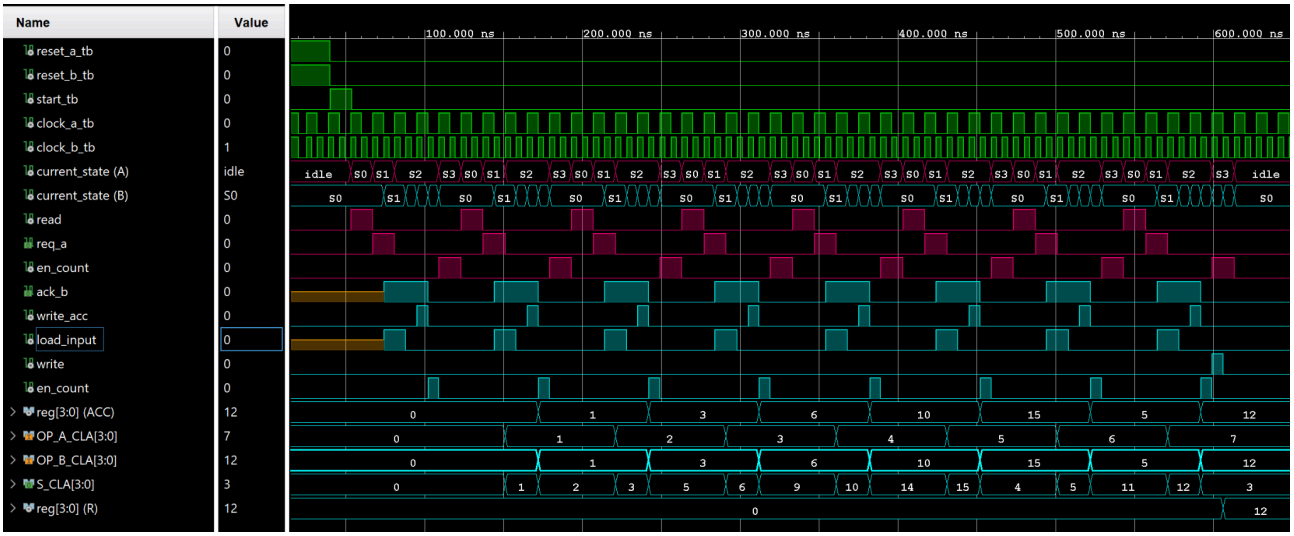



*Figura 10.8 Risultato della simulazione: caso clock_a < clock_b*

# Appendice

## Multiplexer indirizzabile 2:1

### Progetto e architettura

Il multiplexer indirizzabile 2:1 è una macchina combinatoria che seleziona uno tra due ingressi binari (*a0* e *a1*) in base a un segnale di selezione *s* e lo instrada all'uscita. Se *s*=0, l'uscita è *a0*; se *s*=1, l'uscita è *a1*.

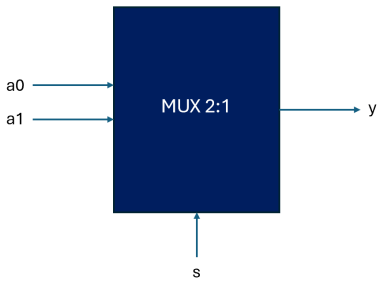



*Figura A.1 Multiplexer indirizzabile 2:1*

### Implementazione

L'entity del multiplexer indirizzabile 2:1 contiene due ingressi dato (*a0* e *a1*) di tipo STD_LOGIC, un ingresso di selezione (*s*) di tipo STD_LOGIC, e un'uscita (*y*) di tipo STD_LOGIC, che rappresenta il valore selezionato tra gli ingressi.

```
entity mux_2_1 is
	port(		a0	: in STD_LOGIC;
			a1	: in STD_LOGIC;
			s	: in STD_LOGIC;
			y	: out STD_LOGIC
	);
end mux_2_1;
```

La descrizione del componente consiste nell'espressione logica corrispondente al multiplexer indirizzabile 2:1, secondo il livello di astrazione dataflow.

```
architecture dataflow of mux_2_1 is
	begin
		y <= ((a0 and (NOT s)) OR (a1 and s));

end dataflow;
```

## Multiplexer indirizzabile 4:1

### Progetto e architettura

Il multiplexer indirizzabile 4:1 è una macchina combinatoria che seleziona uno tra quattro ingressi binari (b*0*, b*1, b2, b3*) in base a due segnali di selezione (*s0* e *s1)* e lo instrada all'uscita.

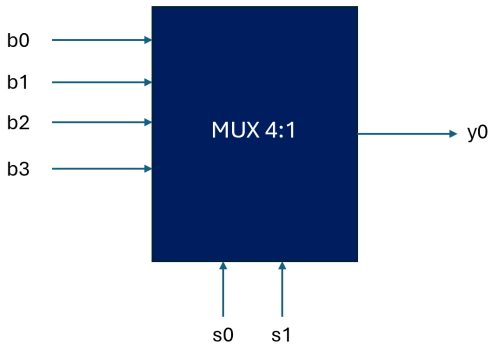



*Figura A.2 Multiplexer indirizzabile 4:1*

Il multiplexer indirizzabile 4:1 è realizzato come una composizione di tre multiplexer 2:1 opportunamente interconnessi.

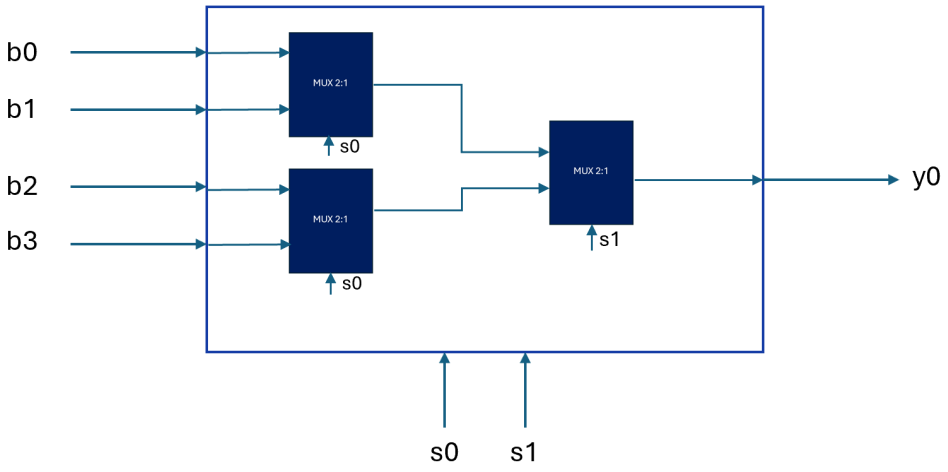



*Figura A.3 Mux 4:1 - Approccio modulare*

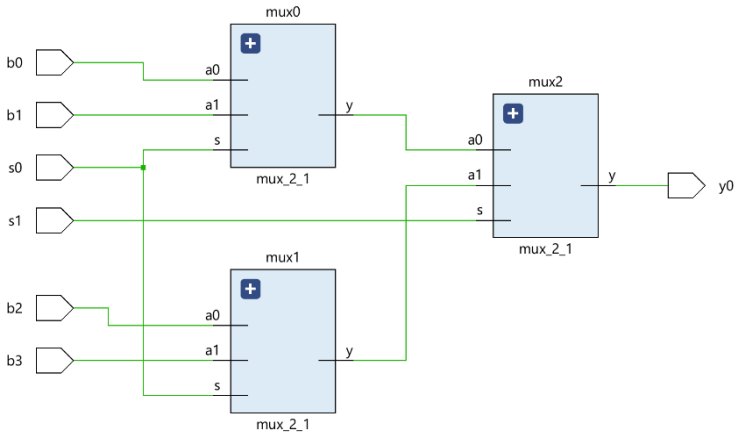



*Figura A.4 Schematic Mux 4:1- Structural*

## Implementazione

L'entity del multiplexer indirizzabile 4:1 contiene quattro ingressi dato (*b0* , *b1, b2, b3*) di tipo STD_LOGIC, due ingressi di selezione (*s0* e *s1*) di tipo STD_LOGIC, e un'uscita (*y0*) di tipo STD_LOGIC, che rappresenta il valore selezionato tra gli ingressi.

```
entity mux_4_1 is
	port(		b0 : in STD_LOGIC;
			b1 : in STD_LOGIC;
			b2 : in STD_LOGIC;
			b3 : in STD_LOGIC;
			s0 : in STD_LOGIC;
			s1 : in STD_LOGIC;
			y0 : out STD_LOGIC
		);
end mux_4_1;
```

La descrizione del componente segue l'approccio strutturale. All'interno del blocco architecture sono dichiarati due segnali (*u0* e *u1*) per realizzare le interconnessioni tra i mux 2:1.

```
architecture structural of mux_4_1 is
	signal u0 : STD_LOGIC := '0';
	signal u1 : STD_LOGIC := '0';


	component mux_2_1
		port(	a0	: in STD_LOGIC;
			a1	: in STD_LOGIC;
			s	: in STD_LOGIC;
			y	: out STD_LOGIC
			);
	end component;

	begin

		mux0: mux_2_1
			Port map(	a0	=> b0,
					a1	=> b1,
```

```
                              s      => s0,
                              y      => u0
                              );
              mux1: mux_2_1
                     Port map(     a0     => b2,
                              a1     => b3,
                              s      => s0,
                              y      => u1
                              );
              mux2: mux_2_1
                     Port map(     a0     => u0,
                              a1     => u1,
                              s      => s1,
                              y      => y0
                           );
end structural;
```

## Demultiplexer indirizzabile 2:1

### Progetto e architettura

Il demultiplexer indirizzabile 1:2 è una macchina combinatoria che seleziona uno tra due ingressi binari (*a0* e *a1*) in base a un segnale di selezione *s* e lo instrada all'uscita. Se *s*=0, l'uscita è *a0*; se *s*=1, l'uscita è *a1*.

Il demultiplexer indirizzabile 1:2 è una macchina combinatoria che riceve un segnale di ingresso binario (*I*) e lo indirizza a una delle due uscite (*Y0* o *Y1*) in base a un segnale di selezione *S*. Se *S*=0, l'ingresso viene instradato all'uscita *Y0*; se *S*=1, l'ingresso viene instradato all'uscita *Y1*.

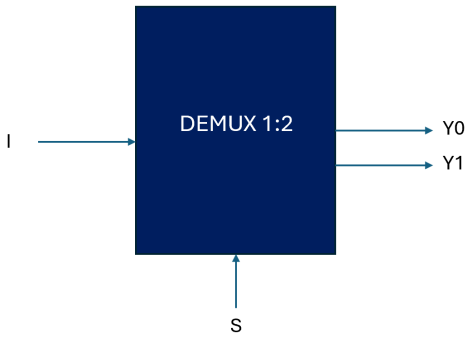



*Figura A.5 Demultiplexer indirizzabile 1:2*

### Implementazione

L'entity del demultiplexer indirizzabile 1:2 contiene un ingresso dato (*I*) di tipo STD_LOGIC, un ingresso di selezione (*S*) di tipo STD_LOGIC, e due uscite (*Y0* e *Y1*) di tipo STD_LOGIC.

```
entity demux1_2 is
       port(  I      : in STD_LOGIC;
                     Y0     : out STD_LOGIC;
```

```
                    Y1      : out STD_LOGIC;
                    S       : in STD_LOGIC
        );
end demux1_2;
```

La descrizione del componente consiste nell'espressione logica corrispondente al demultiplexer indirizzabile 1:2, secondo il livello di astrazione dataflow.

```
architecture dataflow of demux1_2 is
        begin
                Y0<= (I and (not S));
                Y1<= (I and S);
end dataflow;
```

## Button Debouncer

### Progetto e architettura

Il button debouncer prende in input il segnale proveniente dal bottone e genera un segnale “ripulito" che presenta un impulso della durata di un colpo di clock per segnalare l'avvenuta pressione del bottone. Il debouncer implementa un semplice automa di 4 stati: si parte da NOT_PRESSED e, appena si rileva BTN=1, si va in CHK_PRESSED dove si attende un certo tempo in modo da "superare" l'oscillazione: se il bottone è ancora alto si alza il segnale ripulito in output (che avrà la durata di un impulso di clock) e si va in PRESSED; anche qui quando si rileva BTN=0 si va in uno stato intermedio CHK_NOT_PRESSED in cui si aspetta un certo tempo per superare l'oscillazione: se dopo questo tempo è ancora basso si ritorna in NOT_PRESSED.

Con questo automa se si mantiene il bottone premuto non vengono generati più impulsi in uscita.

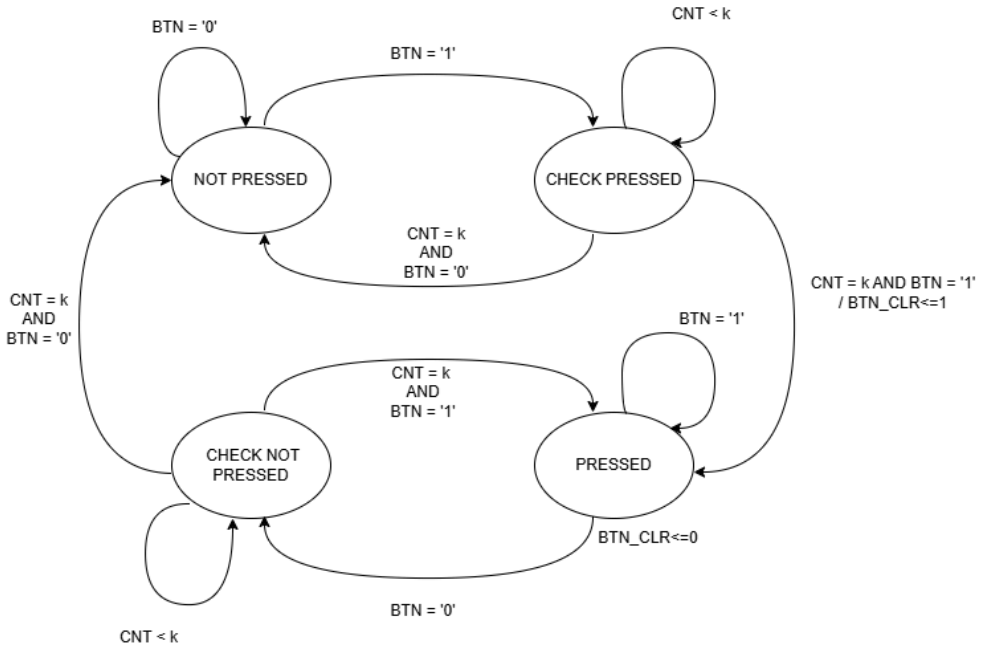



*Figura A.6 Automa del button debouncer*

### Implementazione

L'entity del button debouncer contiene due parametri generici: *CLK_period*, che indica il periodo del clock della scheda in nanosecondi, e *btn_noise_time*, che rappresenta la durata stimata delle oscillazioni del pulsante. Vengono impiegati i generics per rendere il sistema adattabile a diverse frequenze di clock e caratteristiche del pulsante. I segnali in ingresso includono: *RST*, un segnale di reset che riporta il debouncer allo stato iniziale, *CLK*, il segnale di clock che sincronizza l’intero processo, *BTN*, il segnale grezzo proveniente dal pulsante, che potrebbe essere soggetto a rimbalzi. L’uscita è *CLEARED_BTN*, un segnale ripulito che genera un solo impulso per ogni pressione effettiva del pulsante, eliminando i rimbalzi.

L’automa è realizzato mediante un process che si attiva in corrispondenza del fronte di salita del clock.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ButtonDebouncer is
    generic (
        CLK_period: integer := 10;  -- periodo del clock (della board) in nanosecondi
```

```
        btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del
bottone in nanosecondi. il valore di default e’ 10 millisecondi
    );
    Port ( RST : in STD_LOGIC;
           CLK : in STD_LOGIC;
           BTN : in STD_LOGIC;
           CLEARED_BTN : out STD_LOGIC);
end ButtonDebouncer;

architecture Behavioral of ButtonDebouncer is

type stato is (NOT_PRESSED, CHK_PRESSED, PRESSED, CHK_NOT_PRESSED);
signal BTN_state : stato := NOT_PRESSED;

constant max_count : integer := btn_noise_time/CLK_period; -- 10000000/10= conto
1000000 colpi di clock

begin

deb: process (CLK)
variable count: integer := 0;

begin
   if rising_edge(CLK) then

          if( RST = '1') then
              BTN_state <= NOT_PRESSED;
              CLEARED_BTN <= '0';
          else
                case BTN_state is
                     when NOT_PRESSED =>
                         if( BTN = '1' ) then
                                  BTN_state <= CHK_PRESSED;
                            else
                                  BTN_state <= NOT_PRESSED;
                            end if;
            when CHK_PRESSED =>
                if(count = max_count -1) then
                    if(BTN = '1') then --se arrivo a count max ed è  ancora alto vuol
dire che non era un bounce, devo alzare CREARED_BTN
                        count:=0;
                        CLEARED_BTN <= '1';
                        BTN_state <= PRESSED;
                    else
                        count:=0;
                        BTN_state <= NOT_PRESSED;
                    end if;

                else
                    count:= count+1;
                    BTN_state <= CHK_PRESSED;
                end if;

            when PRESSED =>
                CLEARED_BTN<= '0'; --questo lo metto per fare in modo che il segnale
sia alto per un solo impulso di clock
```

```
                        if(BTN = '0') then
                               BTN_state <= CHK_NOT_PRESSED;
                           else
                               BTN_state <= PRESSED;
                           end if;

                    when CHK_NOT_PRESSED =>
                        if(count = max_count -1) then
                    if(BTN = '0') then --se arrivo a count max e BTN è ancora basso
vuol dire che non era un bounce e il bottone è stato rilasciato
                        count:=0;
                        BTN_state <= NOT_PRESSED;
                    else
                        count:=0;
                        BTN_state <= PRESSED;
                    end if;

                when others =>
                    BTN_state <= CHK_NOT_PRESSED;
                end if;

            when others =>
                BTN_state <= NOT_PRESSED;
               end case;
    end if;
  end if;
end process;


end Behavioral;
```

## Ripple Carry Adder

### Progetto e architettura

Il Ripple Carry Adder è un sommatore la cui struttura deriva dall'algoritmo manuale per la somma di due numeri interi positivi di n bit ciascuno. È costituito da n full adder identici connessi in serie.

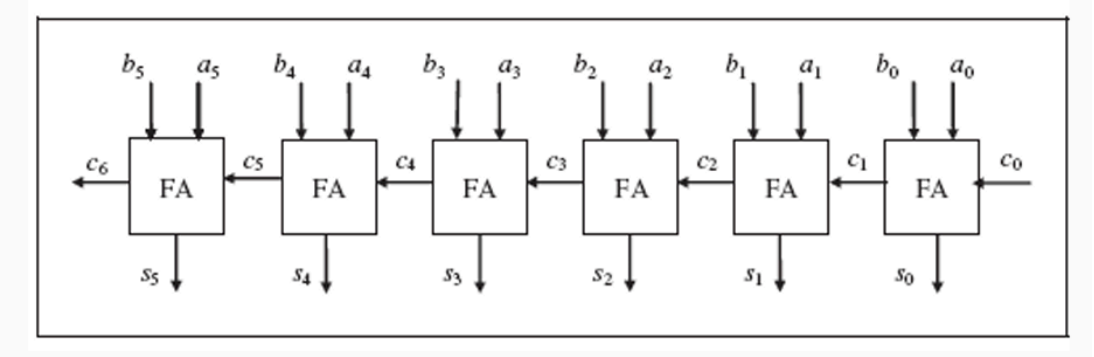



*Figura A.7 Ripple Carry Adder*

La cella a destra riceve i bit meno significativi dei due addendi e un riporto c0=0 e produce la somma s0 e il riporto c1 in ingresso alla seconda cella. La seconda cella da destra riceve i bit a1 e b1 dei due addendi e il riporto in uscita dal full adder precedente, e produce in uscita la somma s1 e il riporto c2, e così via per tutti gli altri stadi.

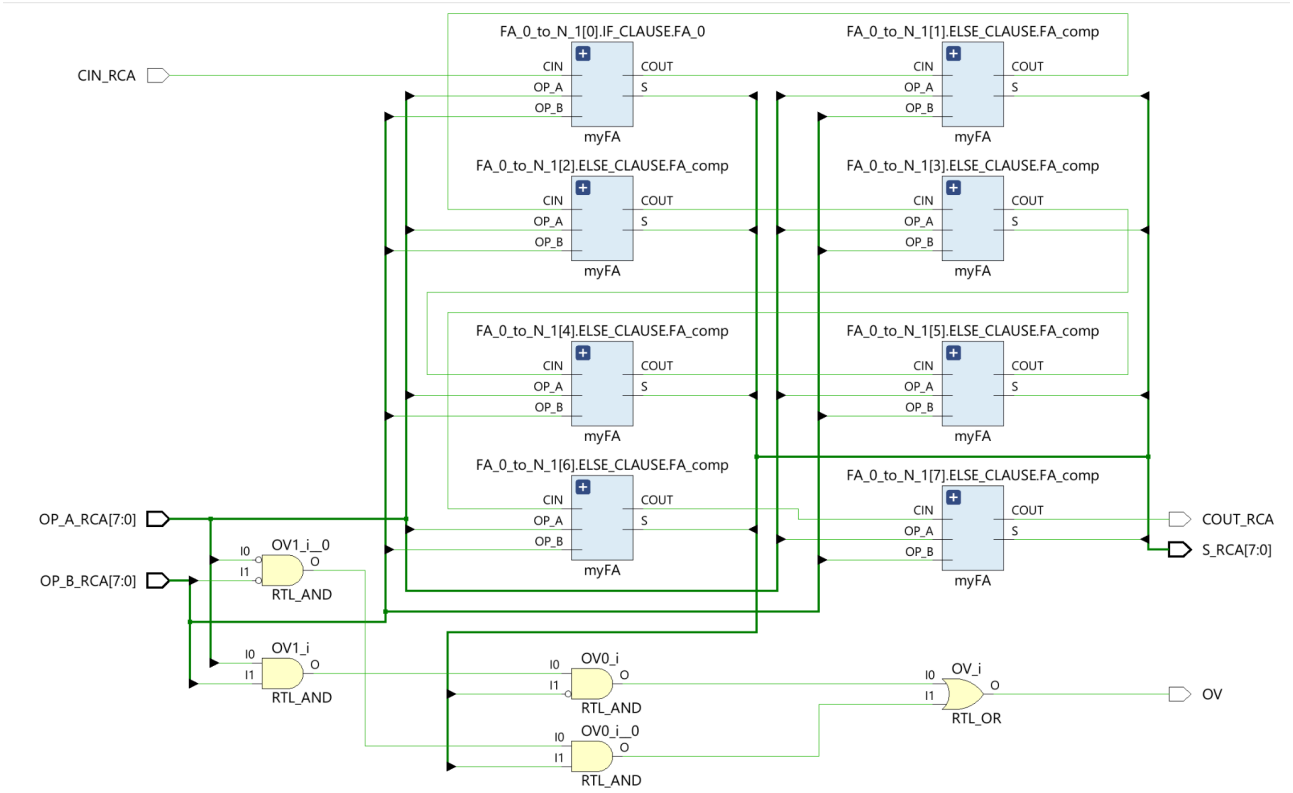



*Figura A.8 Schematic Ripple Carry Adder*

## Implementazione

L'implementazione del ripple carry adder segue un approccio strutturale. Per istanziare gli n full adder, è stato sfruttato il costrutto FOR GENERATE. Poiché, però, il primo full adder è collegato direttamente con l'esterno, è stata usata una if clause per gestire opportunamente la sua istanziazione, differenziandola da quella degli altri, in quanto il carry-in proviene da un ingresso esterno e non dall'uscita del blocco precedente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity RCA_Nbit is
	generic (N: natural range 0 to 32 := 8);
	port(
		OP_A_RCA: in std_logic_vector(N-1 downto 0);
		OP_B_RCA: in std_logic_vector(N-1 downto 0);
		CIN_RCA: in std_logic;

		S_RCA: out std_logic_vector(N-1 downto 0);
		COUT_RCA: out std_logic;
		OV: out std_logic
	);
end RCA_Nbit;

architecture dataflow of RCA_Nbit is

	component myFA is
	   port(
		OP_A: in std_logic;
		OP_B: in std_logic;
		CIN: in std_logic;

		S: out std_logic;
		COUT: out std_logic
		);
	end component;
	signal cout_int: std_logic_vector(N-1 downto 0);
	signal s_int: std_logic_vector(N-1 downto 0);

begin



	FA_0_to_N_1: for i in 0 to N-1 generate

	    IF_CLAUSE: if i=0 generate

	      FA_0: myFA port map(
	      OP_A => OP_A_RCA(0),
	      OP_B => OP_B_RCA(0),
	      CIN => CIN_RCA,
	      S => s_int(0),
	      COUT => cout_int(0)
	      );
	end generate IF_CLAUSE;
```

```
	ELSE_CLAUSE: if i/= 0 generate --non uso ELSIF perchè è supportato in VHDL2008
	                               --che non è la versione di default usata in vivado

	            FA_comp: myFA port map(
	            OP_A => OP_A_RCA(i),
	            OP_B => OP_B_RCA(i),
	            CIN => cout_int(i-1),
	            COUT => cout_int(i),
	            S => s_int(i)
	        );
	    end generate ELSE_CLAUSE;

	end generate FA_0_to_N_1;


	S_RCA <= s_int;
	COUT_RCA <= cout_int(N-1);

	OV <= (OP_A_RCA(N-1) and OP_B_RCA(N-1) and not s_int(N-1)) or (not OP_A_RCA(N-1)
and not OP_B_RCA(N-1) and s_int(N-1));

end dataflow;
```

L'implementazione del full adder segue un approccio data flow ed è riportata di seguito.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity myFA is
	port(
		OP_A: in std_logic;
		OP_B: in std_logic;
		CIN: in std_logic;

		S: out std_logic;
		COUT: out std_logic
	);
end myFA;

architecture dataflow of myFA is
begin
	S <= (OP_A xor OP_B) xor CIN;
	COUT <= (OP_A and OP_B) or (CIN and(OP_A or OP_B));
end dataflow;
```

# Carry Look Ahead

## Progetto e architettura

Il Carry Look Ahead è un sommatore che presenta tempi di propagazione inferiori rispetto al RCA, in quanto, disponendo in anticipo di tutti i riporti, la somma sui vari bit può essere calcolata in parallelo.

Per la realizzazione del Carry Look Ahead, è necessario introdurre le funzioni booleane ausiliare Pi e Gi, che rappresentano rispettivamente la condizione di propagazione e quella di generazione.

$$P_i = x_i + y_i$$
$$G_i = x_i y_i$$

Il riporto in uscita dalla stadio i-esimo assume la forma: $c_{i+1} = G_i + P_i c_i$.

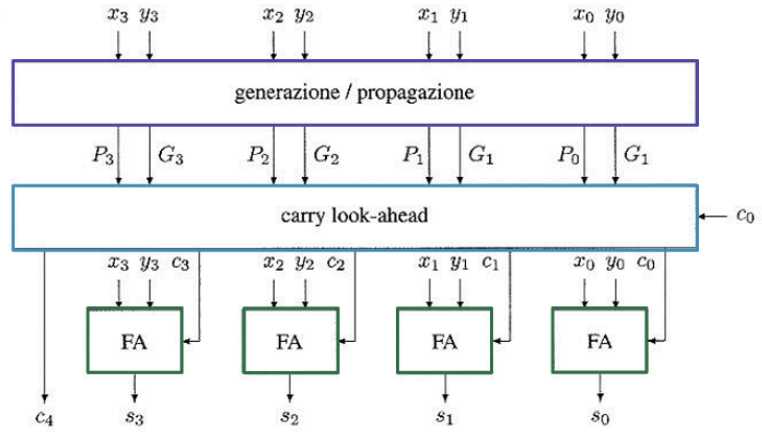



*Figura A.9 Carry Look Ahead*

L'architettura è dunque costituita da tre stadi:

- generazione/propagazione: si occupa semplicemente di calcolare le condizioni di generazione (Gi) e propagazione (Pi), dunque contiene esclusivamente porte AND e OR,
- carry look-ahead: si occupa di calcolare i valori dei riporti (ci),
- full adder: sommano i bit xi e yi con i riporti calcolati dello stadio precedente.

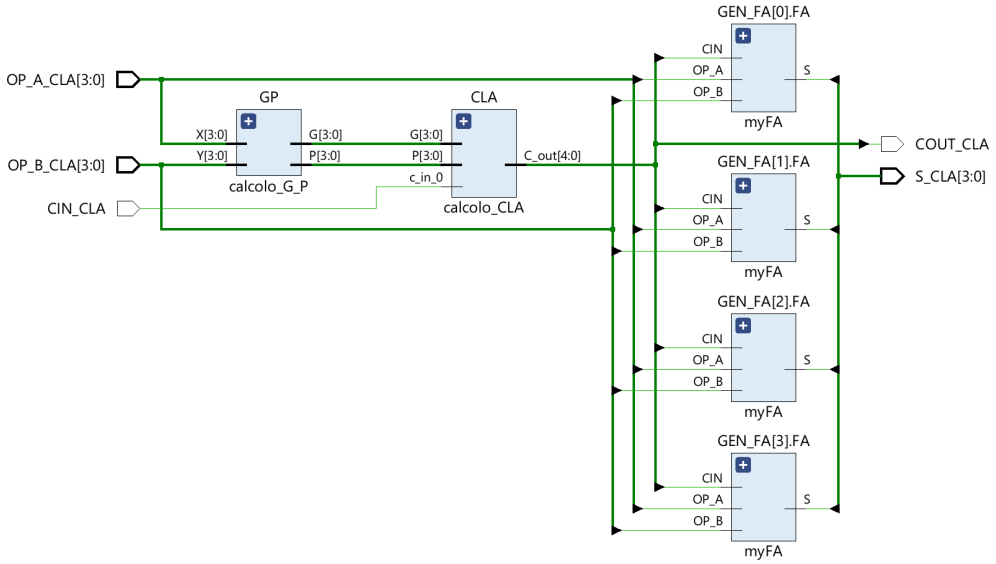



*Figura A.10 Schematic Carry Look Ahead*

## Implementazione

L'implementazione del ripple carry adder segue un approccio strutturale.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity CLA_adder is
	generic (N: natural range 0 to 32 := 4);
	port(
		OP_A_CLA: in std_logic_vector(N-1 downto 0);
		OP_B_CLA: in std_logic_vector(N-1 downto 0);
		CIN_CLA: in std_logic; -- riporto entrante

		S_CLA: out std_logic_vector(N-1 downto 0);
		COUT_CLA: out std_logic -- riporto uscente
	);
end CLA_adder;

architecture Structural of CLA_adder is

     signal G_temp: std_logic_vector(n-1 downto 0); -- condizioni di generazione
     signal P_temp: std_logic_vector(n-1 downto 0);  -- condizioni di propagazione
     signal C_temp: std_logic_vector(n downto 0); -- riporti calcolati dal livello cla
     signal C_FA_temp: std_logic_vector(N-1 downto 0);  -- appoggio per i riporti
uscenti dai FA


component myFA is
	 port(
	    OP_A: in std_logic;
	    OP_B: in std_logic;
	    CIN: in std_logic;

	    S: out std_logic;
	    COUT: out std_logic
```

```
            );
       end component;

component calcolo_G_P is
     generic (      n: integer := 4      );
      Port (
            X: in std_logic_vector(n-1 downto 0);
            Y: in std_logic_vector(n-1 downto 0);
            G: out std_logic_vector(n-1 downto 0); -- condizioni di generazione
            P: out std_logic_vector(n-1 downto 0)  -- condizioni di propagazione

            );
    end component;


component calcolo_CLA is
     generic (      n: integer := 4       );
      Port (
            P: in std_logic_vector(n-1 downto 0);
            G: in std_logic_vector(n-1 downto 0);
            c_in_0: in std_logic; --riporto entrante (c0)
            C_out: out std_logic_vector(n downto 0)  -- riporti calcolati

            );
    end component;

begin

    COUT_CLA<=C_temp(n);

        GP: calcolo_G_P
        Generic map(
                n => 4
                )
        Port map (
                X =>OP_A_CLA,
                Y => OP_B_CLA,
                G => G_temp,
                P => P_temp
                );

       CLA: calcolo_CLA
     Generic map(
             n => 4
             )
      Port map(
            P => P_temp,
            G => G_temp,
            c_in_0 => CIN_CLA,
            C_out => C_temp
            );

             GEN_FA: for i in 0 to N-1 generate

            FA: myFA port map(
                OP_A => OP_A_CLA(i),
```

```
                OP_B => OP_B_CLA(i),
                CIN => C_temp(i),
                S => S_CLA(i),
                COUT => C_FA_temp(i)
            );
        end generate GEN_FA;


       end Structural;
```

Il blocco che calcola le condizioni di generazione e propagazione è il seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity calcolo_G_P is
     generic (      n: integer := 8       );
      Port (
            X: in std_logic_vector(n-1 downto 0);
            Y: in std_logic_vector(n-1 downto 0);
            G: out std_logic_vector(n-1 downto 0); -- condizioni di generazione
            P: out std_logic_vector(n-1 downto 0)  -- condizioni di propagazione

            );
    end calcolo_G_P;

architecture Behavioral of calcolo_G_P is

begin
     G_P: for i in 0 to n-1 generate
                G(i) <= X(i) and Y(i);
                P(i) <= X(i) or Y(i);

            end generate;

end Behavioral;
```

Il blocco che calcola i riporti è il seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity calcolo_CLA is
     generic (      n: integer := 8       );
      Port (
            P: in std_logic_vector(n-1 downto 0);
            G: in std_logic_vector(n-1 downto 0);
            c_in_0: in std_logic; --riporto entrante (c0)
            C_out: out std_logic_vector(n downto 0)  -- riporti calcolati

            );
    end calcolo_CLA;

architecture Behavioral of calcolo_CLA is
    signal carry: std_logic_vector(n downto 0); -- segnale interno
```

```
begin
    carry(0) <= c_in_0;

    cla: for i in 1 to n generate
        carry(i) <= G(i-1) or (P(i-1) and carry(i-1));
    end generate;

    C_out <= carry; -- assegnamento finale all'uscita
end Behavioral;
```

L’implementazione dei full adder è riportata nell’implementazione del ripple carry adder.

## Display a sette segmenti

### Progetto e architettura

La scheda Nexys A7 è dotata di due display a sette segmenti, ciascuno con quattro cifre ad anodo comune. Questi sono configurati per funzionare congiuntamente come un unico display a otto cifre. Ogni cifra è composta da sette segmenti, ciascuno dei quali integra un LED. I sette anodi dei LED che costituiscono ogni cifra sono collegati insieme in un nodo di tipo "anodo comune", mentre i catodi rimangono separati. L'attivazione di una cifra avviene mediante otto segnali di abilitazione digit (digit enable), uno per cifra. Per accendere un determinato segmento, è necessario porre l'anodo a livello logico alto (1) e il corrispondente catodo a livello logico basso (0).

Poiché l'interfaccia hardware consente di impostare simultaneamente i catodi di tutte le cifre, per ottenere una visualizzazione corretta di ciascun numero è necessario implementare una tecnica di multiplexing temporale tramite un circuito di scanning display controller. Questo controller deve gestire in maniera ciclica e continua l'attivazione dei segnali di anodo e i relativi pattern sui catodi per ciascuna cifra, con una frequenza di aggiornamento sufficientemente elevata da risultare impercettibile all'occhio umano.

In pratica, il controller deve:

- attivare temporaneamente un singolo anodo alla volta,
- applicare il pattern dei catodi corrispondente alla cifra da visualizzare,
- disattivare l'anodo corrente e passare a quello successivo.

Questa operazione viene ripetuta ciclicamente su tutte le otto cifre, con una frequenza tale che la persistenza della visione rende l'intera visualizzazione stabile e continua agli occhi dell'osservatore. Generalmente, l'occhio umano non rileva variazioni visive a frequenze superiori a circa 60 Hz. Il manuale della scheda Nexys A7 raccomanda un periodo di aggiornamento compreso tra 1 ms e 16 ms, corrispondente a una frequenza tra 1 kHz e 60 Hz.

Poiché la scheda FPGA dispone di un oscillatore a 100 MHz, è possibile impiegare tale clock come sorgente temporale primaria. Tuttavia, è necessario introdurre un clock divider che consenta di raggiungere il range di frequenze consigliato per il corretto funzionamento del sistema di visualizzazione.

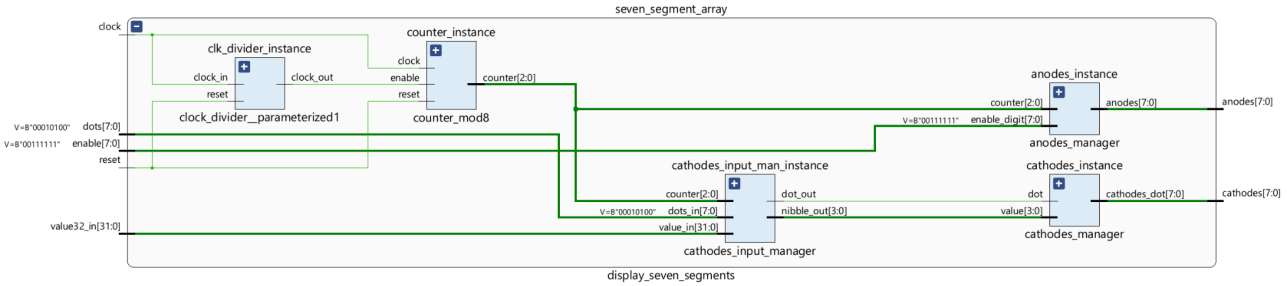



*Figura A.11 Schematic Display Seven Segments*

L'ingresso al sistema di visualizzazione è rappresentato da una parola di 32 bit, dove ogni gruppo di 4 bit codifica una cifra da visualizzare su uno degli 8 display a sette segmenti. Oltre ai dati, il blocco riceve il clock, il reset e due vettori:

- enable: vettore di 8 bit che specifica quali cifre del display devono essere attive. Ogni bit attivo alto (1) abilita la visualizzazione della cifra corrispondente;
- dots: vettore di 8 bit, ciascun bit controlla il punto decimale associato alla rispettiva cifra (bit alto = punto acceso).

Le uscite del blocco sono costituite da:

- un vettore di 8 bit per il controllo degli anodi, utilizzato per abilitare selettivamente una delle otto cifre (*digit enable*),
- un vettore di 8 bit per il controllo dei catodi, dove 7 bit definiscono il pattern di accensione dei segmenti (da *a* a *g*) necessari a rappresentare la cifra, e 1 bit aggiuntivo controlla l'attivazione del punto decimale (*decimal point*).

All'interno, il blocco seven segment display è costituito da:

- un divisore di frequenza, che genera un segnale di abilitazione per il contatore mod8 che viene usato come segnale di conteggio e quindi di fatto fornisce la frequenza con cui viene modificata la cifra da mostrare;
- un contatore modulo 8, il cui valore di conteggio viene usato dal gestore dei catodi e degli anodi per selezionare l'anodo da accendere e il suo rispettivo valore. Il conteggio avviene sul fronte di salita del clock in corrispondenza dell'enable, dato in output dal clock divider;
- un componente denominato "cathodes input manager", che si occupa di selezionare il nibble da mostrare sulla cifra illuminata in base al valore del counter; il componente si occupa anche di accendere o meno il punto;
- un cathodes manager, che codifica i pattern delle cifre selezionate dal contatore;
- un anodes manager, che accende gli anodi in base all'uscita del contatore: a seconda del valore di counter le cifre si illuminano una alla volta da destra a sinistra.

## Implementazione

Il top module è implementato come segue.

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity display_seven_segments is
                  Generic(
                                  clock_frequency_in : integer := 50000000;
                                  clock_frequency_out : integer := 5000000
                                  );
    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC;
           value32_in : in  STD_LOGIC_VECTOR (31 downto 0);
           enable : in  STD_LOGIC_VECTOR (7 downto 0);
           dots : in  STD_LOGIC_VECTOR (7 downto 0);
```

```
           anodes : out  STD_LOGIC_VECTOR (7 downto 0);
           cathodes : out  STD_LOGIC_VECTOR (7 downto 0));
end display_seven_segments;

architecture Structural of display_seven_segments is

signal counter : std_logic_vector(2 downto 0);
signal value_digit : std_logic_vector(3 downto 0);
signal clock_fx,dot_digit : std_logic := '0';

COMPONENT counter_mod8
                  PORT(
                    clock : in  STD_LOGIC;
        reset : in  STD_LOGIC;
                    enable : in STD_LOGIC;
        counter : out  STD_LOGIC_VECTOR (2 downto 0)
                    );
END COMPONENT;


COMPONENT cathodes_input_manager
  PORT (
       counter : in std_logic_vector(2 downto 0);
       value_in: in std_logic_vector(31 downto 0);
       dots_in : in std_logic_vector(7 downto 0);
       nibble_out : out std_logic_vector(3 downto 0);
       dot_out : out std_logic
       );
END COMPONENT;



COMPONENT cathodes_manager
                  PORT(
                    value : in  STD_LOGIC_VECTOR (3 downto 0); --dato da mostrare sulla
cifra del display
        dot : in  STD_LOGIC; --punto
        cathodes_dot : out  STD_LOGIC_VECTOR (7 downto 0)
                    );
END COMPONENT;

COMPONENT anodes_manager
                  PORT(

                    counter : IN std_logic_vector(2 downto 0);
                    enable_digit : IN std_logic_vector(7 downto 0);
                    anodes : OUT std_logic_vector(7 downto 0)
                    );
END COMPONENT;

COMPONENT clock_divider
                  GENERIC(
                                clock_frequency_in : integer := 100000000;--100MHz
                                clock_frequency_out : integer := 500 --500 Hz
                                );
                  PORT(
```

```
                    clock_in : IN std_logic;
        reset : in  STD_LOGIC;
                    clock_out : OUT std_logic
                    );
END COMPONENT;
begin
--il clock divider genera un segnale di abilitazione per il contatore mod8 che viene
usato
--come segnale di conteggio e quindi di fatto fornisce la frequenza con cui viene
modificata
--la cifra da mostrare

clk_divider_instance: clock_divider generic map(
                  clock_frequency_in => clock_frequency_in,
                  clock_frequency_out => clock_frequency_out
                  )
                  port map(
                    clock_in => clock,
                    reset => reset,
                    clock_out => clock_fx
                  );

counter_instance: counter_mod8 port map(
                  clock => clock,
                  enable => clock_fx,
                  reset => reset,
                  counter => counter
);

cathodes_input_man_instance : cathodes_input_manager port map(
       counter => counter,
       value_in => value32_in,
       dots_in => dots,
       nibble_out => value_digit,
       dot_out => dot_digit
);
--il valore di conteggio viene usato dal gestore dei catodi e degli anodi per
--selezionare l'anodo da accendere e il suo rispettivo valore
cathodes_instance: cathodes_manager port map(
                  value => value_digit,
                  dot => dot_digit,
                  cathodes_dot => cathodes
);

anodes_instance: anodes_manager port map(
                  counter => counter,
                  enable_digit => enable,
                  anodes => anodes
);


end Structural;
```

Il divisore di frequenza è il seguente:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- il clock_divider genera un impulso a frequenza inferiore rispetto a quella del clock
-- che viene usato per illuminare i diversi anodi
-- il segnale generato è alto per un solo periodo di clock

entity clock_divider is
                  generic(
                                clock_frequency_in : integer := 100000000;--100MHz
                                clock_frequency_out : integer := 500 --500 Hz
                                );
    Port ( clock_in : in  STD_LOGIC;
                       reset : in STD_LOGIC;
           clock_out : out  STD_LOGIC);
end clock_divider;

architecture Behavioral of clock_divider is

signal clockfx: std_logic := '0';

constant count_max_value : integer := (clock_frequency_in/clock_frequency_out) -1;

begin

clock_out <= clockfx;

count_for_division: process(clock_in)
variable counter : integer range 0 to count_max_value := 0;
begin


                 if clock_in'event and clock_in = '1' then
                  if reset = '1' then
                   counter := 0;
                   clockfx <= '0';
                  else
                   if counter = count_max_value then
                         clockfx <=  '1';
                         counter := 0;
                   else
                         clockfx <=  '0';
                         counter := counter + 1;
                   end if;
                  end if;
                 end if;

end process;


end Behavioral;
```

Il contatore modulo 8 è implementato nel seguente modo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity counter_mod8 is
    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC;
                       enable : in STD_LOGIC; --questo è l'enable del clock, insieme
danno l'impulso di conteggio
           counter : out  STD_LOGIC_VECTOR (2 downto 0));
end counter_mod8;

architecture Behavioral of counter_mod8 is

signal c : std_logic_vector (2 downto 0) := (others => '0');

begin

counter <= c;


counter_process: process(clock)
begin
                  if (clock'event AND clock = '1') then
                    if (reset = '1') then
                    c <= (others => '0');
                    else
                     if (enable='1') then
                    c <= std_logic_vector(unsigned(c) + 1);
                     end if;
                    end if;
                  end if;
end process;

end Behavioral;
```

L’implementazione del componente cathodes input manager è la seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- questo componente si occupa di selezionare il nibble da mostrare sulla cifra
illuminata
-- in base al valore del counter; il componente si occupa anche di accendere o meno il
punto
entity cathodes_input_manager is
  Port (
       counter : in std_logic_vector(2 downto 0);
       value_in: in std_logic_vector(31 downto 0);
       dots_in: in std_logic_vector(7 downto 0); --configurazione dei punti degli 8
display
```

```
       nibble_out : out std_logic_vector(3 downto 0); --nibble da mostrare in base alla
selezione
       dot_out : out std_logic --punto da mostrare in base alla selezione
       );
end cathodes_input_manager;

architecture Behavioral of cathodes_input_manager is

begin

with counter select
   nibble_out <= value_in(31 downto 28) when "111",
            value_in(27 downto 24) when "110",
            value_in(23 downto 20) when "101",
            value_in(19 downto 16) when "100",
            value_in(15 downto 12) when "011",
            value_in(11 downto 8) when "010",
            value_in(7 downto 4) when "001",
            value_in(3 downto 0) when "000",
            "0000" when others;

with counter select
   dot_out <= dots_in(7)when "111",
            dots_in(6) when "110",
            dots_in(5) when "101",
            dots_in(4) when "100",
            dots_in(3) when "011",
            dots_in(2) when "010",
            dots_in(1) when "001",
            dots_in(0) when "000",
            '0' when others;

end Behavioral;
```

Il componente cathodes manager è il seguente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity cathodes_manager is
    Port ( value : in  STD_LOGIC_VECTOR (3 downto 0); --dato da mostrare sulla cifra
del display
           dot : in  STD_LOGIC; --configurazione del punto: acceso o spento
           cathodes_dot : out  STD_LOGIC_VECTOR (7 downto 0)); --indica i 7 catodi più
il punto
end cathodes_manager;

architecture Behavioral of cathodes_manager is

-- i catodi sono collegati nel seguente ordine:
-- cathodes[0]=CA
```

```
-- cathodes[2]=CB
--....
-- i catodi sono 0-attivi
constant zero   : std_logic_vector(6 downto 0) := "1000000";
constant one    : std_logic_vector(6 downto 0) := "1111001";
constant two    : std_logic_vector(6 downto 0) := "0100100";
constant three  : std_logic_vector(6 downto 0) := "0110000";
constant four   : std_logic_vector(6 downto 0) := "0011001";
constant five   : std_logic_vector(6 downto 0) := "0010010";
constant six    : std_logic_vector(6 downto 0) := "0000010";
constant seven  : std_logic_vector(6 downto 0) := "1111000";
constant eight  : std_logic_vector(6 downto 0) := "0000000";
constant nine   : std_logic_vector(6 downto 0) := "0010000";
constant a        : std_logic_vector(6 downto 0) := "0001000";
constant b      : std_logic_vector(6 downto 0) := "0000011";
constant c      : std_logic_vector(6 downto 0) := "1000110";
constant d      : std_logic_vector(6 downto 0) := "0100001";
constant e      : std_logic_vector(6 downto 0) := "0000110";
constant f      : std_logic_vector(6 downto 0) := "0001110";

signal cathodes : std_logic_vector(6 downto 0); --segnale temporaneo per codificare
                                                --il pattern sui 7 led in base alla
cifra da mostrare (value)

begin


seven_segment_decoder_process: process
  begin
    case value is
      when "0000" => cathodes <= zero;
      when "0001" => cathodes <= one;
      when "0010" => cathodes <= two;
      when "0011" => cathodes <= three;
      when "0100" => cathodes <= four;
      when "0101" => cathodes <= five;
      when "0110" => cathodes <= six;
      when "0111" => cathodes <= seven;
      when "1000" => cathodes <= eight;
      when "1001" => cathodes <= nine;
      when "1010" => cathodes <= a;
      when "1011" => cathodes <= b;
      when "1100" => cathodes <= c;
      when "1101" => cathodes <= d;
      when "1110" => cathodes <= e;
      when "1111" => cathodes <= f;
                   when others => cathodes <= (others => '0');
    end case;
  end process seven_segment_decoder_process;

cathodes_dot <= (not dot)&cathodes; --segnale complessivo di 7+1 bit con catodi e punto



end Behavioral;
```

Infine, l'anodes manager è implementato come segue.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity anodes_manager is
    Port ( counter : in  STD_LOGIC_VECTOR (2 downto 0);
           enable_digit : in  STD_LOGIC_VECTOR (7 downto 0);
           anodes : out  STD_LOGIC_VECTOR (7 downto 0)
                     );
end anodes_manager;

architecture Behavioral of anodes_manager is

signal anodes_switching : std_logic_vector(7 downto 0) := (others => '0');

begin

anodes <= not anodes_switching OR not enable_digit;

anodes_process: process(counter)
begin
      --a seconda del valore di counter le cifre si illuminano una alla volta da destra
      a sinistra
      case counter is
            when "000" =>
                  anodes_switching <= "00000001";
            when "001" =>
                  anodes_switching <= "00000010";
            when "010" =>
                  anodes_switching <= "00000100";
            when "011" =>
                  anodes_switching <= "00001000";
            when "100" =>
                  anodes_switching <= "00010000";
            when "101" =>
                  anodes_switching <= "00100000";
            when "110" =>
                  anodes_switching <= "01000000";
            when "111" =>
                  anodes_switching <= "10000000";
            when others =>
                  anodes_switching <= (others => '0');
      end case;

end process;


end Behavioral;
```

## Contatore modulo N

### Progetto e architettura

Il contatore è realizzato utilizzando due processi separati, entrambi con il clock nella sensitivity list. Sono stati utilizzati i generics per rendere il contatore flessibile, permettendo di configurare il numero di bit e il modulo in base al tipo di contatore.Il primo processo si occupa del conteggio vero e proprio: ad ogni fronte di discesa del clock, verifica se il segnale di **reset** è attivo (in tal caso azzera il contatore), se il segnale **set** è attivo (inizializza il contatore con un valore specificato), o se il segnale **enable** è attivo (in tal caso incrementa il contatore, o, se  il contatore raggiunge il valore massimo, lo azzera). Il secondo processo gestisce il segnale **umax**, che indica quando il contatore ha raggiunto il valore massimo. Quando si verifica un fronte di salita del clock aggiorna **umax** a '1' quando il contatore raggiunge il valore massimo, altrimenti lo mantiene a '0'.

### Implementazione

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;



entity counter is
    generic(

        n: integer := 3 ;--numero di bit necessari
        modulo: integer := 8

        );

    Port ( clock : in  STD_LOGIC;
           reset : in  STD_LOGIC; --ingresso di reset per azzerare il contatore
                enable : in STD_LOGIC;
           umax: out STD_LOGIC; -- si alza quando si raggiunge il valore massimo di
conteggio
           counter : out  STD_LOGIC_VECTOR (n-1 downto 0));


end counter;

architecture Behavioral of counter is

signal c : std_logic_vector (n-1 downto 0) := (others => '0');


signal u : std_logic := '0';

begin
```

```
counter <= c; --valore attuale del contatore


counter_process: process(clock)
begin

         if rising_edge(clock) then
          if (reset = '1') then  --sia reset che set sono sincroni con il clock.
            c <= (others => '0');
          elsif (enable='1') then

              if( c=std_logic_vector(TO_UNSIGNED(modulo-1,n))) then
                c <= (others => '0');

               else
                     c <= std_logic_vector(unsigned(c) + 1);

              end if;
           end if;
          end if;
    end process;



     proc_count_max: process(clock)
     begin
        if rising_edge(clock) then
            if (reset = '1') then
                u <= '0'; --reset del segnale
            elsif (c= std_logic_vector(TO_UNSIGNED(modulo-1,n))) then --cast per type
mismatch
                u <= '1'; --attivo quando raggiungo il massimo
            else
                u <= '0'; --se no rimane basso
            end if;
        end if;
     end process;



   umax<=u;

end Behavioral;
```